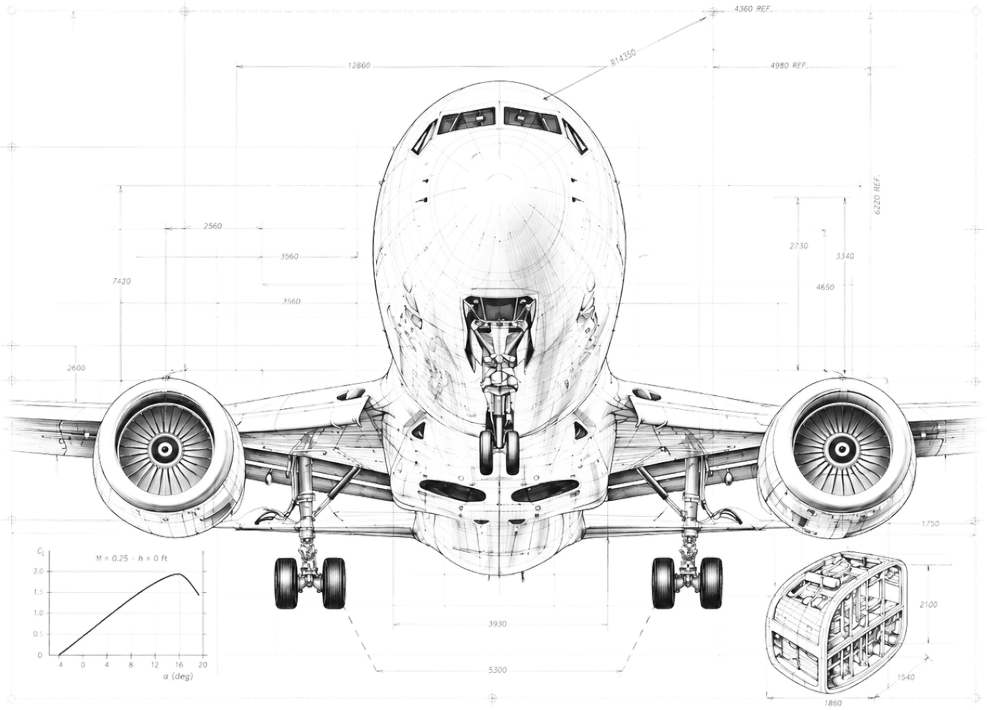
Costruzioni Aeronautiche

ITIS “E. Majorana” — Fluidodinamica numerica

Flussi viscosi

Dallo strato limite alla stabilità idrodinamica: equazioni paraboliche, flussi di Stokes e metodi spettrali



Monografia didattica di fluidodinamica numerica
Modulo 3: flussi viscosi

Prof. Ing. Biagio Raucci

Esempi numerici in Python (NumPy, SciPy, Matplotlib)

Indice

Presentazione **vii**

1 Le equazioni del moto dei fluidi viscosi **1**
1.1 Dal fluido ideale al fluido reale 1
1.2 Gli sforzi in un fluido newtoniano 2
1.3 Le equazioni di bilancio 2
1.3.1 Continuità e quantità di moto 2
1.3.2 Energia e funzione di dissipazione 3
1.4 Il fluido incomprimibile a proprietà costanti 4
1.5 Adimensionalizzazione e numero di Reynolds 4
1.6 Vorticità e funzione di corrente 5
1.7 Soluzioni esatte: le correnti parallele 5
1.7.1 La famiglia di Couette–Poiseuille 5
1.7.2 La corrente di Hagen–Poiseuille 8
1.8 Approfondimenti 8
1.8.1 L'equazione di Cauchy e la simmetria del tensore degli sforzi 8
1.8.2 Il bilancio dell'energia meccanica 9
1.8.3 Le condizioni al contorno 10
1.8.4 La diffusione della vorticità: il vortice di Lamb–Oseen 10
1.8.5 Analisi dimensionale: la resistenza di una sfera 11
1.9 Un panorama dei problemi del modulo 11

2 Lo strato limite laminare sulla lamina piana **13**
2.1 L'idea di Prandtl 13
2.2 Le equazioni dello strato limite 13
2.3 Il problema di Blasius 14
2.3.1 La trasformazione di similitudine 14
2.4 Il metodo di tiro 15
2.4.1 La condizione all'infinito e il dimezzamento dell'intervallo 16
2.5 Le grandezze ingegneristiche dello strato limite 19
2.6 Metodi di tiro più efficienti e studio dell'accuratezza 21
2.7 Approfondimenti 25

2.7.1 L'equazione integrale di von Kármán e il metodo di Pohlhausen 25
2.7.2 La serie di Blasius 28
2.7.3 La resistenza di una lamina e la scia 29
2.8 Oltre la lamina piana 30

3 Lo strato limite termico: il termometro a lamina piana 31
3.1 Il problema fisico 31
3.2 Le equazioni e la loro riduzione 31
3.3 Le differenze finite con una condizione sulla derivata 32
3.3.1 Il sistema tridiagonale 32
3.3.2 Le condizioni al contorno 33
3.4 Il Programma 3.2 33
3.5 Il fattore di recupero 35
3.6 Soluzioni per quadratura e ruolo del numero di Prandtl 36
3.7 Approfondimenti 41
3.7.1 Parete a temperatura imposta e velocità elevata: la temperatura motrice . 41
3.7.2 L'analogia di Reynolds e la correzione di Colburn 42
3.7.3 Gli spessori dei due strati limite 42
3.7.4 Quando conta la dissipazione: il numero di Eckert 43

4 Correnti in condotti e canali a pelo libero 44
4.1 La corrente in un condotto di sezione qualsiasi 44
4.2 Il canale inclinato a pelo libero 45
4.2.1 Sovrarilassamento con una condizione di derivata 45
4.2.2 La portata 46
4.2.3 L'effetto frenante delle pareti laterali 48
4.3 Validazione con la soluzione esatta 49
4.4 Approfondimenti 52
4.4.1 Fattore d'attrito e diametro idraulico 52
4.4.2 L'analogia di Prandtl: dalla portata alla rigidezza torsionale 53
4.4.3 Quante iterazioni servono: la convergenza del sovrarilassamento 53
4.4.4 La lunghezza d'imbocco 54

5 Equazioni paraboliche I: metodi espliciti e il problema di Rayleigh 55
5.1 La lastra messa bruscamente in moto 55
5.2 Lo schema esplicito 56
5.3 Stabilità 56
5.4 La corrente di Couette che si avvia: il Programma 3.4 57
5.5 L'accuratezza dello schema e il caso $R = 1/6$ 61
5.6 La velocità di propagazione dello schema esplicito 63
5.7 Approfondimenti 64

5.7.1 Il metodo delle linee e l'analisi matriciale della stabilità 64
5.7.2 Un'insidia: lo schema di Richardson 65
5.7.3 Coefficienti variabili e forma conservativa 65
5.7.4 La stessa equazione per il calore 65

6 Equazioni paraboliche II: metodi impliciti e l'avviamento di una corrente in un canale 67
6.1 Il metodo implicito 67
6.2 L'avviamento di una corrente in un canale 68
6.3 Il metodo di Crank–Nicolson 71
6.4 Esplicito o implicito? Un bilancio dei costi 76
6.5 Approfondimenti 76
6.5.1 Perché l'algoritmo di Thomas funziona senza pivoting 76
6.5.2 Lo schema ϑ: stabilità, accuratezza e la scelta di Crandall 77
6.5.3 Il rimedio alle oscillazioni di Crank–Nicolson 78
6.5.4 Due dimensioni: il metodo ADI 78

7 Flussi di Stokes e l'equazione biarmonica 83
7.1 Il limite dei piccoli numeri di Reynolds 83
7.2 L'equazione biarmonica 84
7.3 La cavità mossa da una parete 84
7.3.1 La vorticità di parete 85
7.3.2 L'algoritmo iterativo 86
7.4 La soluzione diretta e le griglie fini 90
7.5 Approfondimenti 95
7.5.1 La sfera in moto lento e la legge di Stokes 95
7.5.2 Il paradosso di Stokes 96
7.5.3 La teoria della lubrificazione 96
7.5.4 La derivazione dei vortici di Moffatt 101

8 Stabilità idrodinamica I: la convezione di Rayleigh–Bénard 102
8.1 Stabilità di una soluzione 102
8.2 Il problema di Rayleigh–Bénard 102
8.3 Linearizzazione e modi normali 103
8.3.1 Le condizioni al contorno 104
8.4 La discretizzazione alle differenze finite 105
8.5 Il programma e la sua validazione 106
8.6 Approfondimenti 109
8.6.1 Il principio di scambio delle stabilità: dimostrazione 109
8.6.2 Il Programma 3.7c: tassi di crescita, altre condizioni al contorno, stima variazionale 110
8.6.3 Oltre la soglia 114

9 Stabilità idrodinamica II: Chebyshev e Orr–Sommerfeld 115
9.1 L'idea dei metodi spettrali . . . 115
9.2 I polinomi di Chebyshev . . . 115
9.3 La matrice di differenziazione . . . 116
9.4 Un problema ai limiti con la collocazione di Chebyshev . . . 116
9.5 Perché i nodi di Chebyshev: il fenomeno di Runge e l'accuratezza spettrale . . . 119
9.6 La stabilità della corrente di Poiseuille piana . . . 122
9.6.1 Il problema agli autovalori generalizzato . . . 123
9.7 La curva di stabilità neutra . . . 127
9.8 Approfondimenti sulla stabilità delle correnti parallele . . . 129
9.8.1 Il teorema di Squire . . . 129
9.8.2 Il limite non viscoso: i teoremi di Rayleigh e di Fjørtoft . . . 130
9.8.3 L'equazione dell'energia di Reynolds–Orr . . . 130
9.8.4 Perché la transizione avviene prima: la non normalità . . . 131

10 Metodi pseudo-spettrali di Fourier 132
10.1 Problemi periodici . . . 132
10.2 La trasformata discreta di Fourier . . . 132
10.3 La derivazione nello spazio dei numeri d'onda . . . 133
10.4 Un problema ai limiti periodico . . . 133
10.5 Convezione e diffusione in un campo di moto assegnato . . . 136
10.6 Approfondimenti . . . 140
10.6.1 Perché i coefficienti di Fourier decrescono così in fretta . . . 140
10.6.2 L'aliasing e la regola dei 3/2 . . . 141
10.6.3 Perché la trasformata rapida costa $N \log N$ . . . 146

A Simboli, listati di supporto e istruzioni 147
A.1 Simboli principali . . . 147
A.2 Listati di supporto . . . 147
A.3 Come riprodurre i risultati . . . 151
A.4 Codici in rete e QR code . . . 151
A.5 Come sostituire le figure . . . 152

B Esercizi svolti 155
B.1 Esercizio 1 – Il problema di Blasius senza iterazioni . . . 155
B.2 Esercizio 2 – Lamina con iniezione e aspirazione . . . 156
B.3 Esercizio 3 – Strati limite con gradiente di pressione: Falkner–Skan . . . 159
B.4 Esercizio 4 – L'entalpia totale nello strato limite termico . . . 160
B.5 Esercizio 5 – Il fluido attorno a un cilindro rotante . . . 162
B.6 Esercizio 6 – Canale con parete superiore mobile . . . 164
B.7 Esercizio 7 – Lastra mobile di larghezza parziale . . . 165

B.8 Esercizio 8 – Lo schema esplicito oltre il limite di stabilità 166
B.9 Esercizio 9 – La messa in rotazione del fluido attorno a un cilindro 167
B.10 Esercizio 10 – La lastra oscillante 168
B.11 Esercizio 11 – Lo schema di DuFort–Frankel 169
B.12 Esercizio 12 – L'avviamento della corrente in un tubo 171
B.13 Esercizio 13 – Cavità con superficie libera e fondo mobile 172
B.14 Esercizio 14 – La curva di stabilità neutra di Bénard 174
B.15 Esercizio 15 – La stabilità della corrente di Couette–Poiseuille 175
B.16 Esercizio 16 – Convezione esplicita e diffusione implicita 177
B.17 Esercizio 17 – Progetto: condotti di varie sezioni 179
B.18 Esercizio 18 – Progetto: la corrente pulsante in un tubo 181
Listato completo degli esercizi 183

Bibliografia **207**

Elenco dei listati

1.1 `prog3_0_soluzioni_esatte.py` – famiglia di Couette–Poiseuille 6
2.1 Programma 3.1 – problema di Blasius, tiro e dimezzamento 17
2.2 `prog3_1b_tiro.py` – Newton, secante, accuratezza 21
2.3 `prog3_1c_karman.py` – metodo integrale e serie di Blasius 26
3.1 Programma 3.2 – temperatura di parete adiabatica 34
3.2 `prog3_2b_prandtl.py` – recupero e Nusselt per quadratura 38
4.1 Programma 3.3 – canale a pelo libero, SOR 46
4.2 `prog3_3b_condotti.py` – soluzione esatta e convergenza 49
5.1 Programma 3.4 – metodo esplicito, avvio della corrente di Couette 58
5.2 `prog3_4b_accuratezza.py` – accuratezza e stabilità 61
6.1 Programma 3.5 – metodo implicito, avviamento in un canale 69
6.2 `prog3_5b_crank_nicolson.py` – implicito e Crank–Nicolson 73
6.3 `prog3_5c_adi.py` – metodo ADI in due dimensioni 79
7.1 Programma 3.6 – cavità di Stokes 86
7.2 `prog3_6b_cavita_fine.py` – cavità su griglie fini 90
7.3 `prog3_6c_lubrificazione.py` – pattino e sfera di Stokes 97
8.1 `prog3_7a_benard.py` – stabilità marginale di Bénard 106
8.2 `prog3_7c_benard_varianti.py` – tassi di crescita e condizioni al contorno . 110
9.1 Programma 3.7 – collocazione di Chebyshev per un problema ai limiti 117
9.2 `prog3_7b_runge.py` – fenomeno di Runge e costante di Lebesgue 120
9.3 Programma 3.8 – equazione di Orr–Sommerfeld 124
9.4 `prog3_8b_curva_neutra.py` – curva neutra e punto critico 127
10.1 Programma 3.9 – problema ai limiti con la FFT 134
10.2 Programma 3.10 – convezione–diffusione con Fourier in y 137
10.3 `prog3_9b_burgers.py` – Burgers pseudo-spettrale, aliasing, FFT 141
A.1 `metodi_numerici.py` – modulo comune 147
A.2 `stile_grafici.py` – impostazioni di Matplotlib 150
B.1 `esercizi_svolti.py` (esercizi 1–18) 183

Presentazione

Di che cosa parla questa monografia

Nel secondo modulo del corso la viscosità era stata messa da parte: per un velivolo in volo il numero di Reynolds è dell'ordine di 10^6–10^8 e, fuori da un sottile strato aderente alla parete, il fluido si comporta come se fosse ideale. Con questa ipotesi la teoria del potenziale prevede la portanza di un profilo alare con ottima precisione. Ma non sa dire nulla sulla *resistenza* (paradosso di d'Alembert), sul riscaldamento di una parete lambita da una corrente veloce, sulla portata di un condotto, sull'istante in cui una corrente laminare diventa turbolenta. Per rispondere a queste domande bisogna rimettere in gioco gli sforzi viscosi, cioè risolvere le **equazioni di Navier–Stokes**, o almeno le loro forme semplificate.

Le equazioni complete sono non lineari e accoppiate, e la loro soluzione generale è affrontata nel modulo successivo. Questo terzo modulo percorre invece una strada più graduale: sceglie problemi in cui la geometria o l'ordine di grandezza dei termini permettono di eliminare una parte delle difficoltà, e ne approfitta per introdurre, uno alla volta, gli strumenti numerici che serviranno poi nel caso generale. Ne risulta un percorso attraverso tutte le classi di equazioni della fluidodinamica viscosa:

- lo **strato limite** laminare, in cui un'opportuna variabile di similitudine trasforma le equazioni alle derivate parziali in equazioni *ordinarie* non lineari, risolte con il metodo di tiro, e lo strato limite *termico*, risolto con le differenze finite e condizioni al contorno sulla derivata;
- le correnti nei **condotti**, governate da un'equazione di Poisson (ellittica);
- le correnti viscose non stazionarie fra pareti parallele, governate dall'equazione della diffusione (**parabolica**), con i metodi espliciti e impliciti e la loro stabilità;
- le correnti a bassissimo numero di Reynolds (**flussi di Stokes**), governate dall'equazione biarmonica, con il problema delle condizioni al contorno sulla vorticità;
- la **stabilità idrodinamica**: la convezione di Rayleigh–Bénard e la transizione della corrente di Poiseuille, che conducono a problemi agli *autovalori* generalizzati, e i **metodi spettrali** di Chebyshev e di Fourier che li risolvono con un'accuratezza esponenziale.

Questa monografia costituisce il terzo modulo del corso di fluidodinamica numerica. I temi seguono una progressione classica della letteratura, qui rielaborata con dimostrazioni complete, programmi in Python corredati di grafici ed esercizi interamente svolti. Ogni capitolo si chiude con una sezione di *approfondimenti*, che estende il tema del capitolo con risultati teorici (dimostrazioni di teoremi classici, stime variazionali, analisi di stabilità più generali), applicazioni tecniche (torsione delle travi, lubrificazione, riscaldamento aerodinamico, vortici di scia) e programmi aggiuntivi. Come nei moduli precedenti, ogni risultato numerico viene confrontato con una soluzione esatta o con una soluzione di riferimento ottenuta per estrapolazione: sapere

quanto è sbagliato un risultato è parte integrante del risultato stesso. La bibliografia raccoglie le fonti di riferimento: fra queste, i testi classici di meccanica dei fluidi computazionale [1, 2], Schlichting [3] e White [4] per lo strato limite, Batchelor [5] per le soluzioni esatte, Tannehill, Anderson e Pletcher [17] e Ferziger e Perić [18] per la discretizzazione delle equazioni paraboliche ed ellittiche, Drazin e Reid [28] e Chandrasekhar [29] per la stabilità, Trefethen [37] e Canuto, Hussaini, Quarteroni e Zang [38] per i metodi spettrali.

Come è organizzata

Capitolo 1 – Le equazioni del moto dei fluidi viscosi.
Sforzi viscosi, equazioni di Navier–Stokes e dell'energia, funzione di dissipazione; il fluido incomprimibile, il numero di Reynolds, la vorticità; le soluzioni esatte di Couette e di Poiseuille. Approfondimenti: equazione di Cauchy e simmetria degli sforzi, bilancio dell'energia meccanica, condizioni al contorno, vortice di Lamb–Oseen, analisi dimensionale.

Capitolo 2 – Lo strato limite laminare sulla lamina piana.
Le equazioni di Prandtl, la soluzione di similitudine di Blasius, il metodo di tiro con Runge–Kutta e dimezzamento dell'intervallo; tiro di Newton, accuratezza, spessori e coefficiente d'attrito. Approfondimenti: equazione integrale di von Kármán e metodo di Pohlhausen, serie di Blasius e suo raggio di convergenza, resistenza di una lamina.

Capitolo 3 – Lo strato limite termico: il termometro a lamina piana.
L'equazione dell'energia di similitudine, le condizioni al contorno sulla derivata e il nodo fittizio; il fattore di recupero e la sua dipendenza dal numero di Prandtl; soluzioni per quadratura. Approfondimenti: temperatura motrice e riscaldamento aerodinamico, analogia di Reynolds e correzione di Colburn, spessori dei due strati, numero di Eckert.

Capitolo 4 – Correnti in condotti e canali a pelo libero.
L'equazione di Poisson per la velocità assiale, il sovrarilassamento con condizione di derivata, il calcolo della portata; validazione con la soluzione in serie e influenza delle pareti laterali. Approfondimenti: fattore d'attrito e diametro idraulico, analogia di Prandtl con la torsione delle travi, convergenza del sovrarilassamento, lunghezza d'imbocco.

Capitolo 5 – Equazioni paraboliche I: metodi espliciti.
Il problema di Rayleigh generalizzato; lo schema in avanti nel tempo e centrato nello spazio, la sua stabilità e la sua accuratezza; il caso speciale $R = 1/6$. Approfondimenti: metodo delle linee e analisi matriciale, schema di Richardson, forma conservativa, conduzione del calore.

Capitolo 6 – Equazioni paraboliche II: metodi impliciti.
L'avviamento di una corrente in un canale; il metodo implicito, il metodo di Crank–Nicolson, le loro proprietà di smorzamento. Approfondimenti: algoritmo di Thomas e dominanza diagonale, schema ϑ e scelta di Crandall, avvio di Rannacher, metodo ADI in due dimensioni.

Capitolo 7 – Flussi di Stokes ed equazione biarmonica.
La corrente in una cavità mossa da una parete; le condizioni al contorno sulla vorticità; soluzione iterativa e soluzione diretta, convergenza, vortici di Moffatt negli spigoli. Approfondimenti: legge di Stokes per la sfera, paradosso di Stokes, teoria della lubrificazione e pattino piano, derivazione dei vortici di Moffatt.

Capitolo 8 – Stabilità idrodinamica I: la convezione di Rayleigh–Bénard.
Linearizzazione, modi normali, stabilità marginale; un problema agli autovalori generalizzato discretizzato con le differenze finite. Approfondimenti: dimostrazione dello scambio delle stabilità e quoziente di Rayleigh, tassi di crescita, altre condizioni al contorno, regime oltre la soglia.

Capitolo 9 – Stabilità idrodinamica II: metodi spettrali di Chebyshev.
Collocazione di Chebyshev, accuratezza spettrale; fenomeno di Runge, ellisse di Bernstein e costante di Lebesgue; l'equazione di Orr–Sommerfeld, le onde di Tollmien–Schlichting e la curva di stabilità neutra della corrente di Poiseuille. Approfondimenti: teorema di Squire, teoremi di Rayleigh e Fjørtoft, equazione dell'energia di Reynolds–Orr, crescita transitoria.

Capitolo 10 – Metodi pseudo-spettrali di Fourier.
La trasformata discreta di Fourier e la derivazione nello spazio dei numeri d'onda; un problema ai limiti periodico e la convezione–diffusione bidimensionale. Approfondimenti: decadimento dei coefficienti, aliasing e regola dei 3/2, equazione di Burgers pseudo-spettrale, trasformata rapida.

Appendici.
Simboli e istruzioni; i diciotto esercizi svolti passo per passo, fra cui due progetti (condotti di varie sezioni e corrente pulsante di Womersley).

Ogni capitolo segue lo stesso schema dei moduli precedenti: prima l'*intuizione fisica*, poi il *modello matematico*, quindi l'*algoritmo* e il *programma*, infine i *risultati* discussi criticamente. I riquadri colorati hanno il significato consueto:

- **teoria** (viola): definizioni, enunciati, dimostrazioni;
- **fasi** (blu): procedure e algoritmi passo per passo;
- **ipotesi** (giallo): le assunzioni semplificative del modello;
- **dati** (grigio): valori numerici usati nei calcoli;
- **risultati** (verde): esiti dei calcoli e loro verifica;
- **esempi** (arancio): applicazioni ed esempi numerici;
- **avvisi** (rosso): errori frequenti, limiti dei metodi, trappole numeriche.

I programmi

Tutti i programmi sono scritti in Python 3 e usano soltanto le librerie `numpy`, `scipy` e `matplotlib`. Ciascuno è autonomo, stampa i risultati principali e salva le figure nella cartella `figure/`; i moduli comuni `metodi_numerici.py` (sistemi tridiagonali, Runge–Kutta, metodi iterativi, matrice di Chebyshev) e `stile_grafici.py` sono riportati in appendice. I programmi che traducono gli algoritmi classici del capitolo sono numerati da 3.1 a 3.10; quelli di approfondimento (quattordici, più il programma introduttivo 3.0) portano lo stesso numero seguito da una lettera. I programmi sono disponibili in rete all'indirizzo `https://braucci.github.io/CFD/monografia3/codici/`: il QR code nel margine accanto a ogni listato porta direttamente al file corrispondente.

Capitolo 1

Le equazioni del moto dei fluidi viscosi

1.1 Dal fluido ideale al fluido reale

In un fluido ideale le particelle si scambiano soltanto forze normali alle superfici di contatto, cioè pressioni. In un fluido reale, quando due strati adiacenti scorrono l'uno rispetto all'altro, nasce anche una forza *tangenziale*: lo strato più veloce trascina quello più lento, e viceversa lo frena. Il fenomeno, dovuto su scala microscopica allo scambio di quantità di moto fra le molecole (nei gas) o alle forze intermolecolari (nei liquidi), si chiama **viscosità**.

L'esperimento che la definisce è quello di Couette: un sottile strato di fluido di spessore h è compreso fra una lastra ferma e una lastra che trasla con velocità U. A regime la forza per unità di superficie necessaria a mantenere il moto è proporzionale a U/h:

$$\tau = \mu \frac{U}{h} \qquad \text{e, localmente,} \qquad \tau = \mu \frac{\partial u}{\partial y}, \tag{1.1}$$

dove μ è il **coefficiente di viscosità** (dinamica), in Pa s. I fluidi per cui vale la (1.1) con μ indipendente dalla velocità di deformazione si dicono *newtoniani*: lo sono l'aria, l'acqua, gli oli minerali e, con buona approssimazione, il sangue nei grandi vasi. Il rapporto $\nu = \mu/\rho$, in $\mathrm{m^2\,s^{-1}}$, è la **viscosità cinematica**; come si vedrà, è il coefficiente di diffusione della quantità di moto.

La viscosità ha due conseguenze, una matematica e una fisica, che sono le due facce della stessa medaglia.

- Gli sforzi viscosi contengono le derivate *prime* della velocità; nelle equazioni del moto compaiono quindi le derivate *seconde*, e l'ordine del sistema aumenta di uno.
- Serve allora una condizione al contorno in più: sulla parete di un corpo solido, oltre alla componente normale, si annulla anche la componente *tangenziale* della velocità relativa (**condizione di aderenza**, o di non scorrimento). Nel fluido ideale il fluido scivola sulla parete; nel fluido reale vi aderisce.

È proprio la condizione di aderenza a generare lo strato limite: fra la parete, dove la velocità relativa è nulla, e la corrente esterna, dove vale la soluzione del fluido ideale, deve esistere una regione di raccordo in cui gli sforzi viscosi sono importanti, per quanto piccola sia la viscosità.

Viscosità di alcuni fluidi a 20 °C e pressione atmosferica

fluido	μ (Pa s)	ν ($m^2\,s^{-1}$)	Pr
aria	$1{,}81 \times 10^{-5}$	$1{,}50 \times 10^{-5}$	0,71
acqua	$1{,}00 \times 10^{-3}$	$1{,}00 \times 10^{-6}$	7,0
mercurio	$1{,}55 \times 10^{-3}$	$1{,}15 \times 10^{-7}$	0,025
olio motore SAE 30	$2{,}9 \times 10^{-1}$	$3{,}3 \times 10^{-4}$	3500
glicerina	1,49	$1{,}18 \times 10^{-3}$	12 500

Il numero di Prandtl $Pr = \mu c_p/k$, definito al Cap. 3, confronta la diffusione della quantità di moto con quella del calore. I valori sono indicativi; per i gas μ cresce con la temperatura, per i liquidi diminuisce rapidamente.

1.2 Gli sforzi in un fluido newtoniano

Per scrivere le equazioni del moto serve la generalizzazione tridimensionale della (1.1). Lo stato di sforzo in un punto è descritto dal tensore simmetrico σ_{ij}, la componente j della forza per unità di area agente su una faccia di normale i. In un fluido in quiete $\sigma_{ij} = -p\,\delta_{ij}$. In moto si aggiunge la parte viscosa, che dipende dalla rapidità con cui la particella si deforma, descritta dal tensore della velocità di deformazione

$$e_{ij} = \frac{1}{2}\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right). \tag{1.2}$$

Legge costitutiva del fluido newtoniano

Se il fluido è isotropo e lo sforzo viscoso dipende linearmente dalla velocità di deformazione, i soli tensori simmetrici costruibili linearmente con e_{ij} sono e_{ij} stesso e $e_{kk}\delta_{ij}$ ($e_{kk} = \nabla \cdot \mathbf{V}$, la velocità di dilatazione cubica). Quindi

$$\sigma_{ij} = -p\,\delta_{ij} + 2\mu\, e_{ij} + \lambda\,(\nabla \cdot \mathbf{V})\,\delta_{ij}, \tag{1.3}$$

con due coefficienti scalari: la viscosità μ e il *secondo coefficiente di viscosità* λ. Per un moto di puro scorrimento ($u = u(y)$) la (1.3) restituisce $\sigma_{xy} = \mu\,\mathrm{d}u/\mathrm{d}y$, cioè la (1.1).
La traccia della (1.3) dà la pressione meccanica media $-\sigma_{kk}/3 = p - (\lambda + \frac{2}{3}\mu)\nabla \cdot \mathbf{V}$. L'*ipotesi di Stokes* $\lambda = -\frac{2}{3}\mu$ fa coincidere la pressione media con quella termodinamica anche durante un'espansione o una compressione; è esatta per i gas monoatomici e si usa correntemente per tutti i gas. Nei fluidi incomprimibili il termine in λ scompare comunque, perché $\nabla \cdot \mathbf{V} = 0$.

1.3 Le equazioni di bilancio

1.3.1 Continuità e quantità di moto

La conservazione della massa non dipende dalla viscosità e ha la stessa forma vista per il fluido ideale:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0. \tag{1.4}$$

La seconda legge della dinamica applicata a una particella di fluido, con la forza di superficie $\partial\sigma_{ij}/\partial x_j$ per unità di volume calcolata dalla (1.3), fornisce l'**equazione di Navier–Stokes**. Con μ e λ costanti e trascurando le forze di volume si può scrivere in forma vettoriale compatta:

$$\rho\frac{\mathrm{D}\mathbf{V}}{\mathrm{D}t} = -\nabla p - \nabla\times\left[\mu\left(\nabla\times\mathbf{V}\right)\right] + \nabla\left[\left(2\mu+\lambda\right)\nabla\cdot\mathbf{V}\right], \qquad \frac{\mathrm{D}}{\mathrm{D}t} = \frac{\partial}{\partial t} + \mathbf{V}\cdot\nabla. \tag{1.5}$$

La forma si ottiene dall'identità vettoriale

$$\nabla^2\mathbf{V} = \nabla(\nabla\cdot\mathbf{V}) - \nabla\times(\nabla\times\mathbf{V}), \tag{1.6}$$

applicata alla divergenza del tensore $2\mu e_{ij} + \lambda e_{kk}\delta_{ij}$, che vale $\mu\nabla^2\mathbf{V} + (\mu+\lambda)\nabla(\nabla\cdot\mathbf{V})$. La (1.5) mostra con chiarezza le due parti della forza viscosa: una legata alla rotazione delle particelle, $\nabla\times\mathbf{V}$, e una legata alla loro variazione di volume, $\nabla\cdot\mathbf{V}$. L'ordine delle derivate spaziali sale da uno (equazione di Eulero) a due.

1.3.2 Energia e funzione di dissipazione

Per un gas ideale conviene scrivere il primo principio della termodinamica in termini dell'entalpia specifica $h = c_p T$:

$$\rho\frac{\mathrm{D}h}{\mathrm{D}t} = \frac{\mathrm{D}p}{\mathrm{D}t} + \nabla\cdot\left(k\,\nabla T\right) + \Phi, \tag{1.7}$$

dove k è la conducibilità termica e Φ è la **funzione di dissipazione**, la potenza per unità di volume con cui gli sforzi viscosi trasformano l'energia del moto ordinato in calore. In coordinate cartesiane

$$\Phi = 2\mu\left[\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial z}\right)^2\right] + \mu\left[\left(\frac{\partial v}{\partial x}+\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}+\frac{\partial v}{\partial z}\right)^2 + \left(\frac{\partial u}{\partial z}+\frac{\partial w}{\partial x}\right)^2\right] + \lambda\left(\nabla\cdot\mathbf{V}\right)^2. \tag{1.8}$$

La dissipazione non è mai negativa

Con l'ipotesi di Stokes la (1.8) è una somma di quadrati. Posto $a = \partial u/\partial x$, $b = \partial v/\partial y$, $c = \partial w/\partial z$, vale l'identità algebrica

$$a^2 + b^2 + c^2 - \tfrac{1}{3}(a+b+c)^2 = \tfrac{1}{3}\left[(a-b)^2 + (b-c)^2 + (c-a)^2\right],$$

che si verifica sviluppando i quadrati. Poiché $\nabla\cdot\mathbf{V} = a+b+c$, con $\lambda = -\tfrac{2}{3}\mu$:

$$\Phi = \frac{2\mu}{3}\left[(a-b)^2 + (b-c)^2 + (c-a)^2\right] + \mu\left[\left(\frac{\partial v}{\partial x}+\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}+\frac{\partial v}{\partial z}\right)^2 + \left(\frac{\partial u}{\partial z}+\frac{\partial w}{\partial x}\right)^2\right] \geq 0.$$

Φ si annulla solo nei moti rigidi (traslazione e rotazione), in cui le particelle non si deformano: è l'espressione locale del secondo principio, la viscosità trasforma sempre energia meccanica in calore e mai il contrario.

Con l'equazione di stato del gas ideale,

$$p = \rho R T, \tag{1.9}$$

le (1.4), (1.5) (tre equazioni scalari), (1.7) e (1.9) formano un sistema di sei equazioni nelle sei incognite ρ, p, T, u, v, w. Il sistema è non lineare e, soprattutto, *accoppiato*: in generale le sei equazioni vanno risolte insieme.

1.4 Il fluido incomprimibile a proprietà costanti

Se il numero di Mach è piccolo e le differenze di temperatura sono moderate, la densità e le proprietà del fluido si possono considerare costanti. Allora $\nabla \cdot \mathbf{V} = 0$, il termine in λ scompare, e per l'identità (1.6) $-\nabla \times (\nabla \times \mathbf{V}) = \nabla^2 \mathbf{V}$:

$$\nabla \cdot \mathbf{V} = 0, \tag{1.10}$$

$$\rho \frac{\mathrm{D}\mathbf{V}}{\mathrm{D}t} = -\nabla p + \mu \nabla^2 \mathbf{V}. \tag{1.11}$$

Il vantaggio è grande: l'equazione di stato non serve più e l'equazione dell'energia si *disaccoppia*. Si calcolano prima $\mathbf{V}$ e p dalle (1.10)–(1.11) e poi, a campo di moto noto, la temperatura dalla (1.7), che diventa un'equazione lineare in T. È la strategia seguita nei Cap. 2 e 3 per lo strato limite.

Ipotesi valide in tutta la monografia, salvo avviso contrario

- fluido newtoniano, monofase, continuo;
- densità ρ, viscosità μ, conducibilità k e calore specifico c_p costanti (fluido incomprimibile, numero di Mach piccolo);
- moto laminare;
- forze di volume assenti, oppure conservative (gravità) e incluse nella pressione, tranne nel canale a pelo libero (Cap. 4) e nella convezione naturale (Cap. 8).

1.5 Adimensionalizzazione e numero di Reynolds

Sia L una lunghezza caratteristica del problema (la corda di un profilo, il diametro di un tubo) e U una velocità caratteristica. Con le grandezze adimensionali

$$\mathbf{x}' = \frac{\mathbf{x}}{L}, \qquad \mathbf{V}' = \frac{\mathbf{V}}{U}, \qquad t' = \frac{t}{L/U}, \qquad p' = \frac{p}{\rho U^2}, \tag{1.12}$$

la (1.11) diventa (omettendo gli apici)

$$\frac{\mathrm{D}\mathbf{V}}{\mathrm{D}t} = -\nabla p + \frac{1}{Re}\nabla^2 \mathbf{V}, \qquad Re = \frac{\rho U L}{\mu} = \frac{UL}{\nu}. \tag{1.13}$$

Il **numero di Reynolds** misura il rapporto fra i termini d'inerzia, di ordine $\rho U^2/L$, e quelli viscosi, di ordine $\mu U/L^2$. Due correnti geometricamente simili con lo stesso Re hanno la stessa soluzione adimensionale: è il fondamento delle prove in galleria del vento. I due limiti opposti definiscono due regimi completamente diversi, entrambi studiati in questo modulo.

- $Re \to \infty$: il termine viscoso diventa trascurabile *quasi ovunque*, ma non vicino alle pareti, dove il gradiente di velocità è tanto più intenso quanto più grande è Re. Nasce lo **strato limite** (Cap. 2).
- $Re \ll 1$: il termine d'inerzia è trascurabile e la (1.13) diventa lineare; sono i **flussi di Stokes** o di scorrimento lento (Cap. 7).

Ordini di grandezza del numero di Reynolds

caso	dati	Re
ala di un aereo di linea in crociera	$L = 5\,\mathrm{m}$, $U = 250\,\mathrm{m\,s^{-1}}$, $\nu = 3{,}9 \times 10^{-5}\,\mathrm{m^2\,s^{-1}}$	$3{,}2 \times 10^{7}$
ala di un aliante	$L = 1\,\mathrm{m}$, $U = 25\,\mathrm{m\,s^{-1}}$, $\nu = 1{,}5 \times 10^{-5}\,\mathrm{m^2\,s^{-1}}$	$1{,}7 \times 10^{6}$
sangue nell'aorta	$L = 2{,}5\,\mathrm{cm}$, $U = 0{,}3\,\mathrm{m\,s^{-1}}$, $\nu = 3{,}3 \times 10^{-6}\,\mathrm{m^2\,s^{-1}}$	$2{,}3 \times 10^{3}$
ala di un insetto	$L = 1\,\mathrm{cm}$, $U = 1\,\mathrm{m\,s^{-1}}$ (aria)	$6{,}7 \times 10^{2}$
batterio che nuota in acqua	$L = 1\,\mu\mathrm{m}$, $U = 30\,\mu\mathrm{m\,s^{-1}}$	3×10^{-5}

La viscosità cinematica dell'aria a 11 km di quota è più del doppio di quella al livello del mare, perché la densità diminuisce molto più della viscosità dinamica.

1.6 Vorticità e funzione di corrente

Nel fluido ideale il moto che parte dalla quiete resta irrotazionale e si descrive con un potenziale di velocità. Nel fluido viscoso la vorticità $\boldsymbol{\zeta} = \nabla \times \mathbf{V}$ nasce alle pareti (per la condizione di aderenza) e diffonde nel fluido: il potenziale non esiste più. Prendendo il rotore della (1.11) la pressione scompare; per un moto piano, in cui $\boldsymbol{\zeta}$ ha la sola componente $\zeta = \partial v/\partial x - \partial u/\partial y$ normale al piano, si ottiene l'**equazione del trasporto della vorticità**

$$\frac{\partial \zeta}{\partial t} + u\frac{\partial \zeta}{\partial x} + v\frac{\partial \zeta}{\partial y} = \nu\,\nabla^2\zeta\,: \tag{1.14}$$

la vorticità di una particella varia soltanto per diffusione viscosa, con coefficiente ν. La funzione di corrente, invece, esiste ancora, perché la sua definizione dipende solo dalla continuità: con $u = \partial\psi/\partial y$ e $v = -\partial\psi/\partial x$ la (1.10) è soddisfatta identicamente e

$$\nabla^2\psi = -\zeta. \tag{1.15}$$

La coppia (1.14)–(1.15) sostituisce le tre equazioni in u, v, p (formulazione *vorticità–funzione di corrente*); la si userà nel Cap. 7.

1.7 Soluzioni esatte: le correnti parallele

Le equazioni di Navier–Stokes hanno pochissime soluzioni esatte, ma quelle poche sono preziose: forniscono il riferimento con cui collaudare i programmi e illustrano i meccanismi fisici senza l'ingombro della non linearità. Le più semplici sono le correnti *parallele*, in cui la velocità ha una sola componente, u, funzione delle sole coordinate trasversali. Allora $\partial u/\partial x = 0$, la continuità è soddisfatta e il termine convettivo $u\,\partial u/\partial x$ si annulla: l'equazione diventa *lineare*.

1.7.1 La famiglia di Couette–Poiseuille

Consideriamo un fluido fra due lastre parallele indefinite, distanti h; la lastra inferiore è ferma, quella superiore trasla con velocità U e lungo il canale agisce un gradiente di pressione costante $\mathrm{d}p/\mathrm{d}x$.

Il profilo di Couette–Poiseuille

Con $u = u(y)$ e $v = w = 0$ la componente y della (1.11) dà $\partial p/\partial y = 0$, quindi $p = p(x)$; la componente x si riduce a

$$\frac{\mathrm{d}p}{\mathrm{d}x} = \mu \frac{\mathrm{d}^2 u}{\mathrm{d}y^2}.$$

Il primo membro dipende solo da x, il secondo solo da y: devono essere entrambi costanti. Integrando due volte con $u(0) = 0$, $u(h) = U$ e posto $Y = y/h$:

$$\frac{u}{U} = Y + P\,Y(1-Y), \qquad P = -\frac{h^2}{2\mu U}\frac{\mathrm{d}p}{\mathrm{d}x}. \tag{1.16}$$

Il parametro P confronta l'effetto del gradiente di pressione con quello del trascinamento della parete. Ne seguono:

- sforzo sulla parete inferiore $\tau_0 = \mu\,(U/h)\,(1+P)$ e sulla superiore $\tau_h = \mu\,(U/h)\,(1-P)$;
- portata per unità di larghezza $Q = Uh\,(\tfrac{1}{2} + P/6)$;
- vorticità $\zeta = -\mathrm{d}u/\mathrm{d}y = -(U/h)\,[1 + P(1-2Y)]$, lineare in Y;
- potenza dissipata per unità di superficie $\int_0^h \mu(\mathrm{d}u/\mathrm{d}y)^2\mathrm{d}y = \mu\,(U^2/h)\,(1 + P^2/3)$.

Per $P = 0$ si ha la corrente di Couette pura, a profilo lineare e vorticità uniforme. Per $P > 0$ (gradiente favorevole, $\mathrm{d}p/\mathrm{d}x < 0$) il profilo si gonfia e la portata aumenta. Per $P < 0$ (gradiente avverso) il fluido vicino alla parete ferma viene rallentato e, se $P < -1$, lo sforzo τ_0 cambia segno: compare una zona di *corrente inversa*. È l'analogo, in questa geometria semplicissima, del distacco dello strato limite in un gradiente avverso (Esercizio 3). Per $P = -3$ la portata è nulla: il fluido trascinato in avanti dalla parete mobile è esattamente compensato da quello respinto dalla pressione, come nel meato di una tenuta o nella vite di un estrusore.

prog3_0_soluzioni_esatte.py

```python
"""
prog3_0_soluzioni_esatte.py -- soluzioni esatte delle equazioni di Navier-Stokes
per flussi paralleli: la famiglia di Couette-Poiseuille fra due lastre piane.

Canale 0 <= y <= h, lastra inferiore ferma, lastra superiore con velocita' U,
gradiente di pressione costante dp/dx. Con Y = y/h e il parametro adimensionale
    P = -(h^2 / (2 mu U)) dp/dx
il profilo di velocita' e'   u/U = Y + P Y (1 - Y).
Si calcolano: profili, vorticita' zeta = -du/dy, sforzo alle pareti,
portata e potenza dissipata per unita' di volume (funzione di dissipazione).
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ROSSO, GRIGIO, PALETTE

Y = np.linspace(0.0, 1.0, 201)
valori_P = [-3.0, -2.0, -1.0, 0.0, 1.0, 2.0, 3.0]


def profilo(P, Y):
    return Y + P * Y * (1.0 - Y)


def derivata(P, Y):                    # d(u/U)/dY
    return 1.0 + P * (1.0 - 2.0 * Y)
```

```
26
27
28 print("Famiglia di Couette-Poiseuille:  u/U = Y + P Y (1 - Y)")
29 print("     P    tau_inf/(mu U/h)  tau_sup/(mu U/h)  portata/(U h)
       dissipazione/(mu U^2/h^2)")
30 for P in valori_P:
31     tau_inf = derivata(P, 0.0)       # sforzo sulla lastra inferiore
32     tau_sup = derivata(P, 1.0)       # sforzo esercitato DAL fluido... sulla
       superiore: -du/dy
33     portata = 0.5 + P / 6.0          # integrale di u/U fra 0 e 1
34     diss = np.trapezoid(derivata(P, Y) ** 2, Y)   # Phi/mu = (du/dy)^2, media sul
       canale
35     print(f"  {P:5.1f}   {tau_inf:12.4f}   {tau_sup:14.4f}   {portata:13.4f}
       {diss:16.4f}")
36
37 # controllo: il profilo soddisfa  mu u'' = dp/dx, cioe' (u/U)'' = -2P
38 Pt = 2.0
39 h_y = Y[1] - Y[0]
40 u = profilo(Pt, Y)
41 residuo = (u[2:] - 2 * u[1:-1] + u[:-2]) / h_y**2 + 2 * Pt
42 print(f"\nVerifica (P = {Pt}): max |(u/U)'' + 2P| = {np.max(np.abs(residuo)):.2e}")
43 print("Il flusso inverso alla parete inferiore compare per P < -1 (tau_inf < 0).")
44
45 # ------------------------------------------------------------------ figure
46 fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.3))
47 for k, P in enumerate(valori_P):
48     col = PALETTE[k % len(PALETTE)]
49     ax[0].plot(profilo(P, Y), Y, color=col, label=f"$P={P:+.0f}$")
50     ax[1].plot(-derivata(P, Y), Y, color=col)
51 ax[0].axvline(0, color=GRIGIO, lw=0.8)
52 ax[0].set_xlabel("$u/U$")
53 ax[0].set_ylabel("$y/h$")
54 ax[0].set_title("profili di velocita'")
55 ax[0].legend(loc="upper left", fontsize=7)
56 ax[1].axvline(0, color=GRIGIO, lw=0.8)
57 ax[1].set_xlabel(r"$\zeta h/U = -\,\mathrm{d}(u/U)/\mathrm{d}(y/h)$")
58 ax[1].set_title("vorticita'")
59 fig.tight_layout()
60 salva(fig, "prog3_0_couette_poiseuille")
```

Listato 1.1: `prog3_0_soluzioni_esatte.py`: profili, vorticità, sforzi, portata e dissipazione della famiglia di Couette–Poiseuille.

Risultati del programma

Il programma verifica per differenze finite che la (1.16) soddisfa l'equazione del moto (residuo $\approx 10^{-11}$, cioè l'errore di arrotondamento) e stampa la tabella seguente; la Fig. 1.1 mostra profili e vorticità.

P	$\tau_0/(\mu U/h)$	$\tau_h/(\mu U/h)$	$Q/(Uh)$	dissipazione
-3	-2	4	0,0000	4,000
-1	0	2	0,3333	1,333
0	1	1	0,5000	1,000
1	2	0	0,6667	1,333
3	4	-2	1,0000	4,000

La dissipazione (in unità di $\mu U^2/h$) coincide con $1 + P^2/3$: a parità di velocità della

parete, la corrente di Couette pura è quella che dissipa meno.

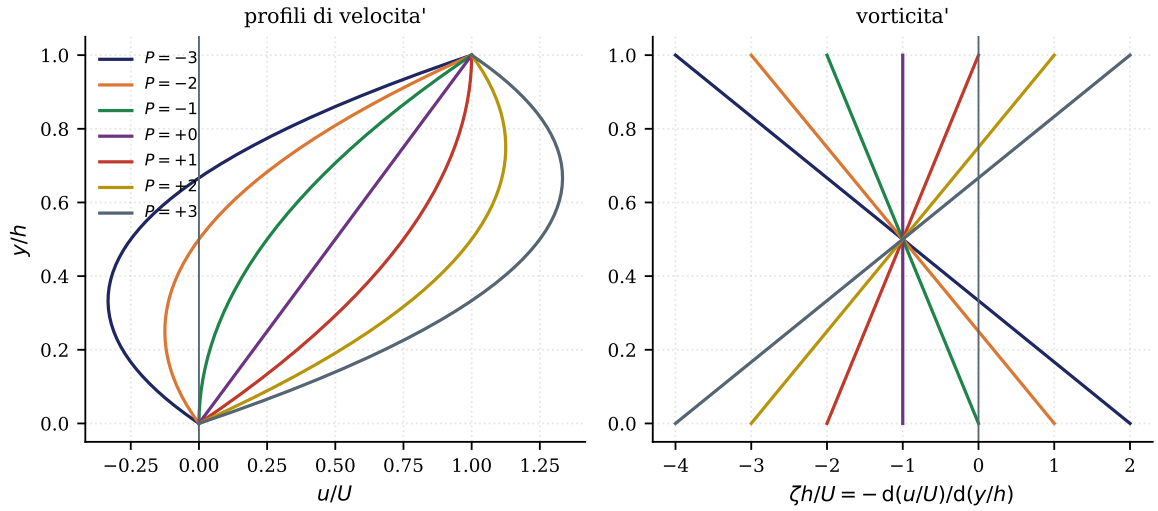


Figura 1.1: Famiglia di Couette–Poiseuille: profili di velocità (a sinistra) e di vorticità (a destra) per $-3 \le P \le 3$. Tutte le rette della vorticità passano per lo stesso punto, $Y = 1/2$, dove il contributo del gradiente di pressione si annulla.

1.7.2 La corrente di Hagen–Poiseuille

Lo stesso ragionamento, ripetuto in coordinate cilindriche per un tubo circolare di raggio R, porta a $\mu\, r^{-1}\, \mathrm{d}(r\, \mathrm{d}u/\mathrm{d}r)/\mathrm{d}r = \mathrm{d}p/\mathrm{d}x$. Imponendo che u sia finita sull'asse e nulla sulla parete:

$$u = -\frac{1}{4\mu}\frac{\mathrm{d}p}{\mathrm{d}x}\left(R^2 - r^2\right), \qquad Q = -\frac{\pi R^4}{8\mu}\frac{\mathrm{d}p}{\mathrm{d}x}. \tag{1.17}$$

La portata cresce con la quarta potenza del raggio: dimezzando il diametro di un tubo, a parità di gradiente di pressione, la portata diventa sedici volte più piccola. In termini di velocità media $\bar{u} = Q/(\pi R^2)$ e di diametro D la (1.17) si scrive $-\mathrm{d}p/\mathrm{d}x = f\,\tfrac{1}{2}\rho\bar{u}^2/D$ con $f = 64/Re_D$, la ben nota legge dell'attrito laminare. Il Cap. 4 e l'Esercizio 17 estendono il calcolo a sezioni di forma qualsiasi, per le quali non esiste una formula chiusa.

1.8 Approfondimenti

1.8.1 L'equazione di Cauchy e la simmetria del tensore degli sforzi

Nel § 1.7 si è usata senza dimostrazione l'affermazione che la forza di superficie per unità di volume vale $\partial\sigma_{ij}/\partial x_j$. Vale la pena ricostruire l'argomento, perché è lo stesso che si usa per le travi e le piastre.

Dal vettore sforzo al tensore degli sforzi

Sia $\mathbf{t}(\mathbf{n})$ la forza per unità di area che il fluido esterno esercita su una superficie di normale uscente $\mathbf{n}$. Si consideri un tetraedro infinitesimo con tre facce sui piani coordinati, di aree $A_i = A n_i$, e la quarta faccia di area A e normale $\mathbf{n}$. L'equilibrio delle forze (le forze di volume e d'inerzia sono proporzionali al volume, di ordine superiore rispetto alle

aree) dà $\mathbf{t}(\mathbf{n})A = \sum_i \mathbf{t}(\mathbf{e}_i)An_i$, cioè

$$t_j(\mathbf{n}) = \sigma_{ij}\, n_i, \qquad \sigma_{ij} = t_j(\mathbf{e}_i)\,:$$

lo sforzo su una faccia qualsiasi è una funzione *lineare* della normale, e i nove coefficienti σ_{ij} formano un tensore (teorema di Cauchy). Applicando la seconda legge della dinamica a un volume V di fluido, con il teorema della divergenza,

$$\int_V \rho\, \frac{\mathrm{D}V_j}{\mathrm{D}t}\,\mathrm{d}V = \oint_{\partial V} \sigma_{ij} n_i\,\mathrm{d}A + \int_V \rho f_j\,\mathrm{d}V = \int_V \left(\frac{\partial \sigma_{ij}}{\partial x_i} + \rho f_j\right)\mathrm{d}V,$$

e poiché V è arbitrario si ottiene l'*equazione di Cauchy* $\rho\,\mathrm{d}V_j/\mathrm{d}t = \partial\sigma_{ij}/\partial x_i + \rho f_j$, valida per qualunque mezzo continuo. La legge costitutiva (1.3) è ciò che la specializza al fluido newtoniano.

Simmetria. Il bilancio del momento della quantità di moto per un cubetto di lato ℓ centrato nell'origine, rispetto all'asse z, contiene i momenti degli sforzi tangenziali sulle facce: $(\sigma_{xy} - \sigma_{yx})\,\ell^3$, mentre i momenti delle forze di volume e d'inerzia sono di ordine ℓ^5 (braccio ℓ per massa ℓ^3 per accelerazione, o momento d'inerzia ℓ^5 per accelerazione angolare). Facendo tendere ℓ a zero deve essere $\sigma_{xy} = \sigma_{yx}$, e analogamente per le altre componenti: in assenza di coppie distribuite il tensore degli sforzi è simmetrico, e dei nove coefficienti ne restano sei indipendenti. È per questo che la legge costitutiva può coinvolgere solo la parte simmetrica del gradiente di velocità, e_{ij}: la parte antisimmetrica (la rotazione rigida locale, cioè metà della vorticità) non produce sforzi.

1.8.2 Il bilancio dell'energia meccanica

Moltiplicando scalarmente l'equazione del moto per la velocità si ottiene il bilancio dell'energia cinetica, che chiarisce il ruolo della funzione di dissipazione.

Energia cinetica, lavoro degli sforzi e dissipazione

Da $\rho\,\mathrm{d}V_j/\mathrm{d}t = \partial\sigma_{ij}/\partial x_i$ (senza forze di volume), moltiplicando per V_j:

$$\rho \frac{\mathrm{D}}{\mathrm{D}t}\left(\tfrac{1}{2}V^2\right) = \frac{\partial(\sigma_{ij}V_j)}{\partial x_i} - \sigma_{ij}\frac{\partial V_j}{\partial x_i}.$$

Il primo termine a destra è la divergenza del flusso di lavoro degli sforzi; il secondo, per la simmetria di σ_{ij}, si può scrivere con e_{ij} al posto di $\partial V_j/\partial x_i$, e con la (1.3)

$$\sigma_{ij}e_{ij} = -p\,\nabla\cdot\mathbf{V} + \underbrace{2\mu e_{ij}e_{ij} + \lambda(\nabla\cdot\mathbf{V})^2}_{\Phi}.$$

Integrando su un volume materiale V (che si muove con il fluido):

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_V \tfrac{1}{2}\rho V^2\,\mathrm{d}V = \oint_{\partial V} \sigma_{ij} n_i V_j\,\mathrm{d}A + \int_V p\,\nabla\cdot\mathbf{V}\,\mathrm{d}V - \int_V \Phi\,\mathrm{d}V. \tag{1.18}$$

L'energia cinetica di una porzione di fluido cambia per tre cause: la potenza delle forze di superficie esercitate dall'esterno (primo termine), il lavoro di compressione (secondo, nullo nell'incomprimibile) e la dissipazione, che è sempre una perdita perché $\Phi \geq 0$. Nell'equazione dell'energia (1.7) la stessa Φ compare con il segno opposto: ciò che l'energia meccanica perde lo guadagna l'energia interna, cioè la temperatura.

Verifica sulla corrente di Couette–Poiseuille

A regime l'energia cinetica è costante e la (1.18) dice che la potenza immessa deve uguagliare la dissipazione. Per unità di superficie delle lastre, la potenza entra da due vie: la lastra mobile, che esercita lo sforzo τ_h e si muove a velocità U, fornisce $\tau_h U = \mu(U^2/h)(1-P)$; il gradiente di pressione, che spinge la portata Q, fornisce $(-\mathrm{d}p/\mathrm{d}x)\,Q = (2\mu UP/h^2)\,Uh(\tfrac{1}{2}+P/6) = \mu(U^2/h)(P+P^2/3)$. La somma vale $\mu(U^2/h)(1+P^2/3)$, esattamente la dissipazione calcolata nel § 1.7 e stampata dal Programma 3.0. Si noti che per $P=1$ la lastra superiore non compie lavoro ($\tau_h = 0$): tutta la potenza viene dalla pompa; per $P=-1$ è la lastra a fornire tutto, e il gradiente avverso assorbe potenza ($P + P^2/3 < 0$) restituendola come energia di pressione.

1.8.3 Le condizioni al contorno

Le equazioni di Navier–Stokes sono del secondo ordine nello spazio e richiedono, su ogni porzione del contorno, condizioni su tutte le componenti della velocità. I casi che si incontrano in questo modulo sono tre.

Parete solida. Aderenza, $\mathbf{V} = \mathbf{V}_{\text{parete}}$: la componente normale esprime l'impermeabilità, quelle tangenziali l'assenza di scorrimento. Quest'ultima è un fatto sperimentale, valido finché il libero cammino medio delle molecole è trascurabile rispetto alle dimensioni del problema (gas rarefatti e microcanali sono le eccezioni). Nessuna condizione va imposta sulla pressione: alla parete essa risulta dall'equazione del moto stessa.

Superficie libera. Se sopra il fluido c'è un gas di viscosità trascurabile, il gas non può esercitare sforzi tangenziali: $\tau_{nt} = 0$, dove n è la normale e t una tangente. Lo sforzo normale uguaglia la pressione del gas (più l'effetto della tensione superficiale, se la superficie è curva). In coordinate cartesiane, con superficie piana $y =$ costante, $\tau_{nt} = \mu(\partial u/\partial y + \partial v/\partial x)$, e per una corrente parallela si riduce a $\partial u/\partial y = 0$; in coordinate curvilinee la forma corretta dello sforzo contiene termini aggiuntivi, e l'Esercizio 5 mostra quanto sia facile sbagliare.

Interfaccia fra due fluidi. Velocità e sforzo tangenziale sono continui attraverso l'interfaccia; la superficie libera è il caso limite in cui uno dei due fluidi ha viscosità nulla.

Nei problemi non stazionari serve inoltre la condizione iniziale; nei domini illimitati, il comportamento all'infinito (per esempio $u \to U$ nello strato limite).

1.8.4 La diffusione della vorticità: il vortice di Lamb–Oseen

L'equazione (1.14) ha un'interpretazione fisica che si coglie meglio su una soluzione esatta.

Un vortice che si attenua per viscosità

Si consideri un vortice piano assialsimmetrico, con la sola velocità azimutale $v_\theta(r,t)$. Il moto è lungo circonferenze e la vorticità $\zeta(r,t)$ viene semplicemente trasportata lungo di esse senza variare: il termine convettivo della (1.14) è nullo e resta l'equazione della diffusione in coordinate polari, $\partial\zeta/\partial t = \nu\, r^{-1}\partial_r(r\,\partial_r\zeta)$. Se all'istante iniziale tutta la circolazione Γ è concentrata sull'asse (vortice potenziale), la soluzione, che si verifica per sostituzione diretta, è

$$\zeta = \frac{\Gamma}{4\pi\nu t}\, e^{-r^2/(4\nu t)}, \qquad v_\theta = \frac{\Gamma}{2\pi r}\left[1 - e^{-r^2/(4\nu t)}\right]. \tag{1.19}$$

La circolazione totale $\int_0^\infty \zeta\, 2\pi r\, \mathrm{d}r = \Gamma$ resta costante: la viscosità non distrugge la vorticità, la *distribuisce*. Lontano dal centro ($r \gg \sqrt{\nu t}$) il vortice è ancora quello potenziale, $v_\theta = \Gamma/(2\pi r)$; vicino al centro la velocità è lineare in r, come in una rotazione rigida. La velocità massima si trova a $r_c = 2{,}24\sqrt{\nu t}$ (raggio del nucleo), che cresce come la radice del tempo: la stessa legge dello strato di Rayleigh del Cap. 5.

I vortici di scia di un velivolo

Un aereo di 60 t in avvicinamento a $70\,\mathrm{m\,s^{-1}}$, con apertura $b = 34\,\mathrm{m}$, lascia due vortici di scia di circolazione $\Gamma \simeq 4W/(\pi\rho U b) \simeq 270\,\mathrm{m^2\,s^{-1}}$. Se il nucleo crescesse per sola diffusione molecolare ($\nu = 1{,}5\times10^{-5}\,\mathrm{m^2\,s^{-1}}$), dopo un minuto avrebbe raggio $r_c = 2{,}24\sqrt{\nu t} \simeq 7\,\mathrm{cm}$ e la velocità al bordo del nucleo sarebbe di centinaia di metri al secondo. I nuclei reali misurano uno o due metri: la turbolenza nel nucleo produce una diffusione *effettiva* centinaia di volte più intensa di quella molecolare. La legge (1.19) resta il modello di riferimento, con ν sostituita da una viscosità turbolenta; è su questo modello che si basano le distanze di separazione fra i velivoli in atterraggio.

1.8.5 Analisi dimensionale: la resistenza di una sfera

La forma adimensionale (1.13) ha una conseguenza pratica immediata. La resistenza D di un corpo di dimensione L in una corrente U di un fluido con ρ e μ assegnati dipende da quattro grandezze dimensionali; per il teorema di Buckingham, con tre unità fondamentali, ne esistono due combinazioni adimensionali indipendenti, e la relazione deve avere la forma

$$C_D = \frac{D}{\frac{1}{2}\rho U^2 L^2} = F(Re). \tag{1.20}$$

Per una sfera di diametro d la curva $F(Re)$ è nota sperimentalmente su dieci decadi di Re e la si ritrova in tutti i manuali. Nei due limiti del § 1.7 si può dire di più. Per $Re \ll 1$ l'inerzia è trascurabile, la resistenza non può dipendere da ρ e la sola combinazione possibile è $D \propto \mu U d$: la costante, 3π, si ottiene dalla soluzione di Stokes del Cap. 7, e corrisponde a $C_D = 24/Re$. Per $Re \gg 1$, se la resistenza non dipende da μ, C_D deve tendere a una costante: è la resistenza di forma, dovuta alla scia che nasce dal distacco dello strato limite, e vale circa 0,4–0,5 fra $Re = 10^3$ e 2×10^5. L'analisi dimensionale non dà il valore delle costanti, ma dice quali esperimenti servono: uno solo per ogni valore di Re, qualunque siano fluido, dimensione e velocità.

1.9 Un panorama dei problemi del modulo

Le semplificazioni che si possono introdurre nelle equazioni di Navier–Stokes conducono a problemi matematici di natura molto diversa. Conviene riassumerle prima di affrontarle una per una.

problema	equazione	metodo numerico
strato limite di similitudine (Cap. 2, 3)	equazioni ordinarie non lineari, problema ai limiti	tiro con Runge–Kutta; differenze finite e sistema tridiagonale
corrente in un condotto (Cap. 4)	Poisson (ellittica)	sovrarilassamento, metodo diretto
correnti parallele non stazionarie (Cap. 5, 6)	diffusione (parabolica)	metodi espliciti e impliciti, Crank–Nicolson
flusso di Stokes (Cap. 7)	biarmonica (ellittica del quarto ordine)	coppia di Poisson accoppiate dalle condizioni di parete
stabilità (Cap. 8, 9)	problemi agli autovalori	differenze finite, collocazione di Chebyshev
problemi periodici (Cap. 10)	ordinarie e paraboliche	trasformata di Fourier

Prima di calcolare, semplificare

Nessuno dei problemi di questo modulo richiede di risolvere le equazioni di Navier–Stokes complete. In ciascun caso un'osservazione fisica (la geometria parallela, lo spessore sottile dello strato limite, il numero di Reynolds molto piccolo, l'ampiezza infinitesima delle perturbazioni) permette di eliminare termini o variabili. È un'abitudine da coltivare anche quando si dispone di un codice commerciale: la soluzione semplificata fornisce l'ordine di grandezza atteso e il caso di verifica con cui controllare il calcolo completo.

Capitolo 2

Lo strato limite laminare sulla lamina piana

2.1 L'idea di Prandtl

Nel 1904 Ludwig Prandtl propose di risolvere il paradosso del fluido ideale dividendo il campo di moto attorno a un corpo affusolato in due regioni. Lontano dalla parete gli sforzi viscosi sono trascurabili e vale la teoria del fluido ideale del modulo precedente. Vicino alla parete, invece, la velocità deve scendere dal valore della corrente esterna a zero, e lo fa in uno strato tanto sottile che i gradienti trasversali di velocità diventano grandissimi: lì gli sforzi viscosi sono dello stesso ordine delle forze d'inerzia, per quanto piccola sia la viscosità. Questo **strato limite** è responsabile della resistenza d'attrito, dello scambio termico con la parete e, quando si distacca, della resistenza di forma.

Lo spessore dello strato limite si stima con un bilancio fra i termini d'inerzia e quelli viscosi. Su una lunghezza L lungo la parete il termine convettivo vale circa $U \cdot U/L$; il termine viscoso, con un gradiente trasversale che si sviluppa in uno spessore δ, vale circa $\nu U/\delta^2$. Uguagliandoli:

$$\frac{U^2}{L} \sim \frac{\nu U}{\delta^2} \quad \Longrightarrow \quad \frac{\delta}{L} \sim \sqrt{\frac{\nu}{UL}} = \frac{1}{\sqrt{Re}}. \tag{2.1}$$

Per l'ala dell'aliante dell'esempio del Cap. 1 ($Re \simeq 1{,}7 \times 10^6$) si ottiene $\delta/L \sim 10^{-3}$: uno strato di frazioni di millimetro, in cui però avviene tutto ciò che la teoria del potenziale non sa spiegare.

2.2 Le equazioni dello strato limite

Consideriamo un moto piano in un sottile strato aderente a una parete, con l'asse x lungo la parete e l'asse y normale. Le stime di ordine di grandezza permettono di semplificare drasticamente le equazioni di Navier–Stokes.

Analisi degli ordini di grandezza

Siano $x \sim L$, $y \sim \delta$, $u \sim U$, con $\delta \ll L$.

1. *Continuità.* Da $\partial u/\partial x + \partial v/\partial y = 0$ i due termini devono essere dello stesso ordine: $U/L \sim v/\delta$, quindi $v \sim U\delta/L \ll U$. La componente normale della velocità è piccola.

2. *Quantità di moto lungo x.* I termini $u\,\partial u/\partial x$ e $v\,\partial u/\partial y$ sono entrambi di ordine U^2/L. Fra i termini viscosi, $\nu\,\partial^2 u/\partial x^2 \sim \nu U/L^2$ è trascurabile rispetto a $\nu\,\partial^2 u/\partial y^2 \sim \nu U/\delta^2$, di un fattore $(\delta/L)^2$. Per la (2.1) quest'ultimo è dell'ordine dei termini d'inerzia: lo si conserva.
3. *Quantità di moto lungo y.* Tutti i termini sono più piccoli di quelli della componente x di un fattore δ/L; resta $\partial p/\partial y = \mathcal{O}\left(\rho U^2 \delta/L^2\right)$. Attraverso lo strato la pressione varia di una quantità di ordine $\rho U^2\,\delta/L$, trascurabile: *la pressione è impressa dalla corrente esterna*.

Ne seguono le **equazioni di Prandtl** per lo strato limite piano incomprimibile:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0, \tag{2.2}$$

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\mathrm{d}p_e}{\mathrm{d}x} + \nu\frac{\partial^2 u}{\partial y^2}, \tag{2.3}$$

dove la pressione $p_e(x)$ si ricava dalla corrente esterna $U_e(x)$ con il teorema di Bernoulli, $-\rho^{-1}\mathrm{d}p_e/\mathrm{d}x = U_e\,\mathrm{d}U_e/\mathrm{d}x$ (in moto stazionario).

Il guadagno è triplice. Le incognite scendono da tre (u, v, p) a due; l'equazione (2.3) è *parabolica* in x (la derivata seconda compare solo rispetto a y), quindi la soluzione si può costruire marciando lungo la parete, come in un problema ai valori iniziali; infine, per alcune distribuzioni di $U_e(x)$, le equazioni ammettono soluzioni di *similitudine* che le riducono a equazioni ordinarie. Con proprietà variabili e un gas comprimibile alle (2.2)–(2.3) si aggiungono l'equazione dell'energia dello strato limite e l'equazione di stato; per l'incomprimibile l'energia si disaccoppia e sarà trattata nel Cap. 3.

2.3 Il problema di Blasius

Il caso più semplice è quello di una lamina piana semi-infinita, allineata con una corrente uniforme di velocità U (Fig. 2.1). La pressione esterna è costante e le equazioni di Prandtl stazionarie diventano

$$u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \nu\frac{\partial^2 u}{\partial y^2}, \qquad \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0, \tag{2.4}$$

con le condizioni di aderenza alla parete e di raccordo con la corrente esterna:

$$u = v = 0 \ \text{ per } y = 0, \qquad u \to U \ \text{ per } y \to \infty. \tag{2.5}$$

Sono tre condizioni per un sistema del terzo ordine complessivo (secondo in u rispetto a y, primo in v): il problema è ben posto. Sulla componente v all'infinito non si può imporre nulla, e si vedrà che infatti non si annulla.

2.3.1 La trasformazione di similitudine

Le misure mostrano che i profili u/U rilevati a distanze diverse dal bordo d'attacco si sovrappongono se l'ordinata viene "stirata" con la legge $y/\sqrt{x}$: il moto è **autosimile**. La (2.1), applicata con $L = x$, dice la stessa cosa: lo spessore locale è proporzionale a $\sqrt{\nu x/U}$. Questo suggerisce di cercare una soluzione in cui u/U dipenda da x e y soltanto attraverso la combinazione $y/\sqrt{\nu x/U}$.

Segnaposto figura

file atteso: `immagini/fig_3_2_1.png`

Lamina piana semi-infinita lungo l'asse x in una corrente uniforme U: profilo di velocità $u(y)$ e bordo dello strato limite $y = \delta(x)$, parabolico (Fig. 3.2.1 della fonte).

Figura 2.1: Strato limite su una lamina piana.

Riduzione a un'equazione ordinaria

Introduciamo la funzione di corrente ($u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$), che soddisfa automaticamente la continuità, e cerchiamo soluzioni della forma

$$\eta = A\,x^m y, \qquad \psi = B\,x^n f(\eta),$$

con A, B costanti, f funzione adimensionale e la richiesta che dopo la sostituzione x e y scompaiano. Poiché $\partial\eta/\partial y = Ax^m$ e $\partial\eta/\partial x = m\eta/x$:

$$u = ABx^{m+n} f', \qquad v = -Bx^{n-1}\left(nf + m\eta f'\right),$$

dove l'apice indica la derivata rispetto a η. Perché $u \to U$ per $\eta \to \infty$ qualunque sia x deve essere $m + n = 0$ e $AB = U$ (con $f' \to 1$). Sostituendo nella (2.4):

$$u\frac{\partial u}{\partial x} = U^2\,\frac{m}{x}\,\eta f' f'', \quad v\frac{\partial u}{\partial y} = -U^2\,\frac{nf + m\eta f'}{x}\,f'', \quad \nu\frac{\partial^2 u}{\partial y^2} = \nu U A^2 x^{2m} f'''.$$

I termini con $\eta f' f''$ si cancellano e resta $-(nU^2/x)\,ff'' = \nu U A^2 x^{2m} f'''$. Perché x scompaia occorre $2m = -1$, cioè $m = -\frac{1}{2}$, $n = \frac{1}{2}$. Scegliendo $A = \sqrt{U/\nu}$ (quindi $B = \sqrt{\nu U}$) si ottiene:

$$\eta = y\sqrt{\frac{U}{\nu x}}, \qquad \psi = \sqrt{\nu U x}\,f(\eta), \tag{2.6}$$

$$f''' + \tfrac{1}{2}\,ff'' = 0, \qquad f(0) = f'(0) = 0, \quad f'(\infty) = 1, \tag{2.7}$$

$$\frac{u}{U} = f'(\eta), \qquad \frac{v}{\sqrt{\nu U/x}} = \tfrac{1}{2}\left(\eta f' - f\right). \tag{2.8}$$

L'**equazione di Blasius** (2.7) è un'equazione ordinaria del terzo ordine, non lineare, con due condizioni nell'origine e una all'infinito: un problema ai limiti su un intervallo illimitato. Si noti che la trasformazione (2.6) non vale nel bordo d'attacco, $x = 0$, dove lo strato limite ha spessore nullo e le ipotesi di Prandtl cadono; la soluzione descrive il moto a distanze dal bordo d'attacco grandi rispetto a ν/U, cioè per $Re_x = Ux/\nu \gg 1$.

2.4 Il metodo di tiro

Blasius risolse la (2.7) raccordando uno sviluppo in serie attorno a $\eta = 0$ con uno sviluppo asintotico per $\eta \to \infty$. Numericamente è più semplice trasformare il problema ai limiti in un

problema ai valori iniziali e usare gli strumenti del primo modulo. Scritta come sistema del primo ordine con $p = f'$ e $q = f''$,

$$\frac{\mathrm{d}f}{\mathrm{d}\eta} = p, \qquad \frac{\mathrm{d}p}{\mathrm{d}\eta} = q, \qquad \frac{\mathrm{d}q}{\mathrm{d}\eta} = -\tfrac{1}{2} f q, \tag{2.9}$$

l'equazione si integra passo per passo con il metodo di Runge–Kutta del quarto ordine, *purché si conoscano* f, p e q nell'origine. Le condizioni al contorno forniscono $f(0) = p(0) = 0$, ma non $q(0) = f''(0)$; in cambio danno una condizione in più all'altro estremo. L'idea del **metodo di tiro** è quella dell'artigliere: si assegna un valore di prova q_0, si "spara" integrando fino a un'ascissa grande e si corregge q_0 secondo l'errore commesso sul bersaglio $p(\infty) = 1$.

Le formule di Runge–Kutta per il sistema (2.9)

Noti f_i, p_i, q_i in η_i, con passo h:

$$\begin{aligned}
&\Delta_1 f = h p_i, && \Delta_1 p = h q_i, && \Delta_1 q = -\tfrac{1}{2} h f_i q_i,\\
&\Delta_2 f = h(p_i + \tfrac{1}{2}\Delta_1 p), && \Delta_2 p = h(q_i + \tfrac{1}{2}\Delta_1 q), && \Delta_2 q = -\tfrac{1}{2} h (f_i + \tfrac{1}{2}\Delta_1 f)(q_i + \tfrac{1}{2}\Delta_1 q),\\
&\Delta_3 f = h(p_i + \tfrac{1}{2}\Delta_2 p), && \Delta_3 p = h(q_i + \tfrac{1}{2}\Delta_2 q), && \Delta_3 q = -\tfrac{1}{2} h (f_i + \tfrac{1}{2}\Delta_2 f)(q_i + \tfrac{1}{2}\Delta_2 q),\\
&\Delta_4 f = h(p_i + \Delta_3 p), && \Delta_4 p = h(q_i + \Delta_3 q), && \Delta_4 q = -\tfrac{1}{2} h (f_i + \Delta_3 f)(q_i + \Delta_3 q),
\end{aligned}$$

e infine $f_{i+1} = f_i + \frac{1}{6}(\Delta_1 f + 2\Delta_2 f + 2\Delta_3 f + \Delta_4 f)$, e analogamente per p e q. In forma vettoriale, con $\mathbf{z} = (f, p, q)$, sono le formule $\mathbf{k}_1 = h\mathbf{F}(\mathbf{z}_i)$, $\mathbf{k}_2 = h\mathbf{F}(\mathbf{z}_i + \mathbf{k}_1/2)$, … del primo modulo, ed è così che sono scritte nel programma.

2.4.1 La condizione all'infinito e il dimezzamento dell'intervallo

Un programma non può integrare fino all'infinito. Si sceglie un'ascissa $\eta_{\max}$ abbastanza grande e si sostituisce la condizione $f'(\infty) = 1$ con

$$0 \le 1 - p(\eta_{\max}) \le \varepsilon, \tag{2.10}$$

dove $\varepsilon \ll 1$ controlla l'accuratezza. La condizione chiede che p si avvicini a 1 *dal basso*, come deve fare la velocità parallela in uno strato limite senza gradiente di pressione.

Metodo del dimezzamento dell'intervallo sul valore iniziale

1. Si sceglie un valore iniziale q_0 e un incremento $\Delta q_0 > 0$.
2. Si pone $q_0 \leftarrow q_0 + \Delta q_0$ e si integra il sistema (2.9) da $\eta = 0$ (con $f = p = 0$, $q = q_0$) fino a $\eta_{\max}$; sia $p_{\max}$ il valore finale di p.
3. Se errore $= 1 - p_{\max}$ soddisfa la (2.10), ci si ferma.
4. Altrimenti, se il punto $(q_0, p_{\max})$ ha "scavalcato" la retta $p_{\max} = 1$ (cioè $p_{\max} > 1$ con $\Delta q_0 > 0$, oppure $p_{\max} < 1$ con $\Delta q_0 < 0$), si inverte il verso e si dimezza il passo: $\Delta q_0 \leftarrow -\Delta q_0/2$. Si torna al punto 2.

È il metodo di bisezione del primo modulo, con la differenza che l'intervallo che contiene la radice non è noto in partenza: lo si cerca avanzando a passi costanti finché non si scavalca il bersaglio (Fig. 2.2).

Segnaposto figura

file atteso: `immagini/fig_3_2_2.png`

Valore finale $p_{\max}$ in funzione del valore di partenza $q_0 = f''(0)$: si avanza con passo $\Delta_1 q_0$ finché non si supera la retta $p_{\max} = 1$, poi si torna indietro con passo $\Delta_2 q_0 = -\Delta_1 q_0/2$, e così via (Fig. 3.2.2 della fonte).

Figura 2.2: Metodo del dimezzamento dell'intervallo per il problema di Blasius.

Dati del Programma 3.1

$\eta_{\max} = 10$; passo $h = 0{,}1$; $\varepsilon = 10^{-6}$; valore di partenza $q_0 = 1$ e incremento iniziale $\Delta q_0 = 0{,}2$ (il primo tentativo è quindi $q_0 = 1{,}2$).

prog3_1_blasius.py

```
"""
prog3_1_blasius.py -- PROGRAMMA 3.1
Strato limite laminare sulla lamina piana semi-infinita (problema di Blasius):
    f''' + (1/2) f f'' = 0,   f(0) = f'(0) = 0,   f'(inf) = 1
risolto come problema ai valori iniziali con Runge-Kutta del quarto ordine
(sistema f' = p, p' = q, q' = -f q / 2) e metodo del dimezzamento
dell'intervallo sul valore iniziale incognito q0 = f''(0).

Condizione all'infinito approssimata:  0 <= 1 - p(eta_max) <= eps.
Uscite: numero di iterazioni, f''(0), tabella f, f', f'' ogni 0,2,
spessori dello strato limite, figura dei profili u/U e v/sqrt(nu U/x).
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import simpson
from stile_grafici import salva, BLU, ARANCIO, GRIGIO

ETAMAX = 10.0
H = 0.1
EPSLON = 1.0e-6


def blasius(z):
    """Secondo membro del sistema: z = (f, p, q)."""
    f, p, q = z
    return np.array([p, q, -0.5 * f * q])


def integra(q0):
    """Integra da eta = 0 fino a superare ETAMAX (come il programma originale)."""
    eta = [0.0]
    Z = [np.array([0.0, 0.0, q0])]
    while eta[-1] <= ETAMAX:
        z = Z[-1]
        k1 = H * blasius(z)
        k2 = H * blasius(z + k1 / 2)
        k3 = H * blasius(z + k2 / 2)
        k4 = H * blasius(z + k3)
        Z.append(z + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0)
```

```
        eta.append(eta[-1] + H)
    return np.array(eta), np.array(Z)


# ------------------------------------------- metodo del dimezzamento
q0, dq0 = 1.0, 0.2
iterazioni = 0
while True:
    iterazioni += 1
    q0 = q0 + dq0
    eta, Z = integra(q0)
    pmax = Z[-1, 1]
    errore = 1.0 - pmax
    if 0.0 <= errore <= EPSLON:
        break
    # si inverte il verso (e si dimezza il passo) quando si e' "scavalcato" 1
    if (pmax > 1.0 and dq0 > 0.0) or (pmax < 1.0 and dq0 < 0.0):
        dq0 = -dq0 / 2.0

f, p, q = Z[:, 0], Z[:, 1], Z[:, 2]
print(f"Numero di iterazioni = {iterazioni}")
print(f"f''(0) = q0 = {q0:.7f}    (1 - p_max = {errore:.2e})")
print("\n    eta        f          f'         f''")
for i in range(0, 101, 2):
    print(f"  {eta[i]:5.2f}  {f[i]:9.5f}  {p[i]:9.5f}  {q[i]:10.3e}")

# ------------------------------------------- grandezze dello strato limite
v = 0.5 * (eta * p - f)                        # v / sqrt(nu U / x)
i99 = np.argmax(p >= 0.99)
eta99 = np.interp(0.99, p[i99 - 1:i99 + 1], eta[i99 - 1:i99 + 1])
i994 = np.argmax(p >= 0.994)
eta994 = np.interp(0.994, p[i994 - 1:i994 + 1], eta[i994 - 1:i994 + 1])
n10 = int(round(10.0 / H))
spost = eta[n10] - f[n10]                      # delta* sqrt(U/(nu x)) = lim (eta - f)
quant = simpson(p[:n10 + 1] * (1 - p[:n10 + 1]), x=eta[:n10 + 1])   # regola di Simpson
print(f"\nu/U = 0,99  per eta = {eta99:.3f};   u/U = 0,994 per eta = {eta994:.3f}")
print(f"spessore di spostamento   delta* sqrt(U/nu x) = {spost:.4f}")
print(f"spessore di quantita' di moto theta sqrt(U/nu x) = {quant:.4f}  (2 f''(0) = {2*q0:.4f})")
print(f"fattore di forma H = delta*/theta = {spost/quant:.4f}")
print(f"velocita' normale all'esterno v/sqrt(nu U/x) = {v[n10]:.4f}")
print(f"coefficiente d'attrito locale  C_f sqrt(Re_x) = 2 f''(0) = {2*q0:.4f}")
print(f"resistenza di una faccia       C_D sqrt(Re_L) = 4 f''(0) = {4*q0:.4f}")

# esempio numerico: aria a 30 m/s, x = 1 m
U, nu, x = 30.0, 1.49e-5, 1.0
Rex = U * x / nu
print(f"\nAria, U = {U} m/s, x = {x} m: Re_x = {Rex:.3e}")
print(f"  delta(0,994) = {eta994*np.sqrt(nu*x/U)*100:.3f} cm,  "
      f"delta* = {spost*np.sqrt(nu*x/U)*1000:.3f} mm,  v_inf = {v[n10]*np.sqrt(nu*U/x):.4f} m/s")
print(f"  tau_w = {q0*np.sqrt(U/(nu*x))*U*1.225*nu:.4f} Pa  (rho = 1,225 kg/m^3)")

# ------------------------------------------- figura
fig, ax = plt.subplots(figsize=(4.4, 3.6))
ax.plot(p[:n10 + 1], eta[:n10 + 1], color=BLU, label=r"$u/U=f'$")
ax.plot(v[:n10 + 1], eta[:n10 + 1], color=ARANCIO, ls="--",
        label=r"$v/\sqrt{\nu U/x}=\frac{1}{2}(\eta f'-f)$")
```

```
ax.plot(p[:n10 + 1:4], eta[:n10 + 1:4], "o", ms=3, color=BLU)
ax.plot(v[:n10 + 1:4], eta[:n10 + 1:4], "+", ms=4, color=ARANCIO)
ax.axhline(eta994, color=GRIGIO, lw=0.8, ls=":")
ax.text(0.55, eta994 + 0.08, r"$u/U=0{,}994$", fontsize=7, color=GRIGIO)
ax.set_xlim(0, 1.0)
ax.set_ylim(0, 6)
ax.set_xlabel("componenti adimensionali della velocita'")
ax.set_ylabel(r"$\eta = y\sqrt{U/(\nu x)}$")
ax.legend(loc="upper left", bbox_to_anchor=(0.0, 0.84))
salva(fig, "prog3_1_blasius")
```

Listato 2.1: Programma 3.1 – strato limite di Blasius con Runge–Kutta e dimezzamento dell'intervallo (`prog3_1_blasius.py`).

Risultati del Programma 3.1

Il programma converge in **31 iterazioni** a

$$f''(0) = 0{,}3320572, \qquad 1 - p_{\max} = 3{,}1 \times 10^{-7}.$$

Un estratto della tabella stampata:

η	f	$f' = u/U$	f''
0,0	0,000 00	0,000 00	$3{,}321 \times 10^{-1}$
1,0	0,165 57	0,329 78	$3{,}230 \times 10^{-1}$
2,0	0,650 02	0,629 77	$2{,}668 \times 10^{-1}$
3,0	1,396 81	0,846 04	$1{,}614 \times 10^{-1}$
4,0	2,305 75	0,955 52	$6{,}423 \times 10^{-2}$
5,0	3,283 27	0,991 54	$1{,}591 \times 10^{-2}$
6,0	4,279 62	0,998 97	$2{,}402 \times 10^{-3}$
7,0	5,279 24	0,999 92	$2{,}202 \times 10^{-4}$

I valori coincidono con quelli ottenuti da Howarth con sviluppi in serie [8]. Oltre $\eta \simeq 6$ la funzione f'' decresce molto rapidamente e f cresce linearmente, $f \simeq \eta - 1{,}72$.

2.5 Le grandezze ingegneristiche dello strato limite

Nota la funzione f, tutte le grandezze di interesse pratico si esprimono con pochi numeri. Alcuni si ottengono in forma chiusa dall'equazione stessa, ed è istruttivo dimostrarlo.

Spessori, velocità normale e sforzo alla parete

Spessore convenzionale. Il profilo tende a U asintoticamente: il "bordo" dello strato limite è una convenzione. Definendo δ come la quota dove $u = 0{,}99\,U$ si trova $\eta_{99} = 4{,}91$, cioè $\delta_{99} = 4{,}91\sqrt{\nu x/U}$; con $u = 0{,}994\,U$ si ha $\eta = 5{,}18$. In ogni caso il bordo è una parabola, $\delta \propto \sqrt{x}$.

Spessore di spostamento. È lo spostamento verso l'esterno delle linee di corrente prodotto dal rallentamento del fluido:

$$\delta^* = \int_0^\infty \left(1 - \frac{u}{U}\right) \mathrm{d}y = \sqrt{\frac{\nu x}{U}} \lim_{\eta\to\infty} (\eta - f) = 1{,}7208\sqrt{\frac{\nu x}{U}}.$$

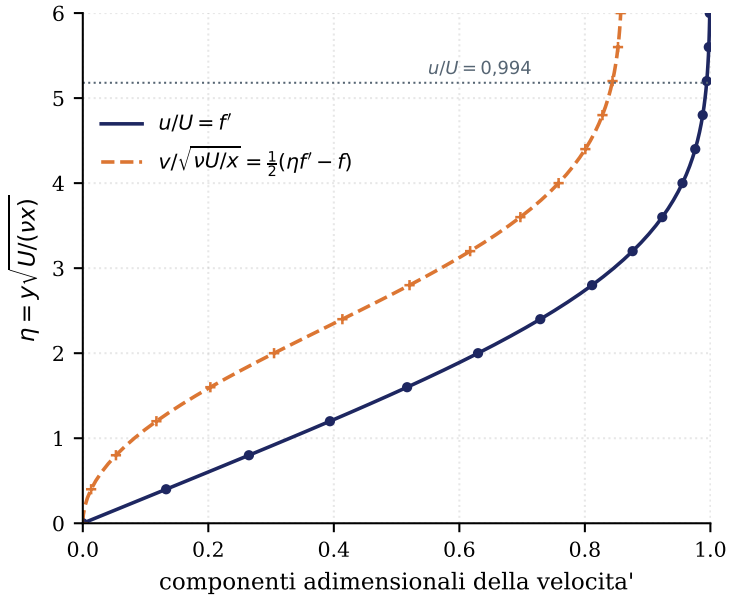


Figura 2.3: Componenti adimensionali della velocità nello strato limite di Blasius. Le due componenti non sono riferite alla stessa velocità: in forma dimensionale v è più piccola di u di un fattore $Re_x^{-1/2}$.

Velocità normale all'esterno. Dalla (2.8), poiché per $\eta \to \infty$ si ha $f' \to 1$ e $\eta - f \to 1{,}7208$: $v_\infty = \frac{1}{2} \cdot 1{,}7208\sqrt{\nu U/x} = 0{,}8604\sqrt{\nu U/x}$. Non è un caso che sia metà del coefficiente di δ^*: la corrente esterna "vede" un corpo ispessito di $\delta^*(x)$, e $v_\infty = U\,\mathrm{d}\delta^*/\mathrm{d}x$.

Spessore di quantità di moto. $\theta = \int_0^\infty \frac{u}{U}\left(1 - \frac{u}{U}\right)\mathrm{d}y = \sqrt{\nu x/U}\int_0^\infty f'(1-f')\,\mathrm{d}\eta$. Poiché $\mathrm{d}[f(1-f')]/\mathrm{d}\eta = f'(1-f') - ff''$ e $ff'' = -2f'''$ per la (2.7),

$$\int_0^\infty f'(1-f')\,\mathrm{d}\eta = \Big[f(1-f')\Big]_0^\infty - 2\int_0^\infty f'''\,\mathrm{d}\eta = 0 + 2f''(0) = 0{,}6641.$$

Sforzo e attrito. $\tau_w = \mu(\partial u/\partial y)_{y=0} = \mu U\sqrt{U/(\nu x)}\,f''(0)$, quindi

$$C_f = \frac{\tau_w}{\frac{1}{2}\rho U^2} = \frac{2f''(0)}{\sqrt{Re_x}} = \frac{0{,}664}{\sqrt{Re_x}}, \qquad C_D = \frac{1}{L}\int_0^L C_f\,\mathrm{d}x = \frac{1{,}328}{\sqrt{Re_L}} \text{ (una faccia).}$$

Il fatto che $\theta = 2f''(0)\sqrt{\nu x/U}$ è l'equazione integrale di von Kármán, $\mathrm{d}\theta/\mathrm{d}x = \tau_w/(\rho U^2)$: la resistenza accumulata fino all'ascissa x è la quantità di moto mancante nello strato limite.

Il rapporto $H = \delta^*/\theta = 2{,}59$ (fattore di forma) è caratteristico del profilo laminare senza gradiente di pressione; nei profili turbolenti vale circa 1,3–1,4.

Lamina piana in aria a $30\,\mathrm{m\,s^{-1}}$

Con $\nu = 1{,}49 \times 10^{-5}\,\mathrm{m^2\,s^{-1}}$ e $\rho = 1{,}225\,\mathrm{kg\,m^{-3}}$, a $x = 1\,\mathrm{m}$ dal bordo d'attacco il programma fornisce $Re_x = 2{,}01 \times 10^6$, $\delta_{0{,}994} = 3{,}65\,\mathrm{mm}$, $\delta^* = 1{,}21\,\mathrm{mm}$, $\tau_w = 0{,}258\,\mathrm{Pa}$ e una velocità normale all'esterno $v_\infty = 0{,}018\,\mathrm{m\,s^{-1}}$, appena lo 0,06% della corrente: lo

strato limite è davvero sottile e la sua presenza perturba pochissimo la corrente esterna. In linea di principio, però, un anemometro posto sopra la lamina ne rileverebbe la presenza misurando la componente verticale della velocità.

Fin dove vale la soluzione di Blasius

Lo strato limite laminare su una lamina reale diventa instabile e transisce alla turbolenza per Re_x compreso fra circa 3×10^5 e 3×10^6, a seconda della turbolenza della corrente esterna e della rugosità della parete. Il valore $Re_x = 2 \times 10^6$ dell'esempio è quindi al limite: in una galleria del vento comune lo strato limite sarebbe già turbolento, con uno sforzo alla parete varie volte maggiore. Le ragioni dell'instabilità sono il tema del Cap. 9.

2.6 Metodi di tiro più efficienti e studio dell'accuratezza

Il dimezzamento dell'intervallo è robusto ma lento: le 31 iterazioni del Programma 3.1 sono quasi tutte spese a "cercare" la radice, perché ogni dimezzamento guadagna un solo bit di precisione. Due alternative molto più rapide sono il metodo della secante e il metodo di Newton.

Il metodo di Newton con le equazioni variazionali

Sia $r(q_0) = p(\eta_{\max}; q_0) - 1$ il residuo del tiro. Il metodo di Newton richiede la derivata $r'(q_0) = \partial p(\eta_{\max})/\partial q_0$. Derivando il sistema (2.9) rispetto a q_0 si ottengono le equazioni per le *sensibilità* $F = \partial f/\partial q_0$, $P = \partial p/\partial q_0$, $Q = \partial q/\partial q_0$:

$$F' = P, \qquad P' = Q, \qquad Q' = -\tfrac{1}{2}\,(Fq + fQ)\,, \qquad F(0) = P(0) = 0,\ Q(0) = 1, \tag{2.11}$$

lineari in (F, P, Q) e con coefficienti dati dalla soluzione corrente. Integrando insieme i sei componenti si ottengono in un colpo solo r e $r' = P(\eta_{\max})$, e si aggiorna

$$q_0 \leftarrow q_0 - \frac{p(\eta_{\max}) - 1}{P(\eta_{\max})}.$$

Vicino alla radice l'errore viene elevato al quadrato a ogni iterazione (convergenza quadratica). Il metodo della secante sostituisce r' con il rapporto incrementale fra le ultime due iterate e converge con ordine 1,62, senza richiedere le (2.11).

Il Programma 3.1b confronta i due metodi, li paragona con un risolutore di problemi ai limiti di libreria (`scipy.integrate.solve_bvp`, basato sulla collocazione) e studia separatamente le due sorgenti di errore: il passo di integrazione h e il troncamento del dominio a $\eta_{\max}$.

```
"""
prog3_1b_tiro.py -- metodi di tiro piu' efficienti per il problema di Blasius
e studio dell'accuratezza.

1) Metodo di Newton con le equazioni variazionali: insieme a (f, p, q) si
   integrano le sensibilita' (F, P, Q) = d(f, p, q)/dq0, che soddisfano
       F' = P,  P' = Q,  Q' = -(F q + f Q)/2,   F(0) = P(0) = 0, Q(0) = 1,
   e si corregge  q0 <- q0 - (p(eta_max) - 1) / P(eta_max).
2) Metodo della secante (non richiede le sensibilita').
3) Confronto con il risolutore di problemi ai limiti scipy.integrate.solve_bvp.
4) Errore su f''(0) al variare del passo h (ordine 4 di RK4) e di eta_max.
```

prog3_1b_tiro.py

```
"""
import time
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_bvp, solve_ivp
from metodi_numerici import rk4_integra
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO


def blasius(eta, z):
    f, p, q = z
    return np.array([p, q, -0.5 * f * q])


def blasius_var(eta, z):
    """Sistema esteso: (f, p, q) e sensibilita' (F, P, Q) rispetto a q0."""
    f, p, q, F, P, Q = z
    return np.array([p, q, -0.5 * f * q, P, Q, -0.5 * (F * q + f * Q)])


def newton(q0, h=0.1, etamax=10.0, tol=1e-12, stampa=False):
    storia = []
    for it in range(1, 30):
        _, Z = rk4_integra(blasius_var, np.array([0, 0, q0, 0, 0, 1.0]), etamax, h)
        res = Z[-1, 1] - 1.0
        storia.append((it, q0, res))
        if stampa:
            print(f"   it {it}:  q0 = {q0:.12f}   p(eta_max) - 1 = {res:+.3e}")
        if abs(res) < tol:
            return q0, it, storia
        q0 -= res / Z[-1, 4]
    raise RuntimeError("Newton non converge")


def secante(qa, qb, h=0.1, etamax=10.0, tol=1e-12):
    def r(q):
        _, Z = rk4_integra(blasius, np.array([0, 0, q]), etamax, h)
        return Z[-1, 1] - 1.0
    ra, rb = r(qa), r(qb)
    for it in range(1, 50):
        qc = qb - rb * (qb - qa) / (rb - ra)
        qa, ra, qb, rb = qb, rb, qc, r(qc)
        if abs(rb) < tol:
            return qb, it
    raise RuntimeError("secante non converge")


# ------------------------------------------------------------- 1) e 2)
print("1) Newton con equazioni variazionali (h = 0,1, eta_max = 10), partenza q0 =
    1:")
t0 = time.perf_counter()
q_n, it_n, storia = newton(1.0, stampa=True)
t_n = time.perf_counter() - t0
print(f"   f''(0) = {q_n:.10f}  in {it_n} iterazioni ({t_n*1e3:.0f} ms)")
t0 = time.perf_counter()
q_s, it_s = secante(0.2, 0.5)
t_s = time.perf_counter() - t0
print(f"2) Secante (partenze 0,2 e 0,5): f''(0) = {q_s:.10f} in {it_s} iterazioni
    ({t_s*1e3:.0f} ms)")

```

```
# ------------------------------------------------------------ valore di riferimento
# integrazione adattiva molto accurata su un dominio ampio, con Newton
def residuo_ivp(q0, etamax=20.0):
    s = solve_ivp(blasius_var, (0, etamax), [0, 0, q0, 0, 0, 1], rtol=1e-13,
    atol=1e-14,
                  method="DOP853")
    return s.y[1, -1] - 1.0, s.y[4, -1]


q_rif = 0.33
for _ in range(12):
    r, dr = residuo_ivp(q_rif)
    q_rif -= r / dr
print(f"   valore di riferimento (DOP853, eta_max = 20): f''(0) = {q_rif:.12f}")

# ------------------------------------------------------------ 3) solve_bvp
def bc(za, zb):
    return np.array([za[0], za[1], zb[1] - 1.0])

eta0 = np.linspace(0, 10, 51)
z0 = np.vstack([eta0 - 1.7 * (1 - np.exp(-eta0)), 1 - np.exp(-eta0), 0.33 *
    np.exp(-eta0)])
t0 = time.perf_counter()
sol = solve_bvp(blasius, bc, eta0, z0, tol=1e-8, max_nodes=20000)
t_b = time.perf_counter() - t0
print(f"3) solve_bvp (collocazione): f''(0) = {sol.sol(0.0)[2]:.10f}, "
      f"{sol.x.size} nodi finali, stato = {sol.status} ({t_b*1e3:.0f} ms)")

# ------------------------------------------------------------ 4) accuratezza
passi = np.array([0.4, 0.2, 0.1, 0.05, 0.025])
err_h = []
print("\n4a) Errore su f''(0) al variare del passo (eta_max = 10):")
q_10, _, _ = newton(0.33, h=0.0025, etamax=10.0)   # riferimento con lo stesso
    eta_max
for h in passi:
    q, _, _ = newton(0.33, h=h, etamax=10.0)
    err_h.append(abs(q - q_10))
for k, h in enumerate(passi):
    ordine = "" if k == 0 else f"   ordine osservato
    {np.log2(err_h[k-1]/err_h[k]):.2f}"
    print(f"   h = {h:6.3f}   |errore| = {err_h[k]:.3e}{ordine}")

emax = np.array([4.0, 5.0, 6.0, 7.0, 8.0, 10.0, 12.0])
err_e = []
print("\n4b) Errore su f''(0) al variare di eta_max (h = 0,01):")
for e in emax:
    q, _, _ = newton(0.33, h=0.01, etamax=e)
    err_e.append(abs(q - q_rif))
    print(f"   eta_max = {e:5.1f}   f''(0) = {q:.10f}   |errore| = {err_e[-1]:.2e}")

# ------------------------------------------------------------ figura
fig, ax = plt.subplots(1, 3, figsize=(7.6, 2.6))
it = [s[0] for s in storia]
ax[0].semilogy(it, [abs(s[2]) + 1e-17 for s in storia], "o-", color=BLU)
ax[0].set_xlabel("iterazione di Newton")
ax[0].set_ylabel(r"$|f'(\eta_{\max})-1|$")
ax[0].set_title("convergenza quadratica")
ax[1].loglog(passi, err_h, "s-", color=ARANCIO, label="RK4")
ax[1].loglog(passi, err_h[2] * (passi / passi[2]) ** 4, ":", color="k",
    label=r"$\propto h^4$")
```

```
ax[1].set_xlabel("passo $h$")
ax[1].set_ylabel(r"errore su $f''(0)$")
ax[1].legend()
ax[1].set_title("errore di discretizzazione")
ax[2].semilogy(emax, err_e, "d-", color=VERDE)
ax[2].set_xlabel(r"$\eta_{\max}$")
ax[2].set_ylabel(r"errore su $f''(0)$")
ax[2].set_title("troncamento del dominio")
fig.tight_layout()
salva(fig, "prog3_1b_tiro")
```

Listato 2.2: `prog3_1b_tiro.py`: tiro di Newton con le equazioni variazionali, secante, `solve_bvp` e studio dell'errore.

Risultati del Programma 3.1b

- *Newton*, partendo dal valore lontanissimo $q_0 = 1$: residui 1,1; $-0{,}24$; $-1{,}6 \times 10^{-2}$; $-6{,}5 \times 10^{-5}$; $-1{,}0 \times 10^{-9}$; 2×10^{-16}. In 6 iterazioni si raggiunge la precisione di macchina; il numero di cifre corrette raddoppia a ogni passo (Fig. 2.4, a sinistra). La *secante* impiega 5 iterazioni partendo da 0,2 e 0,5.
- Con $h = 0{,}1$ e $\eta_{\max} = 10$ entrambi danno $f''(0) = 0{,}3320573456$; `solve_bvp` fornisce 0,3320573372 con 362 nodi. Il valore di riferimento, ottenuto con un integratore adattivo di ordine 8 su $\eta_{\max} = 20$, è

$$f''(0) = 0{,}332057336215.$$

- L'errore dovuto al passo decresce come h^4 (ordine osservato 3,96), come previsto per Runge–Kutta del quarto ordine; con $h = 0{,}1$ vale 8×10^{-9}.
- L'errore dovuto a $\eta_{\max}$ decresce più che esponenzialmente: 2×10^{-2} per $\eta_{\max} = 4$, 5×10^{-4} per 6, $1{,}9 \times 10^{-6}$ per 8, 10^{-9} per 10. Infatti $1 - f'$ si comporta per η grande come $\exp[-(\eta - 1{,}72)^2/4]$.

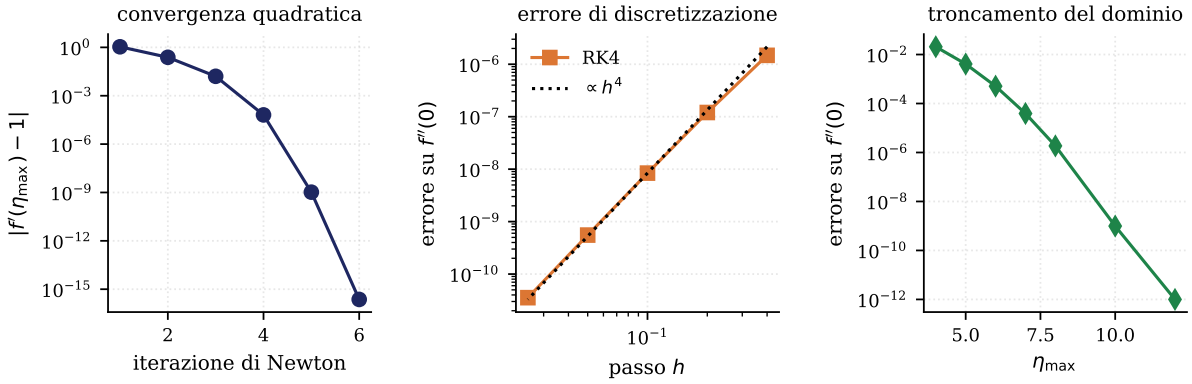


Figura 2.4: Metodo di Newton per il problema di Blasius: convergenza quadratica (a sinistra), errore in funzione del passo (al centro) e dell'estremo $\eta_{\max}$ del dominio di calcolo (a destra).

Il dominio troncato non è un dettaglio

Con $\eta_{\max} = 4$ il metodo converge perfettamente, ma al valore sbagliato 0,3527 (errore del 6%): la condizione $f' = 1$ imposta troppo vicino alla parete costringe il profilo a rag-

giungere la velocità esterna prima del dovuto, e lo sforzo alla parete risulta sovrastimato. Un buon controllo è ripetere il calcolo con $\eta_{\max}$ maggiore e verificare che il risultato non cambi.

2.7 Approfondimenti

2.7.1 L'equazione integrale di von Kármán e il metodo di Pohlhausen

Prima che i calcolatori rendessero banale l'integrazione dell'equazione di Blasius, gli strati limite si calcolavano con un metodo *approssimato* che resta istruttivo e che, per i gradienti di pressione qualsiasi, è ancora la base dei metodi integrali usati nei codici di progetto dei profili. L'idea è rinunciare a soddisfare l'equazione del moto in ogni punto e imporla soltanto *in media* attraverso lo strato.

L'equazione integrale della quantità di moto

Si integri la (2.3), stazionaria e con U_e costante, da $y = 0$ a $y \to \infty$. Il termine $v\,\partial u/\partial y$ si integra per parti usando la continuità, $v = -\int_0^y \partial u/\partial x\,\mathrm{d}y'$:

$$\int_0^\infty v\frac{\partial u}{\partial y}\mathrm{d}y = \Big[vu\Big]_0^\infty + \int_0^\infty u\frac{\partial u}{\partial x}\mathrm{d}y = -U\int_0^\infty \frac{\partial u}{\partial x}\mathrm{d}y + \int_0^\infty u\frac{\partial u}{\partial x}\mathrm{d}y.$$

Il termine viscoso dà $\nu[\partial u/\partial y]_0^\infty = -\tau_w/\rho$. Raccogliendo, e portando la derivata fuori dall'integrale (gli estremi non dipendono da x):

$$\frac{\mathrm{d}}{\mathrm{d}x}\int_0^\infty u\,(U-u)\,\mathrm{d}y = \frac{\tau_w}{\rho} \quad\Longleftrightarrow\quad \frac{\mathrm{d}\theta}{\mathrm{d}x} = \frac{\tau_w}{\rho U^2},$$

che è l'equazione di von Kármán già incontrata a proposito dello spessore di quantità di moto: la crescita dello spessore di quantità di moto misura lo sforzo alla parete. Con un gradiente di pressione compare a secondo membro anche il termine $-(2\theta+\delta^*)U'/U$.

Il metodo di Pohlhausen

Si assume per il profilo una forma $u/U = F(Y)$, con $Y = y/\delta(x)$, che soddisfi le condizioni essenziali: $F(0) = 0$, $F(1) = 1$, $F'(1) = 0$ (raccordo liscio con la corrente esterna) e, volendo, $F''(0) = 0$ (dall'equazione del moto scritta alla parete, dove $u = v = 0$ e $\mathrm{d}p/\mathrm{d}x = 0$) e $F''(1) = 0$. Allora $\theta = I_2\delta$ e $\tau_w = \mu U F'(0)/\delta$, con $I_2 = \int_0^1 F(1-F)\,\mathrm{d}Y$, e l'equazione integrale diventa $I_2\,\delta\,\mathrm{d}\delta/\mathrm{d}x = \nu F'(0)/U$, cioè

$$\delta = \sqrt{\frac{2F'(0)}{I_2}}\sqrt{\frac{\nu x}{U}}, \qquad C_f\sqrt{Re_x} = \sqrt{2F'(0)\,I_2}, \qquad \delta^* = I_1\delta, \quad I_1 = \int_0^1 (1-F)\,\mathrm{d}Y.$$

Il Programma 3.1c applica la formula ai profili lineare, cubico ($F = \frac{3}{2}Y - \frac{1}{2}Y^3$), quartico di Pohlhausen ($F = 2Y - 2Y^3 + Y^4$, l'unico che soddisfa tutte e cinque le condizioni) e sinusoidale.

prog3_1c_karman.py

```python
"""
prog3_1c_karman.py -- metodo integrale di von Karman-Pohlhausen e serie di Blasius.

1) Profili polinomiali u/U = F(Y), Y = y/delta, che soddisfano le condizioni
   F(0) = 0, F(1) = 1, F'(1) = 0 (e, per il quartico, F''(0) = 0, F''(1) = 0):
   l'equazione integrale di von Karman  d(theta)/dx = tau_w/(rho U^2)  fornisce
   delta(x) in forma chiusa e, con essa, C_f, delta*, theta e H, da confrontare
   con i valori esatti di Blasius.
2) Serie di potenze di Blasius  f = sum_k (-1/2)^k A_k alfa^(k+1) eta^(3k+2)/(3k+2)!
   con alfa = f''(0), A_0 = A_1 = 1 e la ricorrenza
   A_k = sum_{j=0}^{k-1} C(3k-1, 3j) A_j A_{k-1-j}:
   stima del raggio di convergenza e confronto con l'integrazione numerica.
"""
import numpy as np
from math import comb, factorial
import matplotlib.pyplot as plt
from metodi_numerici import rk4_integra
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

ALFA = 0.332057336215          # f''(0) di Blasius (riferimento)


def blasius(eta, z):
    f, p, q = z
    return np.array([p, q, -0.5 * f * q])


# ------------------------------------------------------------- 1) Karman-Pohlhausen
# ciascun profilo: nome, F(Y), F'(0), e gli integrali I1 = int(1-F), I2 = int F(1-F)
profili = {
    "lineare  F = Y":            (lambda Y: Y,                      1.0),
    "cubico   F = 3Y/2 - Y^3/2": (lambda Y: 1.5 * Y - 0.5 * Y**3,   1.5),
    "quartico F = 2Y-2Y^3+Y^4":  (lambda Y: 2 * Y - 2 * Y**3 + Y**4, 2.0),
    "seno     F = sin(pi Y/2)":  (lambda Y: np.sin(np.pi * Y / 2),  np.pi / 2),
}
Y = np.linspace(0, 1, 2001)
print("Metodo integrale di von Karman: delta/sqrt(nu x/U), C_f sqrt(Re_x), delta*, theta, H")
print(f"   {'profilo':28s} {'delta':>7s} {'Cf*sqrt(Re)':>12s} {'delta*':>8s} {'theta':>8s} {'H':>6s}")
risultati = {}
for nome, (F, Fp0) in profili.items():
    Fv = F(Y)
    I1 = np.trapezoid(1 - Fv, Y)            # delta*/delta
    I2 = np.trapezoid(Fv * (1 - Fv), Y)     # theta/delta
    # equazione integrale: U^2 d(I2 delta)/dx = nu U F'(0)/delta  ->  delta^2 = 2 nu x F'(0)/(U I2)
    d = np.sqrt(2 * Fp0 / I2)               # delta / sqrt(nu x / U)
    Cf = 2 * Fp0 / d                        # C_f sqrt(Re_x) = 2 F'(0) sqrt(nu x/U)/delta
    risultati[nome] = (d, Cf, I1 * d, I2 * d, I1 / I2)
    print(f"   {nome:28s} {d:7.3f} {Cf:12.4f} {I1*d:8.4f} {I2*d:8.4f} {I1/I2:6.3f}")
print(f"   {'Blasius (esatto)':28s} {'4.910':>7s} {2*ALFA:12.4f} {1.7208:8.4f} {2*ALFA:8.4f} {1.7208/(2*ALFA):6.3f}")

# ------------------------------------------------------------- 2) serie di Blasius
K = 40
A = [1.0, 1.0]
```

```
for k in range(2, K + 1):
    A.append(sum(comb(3 * k - 1, 3 * j) * A[j] * A[k - 1 - j] for j in range(k)))
A = np.array(A, dtype=float)


def serie(eta, nterm):
    s = 0.0
    for k in range(nterm):
        s += (-0.5)**k * A[k] * ALFA**(k + 1) * eta**(3 * k + 2) / factorial(3 * k + 2)
    return s


# raggio di convergenza: rapporto fra termini successivi (test del rapporto)
k = np.arange(5, K)
c_k = np.array([0.5**kk * A[kk] * ALFA**(kk + 1) / factorial(3 * kk + 2) for kk in range(K + 1)])
rho = (c_k[k] / c_k[k + 1])**(1 / 3)          # |c_k eta^(3k)| ~ costante -> eta ~ (c_k/c_(k+1))^(1/3)
print("\nSerie di Blasius: A_k =", [int(a) for a in A[:6]], "...")
print(f"   raggio di convergenza stimato dal test del rapporto: {rho[-1]:.3f} (k = {k[-1]})")
eta_rk, Z = rk4_integra(blasius, np.array([0, 0, ALFA]), 8.0, 0.01)
for eta_t in (2.0, 4.0, 5.0, 5.5, 6.0):
    i = int(round(eta_t / 0.01))
    err = [abs(serie(eta_t, n) - Z[i, 0]) for n in (10, 20, 40)]
    print(f"   eta = {eta_t:3.1f}: |serie - RK4| con 10, 20, 40 termini = "
          + ", ".join(f"{e:.1e}" for e in err))

# ------------------------------------------------------------------ figure
fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.9))
eta = np.linspace(0, 6, 300)
_, Zb = rk4_integra(blasius, np.array([0, 0, ALFA]), 6.0, 0.02)
ax[0].plot(Zb[:, 1], np.linspace(0, 6, Zb.shape[0]), "k", lw=1.8, label="Blasius")
for (nome, (F, _)), col in zip(profili.items(), [GRIGIO, VERDE, ARANCIO, ROSSO]):
    d = risultati[nome][0]
    ax[0].plot(F(np.minimum(eta / d, 1.0)), eta, "--", color=col, lw=1.1,
    label=nome.split()[0])
ax[0].set_xlabel("$u/U$")
ax[0].set_ylabel("$\\eta$")
ax[0].legend(fontsize=7)
ax[0].set_title("profili di prova (scala $\\delta$ di Karman)")
nomi = list(risultati)
ax[1].bar(range(4), [risultati[n][1] for n in nomi], color=[GRIGIO, VERDE, ARANCIO,
    ROSSO], width=0.6)
ax[1].axhline(2 * ALFA, color="k", ls="--", lw=1.0, label="Blasius $0{,}664$")
ax[1].set_xticks(range(4))
ax[1].set_xticklabels([n.split()[0] for n in nomi], fontsize=7)
ax[1].set_ylabel("$C_f\\sqrt{Re_x}$")
ax[1].set_ylim(0.5, 0.8)
ax[1].legend(fontsize=7)
ax[1].set_title("sforzo alla parete")
for n, col in zip((10, 20, 40), (GRIGIO, ARANCIO, BLU)):
    e = np.array([abs(serie(x, n) - Z[int(round(x / 0.01)), 0]) for x in eta])
    ax[2].semilogy(eta, np.maximum(e, 1e-16), color=col, label=f"{n} termini")
ax[2].axvline(rho[-1], color=ROSSO, ls=":", lw=1.0)
ax[2].text(rho[-1] - 0.1, 1e-8, "raggio di\nconvergenza", ha="right", fontsize=7,
    color=ROSSO)
ax[2].set_xlabel("$\\eta$")
ax[2].set_ylabel("$|f_{serie}-f|$")
```

```
ax[2].set_ylim(1e-16, 1e2)
ax[2].legend(fontsize=7, loc="upper left")
ax[2].set_title("serie di Blasius")
fig.tight_layout()
salva(fig, "prog3_1c_karman")
```

Listato 2.3: `prog3_1c_karman.py`: profili di Pohlhausen con l'equazione integrale di von Kármán e serie di potenze di Blasius.

Risultati del Programma 3.1c: metodo integrale

profilo	$\delta/\sqrt{\nu x/U}$	$C_f\sqrt{Re_x}$	$\delta^*/\sqrt{\nu x/U}$	$\theta/\sqrt{\nu x/U}$	H
lineare	3,464	0,5774	1,7321	0,5774	3,000
cubico	4,641	0,6464	1,7404	0,6464	2,692
quartico	5,836	0,6854	1,7507	0,6854	2,554
seno	4,795	0,6551	1,7425	0,6551	2,660
Blasius	4,910	0,6641	1,7208	0,6641	2,591

Perfino il profilo lineare, che non assomiglia affatto a quello vero, sbaglia lo sforzo alla parete solo del 13%; il cubico e il seno lo danno entro il 3% e lo spessore di spostamento entro l'1%. Lo spessore δ dipende invece fortemente dal profilo scelto, perché è una quantità convenzionale.

Il motivo di tanta robustezza sta nella natura integrale del metodo: la quantità calcolata, θ, è una media attraverso lo strato, poco sensibile ai dettagli del profilo. Con un gradiente di pressione il profilo quartico acquista un parametro di forma $\Lambda = \delta^2 U'/\nu$ e il metodo permette di seguire lo strato limite fino al distacco (metodo di Pohlhausen [14], poi raffinato da Thwaites); i codici di progetto dei profili alari usano ancora oggi metodi integrali di questo tipo, accoppiati al calcolo potenziale della corrente esterna.

2.7.2 La serie di Blasius

Blasius risolse la (2.7) nel 1908 con uno sviluppo in serie di potenze attorno alla parete. È un esercizio istruttivo di analisi, e una verifica indipendente del calcolo numerico.

Sviluppo attorno alla parete e raggio di convergenza

Con $f(0) = f'(0) = 0$ e $f''(0) = \alpha$ si cerca $f = \sum_{k\geq 0} c_k \eta^{3k+2}$: sostituendo nell'equazione e uguagliando le potenze si trova che compaiono solo le potenze η^{3k+2} (una conseguenza dell'invarianza di scala dell'Esercizio 1) e che

$$f(\eta) = \sum_{k=0}^{\infty} \left(-\frac{1}{2}\right)^k \frac{A_k\, \alpha^{k+1}}{(3k+2)!}\, \eta^{3k+2}, \qquad A_0 = A_1 = 1, \quad A_k = \sum_{j=0}^{k-1} \binom{3k-1}{3j} A_j A_{k-1-j}.$$

I primi coefficienti sono $A_k = 1, 1, 11, 375, 27\,897, 3\,817\,137, \ldots$ (il programma li ricava dalla ricorrenza). Il test del rapporto sui termini della serie fornisce il raggio di convergenza: $\eta_{\text{conv}} = 5{,}690$. La serie non converge dunque su tutto lo strato limite: la singolarità più vicina della funzione f, nel piano complesso, dista 5,69 dall'origine. Blasius dovette perciò raccordare la serie con uno sviluppo asintotico valido per η grande, della forma

$f \simeq \eta - \beta + \gamma \int_\eta^\infty \int e^{-(s-\beta)^2/4}$, che è anche la ragione del decadimento gaussiano di $1 - f'$ osservato nel Programma 3.1b.

Risultati del Programma 3.1c: serie

Con 40 termini la serie coincide con l'integrazione numerica entro 10^{-11} fino a $\eta = 4$ e entro 3×10^{-7} a $\eta = 5$; a $\eta = 5{,}5$ l'errore è già 3×10^{-2} e a $\eta = 6$ la serie diverge, tanto più violentemente quanti più termini si sommano (9×10^2 con 40 termini). La Fig. 2.5 (a destra) mostra la "parete" verticale dell'errore in corrispondenza di $\eta = 5{,}69$.

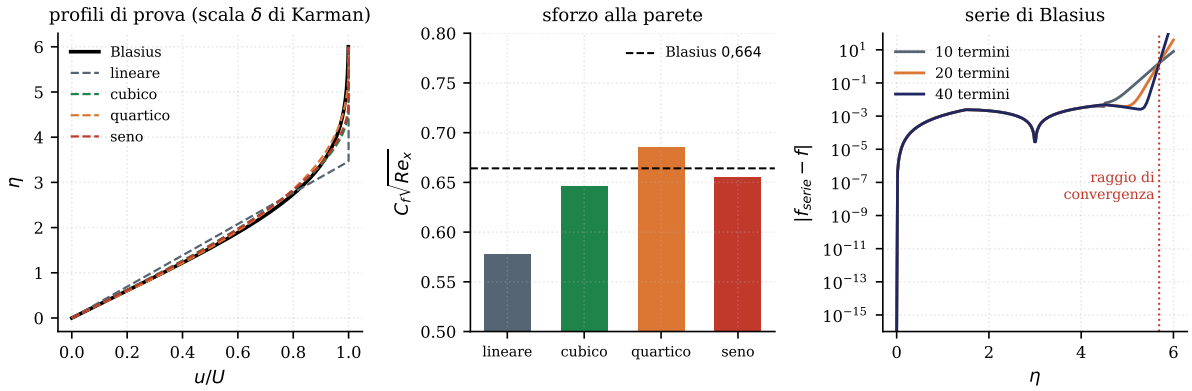


Figura 2.5: A sinistra: profili di prova del metodo integrale confrontati con quello di Blasius. Al centro: sforzo alla parete previsto da ciascun profilo. A destra: errore della serie di Blasius troncata, che diverge oltre il raggio di convergenza 5,69.

2.7.3 La resistenza di una lamina e la scia

L'equazione integrale offre anche un modo per *misurare* la resistenza senza bilance. Integrando $\mathrm{d}\theta/\mathrm{d}x = \tau_w/(\rho U^2)$ da 0 a L, la resistenza per unità di apertura di una faccia della lamina è $D = \rho U^2 \theta(L)$: lo spessore di quantità di moto all'uscita contiene tutta la resistenza accumulata. Poiché θ si conserva anche nella scia a pressione costante, basta rilevare il profilo di velocità nella scia a valle del corpo con un pettine di tubi di Pitot e calcolarne lo spessore di quantità di moto: è il metodo di Jones, usato in galleria del vento per misurare la resistenza dei profili alari con grande precisione.

Resistenza d'attrito dell'ala di un aliante

Per l'ala dell'aliante del Cap. 1 ($c = 1\,\mathrm{m}$, $U = 25\,\mathrm{m\,s^{-1}}$, apertura 15 m, $Re_c = 1{,}7 \times 10^6$) il coefficiente di resistenza d'attrito laminare delle due facce vale $2 \times 1{,}328/\sqrt{Re_c} = 2{,}04 \times 10^{-3}$, cioè $D = 2{,}04 \times 10^{-3} \cdot \frac{1}{2}\rho U^2 \cdot S \simeq 12\,\mathrm{N}$. Se lo strato limite fosse interamente turbolento, con la correlazione $C_D = 0{,}074\, Re_c^{-1/5}$ per faccia si avrebbe $8{,}4 \times 10^{-3}$ e circa 48 N: quattro volte tanto. Qui sta tutto l'interesse dei profili laminari e del controllo dello strato limite dell'Esercizio 2: mantenere laminare la maggior parte possibile della superficie. Su un aliante con $W = 5000\,\mathrm{N}$ ed efficienza 40 la resistenza totale è di circa 125 N: la sola differenza fra attrito laminare e turbolento vale un terzo della resistenza complessiva.

2.8 Oltre la lamina piana

La lamina piana è solo il primo esempio di una famiglia di strati limite di similitudine. Le stesse idee si applicano, con piccole modifiche, a due casi di grande interesse tecnico, sviluppati negli esercizi.

- *Lamina con trasformazione di Töpfer.* L'equazione di Blasius è invariante per il cambio di scala $\eta = kz$, $f = g/k$: si può integrare una volta sola un problema ausiliario con $g''(0) = 1$ e ricavare $f''(0)$ senza iterazioni (Esercizio 1).
- *Aspirazione e soffiamento.* Iniettando o aspirando fluido attraverso una parete porosa con velocità $v_0 \propto x^{-1/2}$ il moto resta autosimile; cambia solo la condizione $f(0)$. L'aspirazione rende il profilo più pieno e stabile, il soffiamento lo allontana dalla parete fino a staccarlo (Esercizio 2).
- *Gradiente di pressione: equazione di Falkner–Skan.* Con una corrente esterna $U_e = U_0 x^m$ (corrente su un cuneo) la similitudine persiste e l'equazione diventa $f''' + ff'' + \beta(1 - f'^2) = 0$, $\beta = 2m/(m+1)$. Il gradiente avverso ($m < 0$) produce profili con flesso e, per $m = -0{,}0904$, il distacco (Esercizio 3).

Capitolo 3

Lo strato limite termico: il termometro a lamina piana

3.1 Il problema fisico

Nello strato limite gli sforzi viscosi compiono lavoro sulle particelle che rallentano, e questo lavoro si trasforma in calore (è la funzione di dissipazione del Cap. 1). Il calore prodotto diffonde per conduzione e, nello stesso tempo, è trasportato a valle dal moto. Se una lamina piana, inizialmente alla temperatura della corrente indisturbata, è isolata termicamente, essa si riscalda finché, a regime, non scambia più calore con il fluido: la sua temperatura T_w si porta a un valore costante, detto **temperatura adiabatica di parete** o di *recupero*, e alla parete vale $(\partial T/\partial y)_{y=0} = 0$.

Viene spontaneo pensare che T_w sia la temperatura di ristagno della corrente, quella che si leggerebbe se il fluido fosse portato a velocità nulla senza perdite. Non è così: nello strato limite il rallentamento avviene con un processo irreversibile, in cui il calore prodotto per attrito viene in parte allontanato per conduzione. Fra la parete e la corrente esterna si forma uno **strato limite termico**, sovrapposto a quello di velocità, di spessore diverso a seconda del fluido. Se se ne conosce la struttura, la lamina diventa un *termometro*: montata su un velivolo, permette di risalire alla temperatura dell'aria T_1 dalla temperatura misurata T_w. È, in sostanza, il principio delle sonde di temperatura totale dei velivoli.

3.2 Le equazioni e la loro riduzione

Ipotesi

Moto stazionario a bassa velocità, proprietà costanti; strato limite termico sottile, con $\partial T/\partial x \ll \partial T/\partial y$ come per la velocità; gradiente di pressione nullo (lamina piana allineata); nella funzione di dissipazione resta soltanto il termine dominante $\mu(\partial u/\partial y)^2$.

Con queste ipotesi le equazioni dello strato limite sono

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0, \qquad u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \nu\frac{\partial^2 u}{\partial y^2}, \tag{3.1}$$

$$\rho c_p \left(u\frac{\partial T}{\partial x} + v\frac{\partial T}{\partial y} \right) = k\frac{\partial^2 T}{\partial y^2} + \mu \left(\frac{\partial u}{\partial y} \right)^2, \tag{3.2}$$

con le condizioni

$$u = v = 0, \quad \frac{\partial T}{\partial y} = 0 \ \text{ per } y = 0; \qquad u \to U, \quad T \to T_1 \ \text{ per } y \to \infty. \tag{3.3}$$

Il campo di velocità è quello di Blasius, già noto dal capitolo precedente; l'equazione dell'energia (3.2) è *lineare* in T e va risolta a velocità assegnata.

L'equazione dell'energia di similitudine

Introduciamo la differenza di temperatura adimensionale

$$\theta(\eta) = \frac{T - T_1}{U^2/(2c_p)}, \tag{3.4}$$

riferita all'aumento di temperatura che il fluido subirebbe se tutta la sua energia cinetica si trasformasse in calore. Con $\eta = y\sqrt{U/(\nu x)}$ e le (2.8):

$$\frac{\partial T}{\partial x} = -\frac{U^2}{2c_p}\frac{\eta}{2x}\theta', \qquad \frac{\partial T}{\partial y} = \frac{U^2}{2c_p}\sqrt{\frac{U}{\nu x}}\,\theta', \qquad \frac{\partial^2 T}{\partial y^2} = \frac{U^2}{2c_p}\frac{U}{\nu x}\theta''.$$

Il primo membro della (3.2) diventa

$$\rho c_p \frac{U^2}{2c_p}\frac{U}{x}\left[-f'\frac{\eta}{2}\theta' + \tfrac{1}{2}(\eta f' - f)\theta'\right] = -\rho\,\frac{U^3}{4x}\,f\theta',$$

mentre al secondo membro $k\,\partial^2 T/\partial y^2 = k\,U^3\theta''/(2c_p\nu x)$ e $\mu(\partial u/\partial y)^2 = \mu U^3 f''^2/(\nu x)$. Moltiplicando tutto per $2c_p\nu x/(kU^3)$ e ricordando che $Pr = \mu c_p/k = \rho\nu c_p/k$:

$$\theta'' + \tfrac{1}{2}\,Pr\,f\,\theta' = -2\,Pr\,(f'')^2, \qquad \theta'(0) = 0, \quad \theta(\infty) = 0. \tag{3.5}$$

Anche il campo di temperatura è autosimile, e dipende da un solo parametro: il **numero di Prandtl**.

Il numero di Prandtl $Pr = \nu/\kappa$, con $\kappa = k/(\rho c_p)$ diffusività termica, è il rapporto fra la diffusività della quantità di moto e quella del calore. Per i gas vale circa 0,7; per l'acqua circa 7 a temperatura ambiente; per i metalli liquidi, ottimi conduttori, è molto minore di uno; per gli oli è dell'ordine delle migliaia.

3.3 Le differenze finite con una condizione sulla derivata

La (3.5) è un'equazione ordinaria lineare del secondo ordine a coefficienti variabili (noti in forma tabulare dalla soluzione di Blasius), con una condizione di *Neumann* alla parete e una di *Dirichlet* all'estremo superiore. La si potrebbe risolvere ancora con il tiro, assegnando $\theta(0)$ e correggendolo; qui la trattiamo invece come un vero problema ai limiti, con il metodo delle differenze finite e il sistema tridiagonale già visti nel modulo precedente, opportunamente modificati per accogliere la condizione sulla derivata.

3.3.1 Il sistema tridiagonale

L'equazione si confronta con la forma generale

$$\theta'' + A(\eta)\,\theta' + B(\eta)\,\theta = D(\eta), \qquad A = \tfrac{1}{2}Prf, \quad B = 0, \quad D = -2Prf''^2. \tag{3.6}$$

Si divide $[0, \eta_{\max}]$ in n intervalli di ampiezza h, con nodi $\eta_1 = 0$, η_2, …, $\eta_{n+1} = \eta_{\max}$, e si sostituiscono le derivate con le differenze centrate $\theta_i' \simeq (\theta_{i+1} - \theta_{i-1})/(2h)$ e $\theta_i'' \simeq (\theta_{i+1} - 2\theta_i + \theta_{i-1})/h^2$, entrambe con errore $\mathcal{O}(h^2)$. Moltiplicando per h^2:

$$C_{i1}\theta_{i-1} + C_{i2}\theta_i + C_{i3}\theta_{i+1} = C_{i4}, \qquad \begin{aligned} C_{i1} &= 1 - \tfrac{1}{2}hA_i = 1 - \tfrac{1}{4}hPrf_i, \quad C_{i2} = -2 + h^2 B_i = -2, \\ C_{i3} &= 1 + \tfrac{1}{2}hA_i = 1 + \tfrac{1}{4}hPrf_i, \quad C_{i4} = h^2 D_i = -2h^2 Prf_i''^2. \end{aligned} \tag{3.7}$$

3.3.2 Le condizioni al contorno

Estremo superiore (Dirichlet). La condizione all'infinito si sposta a $\eta_{\max}$: $\theta_{n+1} = 0$. Nell'equazione scritta nel nodo η_n il termine $C_{n3}\theta_{n+1}$ è noto e passa a secondo membro:

$$C_{n4} \leftarrow C_{n4} - C_{n3}\theta_{n+1}, \qquad C_{n3} \leftarrow 0. \tag{3.8}$$

Parete (Neumann). L'equazione scritta nel nodo di parete η_1 coinvolge θ_0, che appartiene a un *nodo fittizio* a distanza h *dentro* la parete. Il suo valore non è assegnato, ma si ricava dalla condizione sulla derivata scritta con la differenza centrata: $(\theta_2 - \theta_0)/(2h) = 0$, cioè $\theta_0 = \theta_2$. Sostituendo:

$$C_{13} \leftarrow C_{11} + C_{13}, \qquad C_{11} \leftarrow 0. \tag{3.9}$$

Perché il nodo fittizio conserva l'ordine due

Si potrebbe imporre $\theta'(0) = 0$ con la differenza in avanti, $\theta_2 = \theta_1$: è più semplice, ma ha un errore $\mathcal{O}(h)$ che degrada l'intera soluzione al primo ordine. Con il nodo fittizio, invece, sia l'equazione sia la condizione al contorno sono scritte con differenze centrate, e l'errore resta $\mathcal{O}(h^2)$ ovunque. Il prezzo è soltanto quello di scrivere l'equazione differenziale anche nel nodo di parete, cosa lecita perché la soluzione regolare si prolunga per continuità oltre la parete. La tecnica funziona per qualunque condizione del tipo $\theta'(0) = g$: basta porre $\theta_0 = \theta_2 - 2hg$.

Il sistema delle n equazioni nelle incognite $\theta_1, \ldots, \theta_n$ è tridiagonale e si risolve con la subroutine TRID del modulo precedente, riportata nel modulo comune `metodi_numerici.py` (Listato A.1).

3.4 Il Programma 3.2

Il programma calcola prima la soluzione di Blasius integrando la (2.9) con Runge–Kutta a partire da $f''(0) = 0{,}3320572$ (il risultato del Programma 3.1), poi risolve la (3.5) per tre fluidi.

Dati del Programma 3.2

$\eta_{\max} = 10$, $n = 100$ intervalli ($h = 0{,}1$); numeri di Prandtl: acqua 6,75, aria 0,714, mercurio 0,044.


prog3_2_termometro.py

```python
"""
prog3_2_termometro.py -- PROGRAMMA 3.2
Il "termometro a lamina piana": profili di temperatura di similitudine nello
strato limite su una lamina adiabatica, per acqua, aria e mercurio.

Equazione dell'energia ridotta (theta = (T - T1)/(U^2/2cp)):
    theta'' + (Pr/2) f theta' = -2 Pr (f'')^2,   theta'(0) = 0,  theta(eta_max) = 0
discretizzata con differenze centrate su n intervalli; la condizione di
derivata nulla alla parete si impone con un nodo fittizio (theta_0 = theta_2).
Il sistema tridiagonale si risolve con TRID. Le funzioni f e f'' vengono da
un'integrazione di Blasius con f''(0) = 0,3320572 (Programma 3.1).
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import trid, rk4_integra
from stile_grafici import salva, BLU, ARANCIO, VERDE

N = 100
ETAMAX = 10.0
H = ETAMAX / N
PR = {"acqua": 6.75, "aria": 0.714, "mercurio": 0.044}


def blasius(eta, z):
    f, p, q = z
    return np.array([p, q, -0.5 * f * q])


eta, Z = rk4_integra(blasius, np.array([0.0, 0.0, 0.3320572]), ETAMAX, H)
f, q = Z[:, 0], Z[:, 2]


def temperatura(Pr, f, q, H):
    """Risolve l'equazione dell'energia sui nodi eta_1..eta_n (indici 0..n-1);
    theta_(n+1) = 0 e' la condizione 'all'infinito'."""
    n = len(f) - 1
    C = np.zeros((n, 4))
    C[:, 0] = 1.0 - H / 4.0 * Pr * f[:n]       # coefficienti (2.2.11) con A = Pr f/2
    C[:, 1] = -2.0
    C[:, 2] = 1.0 + H / 4.0 * Pr * f[:n]
    C[:, 3] = -2.0 * H**2 * Pr * q[:n] ** 2
    # condizione di derivata alla parete: theta_0 = theta_2
    C[0, 2] = C[0, 0] + C[0, 2]
    C[0, 0] = 0.0
    # condizione all'estremo superiore: theta_(n+1) = 0
    theta_sup = 0.0
    C[n - 1, 3] -= C[n - 1, 2] * theta_sup
    C[n - 1, 2] = 0.0
    theta = np.zeros(n + 1)
    theta[:n] = trid(C)
    theta[n] = theta_sup
    return theta


risultati = {nome: temperatura(Pr, f, q, H) for nome, Pr in PR.items()}

print("     eta    theta(acqua)  theta(aria)  theta(mercurio)")
for i in range(0, N + 1, 5):
    print(f"   {eta[i]:5.2f}   {risultati['acqua'][i]:10.4e}
```

```
    {risultati['aria'][i]:10.4e}"
60          f"  {risultati['mercurio'][i]:10.4e}")
61 print("\nFattore di recupero r = theta(0) e confronto con sqrt(Pr):")
62 for nome, Pr in PR.items():
63     r = risultati[nome][0]
64     print(f"   {nome:9s} Pr = {Pr:6.3f}   r = {r:.5f}   sqrt(Pr) =
    {np.sqrt(Pr):.5f}")
65 print(f"   (Pr = 1: r = {temperatura(1.0, f, q, H)[0]:.5f})")
66
67 # esempio: aria a 100 m/s e T1 = 288 K
68 U, cp = 100.0, 1005.0
69 r = risultati["aria"][0]
70 print(f"\nAria, U = {U} m/s: aumento di temperatura della parete Tw - T1 = r U^2/(2
    cp) = "
71       f"{r*U**2/(2*cp):.2f} K (isentropico: {U**2/(2*cp):.2f} K)")
72
73 fig, ax = plt.subplots(figsize=(4.6, 3.3))
74 for (nome, Pr), col, mk in zip(PR.items(), [BLU, ARANCIO, VERDE], ["d", "x", "o"]):
75     ax.plot(eta, risultati[nome], color=col, label=f"{nome} ($Pr={Pr}$)")
76     ax.plot(eta[::5], risultati[nome][::5], mk, ms=3.5, color=col)
77 ax.set_xlabel(r"$\eta=y\sqrt{U/(\nu x)}$")
78 ax.set_ylabel(r"$\theta=(T-T_1)/(U^2/2c_p)$")
79 ax.set_xlim(0, 10)
80 ax.legend()
81 salva(fig, "prog3_2_termometro")
```

Listato 3.1: Programma 3.2 – profili di temperatura di similitudine su una lamina adiabatica (`prog3_2_termometro.py`).

Risultati del Programma 3.2

fluido	Pr	$r = \theta(0)$	$\sqrt{Pr}$
acqua	6,75	2,487 67	2,598 08
aria	0,714	0,844 28	0,844 99
mercurio	0,044	0,159 00	0,209 76

Con $Pr = 1$ il programma restituisce $\theta(0) = 1{,}0002$. Per l'aria il valore di $\theta(0)$ coincide con $\sqrt{Pr}$ entro lo 0,1%. Lo strato limite termico è tanto più spesso quanto più piccolo è Pr (Fig. 3.1): per il mercurio la temperatura è ancora lontana da zero a $\eta = 10$, e il valore 0,159 è condizionato dalla scelta di $\eta_{\max}$ (si veda il § 3.6).

3.5 Il fattore di recupero

Il valore di θ alla parete ha un significato fisico preciso. In un moto stazionario senza viscosità né conduzione, combinando l'equazione dell'energia con il prodotto scalare della velocità per l'equazione del moto, si trova che l'entalpia totale si conserva lungo una linea di corrente:

$$c_p T + \tfrac{1}{2} V^2 = \text{costante}. \tag{3.10}$$

Una particella che dalla corrente esterna (T_1, U) venisse rallentata fino a fermarsi senza perdite raggiungerebbe la temperatura di ristagno $(T_w)_{\text{isen}} = T_1 + U^2/(2c_p)$: tutta l'energia cinetica verrebbe "recuperata" come calore. Nel fluido reale, per la (3.4),

$$c_p T_w = c_p T_1 + \theta(0)\, \tfrac{1}{2} U^2, \tag{3.11}$$

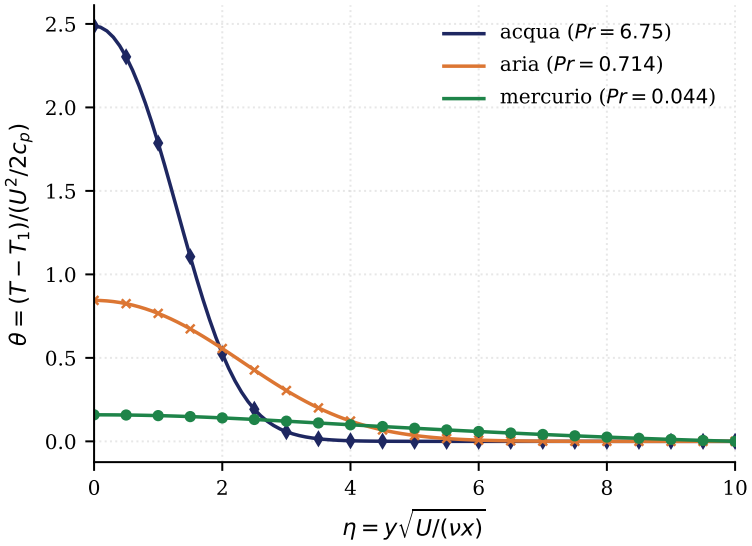


Figura 3.1: Profili di temperatura di similitudine sulla lamina adiabatica per acqua, aria e mercurio.

cioè se ne recupera solo la frazione $r = \theta(0)$, detta **fattore di recupero**.

Il risultato del programma è che $r < 1$ per l'aria e il mercurio ($Pr < 1$) e $r > 1$ per l'acqua ($Pr > 1$). Una spiegazione intuitiva è la seguente. In un fluido con piccolo numero di Prandtl il calore diffonde più facilmente della quantità di moto: da un elemento di fluido vicino alla parete esce per conduzione più calore di quanto vi sia prodotto per attrito, e la sua energia diminuisce. Nel fluido con Pr grande accade il contrario, e l'energia totale vicino alla parete può superare quella della corrente esterna. Per $Pr = 1$ i due effetti si bilanciano esattamente e la temperatura di parete è proprio quella di ristagno, anche se il processo è irreversibile; nell'Esercizio 4 si mostra che in questo caso l'entalpia totale è uniforme in tutto lo strato limite.

Temperatura della pelle di un velivolo

A $100\,\mathrm{m\,s^{-1}}$ in aria ($c_p = 1005\,\mathrm{J\,kg^{-1}\,K^{-1}}$) l'aumento di temperatura isentropico è $U^2/(2c_p) = 4{,}98\,\mathrm{K}$; con $r = 0{,}844$ la parete adiabatica si porta a 4,20 K sopra la temperatura dell'aria. Per un aereo di linea in crociera a $236\,\mathrm{m\,s^{-1}}$ a 11 km di quota ($T_1 = 216{,}7\,\mathrm{K}$) l'aumento isentropico è di 27,7 K e quello della parete di circa 23 K: la pelle si trova a circa $-33\,^\circ\mathrm{C}$ invece che a $-56{,}5\,^\circ\mathrm{C}$. A questi numeri di Mach la comprimibilità non è più trascurabile, ma per strati limite laminari la relazione $r \simeq \sqrt{Pr}$ resta una buona approssimazione; per strati limite turbolenti si usa $r \simeq Pr^{1/3} \simeq 0{,}89$.

3.6 Soluzioni per quadratura e ruolo del numero di Prandtl

L'equazione (3.5) è lineare del primo ordine in θ', e si può integrare esattamente. Il risultato è una formula che fornisce r per quadratura, senza differenze finite e senza dipendere dalla scelta di $\eta_{\max}$: un ottimo strumento di verifica.

Il fattore di recupero per quadratura

Dalla (2.7), $f'''/f'' = -\frac{1}{2}f$, quindi $f''(\eta) = f''(0)\,e^{-G(\eta)}$ con $G(\eta) = \frac{1}{2}\int_0^\eta f\,\mathrm{d}s$. Il fattore integrante della (3.5) è e^{PrG}:

$$\left(e^{PrG}\theta'\right)' = -2Pr\,e^{PrG}f''^2 \quad\Longrightarrow\quad \theta'(\eta) = -2Pr\int_0^\eta e^{-Pr[G(\eta)-G(s)]}f''(s)^2\,\mathrm{d}s,$$

dove si è usato $\theta'(0) = 0$. Poiché $\theta(\infty) = 0$, $r = \theta(0) = -\int_0^\infty \theta'\,\mathrm{d}\eta$:

$$r = 2Pr\,f''(0)^2\int_0^\infty\int_0^\eta e^{-Pr[G(\eta)-G(s)]}\,e^{-2G(s)}\,\mathrm{d}s\,\mathrm{d}\eta. \tag{3.12}$$

Caso $Pr = 1$. L'integrando interno diventa $e^{-G(\eta)}e^{-G(s)}$ e, con $f'' = f''(0)e^{-G}$,

$$r = 2\int_0^\infty f''(\eta)\left[\int_0^\eta f''(s)\,\mathrm{d}s\right]\mathrm{d}\eta = 2\int_0^\infty f''f'\,\mathrm{d}\eta = \left[f'^2\right]_0^\infty = 1.$$

Il risultato $r = 1$ per $Pr = 1$ è dunque *esatto*.

Con la stessa tecnica si risolve il problema complementare della parete *isoterma*, in cui la temperatura di parete T_w è imposta e interessa il flusso di calore. Trascurando la dissipazione (bassa velocità) e posto $\theta_T = (T - T_1)/(T_w - T_1)$, l'equazione è $\theta_T'' + \frac{1}{2}Pr f\theta_T' = 0$ con $\theta_T(0) = 1$, $\theta_T(\infty) = 0$ (problema di Pohlhausen [11]). La soluzione è $\theta_T = 1 - \int_0^\eta e^{-PrG}\,\mathrm{d}s \Big/ \int_0^\infty e^{-PrG}\,\mathrm{d}s$, e il numero di Nusselt locale vale

$$\frac{Nu_x}{\sqrt{Re_x}} = -\theta_T'(0) = \left[\int_0^\infty e^{-PrG(\eta)}\,\mathrm{d}\eta\right]^{-1}. \tag{3.13}$$

Per $Pr = 1$, poiché $\int_0^\infty f''\,\mathrm{d}\eta = 1$, si ottiene $Nu_x/\sqrt{Re_x} = f''(0)$: è l'*analogia di Reynolds*, $Nu_x = \frac{1}{2}C_f Re_x$, fra attrito e scambio termico.

Le leggi asintotiche

$Pr \gg 1$. Lo strato termico è molto più sottile di quello di velocità e vi si può usare l'approssimazione di parete $f \simeq \frac{1}{2}f''(0)\eta^2$, cioè $G \simeq f''(0)\eta^3/12$. Allora

$$\int_0^\infty e^{-Prf''(0)\eta^3/12}\mathrm{d}\eta = \Gamma\left(\tfrac{4}{3}\right)\left(\frac{12}{Prf''(0)}\right)^{1/3}$$

$$\Longrightarrow\quad \frac{Nu_x}{\sqrt{Re_x}} = \frac{[f''(0)/12]^{1/3}}{\Gamma(4/3)}\,Pr^{1/3} = 0{,}3387\,Pr^{1/3}.$$

$Pr \ll 1$. Lo strato termico è molto più spesso di quello di velocità, e il fluido che lo attraversa si muove praticamente con la velocità esterna: $f \simeq \eta$, $G \simeq \eta^2/4$, $\int_0^\infty e^{-Pr\eta^2/4}\mathrm{d}\eta = \sqrt{\pi/Pr}$, quindi $Nu_x/\sqrt{Re_x} \simeq \sqrt{Pr/\pi} = 0{,}564\sqrt{Pr}$.

La correlazione d'uso comune $Nu_x = 0{,}332\,Pr^{1/3}Re_x^{1/2}$ è la prima legge, con il coefficiente aggiustato per essere esatta a $Pr \simeq 1$.

prog3_2b_prandtl.py

```python
"""
prog3_2b_prandtl.py -- influenza del numero di Prandtl sullo strato limite
termico della lamina piana: soluzioni "per quadratura".

Posto G(eta) = (1/2) int_0^eta f, la soluzione di Blasius da' f'' = q0 exp(-G).
1) Lamina adiabatica (termometro): l'equazione theta'' + (Pr/2) f theta' =
   -2 Pr (f'')^2 con theta'(0) = 0 e theta(inf) = 0 si integra esattamente:
       r = theta(0) = 2 Pr q0^2 int_0^inf int_0^eta exp[-Pr (G(eta)-G(s))] exp[-2
    G(s)] ds deta
2) Lamina isoterma (problema di Pohlhausen): theta_T'' + (Pr/2) f theta_T' = 0,
   theta_T(0) = 1, theta_T(inf) = 0, da cui il numero di Nusselt locale
       Nu_x / sqrt(Re_x) = -theta_T'(0) = 1 / int_0^inf exp(-Pr G) deta .
Confronto con le differenze finite del Programma 3.2 e con le leggi
approssimate r ~ sqrt(Pr), Nu_x ~ 0,332 Pr^(1/3) Re_x^(1/2).
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import trid, rk4_integra
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

Q0 = 0.332057336215          # f''(0) (Programma 3.1b)
BETA = 1.7207876575          # f ~ eta - BETA per eta grande
H = 0.005


def blasius(eta, z):
    f, p, q = z
    return np.array([p, q, -0.5 * f * q])


_eta10, _Z = rk4_integra(blasius, np.array([0.0, 0.0, Q0]), 10.0, H)
_f10 = _Z[:, 0]


def griglia(etamax):
    """eta, f e G su [0, etamax]; oltre eta = 10 si usa f = eta - BETA."""
    eta = np.arange(0.0, etamax + H / 2, H)
    f = np.where(eta <= 10.0, np.interp(eta, _eta10, _f10), eta - BETA)
    G = np.concatenate([[0.0], np.cumsum(0.5 * H * (f[1:] + f[:-1]))]) * 0.5
    return eta, f, G


def recupero_quadratura(Pr):
    etamax = max(14.0, 14.0 / np.sqrt(Pr))
    eta, f, G = griglia(etamax)
    # J(eta) = int_0^eta exp[-Pr (G(eta)-G(s))] exp(-2 G(s)) ds, per ricorrenza
    J = np.zeros_like(eta)
    e2 = np.exp(-2.0 * G)
    for k in range(len(eta) - 1):
        fatt = np.exp(-Pr * (G[k + 1] - G[k]))
        J[k + 1] = fatt * J[k] + 0.5 * H * (fatt * e2[k] + e2[k + 1])
    return 2.0 * Pr * Q0**2 * np.trapezoid(J, eta)


def nusselt_quadratura(Pr):
    etamax = max(14.0, 14.0 / np.sqrt(Pr))
    eta, f, G = griglia(etamax)
    return 1.0 / np.trapezoid(np.exp(-Pr * G), eta)


```

```
def recupero_differenze(Pr, etamax, n):
    """Schema del Programma 3.2 su [0, etamax] con n intervalli."""
    eta = np.linspace(0.0, etamax, n + 1)
    h = eta[1]
    eta_f, f_f, G_f = griglia(etamax)
    f = np.interp(eta, eta_f, f_f)
    q = Q0 * np.exp(-np.interp(eta, eta_f, G_f))
    C = np.zeros((n, 4))
    C[:, 0] = 1.0 - h / 4.0 * Pr * f[:n]
    C[:, 1] = -2.0
    C[:, 2] = 1.0 + h / 4.0 * Pr * f[:n]
    C[:, 3] = -2.0 * h**2 * Pr * q[:n] ** 2
    C[0, 2] += C[0, 0]
    C[0, 0] = 0.0
    C[n - 1, 2] = 0.0
    return trid(C)[0]


print("1) Verifica per Pr = 1 (deve essere r = 1 e Nu/sqrt(Re) = f''(0)):")
print(f"   r = {recupero_quadratura(1.0):.6f},  Nu/sqrt(Re) =
    {nusselt_quadratura(1.0):.6f}")

print("\n2) Mercurio (Pr = 0,044): effetto dell'estremo eta_max sulle differenze
    finite (h = 0,1)")
r_merc = recupero_quadratura(0.044)
for emax in [10.0, 20.0, 40.0, 60.0]:
    r = recupero_differenze(0.044, emax, int(round(emax / 0.1)))
    print(f"   eta_max = {emax:5.1f}   r = {r:.5f}   (quadratura: {r_merc:.5f})")

print("\n3) Fattore di recupero e numero di Nusselt al variare di Pr:")
print("      Pr       r(quadr.)   sqrt(Pr)   Nu/sqrt(Re)   0,332 Pr^(1/3)")
Pr_tab = [0.01, 0.044, 0.1, 0.3, 0.714, 1.0, 2.0, 6.75, 10.0, 50.0, 100.0]
for Pr in Pr_tab:
    r = recupero_quadratura(Pr)
    nu = nusselt_quadratura(Pr)
    print(f"   {Pr:7.3f}   {r:9.5f}   {np.sqrt(Pr):8.5f}   {nu:10.5f}
    {0.332*Pr**(1/3):10.5f}")

Pr = np.logspace(-2, 2, 33)
r = np.array([recupero_quadratura(p) for p in Pr])
nu = np.array([nusselt_quadratura(p) for p in Pr])
# esponenti locali alle due estremita'
print(f"\n   pendenza d(ln r)/d(ln Pr):   Pr = 0,01 -> {np.gradient(np.log(r),
    np.log(Pr))[0]:.3f},"
      f"  Pr = 100 -> {np.gradient(np.log(r), np.log(Pr))[-1]:.3f}")
print(f"   pendenza d(ln Nu)/d(ln Pr):  Pr = 0,01 -> {np.gradient(np.log(nu),
    np.log(Pr))[0]:.3f},"
      f"  Pr = 100 -> {np.gradient(np.log(nu), np.log(Pr))[-1]:.3f}")
print(f"   r/Pr^(1/3) a Pr = 100: {r[-1]/100**(1/3):.4f};  Nu/sqrt(Pr Re) a Pr =
    0,01: {nu[0]/np.sqrt(0.01):.4f}"
      f"  (1/sqrt(pi) = {1/np.sqrt(np.pi):.4f})")

fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
ax[0].loglog(Pr, r, color=BLU, label="quadratura esatta")
ax[0].loglog(Pr, np.sqrt(Pr), "--", color=ARANCIO, label=r"$\sqrt{Pr}$")
ax[0].loglog(Pr_tab[1::5], [recupero_differenze(p, max(10, 14/np.sqrt(p)),
    int(max(10, 14/np.sqrt(p))/0.05))
                            for p in Pr_tab[1::5]], "o", ms=4, color=VERDE,
    label="differenze finite")
ax[0].set_xlabel("$Pr$")
```

```
ax[0].set_ylabel(r"fattore di recupero $r=\theta(0)$")
ax[0].legend()
ax[1].loglog(Pr, nu, color=BLU, label="quadratura esatta")
ax[1].loglog(Pr, 0.332 * Pr ** (1 / 3), "--", color=ARANCIO,
    label=r"$0{,}332\,Pr^{1/3}$")
ax[1].loglog(Pr, Pr ** 0.5 / np.sqrt(np.pi), ":", color=ROSSO,
    label=r"$\sqrt{Pr/\pi}$")
ax[1].set_xlabel("$Pr$")
ax[1].set_ylabel(r"$Nu_x/\sqrt{Re_x}$ (parete isoterma)")
ax[1].legend()
fig.tight_layout()
salva(fig, "prog3_2b_prandtl")
```

Listato 3.2: `prog3_2b_prandtl.py`: fattore di recupero e numero di Nusselt in funzione del numero di Prandtl, per quadratura.

Risultati del Programma 3.2b

- Verifica: per $Pr = 1$ la quadratura dà $r = 0{,}999998$ e $Nu_x/\sqrt{Re_x} = 0{,}332058 = f''(0)$.
- Mercurio: il valore esatto è $r = 0{,}20126$. Lo schema del Programma 3.2 dà 0,15900 con $\eta_{\max} = 10$, 0,19998 con 20, 0,20127 con 40: per i piccoli Pr il dominio deve crescere come $Pr^{-1/2}$. Per l'aria il valore esatto è 0,84413, per l'acqua 2,48618 (gli scarti del Programma 3.2, 2×10^{-4} e $1{,}5 \times 10^{-3}$, sono l'errore di discretizzazione con $h = 0{,}1$).
- La tabella seguente riassume la dipendenza da Pr:

Pr	r	$\sqrt{Pr}$	$Nu_x/\sqrt{Re_x}$	$0{,}332\,Pr^{1/3}$
0,01	0,0943	0,100	0,0516	0,0715
0,1	0,3073	0,316	0,1400	0,1541
0,714	0,8441	0,845	0,2948	0,2967
1	1,0000	1,000	0,3321	0,3320
6,75	2,4862	2,598	0,6381	0,6274
100	7,6297	10,000	1,5719	1,5410

- Le pendenze locali delle curve confermano le leggi asintotiche: per $Pr = 100$ il rapporto $Nu_x/(Pr^{1/3}\sqrt{Re_x})$ vale 0,3386, contro il valore teorico 0,3387; il fattore di recupero cresce come $Pr^{1/2}$ per Pr piccolo e tende a crescere come $Pr^{1/3}$ per Pr grande (pendenza 0,39 a $Pr = 100$).

Due lezioni numeriche

La prima: un metodo stabile e consistente può dare un risultato sbagliato del 20% (il mercurio con $\eta_{\max} = 10$) se il dominio di calcolo non contiene la soluzione. Il dominio va dimensionato sulla *scala fisica* del problema, qui lo spessore termico $\propto Pr^{-1/2}$. La seconda: quando esiste una formula per quadratura, anche se poco elegante, conviene sempre implementarla come verifica indipendente del programma alle differenze finite.

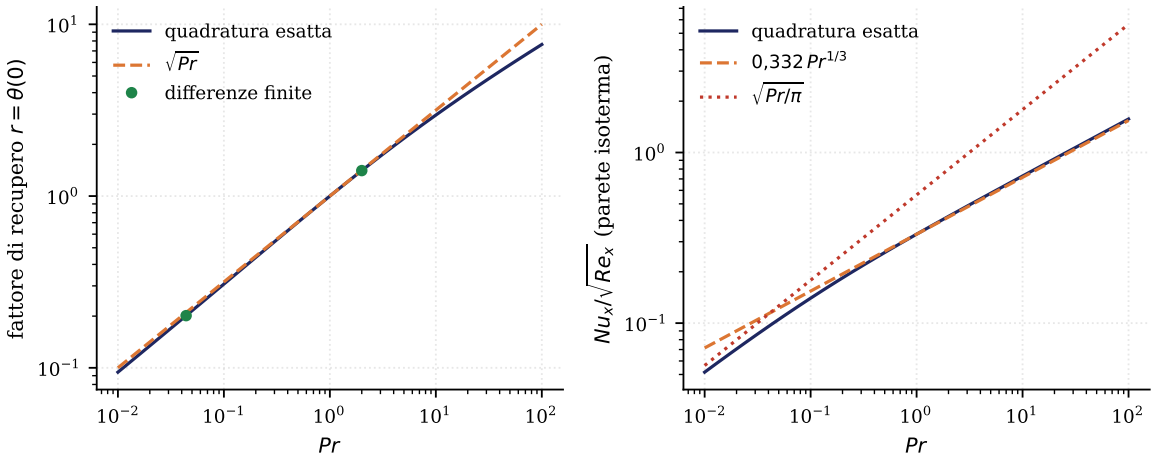


Figura 3.2: Fattore di recupero della lamina adiabatica (a sinistra) e numero di Nusselt della lamina isoterma (a destra) in funzione del numero di Prandtl. La legge $r = \sqrt{Pr}$ è eccellente fra 0,3 e 2.

3.7 Approfondimenti

3.7.1 Parete a temperatura imposta e velocità elevata: la temperatura motrice

Nei due problemi trattati, la lamina adiabatica e la lamina isoterma a bassa velocità, compare uno solo dei due effetti: la dissipazione nel primo, la differenza di temperatura imposta nel secondo. Nel caso generale (una parete a temperatura T_w in una corrente veloce) sono presenti entrambi, e la linearità dell'equazione dell'energia permette di combinarli.

Sovrapposizione e temperatura adiabatica di parete

L'equazione (3.2) è lineare in T, con il termine noto $\mu(\partial u/\partial y)^2$ indipendente da T. Siano $T_{aw}(x,y)$ la soluzione con parete adiabatica (quella del Programma 3.2, con valore di parete $T_{aw} = T_1 + r\,U^2/2c_p$) e $\theta_T(\eta)$ la soluzione *omogenea* di Pohlhausen, con $\theta_T(0) = 1$ e $\theta_T(\infty) = 0$. La funzione

$$T(x,y) = T_{aw}(x,y) + (T_w - T_{aw})\,\theta_T(\eta)$$

soddisfa l'equazione (la parte omogenea non altera il termine noto), tende a T_1 all'infinito e vale T_w alla parete: è la soluzione cercata. Il flusso termico alla parete è allora

$$q_w = -k\left(\frac{\partial T}{\partial y}\right)_w = -k\,(T_w - T_{aw})\,\theta_T'(0)\sqrt{\frac{U}{\nu x}} = h\,(T_w - T_{aw}), \qquad h = \frac{k}{x}\,Nu_x, \quad (3.14)$$

perché il contributo di T_{aw} ha derivata nulla alla parete. La differenza di temperatura che "muove" il calore non è $T_w - T_1$, ma $T_w - T_{aw}$: una parete più fredda della corrente ma più calda della temperatura adiabatica *cede* calore al fluido. Il coefficiente di scambio h è lo stesso del problema a bassa velocità.

Il bilancio termico della pelle di un velivolo

Per l'aereo di linea dell'esempio precedente ($T_1 = 216{,}7\,\mathrm{K}$, $T_{aw} \simeq 240\,\mathrm{K}$) una pelle mantenuta a 230 K dall'interno riceve calore dall'aria ($T_w < T_{aw}$) nonostante sia più

calda dell'aria di 13 K. Con $x = 1\,\mathrm{m}$ e strato limite laminare ($Re_x \simeq 6 \times 10^6$, $Nu_x = 0{,}332\,Pr^{1/3}\sqrt{Re_x} \simeq 730$, $k \simeq 0{,}02\,\mathrm{W\,m^{-1}\,K^{-1}}$) si ha $h \simeq 15\,\mathrm{W\,m^{-2}\,K^{-1}}$ e $q_w \simeq 150\,\mathrm{W\,m^{-2}}$ entranti. Con strato limite turbolento h sarebbe circa dieci volte maggiore. Lo stesso ragionamento, con numeri molto più grandi, governa il riscaldamento aerodinamico dei velivoli supersonici e dei veicoli di rientro, dove T_{aw} supera di migliaia di kelvin la temperatura dell'aria.

3.7.2 L'analogia di Reynolds e la correzione di Colburn

Per $Pr = 1$ si è trovato $Nu_x/\sqrt{Re_x} = f''(0)$, cioè, introducendo il numero di Stanton $St = Nu_x/(Re_x Pr) = h/(\rho c_p U)$,

$$St = \frac{C_f}{2} \qquad (Pr = 1). \tag{3.15}$$

È l'analogia di Reynolds nella sua forma originaria: calore e quantità di moto sono trasportati dallo stesso meccanismo, e il flusso termico adimensionale è uguale all'attrito adimensionale. Per $Pr \neq 1$ la legge asintotica del § 3.6 dà $Nu_x = 0{,}3387\,Pr^{1/3}\sqrt{Re_x}$, quindi

$$St\,Pr^{2/3} = \frac{0{,}3387}{\sqrt{Re_x}} = 1{,}02\,\frac{C_f}{2}\,:$$

il fattore $Pr^{2/3}$ è la *correzione di Colburn*, e il prodotto $St\,Pr^{2/3}$ è il "fattore j" delle correlazioni di scambio termico. La lamina piana laminare è uno dei pochi casi in cui la correzione si può *dimostrare* invece che postulare; l'esperienza mostra che resta valida, approssimativamente, anche per gli strati limite turbolenti e per le correnti nei tubi. La sua utilità pratica è grande: misurare un coefficiente d'attrito è molto più semplice che misurare un coefficiente di scambio termico.

3.7.3 Gli spessori dei due strati limite

Le leggi asintotiche permettono anche di stimare lo spessore dello strato termico rispetto a quello di velocità, cosa utile per dimensionare il dominio di calcolo. Il profilo θ_T decade dove l'esponente $Pr\,G(\eta)$ diventa di ordine uno.

- Per $Pr \gg 1$, con $G \simeq f''(0)\eta^3/12$: $\eta_T \sim [12/(Prf''(0))]^{1/3} = 3{,}3\,Pr^{-1/3}$, cioè $\delta_T/\delta \sim Pr^{-1/3}$. Per l'acqua ($Pr = 7$) lo strato termico è circa la metà di quello di velocità; per un olio con $Pr = 1000$ un decimo. Lo strato termico è tutto contenuto nella regione vicina alla parete dove il profilo di velocità è lineare: è il motivo per cui l'esponente è $1/3$ e non $1/2$.
- Per $Pr \ll 1$, con $G \simeq \eta^2/4$: $\eta_T \sim 2/\sqrt{Pr}$, cioè $\delta_T/\delta \sim Pr^{-1/2}$. Per il mercurio ($Pr = 0{,}025$) si ha $\eta_T \simeq 13$, contro $\eta_{99} = 4{,}9$ dello strato di velocità, e la temperatura "vede" quasi ovunque la velocità esterna U: è il regime dei metalli liquidi, in cui la conduzione prevale sulla convezione.

Il Programma 3.2 con $\eta_{\max} = 10$ conteneva bene lo strato termico dell'acqua e dell'aria, non quello del mercurio: il dominio doveva estendersi a tre o quattro volte η_T, cioè a $\eta \simeq 40$–50: è ciò che si è trovato empiricamente nel Programma 3.2b.

3.7.4 Quando conta la dissipazione: il numero di Eckert

Nel problema generale della parete a temperatura imposta i due effetti, differenza di temperatura e dissipazione, hanno pesi relativi misurati dal **numero di Eckert**

$$Ec = \frac{U^2}{c_p\,(T_w - T_1)},$$

ottenuto confrontando il termine $\mu(\partial u/\partial y)^2 \sim \mu U^2/\delta^2$ con quello convettivo $\rho c_p U \Delta T/x$, tenuto conto che $\delta^2 \sim \nu x/U$. Per una lamina a $30\,\mathrm{m\,s^{-1}}$ con $\Delta T = 10\,\mathrm{K}$ in aria, $Ec = 0{,}09$ e la dissipazione è una correzione del 10%; a $250\,\mathrm{m\,s^{-1}}$ si ha $Ec \simeq 6$ e la dissipazione domina. Nell'acqua (c_p quattro volte più grande, velocità molto minori) Ec è quasi sempre trascurabile: il termometro a lamina piana è un problema aeronautico, non idraulico.

Le condizioni sulla derivata compaiono in molti altri problemi fluidodinamici: sulle superfici libere (sforzo tangenziale nullo), sugli assi o sui piani di simmetria, sulle pareti adiabatiche. L'Esercizio 5 applica la tecnica del nodo fittizio allo strato di fluido attorno a un cilindro rotante, e mette in luce una sottigliezza sulla corretta espressione dello sforzo in coordinate polari.

Capitolo 4

Correnti in condotti e canali a pelo libero

4.1 La corrente in un condotto di sezione qualsiasi

Nel Cap. 1 la geometria parallela ha reso lineari le equazioni di Navier–Stokes e ne ha permesso la soluzione esatta fra due lastre e in un tubo circolare. Lo stesso accade in un condotto rettilineo di sezione *qualsiasi*: la geometria non è più semplice, ma il problema resta lineare.

Consideriamo un condotto indefinito con l'asse parallelo a x e sezione costante. Il moto può essere prodotto da un gradiente di pressione, dalla gravità, dal moto di una parte delle pareti, o da una combinazione di queste cause; la sezione può essere chiusa da pareti rigide, avere una superficie libera, contenere tubi interni (dominio a connessione multipla). Lontano dagli imbocchi la corrente è *sviluppata*: $\partial \mathbf{V}/\partial x = 0$.

L'equazione della corrente sviluppata

Con $\partial/\partial x = 0$ la continuità impone $\partial v/\partial y + \partial w/\partial z = 0$ nella sezione; in assenza di cause che le generino, le componenti trasversali sono nulle e resta la sola componente assiale $u(y,z)$. Il termine convettivo $u\,\partial u/\partial x$ si annulla e la componente x dell'equazione del moto, con una forza di volume f_x per unità di volume, diventa

$$\frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = \frac{1}{\mu}\left(\frac{\mathrm{d}p}{\mathrm{d}x} - f_x\right). \tag{4.1}$$

Come nel Cap. 1, le componenti y e z mostrano che p dipende solo da x, e allora il secondo membro è costante: la (4.1) è un'**equazione di Poisson** con termine noto costante, per la quale valgono tutti i metodi iterativi del modulo precedente.

Le condizioni al contorno sono di due tipi: sulle pareti ferme $u = 0$ (aderenza) e su quelle mobili $u = u_w$; su una superficie libera lo sforzo tangenziale deve essere nullo, $\partial u/\partial n = 0$, perché sopra di essa non c'è nulla che possa equilibrarlo. Se $f_x = 0$ e la sezione è circolare si ritrova la corrente di Hagen–Poiseuille (1.17). Per poche altre sezioni (ellisse, triangolo equilatero, rettangolo con soluzione in serie) esiste una soluzione analitica; in generale la (4.1) va risolta numericamente.

4.2 Il canale inclinato a pelo libero

Come esempio consideriamo un canale aperto di sezione quadrata di lato L, inclinato dell'angolo θ sull'orizzontale (Fig. 4.1). Il moto è prodotto soltanto dalla componente della gravità lungo il canale, $f_x = \rho g \sin\theta$; l'angolo è così piccolo che il pelo libero si può considerare parallelo al fondo, e lungo il canale non c'è gradiente di pressione.

Segnaposto figura

file atteso: `immagini/fig_3_4_1.png`

Canale aperto di sezione quadrata $L \times L$, inclinato dell'angolo θ sull'orizzontale; asse x lungo il canale, y verticale, z trasversale; componente della gravità $\rho g \sin\theta$ lungo il canale e pelo libero parallelo al fondo (Fig. 3.4.1 della fonte).

Figura 4.1: Corrente in un canale a pelo libero.

Con le grandezze adimensionali

$$Y = \frac{y}{L}, \qquad Z = \frac{z}{L}, \qquad U = \frac{u}{L^2 \rho g \sin\theta / \mu} \tag{4.2}$$

la (4.1) diventa

$$\frac{\partial^2 U}{\partial Y^2} + \frac{\partial^2 U}{\partial Z^2} = -1, \tag{4.3}$$

con le condizioni

$$U = 0 \ \text{ per } Y = 0 \text{ e per } Z = 0,\, 1; \qquad \frac{\partial U}{\partial Y} = 0 \ \text{ per } Y = 1. \tag{4.4}$$

4.2.1 Sovrarilassamento con una condizione di derivata

La sezione si ricopre con una griglia quadrata di passo $h = 0{,}05$: $m = 21$ linee verticali (indice i, coordinata Z) e $n = 21$ linee orizzontali (indice j, coordinata Y), con la linea $j = n$ sul pelo libero. La (4.3) discretizzata con la formula a cinque punti e risolta con il **sovrarilassamento successivo** (SOR) dà la formula di aggiornamento

$$U_{i,j} \leftarrow (1-\omega)\, U_{i,j} + \frac{\omega}{4}\left(U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1} + h^2\right), \tag{4.5}$$

applicata successivamente a tutti i nodi interni partendo dall'angolo in basso a sinistra (gli indici di iterazione non servono: si usano sempre i valori più recenti). Il fattore di rilassamento è quello ottimo del modulo precedente,

$$\omega = \frac{8 - 4\sqrt{4 - \alpha^2}}{\alpha^2}, \qquad \alpha = \cos\frac{\pi}{m} + \cos\frac{\pi}{n}. \tag{4.6}$$

Le condizioni (4.4) in forma discreta diventano

$$U_{i,1} = 0, \ \ i = 1, \ldots, m; \qquad U_{1,j} = U_{m,j} = 0, \ \ j = 1, \ldots, n; \tag{4.7}$$

$$U_{i,n+1} = U_{i,n-1}, \ \ i = 2, \ldots, m-1. \tag{4.8}$$

La (4.8) introduce una *riga fittizia* sopra il pelo libero, esattamente come il nodo fittizio del Cap. 3: la derivata $\partial U/\partial Y$ scritta con la differenza centrata nel nodo (i, n) si annulla se i valori sopra e sotto il pelo libero sono uguali. Le (4.7) si assegnano una volta per tutte; la (4.8) va invece aggiornata *durante* ogni iterazione, ogni volta che si sta per applicare la (4.5) a un nodo del pelo libero, perché il valore di $U_{i,n-1}$ è appena cambiato.

4.2.2 La portata

Nota la velocità nei nodi, la portata volumetrica si ottiene come somma dei prodotti fra la velocità di ciascun nodo e l'area di sua competenza (Fig. 4.2): h^2 per i nodi interni, $h^2/2$ per i nodi di contorno, $h^2/4$ per gli spigoli.

Segnaposto figura

file atteso:
`immagini/fig_3_4_2.png`

Griglia sulla sezione del canale (linee continue) e aree di competenza dei nodi delimitate dalle linee tratteggiate che bisecano le maglie: nodo interno (h^2), nodo di contorno ($h^2/2$), spigolo ($h^2/4$) (Fig. 3.4.2 della fonte).

Figura 4.2: Calcolo della portata.

La regola delle aree è la regola dei trapezi

I pesi 1, $\frac{1}{2}$, $\frac{1}{4}$ sono esattamente il prodotto dei pesi della regola dei trapezi monodimensionale nelle due direzioni: $w_i = 1$ all'interno e $\frac{1}{2}$ agli estremi, e $w_i w_j$ vale 1, $\frac{1}{2}$ o $\frac{1}{4}$ a seconda che il nodo sia interno, di lato o di spigolo. Quindi

$$Q \simeq h^2 \sum_i \sum_j w_i w_j U_{i,j} = \int_0^1 \int_0^1 U \, \mathrm{d}Y \, \mathrm{d}Z + \mathcal{O}\left(h^2\right),$$

con lo stesso ordine di accuratezza della soluzione alle differenze finite: non ha senso usare una quadratura più accurata se il campo integrato ha un errore $\mathcal{O}\left(h^2\right)$.

```python
"""
prog3_3_canale.py -- PROGRAMMA 3.3
Corrente laminare in un canale inclinato a pelo libero di sezione quadrata.

Velocita' adimensionale U = u / (L^2 rho g sin(theta) / mu) nella sezione 0 <= Y, Z <= 1:
    U_YY + U_ZZ = -1,   U = 0 su fondo e pareti laterali,  dU/dY = 0 sul pelo libero (Y = 1).
Sovrarilassamento successivo con omega ottimo; la condizione di derivata si
impone con una riga fittizia sopra il pelo libero (U[i, n] = U[i, n-2]).
Portata calcolata pesando i nodi con le aree h^2 (interni), h^2/2 (contorno),
h^2/4 (spigoli), cioe' con la regola dei trapezi bidimensionale.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import omega_ottimo
```

prog3_3_canale.py

```
from stile_grafici import salva, BLU, ROSSO

H = 0.05
M = int(round(1.0 / H)) + 1          # linee verticali (indice i, coordinata Z)
N = int(round(1.0 / H)) + 1          # linee orizzontali (indice j, coordinata Y)
omega = omega_ottimo(M, N)           # come nel programma originale (a = cos(pi/M)
    + cos(pi/N))

U = np.zeros((M, N + 1))             # colonna N = riga fittizia sopra il pelo
    libero
U[1:M - 1, 1:N] = 0.04               # valori di primo tentativo

iterazioni = 0
errore = 1.0
while errore > 0.001:
    iterazioni += 1
    errore = 0.0
    for i in range(1, M - 1):
        for j in range(1, N):        # j = N-1 e' il pelo libero
            vecchio = U[i, j]
            if j == N - 1:
                U[i, j + 1] = U[i, j - 1]          # nodo fittizio: dU/dY = 0
            U[i, j] = (1 - omega) * U[i, j] + omega / 4.0 * (
                H * H + U[i - 1, j] + U[i + 1, j] + U[i, j - 1] + U[i, j + 1])
            errore += abs(U[i, j] - vecchio)
U = U[:, :N]

# portata: regola dei trapezi in due dimensioni
pesi = np.ones((M, N))
pesi[0, :] *= 0.5
pesi[-1, :] *= 0.5
pesi[:, 0] *= 0.5
pesi[:, -1] *= 0.5
portata = np.sum(pesi * U) * H * H

print(f"omega = {omega:.4f};  {iterazioni} iterazioni")
print(f"portata adimensionale Q / (L^4 rho g sin(theta)/mu) = {portata:.6f}")
print(f"velocita' massima (centro del pelo libero) = {U[M // 2, N - 1]:.5f}")
print("\nVelocita' (righe dal pelo libero al fondo, una colonna ogni due):")
for j in range(N - 1, -1, -2):
    print("  " + " ".join(f"{U[i, j]:.4f}" for i in range(0, M, 2)))

# confronto con il canale indefinitamente largo (senza pareti laterali)
print(f"\nCanale senza pareti laterali: U_max = 0,5,  Q = 1/3")
print(f"   rapporti: U_max(largo)/U_max = {0.5 / U[M//2, N-1]:.2f},  Q(largo)/Q =
    {(1/3)/portata:.2f}")

# ------------------------------------------------------------------ figura
Z = np.linspace(0, 1, M)
Y = np.linspace(0, 1, N)
fig, ax = plt.subplots(figsize=(4.0, 3.8))
livelli = [0.0, 0.018, 0.036, 0.054, 0.072, 0.090, 0.108]
cs = ax.contour(Z, Y, U.T, levels=livelli[1:], colors=BLU, linewidths=1.0)
ax.clabel(cs, fmt=lambda v: f"{v:.3f}".replace(".", ","), fontsize=6.5)
cf = ax.contourf(Z, Y, U.T, levels=40, cmap="Blues", alpha=0.35)
ax.plot([0, 0, 1, 1], [1, 0, 0, 1], color="k", lw=2.0)        # pareti e fondo
ax.plot([0, 1], [1, 1], color=ROSSO, lw=1.0, ls="--")         # pelo libero
ax.text(0.5, 1.02, "pelo libero", ha="center", fontsize=7, color=ROSSO)
ax.set_aspect("equal")
ax.set_xlabel("$Z=z/L$")
```

```
ax.set_ylabel("$Y=y/L$")
ax.set_ylim(0, 1.07)
salva(fig, "prog3_3_canale")
```

Listato 4.1: Programma 3.3 – velocità e portata in un canale inclinato a pelo libero (`prog3_3_canale.py`).

Risultati del Programma 3.3

Con $\omega = 1{,}7406$ e partendo da $U = 0{,}04$ nei nodi interni, la somma delle variazioni scende sotto 10^{-3} in **60 iterazioni**. La velocità massima, al centro del pelo libero, e la portata valgono

$$U_{\max} = 0{,}1138 \left(u_{\max} = 0{,}114\,\frac{L^2\rho g\sin\theta}{\mu}\right), \qquad Q = 0{,}056892\,\frac{L^4\rho g\sin\theta}{\mu}.$$

Le isotache (Fig. 4.3) sono simmetriche rispetto alla mezzeria; vicino al pelo libero sono perpendicolari alla superficie, come impone la condizione di sforzo nullo.

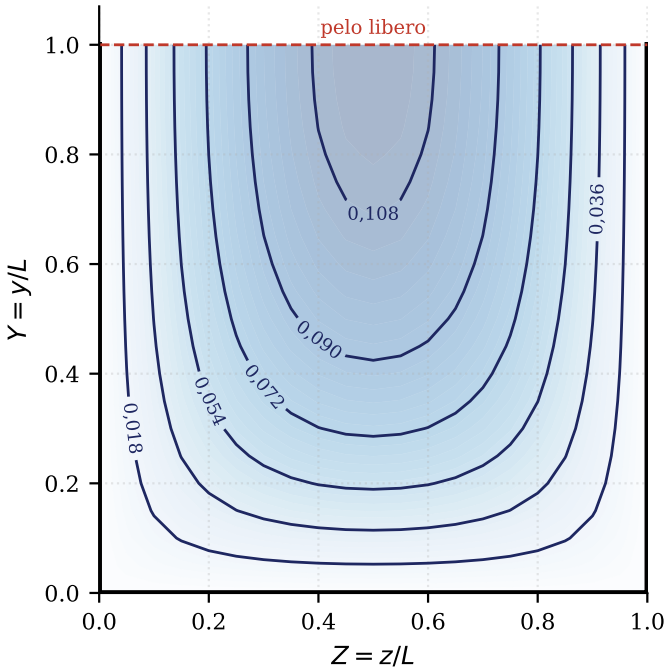


Figura 4.3: Isotache nella sezione del canale a pelo libero, $U = 0{,}018, 0{,}036, \ldots, 0{,}108$.

4.2.3 L'effetto frenante delle pareti laterali

Senza le pareti laterali, cioè in un canale indefinitamente largo con la stessa profondità L, il profilo sarebbe quello di un semicanale di Poiseuille,

$$u = \frac{\rho g\sin\theta}{2\mu}\,y(2L-y), \qquad Q = \frac{\rho g\sin\theta}{3\mu}L^4 \ \text{ per una larghezza } L, \tag{4.9}$$

con velocità massima in superficie $0{,}5\,L^2\rho g\sin\theta/\mu$. La velocità massima è quindi 4,4 volte e la portata 5,9 volte quella del canale quadrato. Le pareti laterali hanno un effetto frenante enorme: in un canale profondo quanto largo, gran parte del fluido si trova vicino a una parete.

4.3 Validazione con la soluzione esatta

Il canale a pelo libero ha una soluzione esatta, che si ottiene con un'osservazione di simmetria.

Il canale aperto come metà di un condotto chiuso

Si consideri un condotto chiuso di larghezza W e altezza $2D$, con pareti ferme. Per simmetria la velocità è pari rispetto al piano di mezzeria $y = D$, dove quindi $\partial u/\partial y = 0$: la metà inferiore del condotto chiuso soddisfa la stessa equazione e le stesse condizioni del canale aperto largo W e profondo D. La soluzione della (4.3) nel rettangolo si trova sviluppando in serie di Fourier in z il termine noto, $1 = \sum_{n \text{ dispari}} \frac{4}{n\pi} \sin\frac{n\pi z}{W}$, e cercando $U = \sum a_n(y) \sin(n\pi z/W)$. Con $k_n = n\pi/W$ ogni coefficiente soddisfa $a_n'' - k_n^2 a_n = -4/(n\pi)$ con $a_n(0) = 0$, $a_n'(D) = 0$:

$$U = \sum_{n \text{ dispari}} \frac{4W^2}{\pi^3 n^3} \sin\frac{n\pi z}{W} \left[1 - \frac{\cosh k_n(y - D)}{\cosh k_n D}\right]. \tag{4.10}$$

Integrando sulla sezione si ottiene la portata. Più comoda per i canali larghi è la serie ottenuta sviluppando invece in y, che conduce alla classica formula per il condotto rettangolare [4, 12]; riferita al canale indefinito della (4.9),

$$\frac{Q}{WD^3/3} = 1 - \frac{384}{\pi^5}\frac{D}{W} \sum_{n \text{ dispari}} \frac{1}{n^5} \tanh\frac{n\pi W}{4D} \xrightarrow{W \gg D} 1 - 1{,}2605\,\frac{D}{W}. \tag{4.11}$$

Il termine correttivo ha un'interpretazione semplice: ciascuna parete laterale "annulla" la portata di una striscia di larghezza $0{,}630\,D$.

Il Programma 3.3b confronta la soluzione alle differenze finite con la (4.10) su griglie sempre più fini. Per non attendere le iterazioni SOR, il sistema lineare viene risolto con un metodo diretto per matrici sparse; il risultato discreto è lo stesso, cambia solo il modo di ottenerlo.

prog3_3b_condotti.py

```
"""
prog3_3b_condotti.py -- validazione del Programma 3.3 con la soluzione esatta
e influenza delle pareti laterali in un canale a pelo libero.

Per simmetria, il canale largo W e profondo D con pelo libero equivale a meta'
di un condotto rettangolare chiuso W x 2D. La soluzione esatta di
U_yy + U_zz = -1 (U = 0 sulle pareti, U_y = 0 per y = D) e' la serie
    U = sum_{n dispari} 4 W^2/(pi^3 n^3) sin(n pi z/W) [1 - cosh(k (y - D))/cosh(k D)],
con k = n pi / W, e la portata
    Q = sum_{n dispari} 4 W^2/(pi^3 n^3) (2 W/(n pi)) [D - tanh(k D)/k].
1) Errore della soluzione alle differenze finite (risolta qui con un metodo
   diretto per matrici sparse) al raffinare della griglia; estrapolazione di
   Richardson.  2) Portata in funzione del rapporto W/D.
"""
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from stile_grafici import salva, BLU, ARANCIO, VERDE, GRIGIO


def serie_U(y, z, W=1.0, D=1.0, nmax=401):
    U = np.zeros(np.broadcast(y, z).shape)
    for n in range(1, nmax + 1, 2):
```

```
        k = n * np.pi / W
        # cosh(k(y-D))/cosh(kD) scritto in forma stabile per k D grande
        rapp = np.exp(-k * y) * (1 + np.exp(-2 * k * (D - y))) / (1 + np.exp(-2 * k * D))
        U += 4 * W**2 / (np.pi**3 * n**3) * np.sin(k * z) * (1 - rapp)
    return U


def serie_Q(W=1.0, D=1.0, nmax=2001):
    Q = 0.0
    for n in range(1, nmax + 1, 2):
        k = n * np.pi / W
        Q += 4 * W**2 / (np.pi**3 * n**3) * (2 * W / (n * np.pi)) * (D - np.tanh(k * D) / k)
    return Q


def differenze_finite(h):
    """Canale quadrato 1 x 1: incognite nei nodi i = 1..M-2 (z), j = 1..N-1 (y);
    la riga j = N-1 e' il pelo libero, con nodo fittizio U[i,N] = U[i,N-2]."""
    n = int(round(1.0 / h))
    nz, ny = n - 1, n                  # incognite per riga e numero di righe
    idx = lambda i, j: (j - 1) * nz + (i - 1)
    A = sp.lil_matrix((nz * ny, nz * ny))
    b = -h * h * np.ones(nz * ny)
    for j in range(1, ny + 1):
        for i in range(1, nz + 1):
            r = idx(i, j)
            A[r, r] = -4.0
            if i > 1:
                A[r, idx(i - 1, j)] = 1.0
            if i < nz:
                A[r, idx(i + 1, j)] = 1.0
            if j > 1:
                A[r, idx(i, j - 1)] += 1.0
            if j < ny:
                A[r, idx(i, j + 1)] = 1.0
            else:                              # pelo libero: il fittizio vale U[i, j-1]
                A[r, idx(i, j - 1)] += 1.0
    u = spla.spsolve(A.tocsr(), b)
    U = np.zeros((n + 1, n + 1))
    U[1:n, 1:n + 1] = u.reshape(ny, nz).T
    pesi = np.ones_like(U)
    pesi[0, :] *= 0.5
    pesi[-1, :] *= 0.5
    pesi[:, 0] *= 0.5
    pesi[:, -1] *= 0.5
    return U[n // 2, n], np.sum(pesi * U) * h * h


Umax_es = serie_U(1.0, 0.5)
Q_es = serie_Q()
print(f"Soluzione esatta (canale quadrato): U_max = {Umax_es:.6f},  Q = {Q_es:.6f}")
print("\n   h        U_max       errore      Q           errore     ordine(Q)")
passi = [1 / 10, 1 / 20, 1 / 40, 1 / 80, 1 / 160]
ris = []
for k, h in enumerate(passi):
    um, Q = differenze_finite(h)
    ris.append((um, Q))
    ordine = "" if k == 0 else f"{np.log2(abs(ris[k-1][1]-Q_es)/abs(Q-Q_es)):6.2f}"
```

```
    print(f"  1/{int(round(1/h)):<4d}  {um:.6f}  {um-Umax_es:+.2e}  {Q:.6f}  {Q-Q_es:+.2e}  {ordine}")
Q_rich = (4 * ris[2][1] - ris[1][1]) / 3
print(f"\nRichardson con h = 1/20 e 1/40: Q = {Q_rich:.6f}  (errore {Q_rich-Q_es:+.1e})")

# influenza del rapporto larghezza/profondita'
WD = np.logspace(-0.6, 1.5, 60)
Qrel = np.array([serie_Q(W=w, D=1.0) / (w / 3.0) for w in WD])   # rispetto al canale indefinito
print("\nPortata per unita' di larghezza rispetto al canale senza pareti (Q/(W D^3/3)):")
for w in [0.5, 1.0, 2.0, 5.0, 10.0, 20.0]:
    print(f"   W/D = {w:5.1f}   {serie_Q(W=w, D=1.0)/(w/3.0):.4f}   "
          f"(stima per W >> D: 1 - 1,2605 D/W = {1 - 1.2605/w:.4f})")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
hh = np.array(passi)
err = np.array([abs(r[1] - Q_es) for r in ris])
err_u = np.array([abs(r[0] - Umax_es) for r in ris])
ax[0].loglog(hh, err, "o-", color=BLU, label="portata $Q$")
ax[0].loglog(hh, err_u, "s-", color=ARANCIO, label=r"$U_{\max}$")
ax[0].loglog(hh, err[0] * (hh / hh[0]) ** 2, ":", color="k", label=r"$\propto h^2$")
ax[0].set_xlabel("passo $h$")
ax[0].set_ylabel("errore rispetto alla serie")
ax[0].legend()
ax[1].semilogx(WD, Qrel, color=VERDE)
ax[1].axhline(1.0, color=GRIGIO, ls=":")
ax[1].axvline(1.0, color=GRIGIO, ls=":")
ax[1].set_xlabel("$W/D$")
ax[1].set_ylabel(r"$Q\,/\,(W D^3/3)$")
ax[1].set_title("effetto delle pareti laterali")
fig.tight_layout()
salva(fig, "prog3_3b_condotti")
```

Listato 4.2: `prog3_3b_condotti.py`: confronto con la soluzione in serie, convergenza e influenza delle pareti laterali.

Risultati del Programma 3.3b

Soluzione esatta del canale quadrato: $U_{\max} = 0{,}113872$, $Q = 0{,}057170$.

h	$U_{\max}$	errore	Q	errore	ordine
1/10	0,113 586	$-2{,}85 \times 10^{-4}$	0,056 131	$-1{,}04 \times 10^{-3}$	–
1/20	0,113 800	$-7{,}15 \times 10^{-5}$	0,056 908	$-2{,}62 \times 10^{-4}$	1,99
1/40	0,113 854	$-1{,}79 \times 10^{-5}$	0,057 105	$-6{,}58 \times 10^{-5}$	2,00
1/80	0,113 867	$-4{,}47 \times 10^{-6}$	0,057 154	$-1{,}65 \times 10^{-5}$	2,00
1/160	0,113 871	$-1{,}12 \times 10^{-6}$	0,057 166	$-4{,}12 \times 10^{-6}$	2,00

L'errore si divide esattamente per quattro a ogni dimezzamento del passo. L'estrapolazione di Richardson con $h = 1/20$ e $1/40$, $Q = (4Q_{1/40} - Q_{1/20})/3 = 0{,}057170$, coincide con il valore esatto entro $2{,}5 \times 10^{-7}$. La portata del Programma 3.3 ($h = 1/20$, 0,056892) differisce da quella del metodo diretto (0,056908) solo per la tolleranza di arresto delle iterazioni.

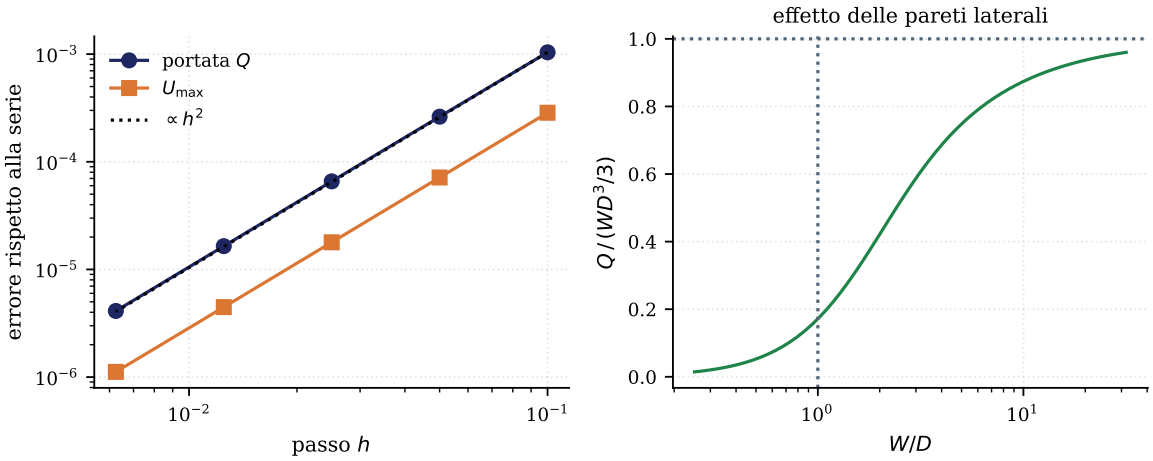


Figura 4.4: A sinistra: errore della soluzione alle differenze finite rispetto alla serie esatta. A destra: portata di un canale largo W e profondo D, riferita a quella del canale senza pareti laterali.

La curva a destra della Fig. 4.4 quantifica l'effetto delle pareti: con $W/D = 1$ la portata è il 17% di quella del canale indefinito, con $W/D = 5$ il 75%, con $W/D = 20$ il 94%, in perfetto accordo con la legge asintotica (4.11) già da $W/D \simeq 5$.

Dove servono questi calcoli

L'equazione (4.1) descrive la corrente laminare in tutti i condotti di sezione non circolare: gli scambiatori di calore compatti a piastre, i canali di raffreddamento delle palette di turbina, i microcanali dei dispositivi di laboratorio, i condotti dei sistemi di lubrificazione. In questi casi interessa soprattutto il prodotto $f \cdot Re_{D_h}$ fra il fattore d'attrito e il numero di Reynolds basato sul diametro idraulico $D_h = 4A/P$, che vale 64 per il tubo circolare, 56,9 per il condotto quadrato, 96 per il canale piano; l'Esercizio 17 lo calcola per altre sezioni. Lo stesso problema matematico (Poisson con termine noto costante) descrive la torsione delle travi prismatiche: la funzione di sforzo di Prandtl soddisfa un'equazione identica, e la rigidezza torsionale è proporzionale alla "portata".

4.4 Approfondimenti

4.4.1 Fattore d'attrito e diametro idraulico

Per confrontare condotti di sezione diversa l'ingegneria usa il fattore d'attrito di Darcy, definito come nel tubo circolare, $-\mathrm{d}p/\mathrm{d}x = f\,\frac{1}{2}\rho\bar{u}^2/D_h$, con la velocità media $\bar{u} = Q/A$ e il diametro idraulico $D_h = 4A/P$ (A area, P perimetro bagnato; per il cerchio $D_h = D$). In regime laminare f è inversamente proporzionale al numero di Reynolds, e il prodotto $f \cdot Re_{D_h}$ dipende solo dalla forma della sezione. Con l'adimensionalizzazione $\nabla^2 U = -1$ di questo capitolo (velocità riferita a $-(\mathrm{d}p/\mathrm{d}x)\ell^2/\mu$, lunghezze a ℓ) si trova subito

$$f \cdot Re_{D_h} = \frac{2\,D_h^2 A}{Q} \qquad \text{(grandezze adimensionali)}, \tag{4.12}$$

dove $Q = \int U\,\mathrm{d}A$ è la "portata" del problema adimensionale. La formula è quella usata nell'Esercizio 17; applicata alla serie (4.11) per il rettangolo chiuso di lati $a \le b$ fornisce la tabella seguente, che riproduce i valori classici [12].

b/a	1	2	4	8	∞
$f \cdot Re_{D_h}$	56,91	62,19	72,93	82,34	96

Il canale piano ($b/a \to \infty$) ha $f \cdot Re = 96$ e il tubo circolare 64: a parità di diametro idraulico e di velocità media il canale piano dissipa il 50% in più, perché il diametro idraulico non "vede" che nel canale il fluido è dappertutto vicino a una parete. Il diametro idraulico è un artificio utile per il regime turbolento, dove f dipende poco dalla forma; in regime laminare la forma conta, e i valori vanno calcolati sezione per sezione, come si è fatto qui.

4.4.2 L'analogia di Prandtl: dalla portata alla rigidezza torsionale

Il problema di Saint-Venant della torsione di una trave prismatica si riduce, con la funzione di sforzo di Prandtl $\phi(y,z)$, a

$$\nabla^2\phi = -2G\vartheta, \qquad \phi = 0 \text{ sul contorno}, \qquad \tau_{xz} = \frac{\partial\phi}{\partial y}, \quad \tau_{xy} = -\frac{\partial\phi}{\partial z}, \qquad M_t = 2\int \phi \,\mathrm{d}A,$$

dove G è il modulo di elasticità tangenziale, ϑ l'angolo di torsione per unità di lunghezza e M_t il momento torcente. È lo stesso problema della corrente sviluppata (4.1) in un condotto chiuso: posto $\phi = 2G\vartheta\,\ell^2 U$, con U la soluzione adimensionale di $\nabla^2 U = -1$, si ha

$$M_t = G\vartheta\, J, \qquad J = 4\ell^4 \int U \,\mathrm{d}A = 4\ell^4\, Q, \qquad \tau_{\max} = 2G\vartheta\,\ell\, \max|\nabla U|. \tag{4.13}$$

La rigidezza torsionale è quattro volte la "portata", e lo sforzo tangenziale massimo è proporzionale alla pendenza massima del profilo di velocità, cioè allo sforzo di parete massimo della corrente. Ogni risultato di questo capitolo si traduce quindi in un risultato sulla torsione.

Torsione di una trave a sezione rettangolare

Dalla serie (4.11) per il rettangolo chiuso $a \times b$ si ottengono i coefficienti della formula dei manuali $J = k_1\, a^3 b$, $\tau_{\max} = k_2\, G\vartheta\, a$ (lato corto a):

b/a	1	2	4	8	∞
k_1	0,1406	0,2287	0,2808	0,3071	0,3333

e $k_2 = 2\max|\nabla U| = 0{,}675$ per il quadrato (lo sforzo massimo è a metà dei lati, dove la velocità della corrente equivalente cresce più ripidamente; è nullo agli spigoli, dove il fluido è fermo). Per la sezione circolare, $U = (1 - r^2)/4$ dà $J = 4 \cdot \pi/8 = \pi/2$ in unità di R^4, il momento d'inerzia polare, come deve. L'analogia dice anche perché una sezione sottile e aperta è così poco rigida a torsione: per $b/a \to \infty$, $J \to ab^3/3$ cresce con il *cubo* dello spessore, esattamente come la portata del canale piano con la profondità. Per le sezioni sottili chiuse (il cassone alare) vale invece la formula di Bredt, che non ha un analogo nella corrente sviluppata, perché la circuitazione dello sforzo attorno alla cavità non è vincolata a zero.

4.4.3 Quante iterazioni servono: la convergenza del sovrarilassamento

Nel modulo precedente si è ricavato il fattore ottimo (4.6) e il fatto che con esso l'errore si riduce a ogni iterazione del fattore $\rho = \omega - 1$ (raggio spettrale della matrice di iterazione), contro $\rho_J^2 \simeq 1 - \pi^2/n^2$ del metodo di Gauss–Seidel. Per il Programma 3.3 ($m = n = 21$)

si ha $\omega = 1{,}7406$ e $\rho = 0{,}741$: per ridurre l'errore di un fattore 10^{-3} dovrebbero bastare $\ln 10^{-3} / \ln 0{,}741 \simeq 23$ iterazioni, ma il programma ne impiega 60. La discrepanza è istruttiva.

La condizione di pelo libero cambia lo spettro

La formula (4.6) vale per un rettangolo con U assegnato su tutto il contorno. Il canale a pelo libero è però, per la simmetria usata nella validazione, la *metà* di un condotto chiuso alto $2L$: la matrice di iterazione è quella di un rettangolo con $m = 21$ linee verticali e $2n - 1 = 41$ linee orizzontali, e il fattore ottimo è $\omega = 1{,}788$ ($\alpha = \cos\frac{\pi}{21} + \cos\frac{\pi}{41}$), con $\rho = 0{,}788$. Con $\omega = 1{,}7406$, minore dell'ottimo, la teoria di Young prevede $\rho = \left[\frac{1}{2}\left(\omega\rho_J + \sqrt{\omega^2\rho_J^2 - 4(\omega-1)}\right)\right]^2 = 0{,}889$, con $\rho_J = \alpha/2$; il raggio spettrale misurato dalla storia della convergenza è 0,876: $\ln 10^{-3} / \ln 0{,}876 \simeq 52$ iterazioni, più le prime iterazioni di assestamento, cioè le 60 osservate. Ripetendo il calcolo con $\omega = 1{,}788$ la somma delle variazioni scende sotto 10^{-3} in 43 iterazioni, con rapporto asintotico misurato 0,789; con $\omega = 1$ (Gauss–Seidel) ne servono 339.

La lezione generale è che il fattore ottimo dipende dalle condizioni al contorno e dalla forma del dominio, non solo dal numero di nodi, e che sbagliarlo per difetto costa più che sbagliarlo per eccesso (con $\omega = 1{,}82$ servono 47 iterazioni). Per domini irregolari non esiste una formula chiusa; si stima ρ_J dalle prime iterazioni di Jacobi, oppure si ricorre ai metodi diretti per matrici sparse, come nel Programma 3.3b, o ai metodi multigriglia.

4.4.4 La lunghezza d'imbocco

Tutto il capitolo riguarda la corrente *sviluppata*. All'imbocco di un condotto il profilo è uniforme e si deforma progressivamente, con uno strato limite che cresce dalle pareti fino a riempire la sezione: solo da lì in poi vale la (4.1). La lunghezza d'imbocco L_e si stima dalla crescita dello strato limite del Cap. 2: lo strato raggiunge il centro quando $\delta \simeq D/2$, cioè, con $\delta \simeq 5\sqrt{\nu x/\bar{u}}$, per $x \simeq D^2\bar{u}/(100\nu)$. Il calcolo completo e gli esperimenti danno, per il tubo circolare,

$$\frac{L_e}{D} \simeq 0{,}06\, Re_D,$$

in accordo con la stima a meno della costante. Per acqua in un tubo di 1 cm a $Re_D = 1000$ ($\bar{u} = 0{,}1\,\mathrm{m\,s^{-1}}$) la lunghezza d'imbocco vale 60 cm; a $Re_D = 2000$, al limite del regime laminare, 1,2 m. Nei microcanali e negli scambiatori compatti, con Re_D di poche decine, l'imbocco è di pochi diametri e la corrente sviluppata descrive quasi tutto il condotto; in regime turbolento L_e/D vale circa 10–40 quasi indipendentemente da Re. La caduta di pressione nel tratto d'imbocco è maggiore di quella della corrente sviluppata (lo sforzo alla parete è più alto dove lo strato limite è sottile, e il fluido al centro viene accelerato): nei condotti corti va aggiunta come perdita concentrata.

Gli Esercizi 6 e 7 affrontano la variante in cui il moto è prodotto non dalla gravità ma da una parete mobile, interamente o parzialmente: l'equazione diventa quella di Laplace e le condizioni al contorno si complicano.

Capitolo 5

Equazioni paraboliche I: metodi espliciti e il problema di Rayleigh

5.1 La lastra messa bruscamente in moto

Studiando come si forma lo strato limite su un corpo che si mette in moto, Rayleigh considerò nel 1911 il problema più semplice possibile: un fluido indefinito, inizialmente in quiete, sopra una lastra piana infinita che all'istante $t = 0$ acquista bruscamente una velocità U nel proprio piano. Se la lastra è normale all'asse y e si muove lungo x, la velocità ha la sola componente $u(y,t)$; la continuità è soddisfatta, il termine convettivo si annulla e l'equazione (1.11) si riduce a

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}. \tag{5.1}$$

In termini di vorticità $\zeta = -\partial u/\partial y$, derivando rispetto a y, si ottiene la stessa equazione:

$$\frac{\partial \zeta}{\partial t} = \nu \frac{\partial^2 \zeta}{\partial y^2}. \tag{5.2}$$

All'istante iniziale tutta la vorticità è concentrata in un foglio di vortici aderente alla lastra; poi diffonde nel fluido come il calore in una barra, con coefficiente di diffusione ν. Secondo la classificazione del modulo precedente le (5.1)–(5.2) sono equazioni **paraboliche**: hanno una sola famiglia di caratteristiche, le rette $t =$ costante, e una perturbazione introdotta in un punto si avverte istantaneamente ovunque, pur con un'ampiezza che decresce rapidissimamente con la distanza.

La soluzione di Rayleigh

Per la lastra singola con $u(y,0) = 0$, $u(0,t) = U$, $u(\infty,t) = 0$ non esiste una lunghezza caratteristica: l'unica combinazione adimensionale di y, t e ν è $\eta = y/(2\sqrt{\nu t})$. Cercando $u = U\,F(\eta)$, con $\partial\eta/\partial t = -\eta/(2t)$ e $\partial\eta/\partial y = 1/(2\sqrt{\nu t})$, la (5.1) diventa $F'' + 2\eta F' = 0$, da cui $F' = C\,e^{-\eta^2}$ e, con $F(0) = 1$, $F(\infty) = 0$,

$$\frac{u}{U} = \operatorname{erfc}\frac{y}{2\sqrt{\nu t}}, \qquad \operatorname{erfc} z = \frac{2}{\sqrt{\pi}} \int_z^\infty e^{-s^2}\,\mathrm{d}s. \tag{5.3}$$

La velocità scende all'1% di U per $\eta = 1{,}82$: lo strato di fluido trascinato ha spessore $\delta \simeq 3{,}6\sqrt{\nu t}$. È la stessa legge dello strato limite stazionario, $\delta \propto \sqrt{\nu x/U}$, se si interpreta x/U come il tempo trascorso dal passaggio del bordo d'attacco.

Le (5.1)–(5.2) valgono per qualunque moto piano non stazionario fra due lastre parallele indefinite che si muovano arbitrariamente nel proprio piano, anche se una delle due è sostituita da una superficie libera; con le opportune modifiche, valgono anche fra cilindri coassiali. Calcolare questi moti si chiama **problema di Rayleigh generalizzato**.

5.2 Lo schema esplicito

In un calcolo numerico il dominio deve essere finito. Supponiamo che il fluido sopra la lastra $y = 0$ sia limitato a una profondità divisa in $m - 1$ intervalli di ampiezza h, e che l'asse dei tempi sia diviso in passi di ampiezza τ: si forma la griglia della Fig. 5.1. Nel nodo (i, j), cioè $y_i = (i-1)h$, $t_j = (j-1)\tau$, si sostituisce la derivata seconda spaziale con la differenza centrata e la derivata temporale con la differenza in avanti:

$$\frac{u_{i,j+1} - u_{i,j}}{\tau} = \nu \, \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2},$$

cioè

$$u_{i,j+1} = u_{i,j} + R\left(u_{i-1,j} - 2u_{i,j} + u_{i+1,j}\right), \qquad R = \frac{\nu\tau}{h^2}. \tag{5.4}$$

Segnaposto figura

file atteso: `immagini/fig_3_5_1.png`

Griglia nel piano y–t con passi h e τ: il valore nel nodo $(i, j+1)$ si calcola dai tre nodi $(i-1, j)$, (i, j), $(i+1, j)$ del livello precedente; le linee tratteggiate delimitano il dominio di dipendenza numerico (Fig. 3.5.1 della fonte).

Figura 5.1: Il metodo esplicito per le equazioni paraboliche.

La (5.4) fornisce la soluzione al tempo t_{j+1} a una certa quota a partire dai valori del tempo t_j nella stessa quota e nelle due vicine: note le condizioni al contorno in funzione del tempo, si calcola il livello t_2 dalla condizione iniziale, poi t_3, e così via, *esplicitamente*, senza risolvere sistemi di equazioni. Per questo si parla di **metodo esplicito** (in inglese FTCS, *forward time, centered space*).

5.3 Stabilità

Come il numero di Courant per le equazioni iperboliche, il parametro R non si può scegliere liberamente.

Analisi di von Neumann dello schema esplicito

Seguendo il modulo precedente, si studia l'evoluzione di una componente di Fourier dell'errore, $u_{i,j} = U_j \, e^{\mathrm{i}kih}$, dove k è il numero d'onda e $\mathrm{i} = \sqrt{-1}$. Sostituendo nella (5.4) e

dividendo per $e^{\mathrm{i}kih}$:

$$U_{j+1} = \left[1 + R\left(e^{-\mathrm{i}kh} - 2 + e^{\mathrm{i}kh}\right)\right] U_j = \left[1 - 2R(1 - \cos kh)\right] U_j = \left[1 - 4R\sin^2\tfrac{kh}{2}\right] U_j. \tag{5.5}$$

La quantità fra parentesi è il fattore di amplificazione G. Se $|G| > 1$ per qualche k, l'ampiezza di quella componente cresce senza limite al crescere di j: lo schema è *instabile*. Poiché $G \leq 1$ sempre, la condizione $|G| \leq 1$ si riduce a $G \geq -1$, cioè $R \leq 1/(1 - \cos kh)$ per ogni k. Il secondo membro è minimo, e vale $1/2$, per $\cos kh = -1$ (l'onda più corta rappresentabile, di lunghezza $2h$). Quindi

$$R = \frac{\nu\tau}{h^2} \leq \frac{1}{2}. \tag{5.6}$$

Lo stesso risultato si ottiene con un argomento diverso, più fisico. Se $R \leq \frac{1}{2}$ la (5.4), scritta come $u_{i,j+1} = R\,u_{i-1,j} + (1 - 2R)\,u_{i,j} + R\,u_{i+1,j}$, esprime il nuovo valore come *media pesata con pesi non negativi* dei tre valori precedenti: il nuovo valore non può superare il massimo né scendere sotto il minimo dei vecchi (*principio del massimo discreto*), e nessuna oscillazione può crescere. Se $R > \frac{1}{2}$ il peso del valore centrale diventa negativo, e un picco produce al passo successivo un avvallamento più profondo: nasce l'oscillazione a dente di sega che si amplifica di un fattore $|1 - 4R|$ a ogni passo (Esercizio 8).

Con il valore limite $R = \frac{1}{2}$ la (5.4) assume la forma particolarmente semplice

$$u_{i,j+1} = \tfrac{1}{2}\left(u_{i-1,j} + u_{i+1,j}\right), \tag{5.7}$$

detta *formula di Bender–Schmidt*: il nuovo valore è la media dei due vicini al tempo precedente. È comoda per il calcolo a mano, ma come si vedrà non è la scelta più accurata.

5.4 La corrente di Couette che si avvia: il Programma 3.4

Consideriamo acqua compresa fra due lastre inizialmente ferme, distanti 1 m. All'istante $t = 0$ la lastra superiore acquista bruscamente la velocità $u_0 = 1\,\mathrm{m\,s^{-1}}$, mentre quella inferiore resta ferma. Il brusco moto della lastra crea un forte gradiente di velocità, un sottile foglio di vortici, che diffonde verso il basso in una regione praticamente priva di vorticità, ridistribuendo la velocità.

Dati del Programma 3.4

$\nu = 1 \times 10^{-6}\,\mathrm{m^2\,s^{-1}}$ (acqua); $m = 21$ nodi, $h = 0{,}05\,\mathrm{m}$; $R = 1/4$, quindi $\tau = Rh^2/\nu = 625\,\mathrm{s}$; condizione iniziale $u_{i,1} = 0$ per $i < m$, $u_{m,1} = u_0$; condizioni al contorno $u_{1,j} = 0$, $u_{m,j} = u_0$ per $j > 1$; stampa ogni 20 passi (12 500 s), grafico ogni 40 passi, fino a $t = 2{,}5 \times 10^5\,\mathrm{s}$.

Il passo temporale di più di dieci minuti può sembrare enorme, ma è adeguato a una corrente laminare in cui la vorticità diffonde soltanto per effetto dell'agitazione molecolare, con un coefficiente piccolissimo. Nel programma la soluzione non viene conservata come matrice $u_{i,j}$: basta un vettore, sovrascritto a ogni passo. Con le operazioni vettoriali di NumPy l'intero livello $j + 1$ si calcola con una sola istruzione, che usa *solo* valori del livello j (il secondo membro viene valutato per intero prima dell'assegnazione).

La soluzione esatta per confronto

Con $u = u_0\, y/d + w$ la parte w soddisfa la (5.1) con condizioni omogenee, $w(0,t) = w(d,t) = 0$, e $w(y,0) = -u_0 y/d$ (per $y < d$). Separando le variabili, $w = \sum_n b_n \sin(n\pi y/d)\, e^{-n^2\pi^2\nu t/d^2}$, e i coefficienti si ricavano dallo sviluppo di $-y/d$ in serie di seni: $b_n = 2u_0(-1)^n/(n\pi)$. Quindi

$$\frac{u}{u_0} = \frac{y}{d} + \sum_{n=1}^{\infty} \frac{2(-1)^n}{n\pi} \sin\frac{n\pi y}{d}\, \exp\left(-\frac{n^2\pi^2\nu t}{d^2}\right). \tag{5.8}$$

Per tempi lunghi sopravvive solo il primo termine, che decade con la costante di tempo $d^2/(\pi^2\nu)$; per tempi brevi, quando la vorticità non ha ancora raggiunto la lastra ferma, vale la soluzione di Rayleigh (5.3) con y sostituito da $d - y$.

prog3_4_rayleigh.py

```
"""
prog3_4_rayleigh.py -- PROGRAMMA 3.4
Problema di Rayleigh generalizzato: acqua fra due lastre distanti 1 m, la
superiore messa bruscamente in moto (u0 = 1 m/s) all'istante t = 0.
    u_t = nu u_yy,   u(0, t) = 0,   u(1, t) = u0,   u(y, 0) = 0 per y < 1.
Metodo esplicito (avanti nel tempo, centrato nello spazio):
    u_i^(j+1) = u_i^j + R (u_(i-1)^j - 2 u_i^j + u_(i+1)^j),   R = nu tau / h^2 <=
    1/2.
Confronto con la soluzione esatta in serie di Fourier
    u/u0 = y/d + sum_n 2 (-1)^n/(n pi) sin(n pi y/d) exp(-n^2 pi^2 nu t/d^2)
e, per tempi brevi, con la soluzione di Rayleigh per la lastra singola
    u/u0 = erfc[(d - y) / (2 sqrt(nu t))].
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erfc
from stile_grafici import salva, PALETTE, GRIGIO

M = 21
D = 1.0
H = D / (M - 1)
NU = 1.0e-6
R = 0.25
TAU = R * H**2 / NU
NPRINT, NPLOT = 20, 40
TMAX = 2.5e5
U0 = 1.0

y = np.linspace(0.0, D, M)


def esatta(y, t, nmax=200):
    u = y / D
    for n in range(1, nmax + 1):
        u = u + 2 * (-1) ** n / (n * np.pi) * np.sin(n * np.pi * y / D) *
    np.exp(-(n * np.pi) ** 2 * NU * t / D**2)
    return U0 * u


u = np.zeros(M)
u[-1] = U0
t, j = 0.0, 0
profili = []
print(f"tau = R h^2 / nu = {TAU:.0f} s")
```

```
print("   tempo (s)                velocita' (m/s) a y = 0; 0,1; ...; 1")
print(f"  {t:9.3e}  " + " ".join(f"{v:.4f}" for v in u[::2]))
while t < TMAX - 1e-9:
    j += 1
    t += TAU
    u[1:-1] = u[1:-1] + R * (u[:-2] - 2 * u[1:-1] + u[2:])     # tutto il vettore
    al livello j
    if j % NPRINT == 0:
        print(f"  {t:9.3e}  " + " ".join(f"{v:.4f}" for v in u[::2]))
    if j % NPLOT == 0 and len(profili) < 5:
        profili.append((t, u.copy()))

print("\nErrore massimo rispetto alla serie esatta:")
for tp, up in profili:
    print(f"   t = {tp:8.0f} s   max|u - u_es| = {np.max(np.abs(up - esatta(y,
    tp))):.2e} m/s")
print(f"   t = {t:8.0f} s   max|u - u_es| = {np.max(np.abs(u - esatta(y, t))):.2e}
    m/s;"
      f"  scarto dal profilo di Couette = {np.max(np.abs(u - y)):.3f} m/s")
tc = D**2 / (np.pi**2 * NU)
print(f"\nTempo caratteristico d^2/(pi^2 nu) = {tc:.3e} s = {tc/86400:.2f} giorni")
for frac in [0.5, 0.1, 0.01]:
    print(f"   lo scarto dal profilo lineare scende sotto {frac*100:.0f}% di u0
    dopo circa "
          f"{tc*np.log(2/np.pi/frac):.3e} s")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.3))
yy = np.linspace(0, D, 201)
simboli = ["+", "o", "s", "d", "v"]
for k, (tp, up) in enumerate(profili):
    col = PALETTE[k]
    ax[0].plot(esatta(yy, tp), yy, color=col, lw=1.0)
    ax[0].plot(up, y, simboli[k], ms=3.5, color=col, mfc="none", label=f"$t$ =
    {tp:,.0f} s".replace(",", "."))
ax[0].plot(yy, yy, ":", color=GRIGIO, label="Couette (regime)")
ax[0].set_xlabel("velocita' (m/s)")
ax[0].set_ylabel("quota (m)")
ax[0].set_xlim(0, 1)
ax[0].set_ylim(0, 1)
ax[0].legend(fontsize=6.5, loc="upper left")
ax[0].set_title("simboli: esplicito; linee: serie esatta")
# tempi brevi: confronto con Rayleigh (lastra singola)
for k, tp in enumerate([625.0 * 4, 625.0 * 16, 625.0 * 40]):
    col = PALETTE[k]
    nst = int(round(tp / TAU))
    uu = np.zeros(M)
    uu[-1] = U0
    for _ in range(nst):
        uu[1:-1] = uu[1:-1] + R * (uu[:-2] - 2 * uu[1:-1] + uu[2:])
    ax[1].plot(erfc((D - yy) / (2 * np.sqrt(NU * tp))), yy, color=col, lw=1.0)
    ax[1].plot(uu, y, "o", ms=3, mfc="none", color=col, label=f"$t$ = {tp:,.0f}
    s".replace(",", "."))
ax[1].set_xlabel("velocita' (m/s)")
ax[1].set_ylim(0.5, 1.0)
ax[1].set_xlim(0, 1)
ax[1].legend(fontsize=6.5, loc="lower right")
ax[1].set_title("tempi brevi: linee = erfc (Rayleigh)")
fig.tight_layout()
salva(fig, "prog3_4_rayleigh")
```

Listato 5.1: Programma 3.4 – problema di Rayleigh generalizzato con il metodo esplicito (`prog3_4_rayleigh.py`).

Risultati del Programma 3.4

Un estratto della tabella (velocità in $\mathrm{m\,s^{-1}}$ alle quote $y = 0;\ 0{,}1;\ \ldots;\ 1$ m):

t (s)	$y = 0{,}5$	0,6	0,7	0,8	0,9	1,0
$1{,}25 \times 10^4$	0,0015	0,0115	0,0596	0,2110	0,5327	1,0000
$5{,}00 \times 10^4$	0,1147	0,2072	0,3443	0,5285	0,7527	1,0000
$1{,}25 \times 10^5$	0,3152	0,4230	0,5484	0,6893	0,8416	1,0000
$2{,}50 \times 10^5$	0,4463	0,5489	0,6565	0,7684	0,8834	1,0000

- L'errore massimo rispetto alla serie (5.8) vale $3{,}1 \times 10^{-3}$ m/s a 25 000 s e scende a $2{,}5 \times 10^{-4}$ m/s a 250 000 s: è più grande all'inizio, quando il gradiente di velocità vicino alla lastra mobile è ripido e la griglia lo risolve male.
- Una discontinuità di velocità non può esistere in un fluido viscoso: viene immediatamente smussata dalla diffusione (Fig. 5.2). Per tempi brevi i profili coincidono con la soluzione erfc di Rayleigh.
- Il profilo tende a quello lineare di Couette, ma molto lentamente: la costante di tempo $d^2/(\pi^2\nu)$ vale $1{,}01 \times 10^5$ s, più di un giorno. A 250 000 s lo scarto massimo dal profilo lineare è ancora di $0{,}054\ \mathrm{m\,s^{-1}}$, esattamente quanto prevede il primo termine della serie, $(2/\pi)\, e^{-t\pi^2\nu/d^2}$; scende sotto l'1% solo dopo circa $4{,}2 \times 10^5$ s.

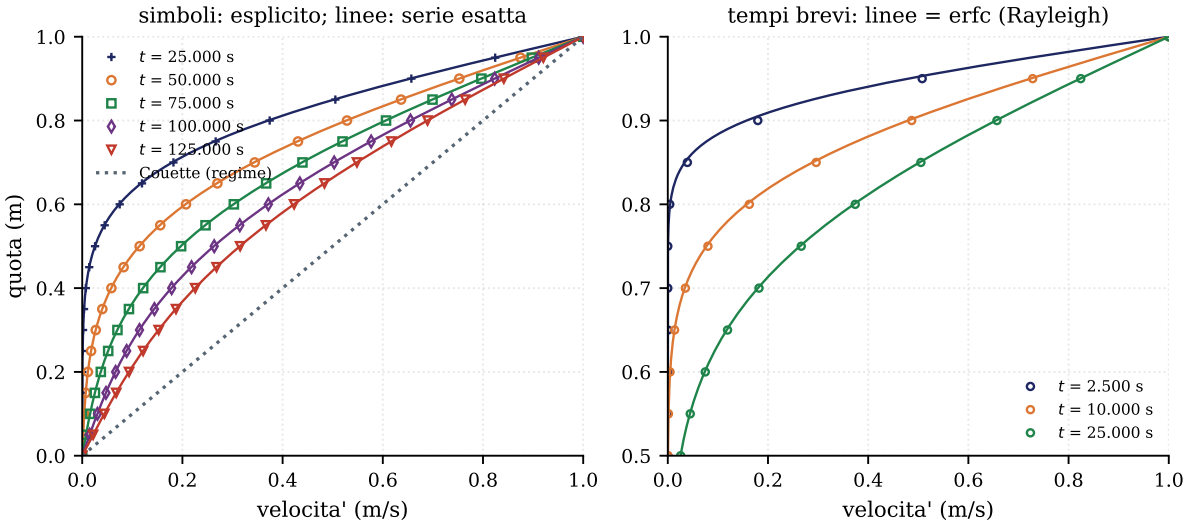


Figura 5.2: Avvio della corrente di Couette. A sinistra: profili calcolati con il metodo esplicito (simboli) e serie esatta (linee) a intervalli di 25 000 s. A destra: per tempi brevi la soluzione coincide con quella di Rayleigh per la lastra singola.

5.5 L'accuratezza dello schema e il caso $R = 1/6$

Lo schema (5.4) usa la differenza in avanti nel tempo e quella centrata nello spazio: il suo errore di troncamento è $\mathcal{O}(\tau, h^2)$, e poiché per la stabilità $\tau \propto h^2$ i due contributi sono dello stesso ordine. C'è però un'osservazione più sottile.

L'errore di troncamento e la scelta $R = 1/6$

Sia u la soluzione esatta. Sviluppando in serie di Taylor attorno a (y_i, t_j):

$$\frac{u_{i,j+1} - u_{i,j}}{\tau} = u_t + \frac{\tau}{2} u_{tt} + \mathcal{O}(\tau^2),$$

$$\nu \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2} = \nu u_{yy} + \frac{\nu h^2}{12} u_{yyyy} + \mathcal{O}\left(h^4\right).$$

L'errore di troncamento è la differenza fra i due membri, calcolata su una soluzione esatta. Poiché $u_t = \nu u_{yy}$, derivando si ha $u_{tt} = \nu^2 u_{yyyy}$, e quindi

$$E = \frac{\tau}{2}\nu^2 u_{yyyy} - \frac{\nu h^2}{12} u_{yyyy} + \cdots = \frac{\nu h^2}{12} (6R - 1)\, u_{yyyy} + \mathcal{O}\left(\tau^2, h^4\right). \tag{5.9}$$

Per $R = 1/6$ il termine principale si annulla: gli errori della derivata temporale e di quella spaziale si compensano esattamente, e lo schema diventa accurato del quarto ordine in h (secondo in τ). Il valore limite $R = 1/2$, invece, è quello con l'errore più grande fra i valori stabili.

Il Programma 3.4b verifica la previsione su un dato iniziale regolare, $u(y, 0) = \sin \pi y$ fra pareti a $u = 0$ (soluzione esatta $e^{-\pi^2 t} \sin \pi y$ in variabili adimensionali), e confronta il fattore di amplificazione dello schema con quello esatto $e^{-R(kh)^2}$.

prog3_4b_accuratezza.py

```
"""
prog3_4b_accuratezza.py -- accuratezza e stabilita' dello schema esplicito
per l'equazione della diffusione u_t = u_yy (variabili adimensionali).

1) Fattore di amplificazione G = 1 - 4 R sin^2(k h/2) confrontato con quello
   esatto exp(-R (k h)^2) per diversi R: per R > 1/2 si ha |G| > 1.
2) Errore al variare di R a passo h fisso, con dato iniziale regolare
   u(y, 0) = sin(pi y) (soluzione esatta exp(-pi^2 t) sin(pi y)): l'errore
   ha un minimo marcato per R = 1/6, dove il termine principale dell'errore di
   troncamento, (h^2/12)(6R - 1) u_yyyy, si annulla.
3) Ordine di convergenza in h per R = 1/4 (ordine 2) e R = 1/6 (ordine 4).
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO


def ftcs(h, R, t_fin):
    n = int(round(1.0 / h))
    y = np.linspace(0.0, 1.0, n + 1)
    u = np.sin(np.pi * y)
    tau = R * h * h
    passi = int(round(t_fin / tau))
    for _ in range(passi):
        u[1:-1] = u[1:-1] + R * (u[:-2] - 2 * u[1:-1] + u[2:])
    t = passi * tau
    return np.max(np.abs(u - np.exp(-np.pi**2 * t) * np.sin(np.pi * y)))
```

```
28
29
30 # 1) fattore di amplificazione
31 kh = np.linspace(0, np.pi, 200)
32 # 2) errore in funzione di R
33 Rv = np.linspace(0.02, 0.5, 49)
34 h0 = 1.0 / 20
35 err_R = []
36 for R in Rv:
37     # tempo finale multiplo del passo, vicino a 0,1
38     err_R.append(ftcs(h0, R, 0.1))
39 err_R = np.array(err_R)
40 print(f"Errore a t = 0,1 con h = 1/20: minimo per R = {Rv[np.argmin(err_R)]:.3f}
       (1/6 = {1/6:.3f})")
41 for R in [0.1, 1 / 6, 0.25, 0.4, 0.5]:
42     print(f"   R = {R:.4f}   max|u - u_es| = {ftcs(h0, R, 0.1):.3e}")
43
44 # 3) ordine di convergenza (tempo finale fisso t = 0,1 raggiunto esattamente)
45 print("\nOrdine di convergenza in h (t = 0,1):")
46 passi = [1 / 10, 1 / 20, 1 / 40, 1 / 80]
47 risultati = {}
48 for R in [0.25, 1 / 6]:
49     e = []
50     for h in passi:
51         # si sceglie R' vicino a R tale che t_fin / tau sia intero
52         n_t = int(round(0.1 / (R * h * h)))
53         e.append(ftcs(h, 0.1 / (n_t * h * h), 0.1))
54     risultati[R] = e
55     ordini = [np.log2(e[k - 1] / e[k]) for k in range(1, len(e))]
56     print(f"   R = {R:.4f}: errori " + ", ".join(f"{x:.2e}" for x in e) +
57           "   ordini " + ", ".join(f"{o:.2f}" for o in ordini))
58
59 fig, ax = plt.subplots(1, 3, figsize=(7.8, 2.7))
60 for R, col in zip([1 / 6, 0.25, 0.5, 0.6], [VERDE, BLU, ARANCIO, ROSSO]):
61     ax[0].plot(kh, 1 - 4 * R * np.sin(kh / 2) ** 2, color=col, label=f"$R={R:.3g}$")
62     ax[0].plot(kh, np.exp(-R * kh**2), ":", color=col)
63 ax[0].axhline(-1, color=GRIGIO, lw=0.8)
64 ax[0].set_xlabel("$kh$")
65 ax[0].set_ylabel("$G$")
66 ax[0].set_title("continue: schema; punteggiate: esatto")
67 ax[0].legend(fontsize=6.5)
68 ax[1].semilogy(Rv, err_R, color=BLU)
69 ax[1].axvline(1 / 6, color=VERDE, ls=":")
70 ax[1].set_xlabel("$R=\\nu\\tau/h^2$")
71 ax[1].set_ylabel("errore massimo a $t=0{,}1$")
72 ax[1].set_title("$h=1/20$")
73 hh = np.array(passi)
74 ax[2].loglog(hh, risultati[0.25], "o-", color=BLU, label="$R=1/4$")
75 ax[2].loglog(hh, risultati[1 / 6], "s-", color=VERDE, label="$R=1/6$")
76 ax[2].loglog(hh, risultati[0.25][0] * (hh / hh[0]) ** 2, ":", color="k",
       label="$h^2$")
77 ax[2].loglog(hh, risultati[1 / 6][0] * (hh / hh[0]) ** 4, "--", color="k",
       label="$h^4$")
78 ax[2].set_xlabel("$h$")
79 ax[2].set_ylabel("errore massimo")
80 ax[2].legend(fontsize=6.5)
81 fig.tight_layout()
82 salva(fig, "prog3_4b_accuratezza")
```

Listato 5.2: `prog3_4b_accuratezza.py`: fattore di amplificazione, errore in funzione di R e ordine di convergenza dello schema esplicito.

Risultati del Programma 3.4b

Con $h = 1/20$, a $t = 0{,}1$:

R	errore massimo
0,1	$3{,}03 \times 10^{-4}$
0,1667	$4{,}16 \times 10^{-7}$
0,25	$3{,}79 \times 10^{-4}$
0,4	$1{,}06 \times 10^{-3}$
0,5	$1{,}52 \times 10^{-3}$

Con $R = 1/6$ l'errore è quasi mille volte più piccolo che con $R = 1/4$. Raffinando la griglia l'errore decresce come h^2 per $R = 1/4$ (ordini osservati 2,00) e come h^4 per $R = 1/6$ (ordini 4,00).

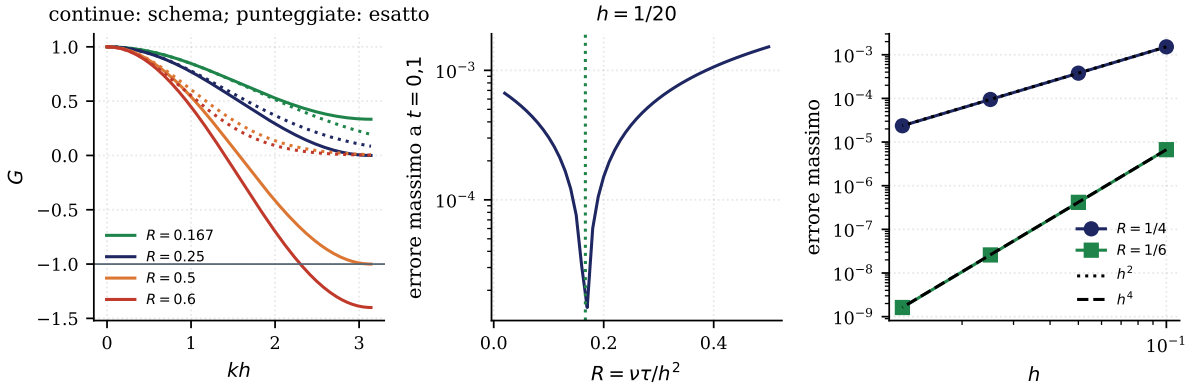


Figura 5.3: A sinistra: fattore di amplificazione dello schema esplicito (linee continue) e fattore esatto (punteggiate); per $R = 0{,}6$ le onde corte hanno $|G| > 1$. Al centro: errore in funzione di R a passo fisso, con il minimo per $R = 1/6$. A destra: ordine di convergenza.

Il risultato vale per il problema modello

La compensazione dell'errore per $R = 1/6$ sfrutta la relazione $u_{tt} = \nu^2 u_{yyyy}$, valida per l'equazione della diffusione a coefficienti costanti e senza termini sorgente. In presenza di termini convettivi, coefficienti variabili o geometrie cilindriche il vantaggio si perde in parte. Resta però una lezione generale: *il passo più grande consentito dalla stabilità non è il più accurato.*

5.6 La velocità di propagazione dello schema esplicito

Si osservi di nuovo la (5.4). Il valore nel nodo $(i, j+1)$ dipende dai valori nei nodi $(i-1, j)$, (i, j), $(i+1, j)$, che a loro volta dipendono dai loro vicini al passo precedente: il *dominio di dipendenza* del nodo è il triangolo delimitato dalle linee tratteggiate della Fig. 5.1, e una perturbazione si propaga nella griglia con velocità finita h/τ. In un fluido incomprimibile, invece, una perturbazione in un punto è avvertita istantaneamente in tutto il fluido. Per migliorare l'accuratezza si può ridurre τ (o R): le linee tratteggiate si avvicinano all'orizzontale, ma servono più passi per raggiungere lo stesso istante. L'accuratezza del metodo esplicito si

paga con il tempo di calcolo, e il vincolo $\tau \le h^2/(2\nu)$ diventa severissimo quando la griglia è fine. È la motivazione dei metodi impliciti del capitolo seguente.

5.7 Approfondimenti

5.7.1 Il metodo delle linee e l'analisi matriciale della stabilità

L'analisi di von Neumann ignora le condizioni al contorno: tratta il dominio come se fosse infinito o periodico. Un secondo punto di vista, più generale, consiste nel discretizzare prima lo spazio e poi il tempo (*metodo delle linee*). Con N nodi interni e condizioni di Dirichlet omogenee, le (5.1) diventano il sistema di equazioni ordinarie

$$\frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = \frac{\nu}{h^2}\,\mathbf{T}\,\mathbf{u}, \qquad \mathbf{T} = \begin{bmatrix} -2 & 1 & & \\ 1 & -2 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & -2 \end{bmatrix}, \tag{5.10}$$

e ogni schema temporale è un integratore per questo sistema: lo schema esplicito (5.4) è il metodo di Eulero in avanti, $\mathbf{u}_{j+1} = (\mathbf{I} + R\mathbf{T})\mathbf{u}_j$.

Gli autovalori della matrice tridiagonale

La matrice $\mathbf{T}$ ha autovettori e autovalori noti in forma chiusa. Si provi $v_i = \sin(ik\pi/(N+1))$, $i = 1,\dots,N$, per $k = 1,\dots,N$: dalla formula di prostaferesi $\sin(i-1)\varphi + \sin(i+1)\varphi = 2\cos\varphi \sin i\varphi$ segue

$$(\mathbf{T}\mathbf{v})_i = v_{i-1} - 2v_i + v_{i+1} = \left(2\cos\frac{k\pi}{N+1} - 2\right) v_i = -4\sin^2\frac{k\pi}{2(N+1)}\, v_i,$$

e la relazione vale anche per $i = 1$ e $i = N$ perché $v_0 = v_{N+1} = 0$. Gli autovalori sono dunque $\lambda_k = -4\sin^2[k\pi/(2(N+1))]$, tutti reali e negativi, compresi fra $\lambda_1 \simeq -\pi^2/(N+1)^2$ e $\lambda_N \simeq -4$. Gli autovettori sono i *seni discreti*: esattamente i modi di Fourier dell'analisi di von Neumann, ma con i numeri d'onda $k\pi/(N+1)$ selezionati dalle condizioni al contorno. Per questo problema le due analisi coincidono.
Il metodo di Eulero amplifica il modo k del fattore $1 + R\lambda_k$, e la stabilità richiede $|1 + R\lambda_k| \le 1$ per tutti i k, cioè $R \le 2/|\lambda_N| = 1/(2\sin^2[N\pi/(2(N+1))])$, che tende a $1/2$ per N grande: la (5.6), con una correzione trascurabile. La stessa formula vale per il problema del Programma 3.4b.

Il punto di vista matriciale rende evidente la natura del vincolo. Il sistema (5.10) è *rigido* (*stiff*): contiene scale di tempo molto diverse, da $h^2/(\pi^2\nu)\cdot(N+1)^2$ per il modo più lento (la costante di tempo fisica $d^2/(\pi^2\nu)$ del Programma 3.4) a $h^2/(4\nu)$ per il più rapido, con un rapporto $\lambda_N/\lambda_1 \simeq 4(N+1)^2/\pi^2$, cioè 160 per i 20 intervalli del Programma 3.4 e 6×10^4 per i 400 del Programma 3.5b. Il passo di un metodo esplicito è dettato dal modo più rapido, che fisicamente non interessa (si smorza in pochi passi), mentre la durata del calcolo è dettata dal più lento. I metodi impliciti del capitolo seguente sono, in questo linguaggio, gli integratori per sistemi rigidi.

5.7.2 Un'insidia: lo schema di Richardson

L'errore $\mathcal{O}(\tau)$ dello schema esplicito suggerisce di usare la differenza *centrata* anche nel tempo, $(u_{i,j+1} - u_{i,j-1})/(2\tau)$, che è del secondo ordine. Lo schema risultante (Richardson, 1910),

$$u_{i,j+1} = u_{i,j-1} + 2R\left(u_{i-1,j} - 2u_{i,j} + u_{i+1,j}\right),$$

è esplicito, del secondo ordine in τ e h, e sembra un miglioramento gratuito. L'analisi di von Neumann, con $u_{i,j} = G^j e^{ikih}$, dà l'equazione di secondo grado $G^2 + 8R\sin^2(kh/2)\,G - 1 = 0$, le cui radici hanno *prodotto* -1: una delle due ha sempre modulo maggiore di uno, qualunque siano R e k (tranne $k = 0$). Lo schema è incondizionatamente *instabile*, e inutilizzabile. L'esercizio 11 mostra come DuFort e Frankel lo abbiano reso stabile con una sostituzione apparentemente innocua, e a quale prezzo. La morale è che l'ordine di accuratezza e la stabilità sono proprietà indipendenti, da verificare separatamente.

5.7.3 Coefficienti variabili e forma conservativa

Se la viscosità dipende dalla quota (fluido stratificato, viscosità dipendente dalla temperatura) l'equazione è $\partial u/\partial t = \partial_y(\nu\,\partial_y u)$, e non va sviluppata come $\nu u_{yy} + \nu' u_y$: conviene discretizzare direttamente il flusso $\nu\,\partial_y u$ nei punti intermedi,

$$u_{i,j+1} = u_{i,j} + \frac{\tau}{h^2}\left[\nu_{i+1/2}(u_{i+1,j} - u_{i,j}) - \nu_{i-1/2}(u_{i,j} - u_{i-1,j})\right], \qquad \nu_{i\pm1/2} = \tfrac{1}{2}(\nu_i + \nu_{i\pm1}).$$

In questa forma *conservativa* la quantità di moto totale $\sum_i u_i$ cambia solo per i flussi alle pareti, come nel continuo, e il principio del massimo discreto vale sotto la condizione $\tau\,(\nu_{i+1/2} + \nu_{i-1/2})/h^2 \le 1$, cioè $R \le 1/2$ con la viscosità *massima*. In coordinate cilindriche il flusso è $r\nu\,\partial_r u$ e il volume di competenza del nodo è $r_i\,h$: è lo schema usato negli Esercizi 9 e 12.

5.7.4 La stessa equazione per il calore

L'equazione (5.1) con ν sostituita dalla diffusività termica $\kappa = k/(\rho c_p)$ descrive la conduzione del calore in una lastra, e tutto il capitolo si applica senza modifiche. Il parametro che decide i tempi è κ, che per i solidi varia di tre ordini di grandezza.

Tempi di diffusione termica

Il tempo caratteristico perché una differenza di temperatura attraversi uno spessore L è $t \sim L^2/\kappa$ (o, con precisione, $L^2/(\pi^2\kappa)$ per il primo modo della serie (5.8)).

- Pelle in lega di alluminio, $L = 2\,\text{mm}$, $\kappa = 7\times10^{-5}\,\text{m}^2\,\text{s}^{-1}$: $t \sim 0{,}06\,\text{s}$. La pelle segue istantaneamente la temperatura adiabatica di parete del Cap. 3; i sensori di temperatura esterna hanno tempi di risposta dettati dallo scambio con l'aria, non dalla conduzione nel metallo.
- Un disco freno in acciaio di 10 mm, $\kappa = 1{,}2\times10^{-5}\,\text{m}^2\,\text{s}^{-1}$: $t \sim 8\,\text{s}$, dell'ordine della durata di una frenata: la superficie si scalda molto più del nucleo, ed è il motivo dei gradienti termici che li deformano.
- Una parete di calcestruzzo di 20 cm, $\kappa = 5\times10^{-7}\,\text{m}^2\,\text{s}^{-1}$: $t \sim 8\times10^4\,\text{s} \simeq 22\,\text{h}$: la parete "ricorda" la temperatura del giorno precedente, e questo è il principio dell'inerzia termica degli edifici. L'Esercizio 10, con una temperatura periodica

al posto della velocità oscillante, descrive esattamente la penetrazione dell'onda termica giornaliera, con profondità di penetrazione $\sqrt{2\kappa/\omega} \simeq 12\,\mathrm{cm}$.

Gli esercizi completano il capitolo: l'Esercizio 8 mostra che cosa succede con $R > 1/2$; l'Esercizio 9 applica lo schema in coordinate cilindriche alla messa in rotazione del fluido attorno a un cilindro; l'Esercizio 10 studia la lastra oscillante (secondo problema di Stokes); l'Esercizio 11 analizza lo schema a tre livelli di DuFort–Frankel, incondizionatamente stabile ma con un'insidia nella consistenza.

Capitolo 6

Equazioni paraboliche II: metodi impliciti e l'avviamento di una corrente in un canale

6.1 Il metodo implicito

Il difetto del metodo esplicito, che la soluzione in un punto non risente subito delle condizioni negli altri punti, si elimina con una modifica in apparenza minima. Si continua a usare la differenza centrata nello spazio, ma la derivata temporale si approssima con la differenza *all'indietro*, scrivendo l'equazione nel nodo (i, j) del nuovo livello:

$$\frac{u_{i,j} - u_{i,j-1}}{\tau} = \nu \, \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2}, \tag{6.1}$$

cioè, raccogliendo,

$$R\, u_{i-1,j} - (1 + 2R)\, u_{i,j} + R\, u_{i+1,j} = -u_{i,j-1}, \qquad R = \frac{\nu\tau}{h^2}. \tag{6.2}$$

La differenza cambia radicalmente la procedura. Ogni equazione (6.2) lega *tre* valori incogniti del nuovo livello: scritta in tutti i nodi interni forma un sistema di equazioni simultanee, che va risolto per ottenere tutte le incognite all'istante t_j. Le velocità alle diverse quote non sono più indipendenti: una variazione in un punto è avvertita immediatamente da tutti gli altri. Lo schema è quindi più aderente alla fisica del moto incomprimibile; poiché la soluzione non si può più calcolare esplicitamente, la (6.2) si chiama formula del **metodo implicito** (o di Eulero all'indietro, in inglese BTCS).

Stabilità incondizionata

Con $u_{i,j} = U_j e^{\mathrm{i}kih}$ la (6.2) dà $U_j\,[1 + 2R(1 - \cos kh)] = U_{j-1}$, cioè

$$U_j = G\, U_{j-1}, \qquad G = \frac{1}{1 + 2R(1 - \cos kh)} = \frac{1}{1 + 4R \sin^2(kh/2)}. \tag{6.3}$$

Al variare di $\cos kh$ fra -1 e 1, G varia fra $1/(1 + 4R)$ e 1: non supera mai l'unità, qualunque sia $R > 0$. Lo schema è *incondizionatamente stabile*.

La stabilità non vincola più R, ma l'accuratezza sì: moltiplicando la (6.1) per τ si vede che i termini trascurati sono moltiplicati per R, e un R più piccolo dà una soluzione più accurata. L'errore di troncamento resta $\mathcal{O}(\tau, h^2)$, come nello schema esplicito.

Il sistema (6.2) è tridiagonale, ed è un caso particolare della forma generale $C_{i1}u_{i-1} + C_{i2}u_i + C_{i3}u_{i+1} = C_{i4}$ con coefficienti costanti al primo membro:

$$C_{i1} = R, \qquad C_{i2} = -1 - 2R, \qquad C_{i3} = R, \qquad C_{i4} = -u_{i,j-1}. \tag{6.4}$$

Lo si risolve con TRID (o con l'algoritmo di Thomas) in circa $8n$ operazioni: un passo implicito costa solo poche volte più di uno esplicito, ma il passo temporale può essere molto più grande.

6.2 L'avviamento di una corrente in un canale

Applichiamo il metodo implicito a un fluido incomprimibile, inizialmente in quiete fra due lastre parallele indefinite distanti $2L$, messo in moto da un gradiente di pressione costante $\mathrm{d}p/\mathrm{d}x$ applicato bruscamente lungo il canale. Con l'origine sulla mezzeria l'equazione del moto è

$$\frac{\partial u}{\partial t} = -\frac{1}{\rho}\frac{\mathrm{d}p}{\mathrm{d}x} + \nu\frac{\partial^2 u}{\partial y^2}, \tag{6.5}$$

con $u = 0$ per $y = \pm L$ e per $t = 0$. Al crescere del tempo il profilo tende a quello di regime di Poiseuille, $u_s = -\frac{1}{2\mu}\frac{\mathrm{d}p}{\mathrm{d}x}(L^2 - y^2)$.

Variabili adimensionali e problema per la differenza dal regime

Con

$$Y = \frac{y}{L}, \qquad T = \frac{t}{L^2/\nu}, \qquad U = \frac{u}{-\frac{L^2}{2\mu}\frac{\mathrm{d}p}{\mathrm{d}x}}, \qquad W = (1 - Y^2) - U, \tag{6.6}$$

la differenza W fra profilo di regime e profilo attuale soddisfa l'equazione della diffusione *omogenea*, perché il termine di pressione è bilanciato dalla parte stazionaria:

$$\frac{\partial W}{\partial T} = \frac{\partial^2 W}{\partial Y^2}, \qquad W(\pm 1, T) = 0, \qquad W(Y, 0) = 1 - Y^2. \tag{6.7}$$

Il problema per W ha anche un significato fisico autonomo: descrive la *decelerazione* di una corrente di Poiseuille quando il gradiente di pressione che la sostiene viene tolto bruscamente.

Per simmetria basta calcolare la metà superiore del canale, con le condizioni

$$W(1, T) = 0, \qquad \frac{\partial W}{\partial Y}(0, T) = 0, \qquad W(Y, 0) = 1 - Y^2 \ \ (0 \le Y \le 1). \tag{6.8}$$

La condizione di simmetria sulla mezzeria richiede, come nel Cap. 3, un nodo fittizio. Si divide $0 \le Y \le 1$ in n intervalli, con nodi $Y_1 = 0, \ldots, Y_{n+1} = 1$, e si aggiunge $Y_0 = -h$: la (6.8) diventa $W_0 = W_2$. Prima di chiamare TRID si modificano i coefficienti come per il termometro a lamina piana:

$$C_{13} \leftarrow C_{11} + C_{13}, \quad C_{11} \leftarrow 0, \qquad C_{n4} \leftarrow C_{n4} - C_{n3}W_{n+1}, \quad C_{n3} \leftarrow 0. \tag{6.9}$$

Gli indici temporali si possono omettere nel programma: si calcolano i C_{i4} con i valori di W disponibili, si chiama TRID, si sovrascrive W con la soluzione e si ripete. La velocità si ricava da $U = (1 - Y^2) - W$.

La soluzione esatta

Separando le variabili nella (6.7), $W = \sum_m A_m \cos(\lambda_m Y)\, e^{-\lambda_m^2 T}$; la condizione $W(1,T) = 0$ impone $\cos\lambda_m = 0$, cioè $\lambda_m = (2m-1)\pi/2$, e la simmetria è automatica. I coefficienti seguono dall'ortogonalità dei coseni su $[0,1]$: $A_m = 2\int_0^1 (1-Y^2)\cos(\lambda_m Y)\,\mathrm{d}Y$. Integrando due volte per parti e usando $\cos\lambda_m = 0$, $\sin\lambda_m = (-1)^{m+1}$:

$$W = \sum_{m=1}^{\infty} \frac{4(-1)^{m+1}}{\lambda_m^3} \cos(\lambda_m Y)\, e^{-\lambda_m^2 T}, \qquad \lambda_m = \frac{(2m-1)\pi}{2}. \tag{6.10}$$

Per $T = 0$ e $Y = 0$ la serie vale $\frac{32}{\pi^3}\sum_m \frac{(-1)^{m+1}}{(2m-1)^3} = \frac{32}{\pi^3}\cdot\frac{\pi^3}{32} = 1$, come deve.

Dati del Programma 3.5

$n = 20$ ($h = 0{,}05$), $R = 0{,}4$, quindi $\tau = Rh^2 = 0{,}001$; $T_{\max} = 1$ (mille passi); stampa ogni 50 passi, grafico ogni 200.

prog3_5_canale_impulsivo.py

```
"""
prog3_5_canale_impulsivo.py -- PROGRAMMA 3.5
Avviamento di una corrente in un canale piano per effetto di un gradiente di
pressione costante applicato bruscamente.

Con Y = y/L, T = t/(L^2/nu), U = u / [-(L^2/(2 mu)) dp/dx] e W = (1 - Y^2) - U:
    W_T = W_YY,   W(1, T) = 0,   W_Y(0, T) = 0 (simmetria),   W(Y, 0) = 1 - Y^2.
Metodo implicito (indietro nel tempo, centrato nello spazio):
    R W_(i-1)^j - (1 + 2R) W_i^j + R W_(i+1)^j = -W_i^(j-1),
sistema tridiagonale risolto con TRID a ogni passo; nodo fittizio W_0 = W_2.
Confronto con la soluzione esatta
    W = sum_m 4 (-1)^(m+1) / lam_m^3 cos(lam_m Y) exp(-lam_m^2 T),  lam_m = (2m -
    1) pi/2.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import trid
from stile_grafici import salva, PALETTE, GRIGIO

N = 20
R = 0.4
TMAX = 1.0
NPRINT, NPLOT = 50, 200
H = 1.0 / N
TAU = R * H**2

Y = np.linspace(0.0, 1.0, N + 1)
US = 1.0 - Y**2                        # profilo di regime (Poiseuille)
W = US.copy()


def W_esatta(Y, T, mmax=200):
    W = np.zeros_like(Y)
    for m in range(1, mmax + 1):
        lam = (2 * m - 1) * np.pi / 2
        W += 4 * (-1) ** (m + 1) / lam**3 * np.cos(lam * Y) * np.exp(-lam**2 * T)
    return W


C = np.zeros((N, 4))
```

```
t, j = 0.0, 0
profili = [(0.0, US - W)]
print(f"tau = R h^2 = {TAU}")
print("     T       U a Y = 0; 0,1; ...; 1")
print(f"  {t:6.3f}  " + " ".join(f"{v:.4f}" for v in (US - W)[::2]))
while t < TMAX - 1e-12:
    C[:, 0] = R
    C[:, 1] = -1.0 - 2.0 * R
    C[:, 2] = R
    C[:, 3] = -W[:N]
    C[0, 2] = C[0, 0] + C[0, 2]       # simmetria in Y = 0 (nodo fittizio)
    C[0, 0] = 0.0
    C[N - 1, 3] -= C[N - 1, 2] * 0.0  # W = 0 alla parete Y = 1
    C[N - 1, 2] = 0.0
    W[:N] = trid(C)
    W[N] = 0.0
    j += 1
    t += TAU
    U = US - W
    if j % NPRINT == 0:
        print(f"  {t:6.3f}  " + " ".join(f"{v:.4f}" for v in U[::2]))
    if j % NPLOT == 0:
        profili.append((t, U.copy()))

print(f"\n{j} passi temporali senza instabilita'.")
print(f"Velocita' al centro a T = 1: U = {U[0]:.4f}  (esatta {1 -
    W_esatta(np.array([0.0]), 1.0)[0]:.4f})")
for tp, up in profili[1:]:
    err = np.max(np.abs(up - (US - W_esatta(Y, tp))))
    print(f"   T = {tp:.1f}   max|U - U_es| = {err:.2e}")
L, nu = 0.01, 1.0e-6
print(f"\nAcqua in un canale alto 2 cm: T = 1 corrisponde a t = L^2/nu =
    {L**2/nu:.0f} s")
for frac in [0.9, 0.99]:
    # tempo per cui la velocita' al centro raggiunge frac del valore di regime
    Ts = np.linspace(0.01, 3, 3000)
    Uc = 1 - np.array([W_esatta(np.array([0.0]), T)[0] for T in Ts])
    print(f"   U_centro = {frac:.2f} per T = {Ts[np.argmax(Uc >= frac)]:.3f}")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(figsize=(4.6, 3.4))
yy = np.linspace(0, 1, 201)
simboli = ["x", "o", "s", "d", "v"]
for k, (tp, up) in enumerate(profili):
    col = PALETTE[k % len(PALETTE)]
    ax.plot(1 - yy**2 - W_esatta(yy, tp), yy, color=col, lw=0.9)
    ax.plot(up, Y, simboli[k % 5], ms=3.5, mfc="none", color=col,
    label=f"$T={tp:.1f}$")
ax.plot(1 - yy**2, yy, "--", color=GRIGIO, label="regime")
ax.set_xlabel("velocita' adimensionale $U$")
ax.set_ylabel("quota adimensionale $Y$")
ax.set_xlim(0, 1.02)
ax.set_ylim(0, 1)
ax.legend(fontsize=7)
salva(fig, "prog3_5_canale_impulsivo")
```

Listato 6.1: Programma 3.5 – avviamento di una corrente in un canale con il metodo implicito (`prog3_5_canale_impulsivo.py`).

Risultati del Programma 3.5

- Mille passi senza alcuna instabilità, come previsto per il metodo implicito.
- A $T = 1$ la velocità al centro vale $U = 0{,}9121$ (esatta 0,9125), cioè il 91% di quella di regime; l'errore massimo rispetto alla (6.10) è compreso fra $3{,}8 \times 10^{-4}$ e $6{,}6 \times 10^{-4}$ in tutto l'intervallo.
- Il centro del canale raggiunge il 90% della velocità di regime per $T = 0{,}946$ e il 99% per $T = 1{,}88$. Per l'acqua in un canale alto 2 cm ($L = 1$ cm) l'unità di tempo adimensionale vale $L^2/\nu = 100$ s: occorrono più di tre minuti perché la corrente si stabilisca.

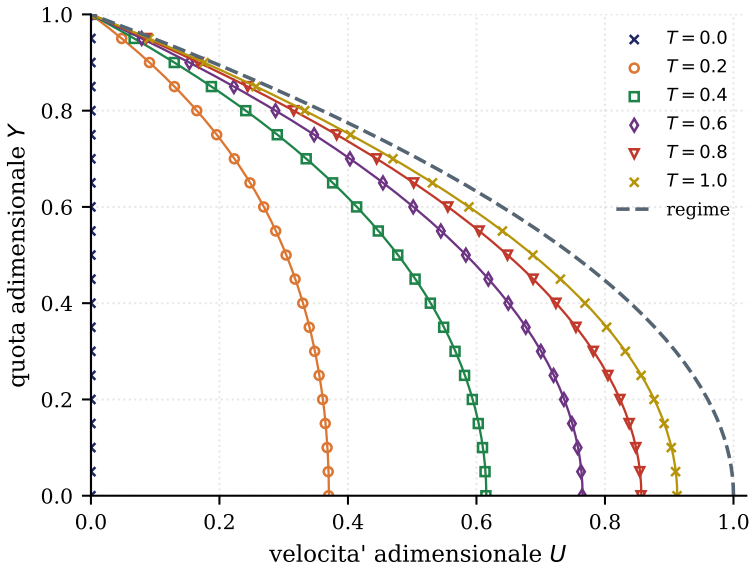


Figura 6.1: Avviamento della corrente nel canale: profili calcolati con il metodo implicito (simboli) e soluzione esatta (linee) per $T = 0$; 0,2; 0,4; 0,6; 0,8; 1; tratteggiato il profilo di regime.

Subito dopo l'applicazione del gradiente di pressione tutto il fluido accelera con la stessa accelerazione, tranne vicino alle pareti, dove l'aderenza lo trattiene: il profilo è piatto al centro e la vorticità è confinata in due strati sottili. Poi la vorticità diffonde verso il centro e l'accelerazione diminuisce. Il lento riassetto della distribuzione di vorticità è caratteristico delle correnti laminari con piccola viscosità cinematica.

6.3 Il metodo di Crank–Nicolson

L'errore di troncamento del metodo implicito è $\mathcal{O}(\tau, h^2)$: del primo ordine nel tempo. Si ottiene uno schema del secondo ordine scrivendo l'equazione nel punto di mezzo $(i, j - \frac{1}{2})$ fra i due livelli (Fig. 6.2).

Derivazione e proprietà

Nel punto $(i, j - \frac{1}{2})$ la derivata temporale si approssima con la differenza centrata su mezzo passo, $(\partial u/\partial t)_{i,j-1/2} = (u_{i,j} - u_{i,j-1})/\tau + \mathcal{O}(\tau^2)$, e la derivata seconda spaziale con la media di quelle ai livelli j e $j - 1$, anch'essa con errore $\mathcal{O}(\tau^2, h^2)$. Sostituendo

Segnaposto figura

file atteso: `immagini/fig_3_6_2.png`

Molecola di calcolo del metodo di Crank–Nicolson: tre nodi al livello j e tre al livello $j-1$, con l'equazione scritta nel punto intermedio $(i, j-\frac{1}{2})$ (Fig. 3.6.2 della fonte).

Figura 6.2: Il metodo di Crank–Nicolson.

nella (5.1):

$$R\,u_{i-1,j} - 2(1+R)\,u_{i,j} + R\,u_{i+1,j} = -R\,u_{i-1,j-1} - 2(1-R)\,u_{i,j-1} - R\,u_{i+1,j-1}. \tag{6.11}$$

Rispetto alla (6.2) cambia solo il secondo membro, che contiene tre valori del livello precedente invece di uno: il costo aggiuntivo è trascurabile. L'analisi di von Neumann dà

$$G = \frac{1 - 2R\sin^2(kh/2)}{1 + 2R\sin^2(kh/2)}, \tag{6.12}$$

e $|G| \le 1$ per ogni $R > 0$: anche il metodo di Crank–Nicolson è incondizionatamente stabile, ed è del secondo ordine nel tempo e nello spazio.

Il metodo implicito e quello di Crank–Nicolson sono i casi $\vartheta = 1$ e $\vartheta = \frac{1}{2}$ dello *schema* ϑ, in cui la derivata spaziale è la media pesata, con pesi ϑ e $1-\vartheta$, di quelle ai livelli nuovo e vecchio; $\vartheta = 0$ restituisce lo schema esplicito. La (6.12) nasconde però un'insidia: per R grande e onde corte ($kh \to \pi$) il fattore di amplificazione tende a -1. Le componenti di lunghezza d'onda dell'ordine del passo non vengono smorzate, come dovrebbero, ma cambiano segno a ogni passo e sopravvivono: se il dato iniziale contiene una discontinuità, la soluzione presenta oscillazioni spurie che si attenuano molto lentamente. Il metodo implicito, con $G \to 0$ per le onde corte, le smorza invece immediatamente.

Il Programma 3.5b mette a confronto i due metodi: l'ordine di convergenza nel tempo sul problema del Programma 3.5, i fattori di amplificazione, e il comportamento con un dato iniziale discontinuo (la lastra di Rayleigh messa in moto) e un passo temporale grande, $R = 10$.

prog3_5b_crank_nicolson.py

```
"""
prog3_5b_crank_nicolson.py -- confronto fra metodo implicito (Eulero all'indietro)
e metodo di Crank-Nicolson per l'equazione della diffusione.

Schema theta (theta = 1: implicito; theta = 1/2: Crank-Nicolson):
  -theta R W_(i-1)^(n+1) + (1 + 2 theta R) W_i^(n+1) - theta R W_(i+1)^(n+1)
     = (1-theta) R W_(i-1)^n + [1 - 2 (1-theta) R] W_i^n + (1-theta) R W_(i+1)^n
1) Ordine nel tempo sul problema del Programma 3.5 (h piccolo, tau variabile).
2) Fattori di amplificazione: G_I = 1/(1 + 4R s^2), G_CN = (1 - 2R s^2)/(1 + 2R
    s^2),
   s = sin(kh/2): per R grande G_CN -> -1 (le componenti brevi non si smorzano).
3) Dato iniziale discontinuo (problema di Rayleigh, R = 10): Crank-Nicolson
   produce oscillazioni spurie, il metodo implicito no.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import thomas
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO


def passo_theta(W, R, theta, simm=True, w_sup=0.0):
    """Un passo dello schema theta su 0 <= Y <= 1, nodi 0..N; W[N] = w_sup.
    Se simm=True in Y = 0 vale W_Y = 0 (nodo fittizio), altrimenti W[0] e' fisso."""
    N = len(W) - 1
    d = W.copy()
    lap = np.zeros_like(W)
    lap[1:-1] = W[:-2] - 2 * W[1:-1] + W[2:]
    if simm:
        lap[0] = 2 * (W[1] - W[0])
    rhs = W + (1 - theta) * R * lap
    a = -theta * R * np.ones(N + 1)
    b = (1 + 2 * theta * R) * np.ones(N + 1)
    c = -theta * R * np.ones(N + 1)
    if simm:
        c[0] = -2 * theta * R                # W_(-1) = W_1
        i0 = 0
    else:
        i0 = 1
    # incognite: i0..N-1; condizione di Dirichlet in N (e in 0 se non simmetrico)
    rhs[N - 1] += theta * R * w_sup
    if not simm:
        rhs[1] += theta * R * W[0]
    sol = thomas(a[i0:N], b[i0:N], c[i0:N], rhs[i0:N])
    nuovo = W.copy()
    nuovo[i0:N] = sol
    nuovo[N] = w_sup
    return nuovo


def W_esatta(Y, T, mmax=400):
    W = np.zeros_like(Y)
    for m in range(1, mmax + 1):
        lam = (2 * m - 1) * np.pi / 2
        W += 4 * (-1) ** (m + 1) / lam**3 * np.cos(lam * Y) * np.exp(-lam**2 * T)
    return W


# ------------------------------------------------ 1) ordine nel tempo
N = 400
Y = np.linspace(0, 1, N + 1)
```

```
h = 1.0 / N
T_fin = 0.5
Wes = W_esatta(Y, T_fin)
print("1) Errore a T = 0,5 (h = 1/400) al variare del passo temporale:")
print("     tau        implicito    ordine    Crank-Nicolson   ordine")
taus = [0.05, 0.025, 0.0125, 0.00625, 0.003125]
err = {1.0: [], 0.5: []}
for tau in taus:
    for th in (1.0, 0.5):
        W = 1 - Y**2
        R = tau / h**2
        for _ in range(int(round(T_fin / tau))):
            W = passo_theta(W, R, th)
        err[th].append(np.max(np.abs(W - Wes)))
for k, tau in enumerate(taus):
    o1 = "" if k == 0 else f"{np.log2(err[1.0][k-1]/err[1.0][k]):6.2f}"
    o2 = "" if k == 0 else f"{np.log2(err[0.5][k-1]/err[0.5][k]):6.2f}"
    print(f"  {tau:8.5f}    {err[1.0][k]:.3e}  {o1:>7s}     {err[0.5][k]:.3e}
    {o2:>7s}")

# ------------------------------------------------ 3) dato discontinuo
N3 = 40
Y3 = np.linspace(0, 1, N3 + 1)
R3 = 10.0
tau3 = R3 / N3**2
n3 = 5
ris = {}
for th in (1.0, 0.5):
    u = np.zeros(N3 + 1)
    u[-1] = 1.0                                  # lastra superiore in moto
    for _ in range(n3):
        u = passo_theta(u, R3, th, simm=False, w_sup=1.0)
    ris[th] = u
from scipy.special import erfc
T3 = n3 * tau3
u_es = np.zeros_like(Y3)
for n in range(1, 400):
    u_es += 2 * (-1) ** n / (n * np.pi) * np.sin(n * np.pi * Y3) * np.exp(-(n *
    np.pi) ** 2 * T3)
u_es += Y3
print(f"\n3) Lastra messa in moto, R = {R3}, {n3} passi (T = {T3:.4f}):")
print(f"   implicito:       min u (interni) = {ris[1.0][1:-1].min():+.4f}, max u
    (interni) = {ris[1.0][:-1].max():.4f}")
print(f"   Crank-Nicolson:  min u (interni) = {ris[0.5][1:-1].min():+.4f}, max u
    (interni) = {ris[0.5][:-1].max():.4f}")
# avvio di Rannacher: i primi due mezzi passi con il metodo implicito, poi
    Crank-Nicolson
u = np.zeros(N3 + 1)
u[-1] = 1.0
for _ in range(2):
    u = passo_theta(u, R3 / 2, 1.0, simm=False, w_sup=1.0)
for _ in range(n3 - 1):
    u = passo_theta(u, R3, 0.5, simm=False, w_sup=1.0)
print(f"   CN con avvio di Rannacher (2 mezzi passi impliciti): max u (interni) =
    {u[:-1].max():.4f}")
print(f"   errore massimo rispetto alla serie esatta: implicito
    {np.max(np.abs(ris[1.0]-u_es)):.2e},"
      f" Crank-Nicolson {np.max(np.abs(ris[0.5]-u_es)):.2e}, Rannacher
    {np.max(np.abs(u-u_es)):.2e}")
print(f"   (la soluzione esatta e' compresa fra 0 e 1)")
```

```
112
113 # ------------------------------------------------ figura
114 fig, ax = plt.subplots(1, 3, figsize=(7.8, 2.7))
115 tt = np.array(taus)
116 ax[0].loglog(tt, err[1.0], "o-", color=BLU, label="implicito")
117 ax[0].loglog(tt, err[0.5], "s-", color=VERDE, label="Crank-Nicolson")
118 ax[0].loglog(tt, err[1.0][0] * (tt / tt[0]), ":", color="k", label=r"$\tau$")
119 ax[0].loglog(tt, err[0.5][0] * (tt / tt[0]) ** 2, "--", color="k",
    label=r"$\tau^2$")
120 ax[0].set_xlabel(r"$\tau$")
121 ax[0].set_ylabel("errore massimo")
122 ax[0].legend(fontsize=6.5)
123 kh = np.linspace(0, np.pi, 300)
124 s2 = np.sin(kh / 2) ** 2
125 for R, ls in [(0.4, "-"), (10.0, "--")]:
126     ax[1].plot(kh, 1 / (1 + 4 * R * s2), ls, color=BLU, label=f"impl., $R={R:g}$")
127     ax[1].plot(kh, (1 - 2 * R * s2) / (1 + 2 * R * s2), ls, color=VERDE,
    label=f"C-N, $R={R:g}$")
128 ax[1].axhline(0, color=GRIGIO, lw=0.6)
129 ax[1].set_xlabel("$kh$")
130 ax[1].set_ylabel("$G$")
131 ax[1].legend(fontsize=6)
132 ax[2].plot(u_es, Y3, color=GRIGIO, lw=2.0, alpha=0.6, label="esatta")
133 ax[2].plot(ris[1.0], Y3, "o-", ms=2.5, color=BLU, label="implicito")
134 ax[2].plot(ris[0.5], Y3, "s-", ms=2.5, color=VERDE, label="Crank-Nicolson")
135 ax[2].set_xlabel("$u/u_0$")
136 ax[2].set_ylabel("$y/d$")
137 ax[2].set_ylim(0.5, 1.0)
138 ax[2].legend(fontsize=6.5)
139 ax[2].set_title(f"$R={R3:g}$, {n3} passi")
140 fig.tight_layout()
141 salva(fig, "prog3_5b_crank_nicolson")
```

Listato 6.2: `prog3_5b_crank_nicolson.py`: confronto fra metodo implicito e metodo di Crank–Nicolson (schema ϑ).

Risultati del Programma 3.5b

- Errore a $T = 0{,}5$ con $h = 1/400$ (l'errore spaziale è trascurabile):

τ	implicito	ordine	Crank–Nicolson	ordine
0,05	$2{,}189 \times 10^{-2}$	–	$5{,}143 \times 10^{-4}$	–
0,025	$1{,}119 \times 10^{-2}$	0,97	$1{,}316 \times 10^{-4}$	1,97
0,0125	$5{,}655 \times 10^{-3}$	0,98	$3{,}295 \times 10^{-5}$	2,00
0,006 25	$2{,}843 \times 10^{-3}$	0,99	$8{,}038 \times 10^{-6}$	2,04

  A parità di passo Crank–Nicolson è da 40 a 350 volte più accurato.

- Lastra messa in moto con $R = 10$ e cinque passi: con il metodo implicito la velocità resta compresa fra 0 e 1; con Crank–Nicolson vicino alla lastra mobile compaiono valori fino a 1,07, fisicamente impossibili (Fig. 6.3, a destra).

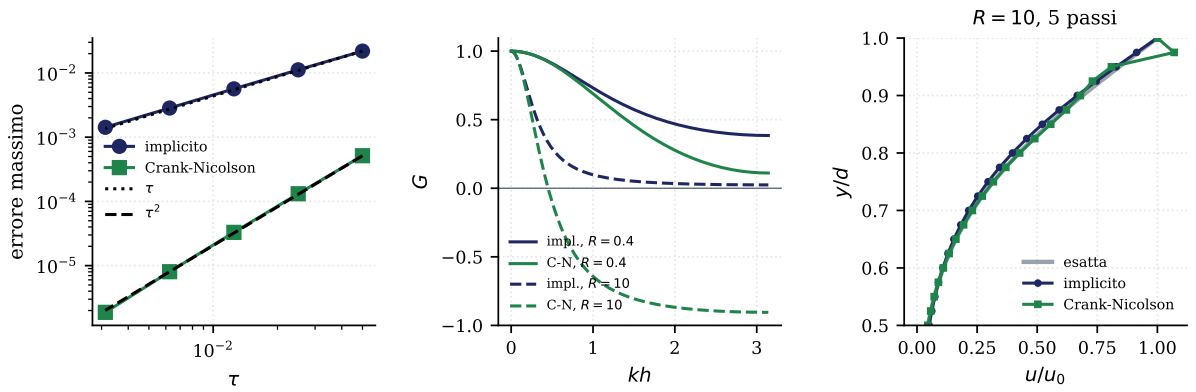


Figura 6.3: Metodo implicito e metodo di Crank–Nicolson: ordine di convergenza nel tempo (a sinistra), fattori di amplificazione (al centro), comportamento con un dato discontinuo e $R = 10$ (a destra).

Quale metodo scegliere

Crank–Nicolson è la scelta naturale quando la soluzione è regolare e interessa l'accuratezza nel tempo. Con dati discontinui (avviamenti bruschi, urti termici) conviene partire con qualche passo implicito, oppure usare ϑ leggermente maggiore di $\frac{1}{2}$, oppure limitare R. È lo stesso compromesso fra accuratezza e smorzamento che si incontra negli schemi per le equazioni iperboliche del modulo precedente.

6.4 Esplicito o implicito? Un bilancio dei costi

Un passo del metodo esplicito costa circa $5n$ operazioni, uno del metodo implicito o di Crank–Nicolson circa $15n$ (preparazione del secondo membro e algoritmo di Thomas). Ma il passo esplicito è limitato da $\tau \leq h^2/(2\nu)$, che decresce col quadrato di h. Con $h = 1/400$, per raggiungere $T = 0{,}5$ lo schema esplicito richiede almeno 160 000 passi; Crank–Nicolson raggiunge un errore di 3×10^{-5} in 40 passi. Per griglie fini e tempi lunghi i metodi impliciti vincono senza discussione; quelli espliciti restano convenienti quando interessa seguire fenomeni rapidi, che richiedono comunque passi piccoli, o quando il sistema da risolvere sarebbe troppo costoso (problemi multidimensionali non lineari).

6.5 Approfondimenti

6.5.1 Perché l'algoritmo di Thomas funziona senza pivoting

L'algoritmo di Thomas è l'eliminazione di Gauss specializzata alle matrici tridiagonali: una passata in avanti che elimina la sottodiagonale e una sostituzione all'indietro, in circa $8n$ operazioni invece delle $2n^3/3$ dell'eliminazione completa. L'eliminazione di Gauss può però fallire, o amplificare gli errori di arrotondamento, se un elemento diagonale diventa piccolo; l'algoritmo di Thomas non prevede scambi di righe, e bisogna assicurarsi che non ne abbia bisogno.

Dominanza diagonale

Sia il sistema $a_i u_{i-1} + b_i u_i + c_i u_{i+1} = d_i$ con $|b_i| \geq |a_i| + |c_i|$ e $|b_i| > |a_i|$ per ogni i (dominanza diagonale). L'eliminazione in avanti sostituisce b_i con $\beta_i = b_i - a_i c_{i-1}/\beta_{i-1}$. Per induzione, se $|c_{i-1}/\beta_{i-1}| < 1$ allora

$$|\beta_i| \geq |b_i| - |a_i| \left|\frac{c_{i-1}}{\beta_{i-1}}\right| > |b_i| - |a_i| \geq |c_i|,$$

quindi $|c_i/\beta_i| < 1$ e $\beta_i \neq 0$: i pivot non si annullano mai e i moltiplicatori restano minori di uno in modulo, il che impedisce la crescita degli errori di arrotondamento. Per lo schema implicito (6.4) si ha $|b_i| = 1 + 2R > 2R = |a_i| + |c_i|$ per ogni $R > 0$: la condizione è sempre soddisfatta, con margine 1. Per Crank–Nicolson $|b_i| = 2(1+R) > 2R$; per il sistema del termometro (3.7), $|C_{i2}| = 2 = |C_{i1}| + |C_{i3}|$, al limite ma sufficiente. È la stessa proprietà che fa convergere i metodi iterativi del Cap. 4: la dominanza diagonale è ciò che rende "benigne" le matrici delle equazioni di diffusione.

6.5.2 Lo schema ϑ: stabilità, accuratezza e la scelta di Crandall

Lo schema ϑ comprende i tre metodi visti e vale la pena analizzarlo una volta per tutte. Con $s = \sin^2(kh/2) \in [0,1]$ il fattore di amplificazione è

$$G(\vartheta) = \frac{1 - 4(1-\vartheta)Rs}{1 + 4\vartheta Rs}. \tag{6.13}$$

Il numeratore è sempre minore del denominatore, quindi $G < 1$; la condizione $G \geq -1$ equivale a $4Rs(1-2\vartheta) \leq 2$. Se $\vartheta \geq \frac{1}{2}$ il primo membro è non positivo e la condizione vale sempre: *stabilità incondizionata per* $\vartheta \geq \frac{1}{2}$. Se $\vartheta < \frac{1}{2}$ deve essere, nel caso peggiore $s = 1$,

$$R \leq \frac{1}{2(1-2\vartheta)}, \tag{6.14}$$

che per $\vartheta = 0$ restituisce $R \leq \frac{1}{2}$ e per $\vartheta = \frac{1}{4}$ dà $R \leq 1$.

Errore di troncamento e schema di Crandall

Si sviluppi attorno al punto intermedio $(y_i, t_{j+1/2})$. La differenza temporale centrata è $u_t + \mathcal{O}(\tau^2)$; la media pesata delle derivate seconde discrete vale $\vartheta\,\delta^2 u^{j+1} + (1-\vartheta)\,\delta^2 u^j = \delta^2 u^{j+1/2} + (\vartheta - \frac{1}{2})\tau\,\delta^2 u_t + \mathcal{O}(\tau^2)$, e $\delta^2 u/h^2 = u_{yy} + \frac{h^2}{12} u_{yyyy} + \mathcal{O}(h^4)$. Il residuo della soluzione esatta nello schema è dunque, usando $u_{yyt} = \nu u_{yyyy}$,

$$E = -\frac{\nu h^2}{12}\left[1 + 12\left(\vartheta - \tfrac{1}{2}\right) R\right] u_{yyyy} + \mathcal{O}\left(\tau^2, h^4\right).$$

Per $\vartheta = 0$ si ritrova la (5.9); per $\vartheta = \frac{1}{2}$ resta il solo errore spaziale $\mathcal{O}(h^2)$, con il tempo al secondo ordine. Il termine principale si annulla per

$$\vartheta = \frac{1}{2} - \frac{1}{12R} \qquad (R \geq \tfrac{1}{6}),$$

che è la *scelta di Crandall*: uno schema implicito accurato al quarto ordine in h (e al secondo in τ). Poiché $1 - 2\vartheta = 1/(6R)$, la condizione di stabilità (6.14) diventa $R \leq 3R$, sempre vera: lo schema di Crandall è incondizionatamente stabile. Per $R = 1/6$ coincide con lo schema esplicito, per $R \to \infty$ tende a Crank–Nicolson.

6.5.3 Il rimedio alle oscillazioni di Crank–Nicolson

Le oscillazioni osservate con $R = 10$ e dato discontinuo si spiegano con la (6.12): per $R \gg 1$ e onde corte $G \to -1$, e il modo a dente di sega contenuto nella discontinuità iniziale si propaga senza smorzarsi cambiando segno a ogni passo. Il metodo implicito ha invece $G \to 0$ per gli stessi modi. Un rimedio semplice ed efficace, dovuto a Rannacher [25], consiste nell'eseguire i primi *due mezzi passi* con il metodo implicito e proseguire con Crank–Nicolson: le componenti corte vengono smorzate del fattore $[1/(1+2R)]^2$ nel primo passo, e dopo di esse resta un dato iniziale regolare, su cui Crank–Nicolson è accurato al secondo ordine. Il costo è nullo. Il Programma 3.5b, ripetendo il caso della lastra con questo avvio, dà:

schema (5 passi, $R = 10$)	$\max u$ nei nodi interni	errore massimo
implicito	0,9137	$2{,}75 \times 10^{-2}$
Crank–Nicolson	1,0699	$1{,}50 \times 10^{-1}$
Crank–Nicolson con avvio di Rannacher	0,9178	$2{,}52 \times 10^{-3}$

L'avvio di Rannacher elimina il valore non fisico 1,07 e riduce l'errore di sessanta volte rispetto a Crank–Nicolson puro e di dieci rispetto al metodo implicito. È la soluzione adottata nei codici di calcolo per gli avviamenti bruschi e, con lo stesso nome, nella finanza computazionale per le equazioni paraboliche con dati non regolari.

6.5.4 Due dimensioni: il metodo ADI

Se la corrente non stazionaria dipende da due coordinate trasversali, come nell'avviamento in un condotto di sezione qualsiasi, l'equazione è $\partial W/\partial T = \nabla^2 W$ nella sezione. Lo schema esplicito ha il vincolo $R \le 1/4$ (ogni direzione contribuisce $4Rs$ al fattore di amplificazione); lo schema implicito o Crank–Nicolson richiedono a ogni passo la soluzione di un sistema con la matrice a cinque diagonali del Cap. 4, che non è più tridiagonale. Il metodo delle *direzioni alternate* (ADI, Peaceman e Rachford [24]) recupera la struttura tridiagonale dividendo il passo in due metà:

$$\begin{aligned} \frac{W^{*} - W^{n}}{\tau/2} &= \delta_x^2 W^{*} + \delta_y^2 W^{n} \qquad \text{(implicito in } x\text{, esplicito in } y\text{)}, \\ \frac{W^{n+1} - W^{*}}{\tau/2} &= \delta_x^2 W^{*} + \delta_y^2 W^{n+1} \qquad \text{(esplicito in } x\text{, implicito in } y\text{)}, \end{aligned} \tag{6.15}$$

dove δ_x^2 e δ_y^2 sono le differenze centrate seconde divise per h^2. Il primo mezzo passo richiede un sistema tridiagonale per ogni riga della griglia, il secondo uno per ogni colonna: il costo per passo è $\mathcal{O}(N^2)$, come per lo schema esplicito.

Stabilità e accuratezza del metodo ADI

Con $W = G^n e^{\mathrm{i}(k_x x + k_y y)}$, $s_x = \sin^2(k_x h/2)$, $s_y = \sin^2(k_y h/2)$ e $r = \tau/h^2$, il primo mezzo passo ha fattore $(1 - 2rs_y)/(1 + 2rs_x)$ e il secondo $(1 - 2rs_x)/(1 + 2rs_y)$; il prodotto è

$$G = \frac{(1 - 2rs_x)(1 - 2rs_y)}{(1 + 2rs_x)(1 + 2rs_y)}, \qquad |G| \le 1 \ \text{ per ogni } r > 0,$$

perché ciascuna frazione $|1 - a|/(1 + a)$ con $a \ge 0$ non supera uno. Lo schema è incondizionatamente stabile. Eliminando W^{*} dalle (6.15) si trova che lo schema coincide con Crank–Nicolson a meno di un termine $\frac{\tau^2}{4}\delta_x^2\delta_y^2(W^{n+1} - W^{n})$, che è $\mathcal{O}(\tau^3)$: l'accuratezza

è del secondo ordine nel tempo. Si noti che i singoli mezzi passi *non* sono accurati né stabili da soli: è la coppia a esserlo.

Il Programma 3.5c applica il metodo all'avviamento della corrente in un condotto quadrato, per il quale la soluzione esatta è una serie doppia di Fourier (l'analogo bidimensionale della (6.10)), e separa l'errore temporale da quello spaziale confrontando lo schema con la soluzione esatta del problema *semidiscreto* (5.10), ottenuta con gli autovettori della matrice tridiagonale.


prog3_5c_adi.py

```
"""
prog3_5c_adi.py -- equazione della diffusione in due dimensioni: metodo ADI
(Peaceman-Rachford) confrontato con lo schema esplicito.

Problema: avviamento della corrente in un condotto quadrato 0 <= x, y <= 1
per un gradiente di pressione applicato bruscamente. Con U = u/[-(dp/dx) L^2/mu]
e T = nu t/L^2, la differenza W = U_s - U fra il profilo di regime e quello
attuale soddisfa  W_T = W_xx + W_yy,  W = 0 sulle pareti,  W(x, y, 0) = U_s,
con U_s soluzione di  lap U_s = -1  (serie doppia di Fourier):
    U_s = sum_{m,n dispari} 16/(pi^4 m n (m^2+n^2)) sin(m pi x) sin(n pi y),
    W   = sum_{m,n dispari} 16/(pi^4 m n (m^2+n^2)) sin(m pi x) sin(n pi y)
    exp[-(m^2+n^2) pi^2 T].

Schema ADI: ogni passo e' diviso in due mezzi passi, implicito in x (poi in y)
ed esplicito nell'altra direzione; ogni mezzo passo richiede solo sistemi
tridiagonali lungo le righe (o le colonne). E' incondizionatamente stabile e
del secondo ordine nel tempo.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import thomas
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO


def serie(x, y, T, nmax=99):
    X, Y = np.meshgrid(x, y, indexing="ij")
    W = np.zeros_like(X)
    for m in range(1, nmax + 1, 2):
        for n in range(1, nmax + 1, 2):
            W += 16 / (np.pi**4 * m * n * (m * m + n * n)) * np.sin(m * np.pi * X)
    * np.sin(n * np.pi * Y) \
                * np.exp(-(m * m + n * n) * np.pi**2 * T)
    return W


def adi(W, h, tau, passi):
    """Peaceman-Rachford: (1 - r/2 d_xx) W* = (1 + r/2 d_yy) W^n,
                          (1 - r/2 d_yy) W^(n+1) = (1 + r/2 d_xx) W*."""
    n = W.shape[0] - 1
    r = tau / h**2
    a = np.full(n - 1, -r / 2)
    b = np.full(n - 1, 1 + r)
    for _ in range(passi):
        # primo mezzo passo: implicito in x, riga per riga (y fissato)
        Ws = np.zeros_like(W)
        rhs = W[1:-1, 1:-1] + r / 2 * (W[1:-1, 2:] - 2 * W[1:-1, 1:-1] + W[1:-1,
    :-2])
        for j in range(n - 1):
            Ws[1:-1, j + 1] = thomas(a, b, a, rhs[:, j])
        # secondo mezzo passo: implicito in y, colonna per colonna (x fissato)
```

```
        Wn = np.zeros_like(W)
        rhs = Ws[1:-1, 1:-1] + r / 2 * (Ws[2:, 1:-1] - 2 * Ws[1:-1, 1:-1] + Ws[:-2, 1:-1])
        for i in range(n - 1):
            Wn[i + 1, 1:-1] = thomas(a, b, a, rhs[i, :])
        W = Wn
    return W


def esplicito(W, h, tau, passi):
    r = tau / h**2
    for _ in range(passi):
        W = W.copy()
        W[1:-1, 1:-1] += r * (W[2:, 1:-1] + W[:-2, 1:-1] + W[1:-1, 2:] + W[1:-1, :-2] - 4 * W[1:-1, 1:-1])
    return W


def semidiscreta(W0, h, T):
    """Soluzione esatta del problema semidiscreto (differenze centrate in x e y, tempo continuo): W(T) = V [ (V^T W0 V) * exp((lam_i + lam_j) T) ] V^T, con V autovettori (seni discreti) e lam autovalori del laplaciano discreto 1D."""
    n = W0.shape[0] - 1
    i = np.arange(1, n)
    lam = -4 / h**2 * np.sin(i * np.pi * h / 2)**2
    V = np.sqrt(2 * h) * np.sin(np.outer(i, i) * np.pi * h)      # ortonormale
    C = V.T @ W0[1:-1, 1:-1] @ V
    W = np.zeros_like(W0)
    W[1:-1, 1:-1] = V @ (C * np.exp(np.add.outer(lam, lam) * T)) @ V.T
    return W


n = 40
h = 1.0 / n
x = np.linspace(0, 1, n + 1)
W0 = serie(x, x, 0.0)
T_fin = 0.1
print(f"Condotto quadrato, griglia {n+1}x{n+1}, T = {T_fin}; U_s al centro = {W0[n//2, n//2]:.6f} (esatto 0,073671)")
W_es = serie(x, x, T_fin)
W_sd = semidiscreta(W0, h, T_fin)
print(f"   errore di discretizzazione spaziale (semidiscreta - esatta): {np.max(np.abs(W_sd - W_es)):.3e}")
print("   schema       tau       r=tau/h^2   passi   err. temporale   ordine   err. totale")
err_prec = None
for tau in (0.02, 0.01, 0.005, 0.0025, 0.00125):
    passi = int(round(T_fin / tau))
    W = adi(W0, h, tau, passi)
    err = np.max(np.abs(W - W_sd))
    ordine = f"{np.log2(err_prec / err):.2f}" if err_prec else "--"
    print(f"   ADI        {tau:8.5f}   {tau/h**2:8.2f}  {passi:6d}   {err:.3e}   {ordine:>5s}    {np.max(np.abs(W - W_es)):.3e}")
    err_prec = err
for r in (0.25, 0.26):
    tau = r * h * h
    passi = int(round(T_fin / tau))
    W = esplicito(W0, h, tau, passi)
    err = np.max(np.abs(W - W_sd))
```

```
    print(f"   esplicito  {tau:8.5f}   {r:8.2f}  {passi:6d}   {err:.3e}" + ("
    (instabile)" if err > 1 else f"                {np.max(np.abs(W -
    W_es)):.3e}"))
# errore spaziale: raffinando la griglia con tau = h/4 (r = n/4, grande)
print("\nConvergenza spaziale con tau = h/4 (ADI, errore rispetto alla serie
    esatta):")
err_prec = None
for nn in (10, 20, 40, 80):
    hh = 1.0 / nn
    xx = np.linspace(0, 1, nn + 1)
    tau = hh / 4
    W = adi(serie(xx, xx, 0.0), hh, tau, int(round(T_fin / tau)))
    err = np.max(np.abs(W - serie(xx, xx, T_fin)))
    print(f"   n = {nn:3d}: r = {tau/hh**2:5.1f}, errore max {err:.3e}" + (f",
    ordine {np.log2(err_prec/err):.2f}" if err_prec else ""))
    err_prec = err

# velocita' al centro e portata in funzione del tempo
Ts = np.linspace(0, 0.6, 61)
Uc = [W0[n//2, n//2] - serie(np.array([0.5]), np.array([0.5]), T)[0, 0] for T in Ts]
Q0 = np.sum(W0) * h * h
Q = [Q0 - np.sum(serie(x, x, T)) * h * h for T in Ts]
Uc = np.array(Uc) / W0[n//2, n//2]
T90, T99 = np.interp(0.9, Uc, Ts), np.interp(0.99, Uc, Ts)
print(f"\nVelocita' al centro: 90% del regime per T = {T90:.3f}, 99% per T =
    {T99:.3f} (T = nu t/L^2, L = lato);")
print(f"   con la semi-larghezza L/2 come nel canale piano: {4*T90:.3f} e
    {4*T99:.3f}"
      f" (canale piano 0,947 e 1,881; tubo circolare 0,417 e 0,814)")

fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.8))
for T, col in zip((0.005, 0.02, 0.05, 0.1, 0.3), [GRIGIO, VERDE, ARANCIO, ROSSO,
    BLU]):
    U = W0 - adi(W0, h, 0.0025, int(round(T / 0.0025)))
    ax[0].plot(x, U[:, n // 2], color=col, label=f"$T={T:g}$".replace(".", "{,}"))
ax[0].plot(x, W0[:, n // 2], "k--", lw=1.0, label="regime")
ax[0].set_xlabel("$x$ (mezzeria $y=0{,}5$)")
ax[0].set_ylabel("$U$")
ax[0].legend(fontsize=6.5)
ax[0].set_title("avviamento (ADI)")
ax[1].plot(Ts, Uc, color=BLU, label="centro")
ax[1].plot(Ts, np.array(Q) / Q0, "--", color=ARANCIO, label="portata")
ax[1].set_xlabel("$T=\\nu t/L^2$")
ax[1].set_ylabel("frazione del regime")
ax[1].legend(fontsize=7)
taus = np.array([0.02, 0.01, 0.005, 0.0025, 0.00125])
errs = [np.max(np.abs(adi(W0, h, t, int(round(T_fin / t))) - W_sd)) for t in taus]
ax[2].loglog(taus, errs, "o-", color=BLU, label="ADI")
ax[2].loglog(taus, errs[0] * (taus / taus[0])**2, ":", color=GRIGIO,
    label="$\\tau^2$")
ax[2].set_xlabel("$\\tau$")
ax[2].set_ylabel("errore temporale")
ax[2].legend(fontsize=7)
ax[2].set_title("ordine nel tempo")
fig.tight_layout()
salva(fig, "prog3_5c_adi")
```

Listato 6.3: `prog3_5c_adi.py`: avviamento della corrente in un condotto quadrato con il metodo delle direzioni alternate.

Risultati del Programma 3.5c

Griglia 41×41, $T = 0{,}1$; errore di discretizzazione spaziale (semidiscreto meno esatto) $1{,}2 \times 10^{-5}$.

schema	τ	$r = \tau/h^2$	passi	errore temporale	ordine
ADI	0,02	32	5	$9{,}92 \times 10^{-5}$	–
ADI	0,01	16	10	$1{,}80 \times 10^{-5}$	2,46
ADI	0,005	8	20	$4{,}49 \times 10^{-6}$	2,01
ADI	0,0025	4	40	$1{,}12 \times 10^{-6}$	2,00
ADI	0,001 25	2	80	$2{,}80 \times 10^{-7}$	2,00
esplicito	0,000 16	0,25	640	$3{,}47 \times 10^{-5}$	–
esplicito	0,000 16	0,26	615	2×10^{10}	instabile

Con 20 passi ADI l'errore temporale è già inferiore a quello spaziale; lo schema esplicito ne richiede 640 e diventa instabile appena oltre $r = 1/4$. Raffinando la griglia con $\tau = h/4$ (quindi $r = n/4$, fino a 20) l'errore totale decresce come h^2 (ordini 2,10; 2,01; 1,99). Il centro del condotto quadrato raggiunge il 90% della velocità di regime per $T = 0{,}122$ e il 99% per $T = 0{,}239$ (con $T = \nu t/L^2$, L lato); riferendo il tempo alla semi-larghezza, 0,49 e 0,96: fra il tubo circolare (0,42 e 0,81, Esercizio 12) e il canale piano (0,95 e 1,88), come ci si aspetta da una sezione che ha "più parete" del canale e meno del tubo.

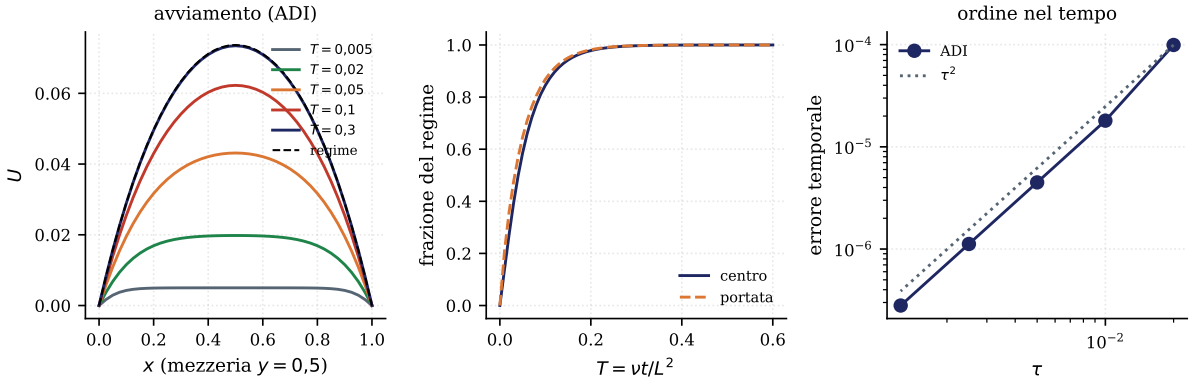


Figura 6.4: Avviamento della corrente in un condotto quadrato con il metodo ADI: profili lungo la mezzeria (a sinistra), velocità al centro e portata (al centro), ordine di convergenza nel tempo (a destra).

In tre dimensioni la versione di Peaceman–Rachford non è più incondizionatamente stabile e si usa la variante di Douglas–Gunn; i codici moderni preferiscono spesso trattare il laplaciano implicito con metodi iterativi (gradiente coniugato precondizionato, multigriglia), ma l'idea di *fattorizzare* un operatore multidimensionale in operatori monodimensionali resta uno strumento fondamentale, e ricompare nei metodi a passi frazionari per le equazioni di Navier–Stokes.

Gli Esercizi 12 e 18 estendono il metodo alla geometria cilindrica: l'avviamento della corrente in un tubo circolare e la corrente pulsante prodotta da un gradiente di pressione alternato, con la soluzione di Womersley che collega questo capitolo all'emodinamica del modulo precedente.

Capitolo 7

Flussi di Stokes e l'equazione biarmonica

7.1 Il limite dei piccoli numeri di Reynolds

Nei tre capitoli precedenti i termini non lineari dell'equazione di Navier–Stokes sono scomparsi grazie alla geometria. Lo stesso risultato si ottiene, in una geometria qualsiasi, quando il numero di Reynolds è molto piccolo. Nella forma adimensionale (1.13),

$$\frac{\mathrm{D}\mathbf{V}}{\mathrm{D}t} = -\nabla p + \frac{1}{Re}\nabla^2\mathbf{V},$$

il secondo termine a destra domina sul primo membro quando $Re \ll 1$. Conviene allora riscalare la pressione sugli sforzi viscosi, $p \sim \mu U/L$ invece di ρU^2; trascurando l'inerzia e tornando alle grandezze dimensionali si ottiene, per il moto stazionario, l'equazione *lineare*

$$\mu\,\nabla^2\mathbf{V} = \nabla p, \qquad \nabla\cdot\mathbf{V} = 0. \tag{7.1}$$

Le correnti con $Re \ll 1$ si chiamano **flussi di Stokes** o di scorrimento lento (*creeping flows*): il colare del miele, la sedimentazione delle polveri fini, il nuoto dei microrganismi, il film d'olio nei cuscinetti, la circolazione nei capillari. La soluzione più celebre è quella della sfera di raggio a in moto lento, la cui resistenza vale $6\pi\mu a U$ (legge di Stokes), utilizzata nel primo modulo per la caduta delle goccioline.

Prendendo il rotore della (7.1) la pressione scompare e, poiché $\nabla\times\nabla^2\mathbf{V} = \nabla^2(\nabla\times\mathbf{V})$,

$$\nabla^2\zeta = 0; \tag{7.2}$$

prendendo invece la divergenza, con $\nabla\cdot\mathbf{V}=0$,

$$\nabla^2 p = 0\,: \tag{7.3}$$

in un flusso di Stokes sia la vorticità sia la pressione sono funzioni armoniche.

Reversibilità e simmetria

L'equazione (7.1) è lineare e non contiene il tempo. Se $(\mathbf{V}, p)$ è una soluzione, lo è anche $(-\mathbf{V}, -p)$: invertendo il moto delle pareti si inverte esattamente il moto del fluido, che ripercorre le stesse traiettorie all'indietro. È la celebre reversibilità dei flussi di Stokes (un colorante mescolato lentamente in un fluido molto viscoso si "smescola" invertendo

il moto) e il motivo per cui un microrganismo non può nuotare con un moto reciproco, come chiudere e riaprire una conchiglia. Una conseguenza geometrica si vedrà più avanti: la corrente nella cavità mossa da una parete è simmetrica rispetto alla mezzeria.

7.2 L'equazione biarmonica

Per un moto piano parallelo al piano x–y si introduce la funzione di corrente, $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$. La sola componente non nulla della vorticità è legata a ψ dalla (1.15), $\nabla^2\psi = -\zeta$, e la (7.2) diventa $\nabla^2\zeta = 0$. Combinandole:

$$\nabla^4\psi = 0, \qquad \nabla^4 = \nabla^2\nabla^2 = \frac{\partial^4}{\partial x^4} + 2\frac{\partial^4}{\partial x^2\partial y^2} + \frac{\partial^4}{\partial y^4}. \tag{7.4}$$

Poiché l'equazione di Laplace si chiama anche equazione armonica, la (7.4) è detta **equazione biarmonica**; la stessa equazione governa la funzione di sforzo di Airy nell'elasticità piana e l'inflessione delle piastre sottili.

La soluzione numerica sembra semplice: la biarmonica si scrive come coppia di equazioni di Poisson,

$$\nabla^2\psi = -\zeta, \qquad \nabla^2\zeta = 0, \tag{7.5}$$

a ciascuna delle quali si applicano i metodi iterativi del modulo precedente. La difficoltà sta nelle **condizioni al contorno**. Sui bordi del dominio si conoscono le velocità, cioè ψ e la sua derivata normale: *due* condizioni per ψ e *nessuna* per ζ. La vorticità alla parete non è un dato, ma un risultato del calcolo. Servono tecniche speciali per trasferire la condizione in eccesso su ψ in una condizione sulla vorticità.

7.3 La cavità mossa da una parete

Un esempio di geometria semplicissima mostra come si affronta il problema. Una cavità quadrata $DABC$ contiene un fluido messo in moto da una lastra indefinita che scorre sopra di essa (Fig. 7.1). Le grandezze sono adimensionali: la cavità ha lato 1, la lastra si muove con velocità 1 verso x positivo, e il numero di Reynolds è così piccolo che il moto è un flusso di Stokes.

Segnaposto figura

file atteso: `immagini/fig_3_7_1.png`

Cavità quadrata $DABC$ di lato 1 con parete superiore CD in moto verso destra con velocità 1; griglia con m linee in ciascuna direzione e nodi (i, m), $(i \pm 1, m-1)$, $(i, m-1)$, $(i, m-2)$ usati per la vorticità di parete; condizioni $\psi = 0$, $\partial\psi/\partial y = 1$ su CD, $\psi = 0$ e derivata normale nulla sulle altre pareti (Fig. 3.7.1 della fonte).

Figura 7.1: Corrente in una cavità prodotta da una lastra mobile.

Se sotto la lastra il fluido non esce dalla cavità, le linee di corrente sono chiuse e le quattro pareti appartengono alla linea di corrente $\psi = 0$: le velocità normali sono nulle. Imponendo

anche le condizioni sulle velocità tangenziali, nulle tranne che sulla lastra dove valgono 1:

$$\begin{aligned} &\psi = 0,\ \frac{\partial\psi}{\partial x} = 0 \ \text{ su } DA \text{ e } BC; \qquad \psi = 0,\ \frac{\partial\psi}{\partial y} = 0 \ \text{ su } AB; \\ &\psi = 0,\ \frac{\partial\psi}{\partial y} = 1 \ \text{ su } CD. \end{aligned} \tag{7.6}$$

7.3.1 La vorticità di parete

La regione si ricopre con una griglia quadrata di passo h con m linee orizzontali e m verticali. Consideriamo un nodo (i,m) sulla lastra mobile. Vogliamo calcolare la vorticità in quel nodo dalla velocità locale e dai valori di ψ in quattro nodi vicini (Fig. 7.1), cercando una formula del tipo

$$\zeta_{i,m} = -\left(\frac{\partial^2\psi}{\partial x^2} + \frac{\partial^2\psi}{\partial y^2}\right)_{i,m} = \alpha_1\psi_{i-1,m-1} + \alpha_2\psi_{i,m-1} + \alpha_3\psi_{i+1,m-1} + \alpha_4\psi_{i,m-2} + \alpha_5\left(\frac{\partial\psi}{\partial y}\right)_{i,m}. \tag{7.7}$$

Derivazione della formula di parete

Si sviluppano in serie di Taylor attorno a (i,m) i valori nei nodi vicini, fino ai termini $\mathcal{O}(h^2)$:

$$\begin{aligned} \psi_{i\pm1,m-1} &= \psi \pm h\psi_x - h\psi_y + \tfrac{h^2}{2}\psi_{xx} \mp h^2\psi_{xy} + \tfrac{h^2}{2}\psi_{yy} + \mathcal{O}(h^3), \\ \psi_{i,m-n} &= \psi - nh\psi_y + \tfrac{1}{2}n^2h^2\psi_{yy} + \mathcal{O}(h^3), \qquad n = 1,2, \end{aligned}$$

con tutte le derivate calcolate in (i,m). Sostituendo nella (7.7) e uguagliando i coefficienti dei termini simili ai due membri, si ottiene un sistema per le α:

$$\begin{aligned} \psi:&\ \alpha_1+\alpha_2+\alpha_3+\alpha_4 = 0, & \psi_x:&\ (\alpha_3-\alpha_1)h = 0, \\ \psi_{xx}:&\ (\alpha_1+\alpha_3)\tfrac{h^2}{2} = -1, & \psi_{yy}:&\ (\alpha_1+\alpha_2+\alpha_3+4\alpha_4)\tfrac{h^2}{2} = -1, \\ \psi_y:&\ \alpha_5 - h(\alpha_1+\alpha_2+\alpha_3+2\alpha_4) = 0. \end{aligned}$$

(Il termine in ψ_{xy} si cancella da solo, perché $\alpha_1 = \alpha_3$.) Dalla seconda e dalla terza $\alpha_1 = \alpha_3 = -1/h^2$; dalla prima $\alpha_2 + 4\alpha_4 = 2/h^2$; dalla quarta $\alpha_2 + 4\alpha_4 = 0$, quindi $\alpha_4 = -2/(3h^2)$, $\alpha_2 = 8/(3h^2)$; infine $\alpha_5 = -2/(3h)$. La formula è

$$\zeta_{i,m} = \frac{1}{h^2}\left(-\psi_{i-1,m-1} + \frac{8}{3}\psi_{i,m-1} - \psi_{i+1,m-1} - \frac{2}{3}\psi_{i,m-2}\right) - \frac{2}{3h}\left(\frac{\partial\psi}{\partial y}\right)_{i,m}, \tag{7.8}$$

con $(\partial\psi/\partial y)_{i,m} = 1$ sulla lastra mobile. Per analogia, sulle pareti ferme (dove la derivata normale è nulla):

$$\begin{aligned} \zeta_{1,j} &= \frac{1}{h^2}\left(-\psi_{2,j-1} + \frac{8}{3}\psi_{2,j} - \psi_{2,j+1} - \frac{2}{3}\psi_{3,j}\right), \\ \zeta_{m,j} &= \frac{1}{h^2}\left(-\psi_{m-1,j-1} + \frac{8}{3}\psi_{m-1,j} - \psi_{m-1,j+1} - \frac{2}{3}\psi_{m-2,j}\right), \\ \zeta_{i,1} &= \frac{1}{h^2}\left(-\psi_{i-1,2} + \frac{8}{3}\psi_{i,2} - \psi_{i+1,2} - \frac{2}{3}\psi_{i,3}\right). \end{aligned} \tag{7.9}$$

Una formula più semplice, molto usata, si ottiene sviluppando solo lungo la normale alla parete. Sia n la coordinata normale diretta verso il fluido e ψ_1 il valore nel primo nodo interno.

Da $\psi_1 = \psi_w + h\,(\partial\psi/\partial n)_w + \frac{1}{2}h^2(\partial^2\psi/\partial n^2)_w + \ldots$, con $\psi_w = 0$ e $\zeta_w = -(\partial^2\psi/\partial n^2)_w$ (lungo la parete ψ è costante e la derivata seconda tangenziale è nulla), si ricava la *formula di Thom*

$$\zeta_w = -\frac{2}{h^2}\left[\psi_1 - h\left(\frac{\partial\psi}{\partial n}\right)_w\right]. \tag{7.10}$$

Sulla lastra mobile la normale verso il fluido è orientata come $-y$, quindi $(\partial\psi/\partial n)_w = -1$ e $\zeta_w = -2\psi_{i,m-1}/h^2 - 2/h$; sulle pareti ferme $\zeta_w = -2\psi_1/h^2$. È localmente meno accurata, ma il suo effetto sulla soluzione è sorprendentemente simile, come si vedrà.

7.3.2 L'algoritmo iterativo

Le condizioni di parete (7.8)–(7.9) permettono di risolvere $\nabla^2\zeta = 0$ se ψ è noto nei nodi interni; ma per calcolare ψ da $\nabla^2\psi = -\zeta$ serve la vorticità interna. Le due equazioni sono accoppiate attraverso il contorno, e la soluzione si costruisce per iterazioni.

Iterazione vorticità–funzione di corrente

1. Si parte dalla quiete: $\psi = 0$ e $\zeta = 0$ ovunque.
2. Con le (7.8)–(7.9) si calcola la vorticità di parete. Alla prima iterazione è diversa da zero solo sulla lastra mobile: la vorticità *nasce* lì.
3. Si risolve $\nabla^2\zeta = 0$ con queste condizioni (metodo di Liebmann): la vorticità "diffonde" nella cavità. Si registra ERZETA, la somma dei valori assoluti delle variazioni di ζ in tutti i nodi.
4. Si risolve $\nabla^2\psi = -\zeta$ con $\psi = 0$ sul contorno, e si registra analogamente ERPSI.
5. Se ERZETA ed ERPSI sono entrambi minori di una tolleranza EPSLON ci si ferma; altrimenti si torna al punto 2 con il nuovo ψ.
6. La vorticità nei quattro spigoli, che non interviene nel calcolo, si ricava a posteriori come media dei due valori adiacenti.

Le due equazioni di Poisson si risolvono con la stessa subroutine LIEBMN, che applica la formula di Liebmann (Gauss–Seidel) ai nodi interni di un dominio quadrato finché la somma delle variazioni non scende sotto una tolleranza. Come nell'originale, la subroutine riparte ogni volta da valori interni nulli; è una scelta semplice ma inefficiente, che si potrebbe migliorare partendo dalla soluzione dell'iterazione precedente.

Dati del Programma 3.6

$m = 11$ ($h = 0{,}1$, 121 nodi); EPSLON= 10^{-3}; tolleranza delle iterazioni di Liebmann 10^{-3}; fattore di rilassamento delle iterazioni esterne RF= 1 (nessun rilassamento).

```
"""
prog3_6_cavita_stokes.py -- PROGRAMMA 3.6
Flusso di Stokes in una cavita' quadrata 1 x 1 mossa dalla parete superiore
(velocita' 1 verso x positivo). Equazione biarmonica scritta come coppia
    lap(psi) = -zeta,    lap(zeta) = 0,
con psi = 0 su tutto il contorno e vorticita' di parete calcolata da psi con
le formule a cinque punti (accuratezza O(h^2)), per esempio sulla parete mobile
    zeta_(i,m) = [ -psi_(i-1,m-1) + 8/3 psi_(i,m-1) - psi_(i+1,m-1) - 2/3
    psi_(i,m-2) ]/h^2
                - 2/(3h) (dpsi/dy)_(i,m),      (dpsi/dy) = 1.
```

prog3_6_cavita_stokes.py

```
Iterazione esterna: (1) vorticita' di parete da psi; (2) Liebmann per zeta;
(3) Liebmann per psi; finche' le somme delle variazioni ERZETA ed ERPSI non
scendono sotto EPSLON. Come nella subroutine originale, ogni chiamata di
Liebmann riparte da valori interni nulli.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import liebmann
from stile_grafici import salva, BLU, ROSSO, GRIGIO

M = 11
H = 1.0 / (M - 1)
EPSLON = 1.0e-3
RF = 1.0                                    # fattore di rilassamento (1: nessuno)

psi = np.zeros((M, M))                      # psi[i, j]: i -> x, j -> y
zeta = np.zeros((M, M))


def vorticita_parete(psi, zeta):
    i = np.arange(1, M - 1)
    j = np.arange(1, M - 1)
    zeta[0, j] = (-psi[1, j + 1] + 8 / 3 * psi[1, j] - psi[1, j - 1] - 2 / 3 *
    psi[2, j]) / H**2
    zeta[M - 1, j] = (-psi[M - 2, j + 1] + 8 / 3 * psi[M - 2, j] - psi[M - 2, j - 1]
                      - 2 / 3 * psi[M - 3, j]) / H**2
    zeta[i, 0] = (-psi[i + 1, 1] + 8 / 3 * psi[i, 1] - psi[i - 1, 1] - 2 / 3 *
    psi[i, 2]) / H**2
    zeta[i, M - 1] = ((-psi[i + 1, M - 2] + 8 / 3 * psi[i, M - 2] - psi[i - 1, M -
    2]
                       - 2 / 3 * psi[i, M - 3]) / H**2 - 2 / (3 * H) * 1.0)


iterazioni = 0
erzeta = erpsi = 1.0
storia = []
while erzeta > EPSLON or erpsi > EPSLON:
    iterazioni += 1
    vorticita_parete(psi, zeta)
    zeta_vecchia = zeta.copy()
    zeta, _ = liebmann(zeta, np.zeros((M, M)), H, 0.001)
    zeta = RF * zeta + (1 - RF) * zeta_vecchia
    erzeta = np.sum(np.abs(zeta - zeta_vecchia))
    psi_vecchia = psi.copy()
    psi, _ = liebmann(psi, -zeta, H, 0.001)
    psi = RF * psi + (1 - RF) * psi_vecchia
    erpsi = np.sum(np.abs(psi - psi_vecchia))
    storia.append((iterazioni, erzeta, erpsi))

# vorticita' negli spigoli: media dei due valori adiacenti
zeta[0, 0] = 0.5 * (zeta[0, 1] + zeta[1, 0])
zeta[M - 1, 0] = 0.5 * (zeta[M - 1, 1] + zeta[M - 2, 0])
zeta[0, M - 1] = 0.5 * (zeta[1, M - 1] + zeta[0, M - 2])
zeta[M - 1, M - 1] = 0.5 * (zeta[M - 1, M - 2] + zeta[M - 2, M - 1])

print(f"Funzione di corrente dopo {iterazioni} iterazioni, ERPSI = {erpsi:.4e}")
for j in range(M - 1, -1, -1):
    print("  " + " ".join(f"{psi[i, j]:+.4f}" for i in range(M)))
print(f"\nVorticita', ERZETA = {erzeta:.4e}")
for j in range(M - 1, -1, -1):
```

```
    print("  " + " ".join(f"{zeta[i, j]:+7.3f}" for i in range(M)))
i0, j0 = np.unravel_index(np.argmin(psi), psi.shape)
print(f"\npsi minimo = {psi.min():.5f} nel nodo x = {i0*H:.1f}, y = {j0*H:.1f}")
print(f"asimmetria massima |psi(x) - psi(1-x)| = {np.max(np.abs(psi - psi[::-1,
    :])):.1e}")
print(f"asimmetria massima |zeta(x) - zeta(1-x)| = {np.max(np.abs(zeta - zeta[::-1,
    :])):.1e}")

# ------------------------------------------------------------------ figura
x = np.linspace(0, 1, M)
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.4))
livelli = [-0.090, -0.075, -0.060, -0.045, -0.030, -0.015, 0.0]
cs = ax[0].contour(x, x, psi.T, levels=livelli, colors=BLU, linewidths=1.0,
    linestyles="solid")
ax[0].clabel(cs, fmt=lambda v: f"{v:.3f}".replace(".", ","), fontsize=6)
ax[0].set_title(r"funzione di corrente $\psi$ (griglia $11\times 11$)")
cz = ax[1].contour(x, x, zeta.T, levels=np.arange(-7, 2, 1), colors=ROSSO,
    linewidths=0.9,
                   linestyles="solid")
ax[1].clabel(cz, fmt="%d", fontsize=6)
ax[1].set_title(r"vorticita' $\zeta$")
for a in ax:
    a.set_aspect("equal")
    a.set_xlabel("$x$")
    a.set_ylabel("$y$")
    a.annotate("", xy=(0.7, 1.0), xytext=(0.3, 1.0),
               arrowprops=dict(arrowstyle="->", color="k", lw=1.2),
    annotation_clip=False)
    a.set_title(a.get_title(), pad=10)
fig.tight_layout()
salva(fig, "prog3_6_cavita_stokes")

fig, ax = plt.subplots(figsize=(4.0, 2.6))
it = [s[0] for s in storia]
ax.semilogy(it, [s[1] for s in storia], color=ROSSO, label="ERZETA")
ax.semilogy(it, [s[2] for s in storia], color=BLU, label="ERPSI")
ax.axhline(EPSLON, color=GRIGIO, ls=":")
ax.set_xlabel("iterazione esterna")
ax.set_ylabel("somma delle variazioni")
ax.legend()
salva(fig, "prog3_6_convergenza")
```

Listato 7.1: Programma 3.6 – flusso di Stokes in una cavità quadrata mossa dalla parete superiore (`prog3_6_cavita_stokes.py`).

Risultati del Programma 3.6

- Convergenza in **109 iterazioni**; nell'ultima ERPSI$=$ 4,07 $\times 10^{-5}$ ed ERZETA$=$ 9,45 $\times 10^{-4}$. Distribuiti su 121 nodi, sono errori medi davvero piccoli. Le due somme decrescono in modo geometrico, con lo stesso rapporto a ogni iterazione (Fig. 7.3).
- La funzione di corrente è ovunque negativa o nulla; il minimo, $\psi_{\min} = -0{,}0933$, si trova sulla mezzeria a $y = 0{,}7$, cioè il centro del vortice è spostato verso la lastra mobile.
- ψ è simmetrica rispetto alla mezzeria entro 3,5 $\times 10^{-5}$; nella vorticità si nota una lieve asimmetria (4,8 $\times 10^{-4}$), dovuta alla tolleranza delle iterazioni e al verso di

scansione del metodo di Gauss–Seidel. Si riduce diminuendo le tolleranze, al prezzo di un tempo di calcolo maggiore.

- La vorticità è massima in valore assoluto sulla lastra mobile vicino agli spigoli superiori ($\zeta \simeq -7{,}5$), dove la velocità passa bruscamente da 1 a 0; sulla parete di fondo è piccola e positiva.

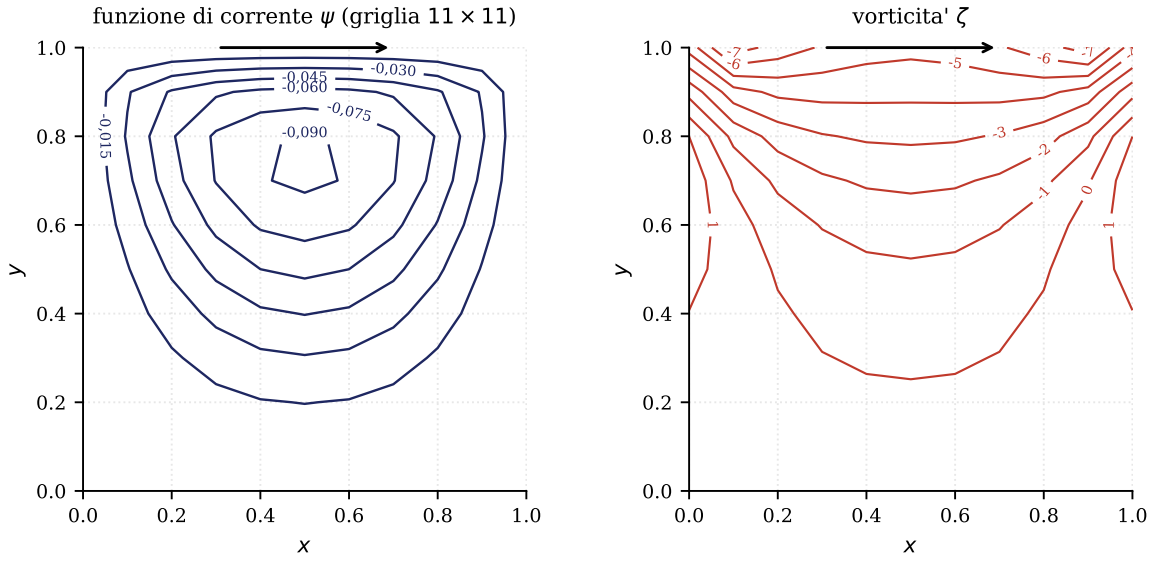


Figura 7.2: Cavità di Stokes su una griglia 11×11: linee di corrente $\psi = -0{,}090, -0{,}075, \ldots, 0$ (a sinistra) e linee di vorticità costante (a destra).

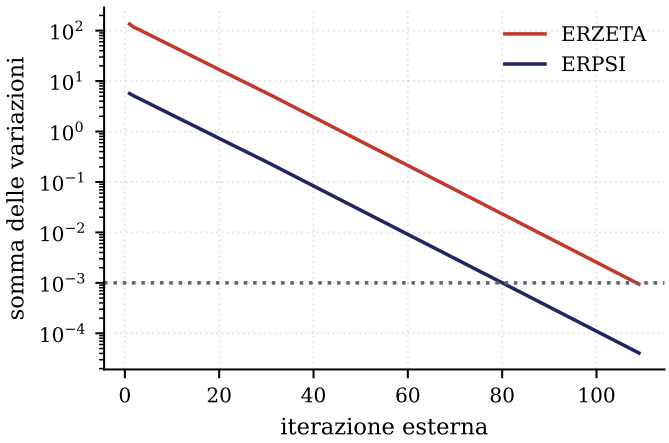


Figura 7.3: Storia della convergenza delle iterazioni esterne del Programma 3.6.

Perché la corrente è simmetrica

Sia $\psi(x,y)$ la soluzione con lastra in moto verso destra, e si consideri $\tilde{\psi}(x,y) = -\psi(1-x,y)$. È ancora biarmonica, è nulla sul contorno, ha derivata normale nulla sulle pareti laterali e sul fondo, e sulla lastra $\partial\tilde{\psi}/\partial y = -1$: è la soluzione con lastra in moto verso *sinistra*. Per la linearità, quella soluzione è $-\psi$. Quindi $-\psi(1-x,y) = -\psi(x,y)$: la funzione di corrente è simmetrica rispetto alla mezzeria $x = \frac{1}{2}$. A numero di Reynolds finito la convezione rompe la simmetria e il centro del vortice si sposta a valle, verso lo

spigolo in cui la lastra "spinge" il fluido contro la parete.

7.4 La soluzione diretta e le griglie fini

Il problema è lineare: le equazioni discrete

$$\nabla_h^2\psi + \zeta = 0, \qquad \nabla_h^2\zeta = 0 \ \text{(nodi interni)}, \qquad \zeta_{\text{parete}} = \text{formula di parete (lineare in } \psi)$$

formano un *unico sistema lineare sparso* nelle incognite ψ (nodi interni) e ζ (nodi interni e di parete). Invece di iterare fra le due equazioni lo si può risolvere con un metodo diretto, che fornisce in una frazione di secondo la soluzione esatta del problema discreto, anche su griglie di centinaia di nodi per lato. Il Programma 3.6b lo fa per le due formule di parete e su griglie sempre più fini.

prog3_6b_cavita_fine.py

```
"""
prog3_6b_cavita_fine.py -- la cavita' di Stokes su griglie fini.

Poiche' il problema e' lineare, le equazioni discrete
    lap_h(psi) + zeta = 0,   lap_h(zeta) = 0   (nodi interni)
    zeta_parete = formula di parete (lineare in psi)
formano UN sistema lineare sparso, risolto qui con un metodo diretto invece
che con le iterazioni annidate di Liebmann. Si confrontano due formule di parete:
  * formula a cinque punti del Programma 3.6 (errore O(h^2));
  * formula di Thom, zeta_w = -2 psi_1/h^2 - 2 U_w/h (ottenuta da Taylor lungo la
    normale).
Si studiano la convergenza di psi_min, l'estrapolazione di Richardson e i
vortici secondari (di Moffatt) negli spigoli inferiori.
"""
import time
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from stile_grafici import salva, BLU, ARANCIO, ROSSO, VERDE, GRIGIO


def risolvi(n, formula="cinque"):
    """Griglia di (n+1) x (n+1) nodi, h = 1/n. Incognite: psi nei nodi interni,
    zeta in tutti i nodi tranne gli spigoli. Restituisce psi, zeta (n+1 x n+1)."""
    h = 1.0 / n
    ip = -np.ones((n + 1, n + 1), dtype=int)        # indici delle incognite psi
    iz = -np.ones((n + 1, n + 1), dtype=int)        # indici delle incognite zeta
    k = 0
    for i in range(1, n):
        for j in range(1, n):
            ip[i, j] = k
            k += 1
    for i in range(n + 1):
        for j in range(n + 1):
            spigolo = (i in (0, n)) and (j in (0, n))
            if not spigolo:
                iz[i, j] = k
                k += 1
    righe, colonne, valori = [], [], []
    b = np.zeros(k)
    r = 0

```

```
    def aggiungi(riga, col, val):
        righe.append(riga)
        colonne.append(col)
        valori.append(val)

    def P(i, j):           # contributo di psi[i,j] (zero se sul contorno)
        return ip[i, j]

    # equazioni nei nodi interni
    for i in range(1, n):
        for j in range(1, n):
            # (1) lap psi + zeta = 0
            aggiungi(r, ip[i, j], -4.0 / h**2)
            for (a, c) in ((i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)):
                if P(a, c) >= 0:
                    aggiungi(r, P(a, c), 1.0 / h**2)
            aggiungi(r, iz[i, j], 1.0)
            r += 1
            # (2) lap zeta = 0
            aggiungi(r, iz[i, j], -4.0)
            for (a, c) in ((i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)):
                aggiungi(r, iz[a, c], 1.0)
            r += 1
    # equazioni di parete per la vorticita'
    pareti = []
    for t in range(1, n):
        pareti.append(((0, t), (1, 0), 0.0))       # parete sinistra, normale verso
    +x
        pareti.append(((n, t), (-1, 0), 0.0))      # parete destra
        pareti.append(((t, 0), (0, 1), 0.0))       # fondo
        pareti.append(((t, n), (0, -1), 1.0))      # coperchio mobile, u = 1
    for (i, j), (ni, nj), uw in pareti:
        aggiungi(r, iz[i, j], 1.0)
        ti, tj = nj, ni                           # direzione tangente
        i1, j1 = i + ni, j + nj                    # primo nodo interno
        i2, j2 = i + 2 * ni, j + 2 * nj            # secondo nodo interno
        if formula == "cinque":
            # zeta = [-psi(t-) + 8/3 psi_1 - psi(t+) - 2/3 psi_2]/h^2 - (2/3h)
    (dpsi/dn)*segno
            for (a, c, w) in ((i1 - ti, j1 - tj, -1.0), (i1, j1, 8 / 3), (i1 + ti,
    j1 + tj, -1.0),
                              (i2, j2, -2 / 3)):
                if P(a, c) >= 0:
                    aggiungi(r, P(a, c), -w / h**2)
            b[r] = -2.0 / (3.0 * h) * uw
        else:                                      # Thom
            aggiungi(r, P(i1, j1), 2.0 / h**2)
            b[r] = -2.0 * uw / h
        r += 1
    A = sp.csr_matrix((valori, (righe, colonne)), shape=(k, k))
    x = spla.spsolve(A, b)
    psi = np.zeros((n + 1, n + 1))
    zeta = np.zeros((n + 1, n + 1))
    m = ip >= 0
    psi[m] = x[ip[m]]
    m = iz >= 0
    zeta[m] = x[iz[m]]
    return psi, zeta


```

```
def minimo_interpolato(psi):
    """Minimo di psi con un paraboloide sui 3x3 nodi attorno al minimo discreto."""
    n = psi.shape[0] - 1
    i, j = np.unravel_index(np.argmin(psi), psi.shape)
    f = psi[i - 1:i + 2, j - 1:j + 2]
    fx = (f[2, 1] - f[0, 1]) / 2
    fy = (f[1, 2] - f[1, 0]) / 2
    fxx = f[2, 1] - 2 * f[1, 1] + f[0, 1]
    fyy = f[1, 2] - 2 * f[1, 1] + f[1, 0]
    fxy = (f[2, 2] - f[2, 0] - f[0, 2] + f[0, 0]) / 4
    Hm = np.array([[fxx, fxy], [fxy, fyy]])
    d = -np.linalg.solve(Hm, [fx, fy])
    val = f[1, 1] + fx * d[0] + fy * d[1] + 0.5 * d @ Hm @ d
    return val, (i + d[0]) / n, (j + d[1]) / n


print("  n     formula     psi_min       x_c       y_c       zeta_c     tempo")
risultati = {"cinque": [], "Thom": []}
griglie = [10, 20, 40, 80, 160]
for formula in ("cinque", "Thom"):
    for n in griglie:
        t0 = time.perf_counter()
        psi, zeta = risolvi(n, formula)
        dt = time.perf_counter() - t0
        val, xc, yc = minimo_interpolato(psi)
        zc = zeta[int(round(xc * n)), int(round(yc * n))]
        risultati[formula].append(val)
        print(f"  {n:3d}   {formula:7s}   {val:.6f}   {xc:.4f}   {yc:.4f}
    {zc:8.4f}   {dt:5.2f} s")

for formula in ("cinque", "Thom"):
    v = risultati[formula]
    ordini = [np.log2(abs(v[k] - v[k - 1]) / abs(v[k + 1] - v[k])) for k in
    range(1, len(v) - 1)]
    rich = v[-1] + (v[-1] - v[-2]) / 3
    print(f"\n{formula}: ordini osservati " + ", ".join(f"{o:.2f}" for o in ordini)
    +
          f";  Richardson (80, 160): psi_min = {rich:.6f}")

# vortici di Moffatt sulla griglia piu' fine
n = 320
psi, zeta = risolvi(n, "cinque")
x = np.linspace(0, 1, n + 1)
basso = psi[: n // 4, : n // 4]
i, j = np.unravel_index(np.argmax(basso), basso.shape)
print(f"\nGriglia {n}x{n}: psi_min = {minimo_interpolato(psi)[0]:.6f}")
print(f"vortice secondario nello spigolo in basso a sinistra: psi_max =
    {basso.max():.3e} "
      f"in x = {x[i]:.4f}, y = {x[j]:.4f}  (rapporto con |psi_min|:
    {basso.max()/abs(psi.min()):.2e})")
# estensione del vortice secondario lungo il fondo (dove psi cambia segno sulla
    riga j = 1)
riga = psi[:, 1]
xs = x[np.nonzero(np.diff(np.sign(riga[1:n // 2])))[0] + 1]
print(f"cambio di segno di psi accanto al fondo: x = {xs[:2]}")

# teoria di Moffatt per uno spigolo retto: psi ~ r^lam f(theta) con f simmetrica
# rispetto alla bisettrice; lam - 1 = z risolve sin(z pi/2) = -z (radice complessa)
from scipy.optimize import fsolve
def _eq(v):
```

```
    z = v[0] + 1j * v[1]
    r = np.sin(z * np.pi / 2) + z
    return [r.real, r.imag]
p, q = fsolve(_eq, [2.7, 1.1])
print(f"\nMoffatt (spigolo di 90 gradi): lam = {p+1:.4f} + {q:.4f} i;  rapporto fra le dimensioni"
      f" di vortici successivi = exp(pi/q) = {np.exp(np.pi/q):.2f};")
print(f"   rapporto fra le intensita' di psi = exp(pi Re(lam)/q) = {np.exp(np.pi*(p+1)/q):.3g},"
      f"  fra le velocita' = {np.exp(np.pi*p/q):.3g}")

# ------------------------------------------------------------------- figura
fig, ax = plt.subplots(1, 3, figsize=(8.0, 2.9))
nn = np.array(griglie)
for formula, col, mk in (("cinque", BLU, "o"), ("Thom", ARANCIO, "s")):
    v = np.array(risultati[formula])
    rif = risultati["cinque"][-1] + (risultati["cinque"][-1] - risultati["cinque"][-2]) / 3
    ax[0].loglog(1 / nn, np.abs(v - rif), mk + "-", color=col, label=f"formula {formula}")
ax[0].loglog(1 / nn, 0.5 * (1 / nn) ** 2, ":", color="k", label="$h^2$")
ax[0].set_xlabel("$h$")
ax[0].set_ylabel(r"$|\psi_{\min}-\psi_{\min,\infty}|$")
ax[0].legend(fontsize=6.5)
lv = np.linspace(psi.min(), 0, 11)[:-1]
ax[1].contour(x, x, psi.T, levels=lv, colors=BLU, linewidths=0.8, linestyles="solid")
ax[1].contour(x, x, psi.T, levels=[0.0], colors=GRIGIO, linewidths=0.6)
pos = basso.max() * np.array([0.01, 0.1, 0.3, 0.6, 0.9])
ax[1].contour(x, x, psi.T, levels=pos, colors=ROSSO, linewidths=0.7, linestyles="solid")
ax[1].set_aspect("equal")
ax[1].set_title(f"$\\psi$, griglia ${n}\\times{n}$")
ax[2].contour(x, x, psi.T, levels=[0.0], colors=GRIGIO, linewidths=0.6)
ax[2].contour(x, x, psi.T, levels=pos, colors=ROSSO, linewidths=0.8, linestyles="solid")
ax[2].contour(x, x, psi.T, levels=-np.logspace(-7, -3, 5)[::-1], colors=BLU, linewidths=0.6,
              linestyles="solid")
ax[2].set_xlim(0, 0.25)
ax[2].set_ylim(0, 0.25)
ax[2].set_aspect("equal")
ax[2].set_title("spigolo in basso a sinistra")
for a in ax[1:]:
    a.set_xlabel("$x$")
    a.set_ylabel("$y$")
fig.tight_layout()
salva(fig, "prog3_6b_cavita_fine")
```

Listato 7.2: `prog3_6b_cavita_fine.py`: soluzione diretta del sistema accoppiato, convergenza e vortici di Moffatt.

Risultati del Programma 3.6b

Minimo della funzione di corrente (interpolato con un paraboloide sui nodi vicini) e posizione del centro del vortice:

	formula a cinque punti		formula di Thom	
griglia	$\psi_{\min}$	y_c	$\psi_{\min}$	y_c
11×11	$-0{,}094\,325$	0,7384	$-0{,}098\,102$	0,7574
21×21	$-0{,}099\,108$	0,7565	$-0{,}099\,412$	0,7622
41×41	$-0{,}099\,949$	0,7633	$-0{,}099\,952$	0,7646
81×81	$-0{,}100\,040$	0,7645	$-0{,}100\,039$	0,7648
161×161	$-0{,}100\,065$	0,7649	$-0{,}100\,067$	0,7650

- Le due formule convergono allo stesso limite. L'estrapolazione di Richardson con le due griglie più fini dà $\psi_{\min} = -0{,}100074$ (cinque punti) e $-0{,}100076$ (Thom), con il centro del vortice in $(0{,}5;\ 0{,}765)$. Sulla griglia 11×11 la formula di Thom è perfino più vicina al limite, a conferma che la sua minore accuratezza locale non si trasmette alla soluzione.
- L'ordine di convergenza osservato è irregolare (fra 1,7 e 3): nei due spigoli superiori la velocità è discontinua e la vorticità tende all'infinito come $1/r$; queste singolarità inquinano la convergenza, che resta comunque prossima al secondo ordine.
- Sulla griglia 321×321 compaiono, negli spigoli inferiori, piccoli vortici controrotanti: in quello di sinistra $\psi_{\max} = +2{,}5 \times 10^{-6}$, 4×10^4 volte meno intenso del vortice principale, esteso lungo il fondo fino a $x = 0{,}103$.

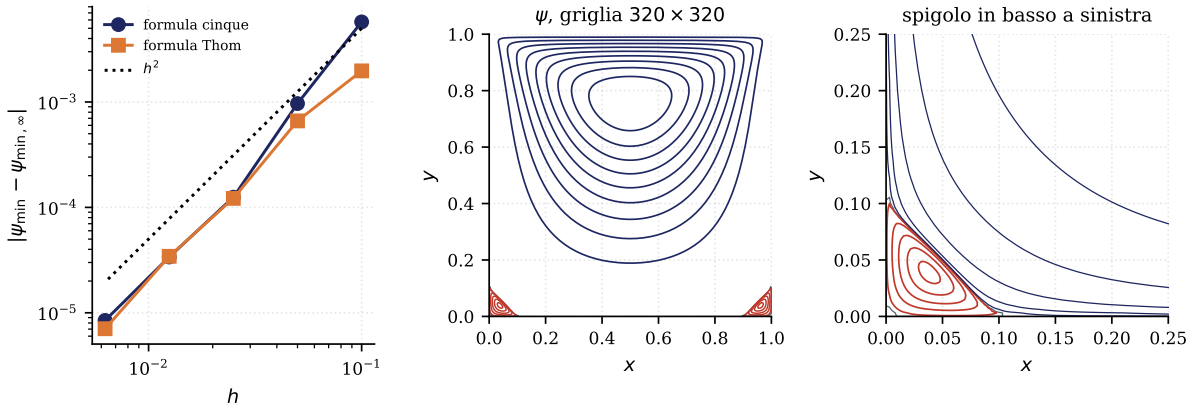


Figura 7.4: Cavità di Stokes su griglie fini. A sinistra: convergenza di $\psi_{\min}$ per le due formule di parete. Al centro: linee di corrente sulla griglia 321×321, con i vortici secondari negli spigoli inferiori (in rosso). A destra: ingrandimento dello spigolo inferiore sinistro.

I vortici di Moffatt

Vicino a uno spigolo di apertura 2α la soluzione biarmonica si può cercare nella forma $\psi = r^\lambda f(\vartheta)$, con r distanza dallo spigolo. Imponendo le condizioni di aderenza sulle due pareti si trova un'equazione trascendente per l'esponente λ; per la componente simmetrica rispetto alla bisettrice (quella che attraversa la bisettrice, come il moto indotto dal vortice principale) si ha, posto $z = \lambda - 1$,

$$\sin(2\alpha z) = -z \sin 2\alpha.$$

Moffatt (1964) [13] osservò che per $2\alpha < 146°$ le radici sono *complesse*: $\lambda = p + \mathrm{i}q$ e $r^\lambda =$

$r^p\, e^{iq\ln r}$ cambia segno infinite volte avvicinandosi allo spigolo. Nello spigolo esiste una successione infinita di vortici controrotanti, ciascuno più piccolo del precedente del fattore $e^{\pi/q}$. Per lo spigolo retto ($\sin(z\pi/2) = -z$) il programma trova $\lambda = 3{,}7396 + 1{,}1190\,\mathrm{i}$: rapporto fra le dimensioni 16,6, fra le intensità di ψ $3{,}6 \times 10^4$, fra le velocità $2{,}2 \times 10^3$.

La previsione si può confrontare con il calcolo: il primo vortice si estende sul fondo fino a $x = 0{,}103$, e il secondo dovrebbe arrivare a $0{,}103/16{,}6 \simeq 0{,}0062$. Sulla griglia più fine la funzione di corrente accanto al fondo cambia di nuovo segno proprio a $x \simeq 0{,}006$; il secondo vortice è però coperto da appena due intervalli di griglia, e la coincidenza va presa come un indizio, non come una verifica. Per vederne di più servirebbero griglie raffinate localmente verso lo spigolo, o la soluzione analitica locale.

Quando la soluzione ha singolarità

La cavità mostra un fenomeno frequente nei calcoli fluidodinamici: le singolarità geometriche (spigoli, discontinuità delle condizioni al contorno) riducono l'ordine di convergenza osservato rispetto a quello nominale dello schema. Il rimedio non è aumentare l'ordine dello schema, ma raffinare la griglia dove la soluzione è singolare, oppure sottrarre analiticamente la parte singolare. In ogni caso, uno studio di convergenza su almeno tre griglie è indispensabile per stimare l'incertezza del risultato.

7.5 Approfondimenti

7.5.1 La sfera in moto lento e la legge di Stokes

La legge di Stokes, citata all'inizio del capitolo e usata nel primo modulo, si ricava dalla stessa equazione biarmonica, in coordinate sferiche. È un calcolo classico che conviene ripercorrere, anche perché mostra che la resistenza di un corpo in moto lento è per due terzi attrito e per un terzo pressione.

La funzione di corrente di Stokes

Per un moto assialsimmetrico attorno all'asse x, in coordinate sferiche (r, θ), si definisce la funzione di corrente ψ con $u_r = \frac{1}{r^2 \sin\theta}\,\partial\psi/\partial\theta$, $u_\theta = -\frac{1}{r\sin\theta}\,\partial\psi/\partial r$, che soddisfa la continuità. Le equazioni di Stokes diventano $E^4\psi = 0$, con l'operatore $E^2 = \partial_r^2 + \frac{\sin\theta}{r^2}\partial_\theta\left(\frac{1}{\sin\theta}\partial_\theta\right)$ (l'analogo della biarmonica piana). Per una sfera ferma di raggio a in una corrente U la soluzione che vale $\frac{1}{2}Ur^2\sin^2\theta$ all'infinito (corrente uniforme) e soddisfa $u_r = u_\theta = 0$ su $r = a$ è

$$\psi = \frac{Ur^2\sin^2\theta}{4}\left(2 - \frac{3a}{r} + \frac{a^3}{r^3}\right). \tag{7.11}$$

Il Programma 3.6c la verifica simbolicamente: $E^4\psi = 0$ e le due componenti di velocità si annullano sulla sfera. Dalla velocità si ricavano la vorticità, $\omega_\varphi = -\frac{3Ua}{2r^2}\sin\theta$, e, integrando $\mu\nabla^2\mathbf{V} = \nabla p$, la pressione $p - p_\infty = -\frac{3}{2}\mu U a\cos\theta/r^2$: massima nel punto di ristagno anteriore, minima in quello posteriore, antisimmetrica. Lo sforzo tangenziale sulla sfera vale $\tau_{r\theta} = -\frac{3}{2}\mu U\sin\theta/a$. Integrando le componenti lungo la corrente:

$$D_p = \int(-p\cos\theta)\,\mathrm{d}A = 2\pi\mu a U, \quad D_\tau = \int(-\tau_{r\theta}\sin\theta)\,\mathrm{d}A = 4\pi\mu a U, \quad D = 6\pi\mu a U.$$

La resistenza è proporzionale alla velocità e non al suo quadrato, e al raggio e non alla sezione: nel moto lento conta il perimetro "visto" dalla viscosità, non l'area frontale. La correzione di Oseen [15] per l'inerzia, $D = 6\pi\mu a U\,(1 + \frac{3}{16}Re)$ con $Re = 2aU/\nu$, estende la validità della legge fino a $Re \simeq 1$.

Sedimentazione delle polveri

Una particella sferica di diametro d e densità ρ_p cade in aria alla velocità limite in cui il peso meno la spinta è equilibrato dalla legge di Stokes: $v = d^2(\rho_p - \rho)g/(18\mu)$. Per polvere minerale ($\rho_p = 2500\,\mathrm{kg\,m^{-3}}$) il programma dà

d (µm)	v (mm s^{-1})	Re
1	0,076	5×10^{-6}
10	7,6	5×10^{-3}
50	189	$6{,}3 \times 10^{-1}$
100	757	5,0

Una particella di 10 µm impiega quattro minuti a scendere di due metri, una di 1 µm più di sette ore: è il motivo per cui le polveri sottili restano in sospensione, e per cui le prese d'aria dei motori in ambienti polverosi hanno separatori inerziali. Oltre $Re \simeq 1$ la legge di Stokes sottostima la resistenza, e la velocità limite cresce meno che con d^2.

7.5.2 Il paradosso di Stokes

Si potrebbe ripetere il calcolo per il cilindro circolare indefinito, con la biarmonica piana di questo capitolo. Non esiste però alcuna soluzione: cercando $\psi = f(r)\sin\theta$ si trova $f = Ar^3 + Br\ln r + Cr + D/r$, e non è possibile soddisfare insieme l'aderenza sul cilindro e la corrente uniforme all'infinito (il termine $r\ln r$, necessario per l'aderenza, diverge). È il *paradosso di Stokes* (1851): in due dimensioni le perturbazioni prodotte da un corpo decadono così lentamente con la distanza che l'inerzia, per quanto piccolo sia Re, non è mai trascurabile lontano dal corpo. Oseen risolse il paradosso conservando l'inerzia linearizzata attorno alla corrente uniforme, e trovò per il cilindro $C_D = 8\pi/[Re\,(2{,}002 - \ln Re)]$: il logaritmo nel denominatore è la traccia del termine $r\ln r$. Nei domini *limitati*, come la cavità di questo capitolo, il paradosso non si presenta, perché le pareti tagliano la lenta decrescita all'infinito; è il motivo per cui la cavità ha una soluzione di Stokes perfettamente regolare e il cilindro in un canale (modulo precedente, a Re piccolo) anche.

7.5.3 La teoria della lubrificazione

Il flusso di Stokes più importante per le macchine è quello nei *meati*: film d'olio di spessore h piccolissimo rispetto alla lunghezza L lungo la quale varia. In questa geometria le equazioni si semplificano ulteriormente, in modo analogo a quanto fa lo strato limite ad alto Reynolds.

L'equazione di Reynolds per il pattino piano

Si consideri un pattino di lunghezza L inclinato di un piccolo angolo, che scorre con velocità U sopra un piano, con il meato di altezza $h(x)$ decrescente da h_1 all'ingresso a h_2 all'uscita (Fig. 7.5, schema). Per $h \ll L$ le stime d'ordine di grandezza danno $v \ll u$,

$\partial p/\partial y \simeq 0$ e, se il numero di Reynolds ridotto $\rho U h^2/(\mu L)$ è piccolo, inerzia trascurabile: localmente la corrente è quella di Couette–Poiseuille del Cap. 1,

$$u = U\left(1-\frac{y}{h}\right) - \frac{1}{2\mu}\frac{\mathrm{d}p}{\mathrm{d}x}\,y(h-y), \qquad Q = \int_0^h u\,\mathrm{d}y = \frac{Uh}{2} - \frac{h^3}{12\mu}\frac{\mathrm{d}p}{\mathrm{d}x}.$$

La portata Q deve essere la stessa in ogni sezione (il fluido non attraversa le pareti): derivando rispetto a x,

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(h^3\frac{\mathrm{d}p}{\mathrm{d}x}\right) = 6\mu U\frac{\mathrm{d}h}{\mathrm{d}x}, \qquad p(0) = p(L) = p_a, \tag{7.12}$$

che è l'equazione di Reynolds (1886). Dove il meato si restringe ($\mathrm{d}h/\mathrm{d}x < 0$) la pressione deve salire per rallentare il fluido trascinato dalla parete: nasce una sovrappressione che *sostiene* il pattino senza che le superfici si tocchino. Per il meato lineare $h = h_2[k-(k-1)x/L]$, $k = h_1/h_2$, l'integrazione dà in forma chiusa, con $X = x/L$ e $H = h/h_2$,

$$p - p_a = \frac{\mu U L}{h_2^2}\,\frac{6(k-1)X(1-X)}{(k+1)H^2},$$

$$W = \int_0^L (p-p_a)\,\mathrm{d}x = \frac{\mu U L^2}{h_2^2}\,\frac{6}{(k-1)^2}\left[\ln k - \frac{2(k-1)}{k+1}\right],$$

e per l'attrito sul pattino $F = \frac{\mu U L}{h_2}\left[\frac{4\ln k}{k-1} - \frac{6}{k+1}\right]$. Il carico sopportato per unità di larghezza, W, è massimo per $k = 2{,}19$.


prog3_6c_lubrificazione.py

```python
"""
prog3_6c_lubrificazione.py -- teoria della lubrificazione: il pattino piano
(cuscinetto di Michell) e la legge di Stokes per la sfera.

1) Pattino inclinato di lunghezza L che scorre con velocita' U su un piano,
   meato di altezza h(x) = h2 [k - (k - 1) x/L], k = h1/h2 > 1 (ingresso in x = 0).
   Equazione di Reynolds (film sottile, flusso di Stokes locale):
       d/dx [ h^3 dp/dx ] = 6 mu U dh/dx,   p(0) = p(L) = 0.
   Con X = x/L, H = h/h2, P = p h2^2/(mu U L):
       P(X) = 6 (k - 1) X (1 - X) / [ (k + 1) H(X)^2 ].
   Portanza per unita' di larghezza W = int p dx e attrito F = int tau dx:
       W h2^2/(mu U L^2) = 6/(k-1)^2 [ ln k - 2 (k-1)/(k+1) ],
       F h2/(mu U L)     = 4 ln k/(k-1) - 6/(k+1).
   Il programma verifica le formule risolvendo l'equazione di Reynolds alle
   differenze finite, traccia P(X) e W(k), e trova il k ottimo.
2) Sfera di raggio a in moto lento: verifica che la funzione di corrente di
   Stokes soddisfa E^4 psi = 0 e calcolo della resistenza 6 pi mu a U; velocita'
   di sedimentazione di una particella di polvere.
"""
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar
from metodi_numerici import thomas
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

# ------------------------------------------------------------------ 1) pattino
def pressione_esatta(X, k):
    H = k - (k - 1) * X
    return 6 * (k - 1) * X * (1 - X) / ((k + 1) * H**2)
```

```
31
32
33  def portanza(k):
34      return 6 / (k - 1)**2 * (np.log(k) - 2 * (k - 1) / (k + 1))
35
36
37  def attrito(k):
38      return 4 * np.log(k) / (k - 1) - 6 / (k + 1)
39
40
41  def pressione_fd(k, n=200):
42      """Risolve d/dX (H^3 dP/dX) = 6 dH/dX = -6 (k-1) con P(0) = P(1) = 0 (forma
        conservativa)."""
43      X = np.linspace(0, 1, n + 1)
44      hX = 1.0 / n
45      H = k - (k - 1) * X
46      Hm = (0.5 * (H[:-1] + H[1:]))**3          # H^3 nei punti intermedi
47      a = Hm[:-1] / hX**2
48      c = Hm[1:] / hX**2
49      b = -(a + c)
50      d = np.full(n - 1, -6 * (k - 1))
51      P = np.zeros(n + 1)
52      P[1:-1] = thomas(a, b, c, d)
53      return X, P
54
55
56  print("Pattino piano: k = h1/h2")
57  print("   k      W (FD)     W (esatta)   F (esatta)   (W/F) h2/L")
58  for k in (1.5, 2.0, 2.2, 3.0, 4.0):
59      X, P = pressione_fd(k)
60      Wfd = np.trapezoid(P, X)
61      print(f"   {k:3.1f}   {Wfd:.5f}    {portanza(k):.5f}      {attrito(k):.4f}
        {portanza(k)/attrito(k):.4f}")
62  kopt = minimize_scalar(lambda k: -portanza(k), bounds=(1.2, 5), method="bounded").x
63  print(f"   portanza massima per k = {kopt:.4f}: W h2^2/(mu U L^2) =
        {portanza(kopt):.5f}")
64  # esempio numerico: pattino di una ralla, olio SAE 30
65  mu, U, L, h2, rho = 0.05, 5.0, 0.05, 50e-6, 900.0
66  k = kopt
67  W = portanza(k) * mu * U * L**2 / h2**2
68  F = attrito(k) * mu * U * L / h2
69  print(f"   esempio: olio a caldo mu = {mu} Pa s, U = {U} m/s, L = {L*1e3:.0f} mm,
        h2 = {h2*1e6:.0f} um:")
70  print(f"      portanza {W/1e3:.1f} kN per metro di larghezza, attrito {F:.0f} N/m,
        W/F = {W/F:.0f},"
71        f" p_max =
        {pressione_esatta(np.linspace(0,1,1001),k).max()*mu*U*L/h2**2/1e6:.2f} MPa,"
72        f" Re ridotto rho U h2^2/(mu L) = {rho*U*h2**2/(mu*L):.1e}")
73
74  # ------------------------------------------------------------- 2) sfera di
        Stokes
75  r, th, a, Us = sp.symbols("r theta a U", positive=True)
76  psi = sp.Rational(1, 4) * Us * r**2 * sp.sin(th)**2 * (2 - 3 * a / r + a**3 / r**3)
77  E2 = lambda f: sp.diff(f, r, 2) + sp.sin(th) / r**2 * sp.diff(sp.diff(f, th) /
        sp.sin(th), th)
78  print("\nSfera in moto lento (sfera ferma, corrente U all'infinito):")
79  print("   E^4 psi =", sp.simplify(E2(E2(psi))))
80  ur = sp.simplify(sp.diff(psi, th) / (r**2 * sp.sin(th)))
81  ut = sp.simplify(-sp.diff(psi, r) / (r * sp.sin(th)))
82  print("   u_r =", ur, ";  u_theta =", ut)
```

```
print("   sulla sfera (r = a): u_r =", sp.simplify(ur.subs(r, a)), ", u_theta =", sp.simplify(ut.subs(r, a)))
mu_s = sp.symbols("mu", positive=True)
# vorticita' omega_phi = (1/r) d(r u_theta)/dr - (1/r) d u_r/d theta ; pressione da mu lap V = grad p
om = sp.simplify(sp.diff(r * ut, r) / r - sp.diff(ur, th) / r)
print("   vorticita' omega_phi =", om)
# pressione: p = -3 mu a U cos(theta)/(2 r^2) (da integrare mu rot(omega) = -grad p)
p = -sp.Rational(3, 2) * mu_s * a * Us * sp.cos(th) / r**2
# sforzo tangenziale sulla sfera: tau = mu * (d u_theta/dr - u_theta/r + (1/r) d u_r/d theta) in r = a
tau = sp.simplify((mu_s * (sp.diff(ut, r) - ut / r + sp.diff(ur, th) / r)).subs(r, a))
print("   sforzo tangenziale sulla sfera tau_rtheta =", tau)
# resistenza = int (-p cos + tau_rtheta sin ... ) dA ; con segno: forza lungo la corrente
Fp = sp.integrate(-p.subs(r, a) * sp.cos(th) * 2 * sp.pi * a**2 * sp.sin(th), (th, 0, sp.pi))
Ft = sp.integrate(-tau * sp.sin(th) * 2 * sp.pi * a**2 * sp.sin(th), (th, 0, sp.pi))
print("   resistenza di pressione =", sp.simplify(Fp), ";  di attrito =", sp.simplify(Ft), ";  totale =", sp.simplify(Fp + Ft))
# sedimentazione di una particella di polvere in aria
rho_p, rho_a, mu_a, nu_a, g = 2500.0, 1.2, 1.8e-5, 1.5e-5, 9.81
for d in (1e-6, 10e-6, 50e-6, 100e-6):
    ap = d / 2
    v = 2 * ap**2 * (rho_p - rho_a) * g / (9 * mu_a)
    Re = d * v / nu_a
    print(f"   particella di {d*1e6:5.0f} um: v = {v*1e3:8.3f} mm/s, Re = {Re:.2e}"
          + ("  (legge di Stokes non piu' valida)" if Re > 1 else ""))

# ------------------------------------------------------------------ figure
fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.8))
X = np.linspace(0, 1, 201)
for k, col in zip((1.5, 2.2, 4.0), (VERDE, BLU, ARANCIO)):
    ax[0].plot(X, pressione_esatta(X, k), color=col,
    label=f"$k={k:g}$".replace(".", "{,}"))
    Xf, Pf = pressione_fd(k, 20)
    ax[0].plot(Xf, Pf, "o", color=col, ms=2.5)
ax[0].set_xlabel("$X=x/L$")
ax[0].set_ylabel("$P=p\\,h_2^2/(\\mu UL)$")
ax[0].legend(fontsize=7)
ax[0].set_title("pressione nel meato")
ks = np.linspace(1.05, 6, 300)
ax[1].plot(ks, portanza(ks), color=BLU, label="portanza $W$")
ax[1].plot(ks, attrito(ks) / 10, "--", color=ARANCIO, label="attrito $F/10$")
ax[1].axvline(kopt, color=GRIGIO, ls=":", lw=1)
ax[1].set_xlabel("$k=h_1/h_2$")
ax[1].set_ylabel("$W h_2^2/(\\mu UL^2)$, $F h_2/(\\mu UL)$")
ax[1].legend(fontsize=7)
ax[1].set_title("carico e attrito")
# linee di corrente attorno alla sfera (soluzione di Stokes nel piano meridiano)
R, TH = np.meshgrid(np.linspace(1, 4, 200), np.linspace(0, 2 * np.pi, 361), indexing="ij")
PSI = 0.25 * R**2 * np.sin(TH)**2 * (2 - 3 / R + 1 / R**3)
Xs, Ys = R * np.cos(TH), R * np.sin(TH)
ax[2].contour(Xs, Ys, PSI, levels=np.linspace(0.1, 4.0, 14), colors=BLU, linewidths=0.7)
ax[2].add_patch(plt.Circle((0, 0), 1, color=GRIGIO))
ax[2].set_aspect("equal")
ax[2].set_xlim(-3, 3); ax[2].set_ylim(-3, 3)
```

```
ax[2].set_xticks([]); ax[2].set_yticks([])
ax[2].set_title("sfera di Stokes: linee di corrente")
fig.tight_layout()
salva(fig, "prog3_6c_lubrificazione")
```

Listato 7.3: `prog3_6c_lubrificazione.py`: equazione di Reynolds per il pattino piano (soluzione esatta e alle differenze finite), verifica simbolica della soluzione di Stokes per la sfera, sedimentazione.

Risultati del Programma 3.6c: il pattino

La soluzione alle differenze finite della (7.12) (forma conservativa, 200 intervalli) coincide con quella esatta a cinque cifre. Carico e attrito adimensionali:

k	$Wh_2^2/(\mu U L^2)$	$Fh_2/(\mu U L)$	$(W/F)\,h_2/L$
1,5	0,131 16	0,8437	0,1555
2,0	0,158 88	0,7726	0,2057
2,2	0,160 24	0,7532	0,2127
3,0	0,147 92	0,6972	0,2122
4,0	0,124 20	0,6484	0,1915

Esempio: pattino di 50 mm, $U = 5\,\mathrm{m\,s^{-1}}$, olio a caldo con $\mu = 0{,}05\,\mathrm{Pa\,s}$, $h_2 = 50\,\mu\mathrm{m}$, $k = 2{,}19$: carico 40 kN per metro di larghezza con una pressione massima di 1,3 MPa, attrito $190\,\mathrm{N\,m^{-1}}$, rapporto $W/F = 212$ (coefficiente d'attrito effettivo 0,005, dieci volte minore di quello dell'attrito secco fra metalli lubrificati); numero di Reynolds ridotto $4{,}5 \times 10^{-3}$.

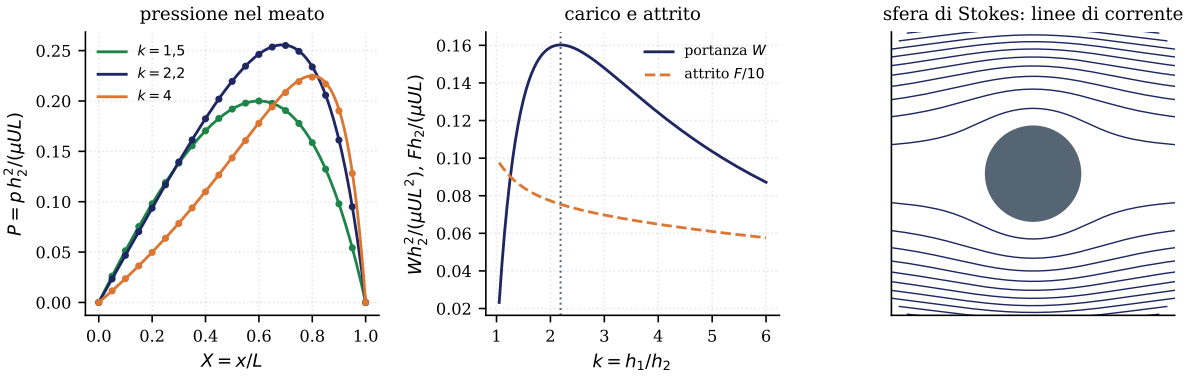


Figura 7.5: A sinistra: pressione nel meato del pattino piano per tre rapporti $k = h_1/h_2$ (linee: soluzione esatta; simboli: differenze finite con 20 intervalli). Al centro: carico e attrito in funzione di k. A destra: linee di corrente della soluzione di Stokes attorno a una sfera, nel piano meridiano.

La stessa equazione, scritta in due dimensioni e in coordinate cilindriche, governa i cuscinetti radiali (dove $h(\theta)$ dipende dall'eccentricità dell'albero), le tenute, le guide idrostatiche e i cuscinetti ad aria; la condizione $\rho U h^2/(\mu L) \ll 1$ è di norma soddisfatta anche a velocità elevate, perché h è piccolissimo. In aeronautica compare nei cuscinetti dei motori e, con la densità variabile, nei cuscinetti a gas delle turbomacchine ad alta velocità.

7.5.4 La derivazione dei vortici di Moffatt

L'equazione trascendente citata per i vortici d'angolo si ricava con un calcolo breve, che illustra come si studia una soluzione *localmente*, vicino a una singolarità geometrica.

Soluzioni della biarmonica in uno spigolo

Con l'origine nello spigolo e le pareti in $\vartheta = \pm\alpha$, si cercano soluzioni separabili $\psi = r^\lambda f(\vartheta)$. In coordinate polari la biarmonica dà per f un'equazione ordinaria del quarto ordine a coefficienti costanti, la cui soluzione generale è $f = A\cos\lambda\vartheta + B\sin\lambda\vartheta + C\cos(\lambda-2)\vartheta + D\sin(\lambda-2)\vartheta$. La componente *pari* in ϑ (simmetrica rispetto alla bisettrice, quella indotta dal vortice principale della cavità che "spinge" sulla bisettrice dello spigolo) è $f = A\cos\lambda\vartheta + C\cos(\lambda-2)\vartheta$. L'aderenza, $f(\alpha) = f'(\alpha) = 0$, dà il sistema omogeneo

$$A\cos\lambda\alpha + C\cos(\lambda-2)\alpha = 0, \qquad A\lambda\sin\lambda\alpha + C(\lambda-2)\sin(\lambda-2)\alpha = 0,$$

che ha soluzioni non nulle solo se il determinante si annulla: $\lambda\sin\lambda\alpha\cos(\lambda-2)\alpha - (\lambda-2)\cos\lambda\alpha\sin(\lambda-2)\alpha = 0$. Con le formule di addizione e posto $z = \lambda - 1$ si ottiene

$$\sin 2\alpha z = -z\sin 2\alpha,$$

l'equazione citata nel testo. Per lo spigolo retto, $2\alpha = \pi/2$: $\sin(\pi z/2) = -z$. Oltre alla radice banale $z = 0$, l'equazione non ha radici reali positive (per $z > 0$, $|\sin(\pi z/2)| \le 1 < z$ appena $z > 1$, e per $0 < z \le 1$ i due membri hanno segno opposto); la radice con parte reale minima è complessa, $z = 2{,}7396 + 1{,}1190\,\mathrm{i}$, cioè $\lambda = 3{,}7396 + 1{,}1190\,\mathrm{i}$. La velocità, proporzionale a $r^{\lambda-1} = r^{2{,}74}e^{1{,}119\,\mathrm{i}\ln r}$, tende a zero nello spigolo e cambia segno ogni volta che $\ln r$ diminuisce di $\pi/1{,}119 = 2{,}81$, cioè ogni volta che r si riduce del fattore $e^{2{,}81} = 16{,}6$; l'intensità di ψ si riduce del fattore $16{,}6^{3{,}74} = 3{,}6\times10^4$. L'angolo critico oltre il quale le radici diventano reali (e i vortici scompaiono) è $2\alpha = 146{,}3°$. Il risultato vale per qualunque flusso di Stokes in uno spigolo, non solo per la cavità: i vortici di Moffatt si osservano negli angoli dei canali, nelle fessure, attorno alle particelle che sedimentano vicino a una parete.

L'Esercizio 13 risolve il problema della cavità con la superficie superiore libera e il fondo in moto, che richiede una diversa condizione di parete per la vorticità sulla superficie libera.

Capitolo 8

Stabilità idrodinamica I: la convezione di Rayleigh–Bénard

8.1 Stabilità di una soluzione

Fin qui si sono cercate soluzioni delle equazioni del moto. Ma una soluzione esatta può non osservarsi mai in natura: la corrente di Poiseuille in un tubo esiste per qualunque numero di Reynolds, eppure oltre $Re \simeq 2000$ si vede di solito una corrente turbolenta. La ragione è che le piccole perturbazioni, sempre presenti, possono smorzarsi oppure amplificarsi. Una soluzione è **stabile** se ogni perturbazione abbastanza piccola si smorza nel tempo, **instabile** se qualche perturbazione cresce. Le perturbazioni crescenti cambiano la natura del moto: possono portare a un nuovo moto ordinato (le celle di convezione di questo capitolo) oppure, attraverso instabilità successive, alla turbolenza (le onde di Tollmien–Schlichting del capitolo seguente).

Lo strumento di base è la **teoria lineare**: si sovrappone alla soluzione di base una perturbazione di ampiezza infinitesima, si trascurano i termini quadratici nella perturbazione e si studiano le equazioni lineari che ne risultano. Poiché i coefficienti dipendono da una sola coordinata, la perturbazione si può decomporre in *modi normali*, ciascuno dei quali cresce o decresce esponenzialmente nel tempo: la stabilità si riduce a un **problema agli autovalori**. In questo e nel prossimo capitolo si risolvono due problemi classici di questo tipo.

8.2 Il problema di Rayleigh–Bénard

Uno strato orizzontale di fluido di spessore H, compreso fra due lastre piane, è riscaldato dal basso: la lastra inferiore è alla temperatura T_0, quella superiore alla temperatura $T_1 < T_0$ (Fig. 8.1). Il fluido caldo in basso è più leggero di quello freddo in alto: la configurazione è potenzialmente instabile, come uno strato d'acqua sopra uno d'olio. Finché la differenza di temperatura è piccola, viscosità e conduzione termica riescono a smorzare i moti nascenti e il calore attraversa lo strato per pura conduzione. Oltre una soglia, il fluido caldo sale e quello freddo scende in un moto ordinato di rulli controrotanti: le **celle di Bénard**. Lo stesso meccanismo si osserva in una pentola d'acqua sul fuoco, nella granulazione della superficie solare, nella circolazione del mantello terrestre, nelle intercapedini dei doppi vetri.

Il parametro che governa il fenomeno è il **numero di Rayleigh**

$$Ra = \frac{g\,\alpha\,\Delta T\,H^3}{\kappa\,\nu}, \tag{8.1}$$

Segnaposto figura

file atteso: `immagini/fig_3_8_1.png`

Strato di fluido di spessore H fra una lastra inferiore calda a temperatura T_0 e una lastra superiore fredda a temperatura $T_1 < T_0$; asse x orizzontale e asse z verticale (Fig. 3.8.1 della fonte).

Figura 8.1: Schema del problema di Bénard.

dove g è l'accelerazione di gravità, α il coefficiente di dilatazione termica, $\Delta T = T_0 - T_1$, κ la diffusività termica e ν la viscosità cinematica. Al numeratore c'è ciò che produce il moto (la spinta di galleggiamento), al denominatore ciò che lo ostacola (la diffusione della quantità di moto e del calore).

Approssimazione di Boussinesq

Le variazioni di densità sono piccole e si trascurano ovunque tranne che nel termine della forza peso, dove si linearizzano, $\rho = \rho_0[1 - \alpha(T - T_1)]$; tutte le altre proprietà sono costanti.

Con le grandezze adimensionali (lunghezze riferite a H, tempi a H^2/κ, velocità a κ/H, pressione a $\rho_0\kappa^2/H^2$) e la temperatura adimensionale $\theta = (T - T_1)/(T_0 - T_1)$, le equazioni di continuità, della quantità di moto (orizzontale x e verticale z) e dell'energia per un moto piano sono

$$\frac{\partial u}{\partial x} + \frac{\partial w}{\partial z} = 0, \tag{8.2}$$

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + w\frac{\partial u}{\partial z} = -\frac{\partial p}{\partial x} + Pr\,\nabla^2 u, \tag{8.3}$$

$$\frac{\partial w}{\partial t} + u\frac{\partial w}{\partial x} + w\frac{\partial w}{\partial z} = -\frac{\partial p}{\partial z} + Pr\,\nabla^2 w + Ra\,Pr\,\theta, \tag{8.4}$$

$$\frac{\partial \theta}{\partial t} + u\frac{\partial \theta}{\partial x} + w\frac{\partial \theta}{\partial z} = \nabla^2 \theta, \tag{8.5}$$

dove $Pr = \nu/\kappa$ e la pressione è quella in eccesso rispetto alla distribuzione idrostatica di riferimento.

8.3 Linearizzazione e modi normali

Lo stato di base è il fluido in quiete con il profilo di temperatura conduttivo:

$$\bar{u} = \bar{w} = 0, \qquad \bar{\theta} = 1 - z, \qquad \frac{\mathrm{d}\bar{p}}{\mathrm{d}z} = Ra\,Pr\,\bar{\theta}. \tag{8.6}$$

Si pone $u = \hat{u}$, $w = \hat{w}$, $\theta = \bar{\theta} + \hat{\theta}$, $p = \bar{p} + \hat{p}$, si sottrae lo stato di base e si trascurano i prodotti delle perturbazioni. Il termine convettivo dell'energia lascia il contributo $\hat{w}\,\mathrm{d}\bar{\theta}/\mathrm{d}z = -\hat{w}$: la perturbazione di velocità verticale porta fluido caldo verso l'alto, dove l'ambiente è più freddo.

$$\frac{\partial \hat{u}}{\partial t} = -\frac{\partial \hat{p}}{\partial x} + Pr\nabla^2\hat{u}, \qquad \frac{\partial \hat{w}}{\partial t} = -\frac{\partial \hat{p}}{\partial z} + Pr\nabla^2\hat{w} + RaPr\,\hat{\theta}, \qquad \frac{\partial \hat{\theta}}{\partial t} - \hat{w} = \nabla^2\hat{\theta}. \tag{8.7}$$

Eliminazione della pressione

Derivando la prima delle (8.7) rispetto a z e la seconda rispetto a x e sottraendo, la pressione scompare (è l'equazione della vorticità). Derivando ancora rispetto a x e usando la continuità $\partial\hat{u}/\partial x = -\partial\hat{w}/\partial z$ si ottiene un'equazione nella sola $\hat{w}$:

$$\frac{\partial}{\partial t}\left(\nabla^2\hat{w}\right) = Pr\,\nabla^4\hat{w} + Ra\,Pr\,\frac{\partial^2\hat{\theta}}{\partial x^2}, \qquad \frac{\partial\hat{\theta}}{\partial t} - \nabla^2\hat{\theta} = \hat{w}. \tag{8.8}$$

I coefficienti dipendono soltanto da z (anzi sono costanti) e il problema è indefinito in x: si cercano soluzioni periodiche in x con andamento esponenziale nel tempo,

$$\hat{w} = w'(z)\,e^{\sigma t}e^{ikx}, \qquad \hat{\theta} = \theta'(z)\,e^{\sigma t}e^{ikx}, \tag{8.9}$$

con k numero d'onda reale. Posto $D = \mathrm{d}/\mathrm{d}z$, si ottiene

$$\left[\frac{\sigma}{Pr} - (D^2 - k^2)\right](D^2 - k^2)\,w' = -k^2 Ra\,\theta', \qquad \left[\sigma - (D^2 - k^2)\right]\theta' = w'. \tag{8.10}$$

Se $\sigma < 0$ la perturbazione si smorza, se $\sigma > 0$ cresce e si formano le celle, con lunghezza d'onda orizzontale $2\pi/k$ e ampiezza verticale descritta da $w'(z)$. Si dimostra che per questo problema σ è reale (*principio di scambio delle stabilità* [29]): il passaggio dalla stabilità all'instabilità avviene attraverso $\sigma = 0$, e non attraverso oscillazioni crescenti. Basta allora studiare la **stabilità marginale**, $\sigma = 0$:

$$(D^2 - k^2)\,\theta' + w' = 0, \qquad (D^2 - k^2)^2\,w' - k^2 Ra\,\theta' = 0. \tag{8.11}$$

Sono due equazioni ordinarie omogenee, di quarto e secondo ordine, che con condizioni al contorno omogenee hanno soluzioni non nulle solo per valori particolari di Ra: per ogni k, il *più piccolo* di questi autovalori è il numero di Rayleigh oltre il quale i disturbi di numero d'onda k crescono. La curva $Ra(k)$ è la **curva di stabilità neutra**, e il suo minimo Ra_c è il numero di Rayleigh critico.

8.3.1 Le condizioni al contorno

Su pareti rigide e isoterme ($z = 0$ e $z = 1$)

$$w' = 0, \qquad Dw' = 0, \qquad \theta' = 0. \tag{8.12}$$

La prima è l'impermeabilità, la terza dice che la parete impone la sua temperatura. La seconda viene dalla continuità: per l'aderenza $\hat{u} = 0$ su tutta la parete, quindi anche $\partial\hat{u}/\partial x = 0$, e allora $\partial\hat{w}/\partial z = 0$. Se invece le superfici fossero libere (sforzo tangenziale nullo, $\partial\hat{u}/\partial z = 0$), derivando la continuità rispetto a z si otterrebbe $D^2w' = 0$.

Il caso delle superfici libere: una soluzione esatta

Con $w' = D^2w' = \theta' = 0$ alle due superfici, le funzioni $w' = \sin\pi z$, $\theta' = \sin(\pi z)/(\pi^2 + k^2)$ soddisfano la prima delle (8.11), perché $(D^2 - k^2)\sin\pi z = -(\pi^2 + k^2)\sin\pi z$. La seconda dà $(\pi^2 + k^2)^2 = k^2 Ra/(\pi^2 + k^2)$, cioè

$$Ra = \frac{(\pi^2 + k^2)^3}{k^2}. \tag{8.13}$$

Posto $a = k^2$, la derivata di $(\pi^2 + a)^3/a$ si annulla per $3a = \pi^2 + a$, cioè $k_c = \pi/\sqrt{2} = 2{,}221$, e il minimo vale $Ra_c = 27\pi^4/4 = 657{,}5$. È il risultato di Rayleigh (1916). Per pareti rigide non esiste una soluzione così semplice e serve un calcolo numerico: il valore classico è $Ra_c = 1707{,}76$ per $k_c = 3{,}117$; le pareti rigide, frenando il moto, stabilizzano lo strato.

8.4 La discretizzazione alle differenze finite

Il problema (8.11)–(8.12) si risolve numericamente sostituendo l'operatore D con differenze finite su n nodi equispaziati $z_i = (i-1)h$, $h = 1/(n-1)$, con $i = 1$ sulla parete inferiore e $i = n$ su quella superiore. Servono le formule:

derivata	formula	tipo	errore
Dw	$(-3w_i + 4w_{i+1} - w_{i+2})/(2h)$	in avanti	$\mathcal{O}(h^2)$
Dw	$(3w_i - 4w_{i-1} + w_{i-2})/(2h)$	all'indietro	$\mathcal{O}(h^2)$
D^2w	$(w_{i+1} - 2w_i + w_{i-1})/h^2$	centrata	$\mathcal{O}(h^2)$
D^4w	$(w_{i+2} - 4w_{i+1} + 6w_i - 4w_{i-1} + w_{i-2})/h^4$	centrata	$\mathcal{O}(h^2)$

Le formule unilatere del secondo ordine per la derivata prima servono alle pareti, dove non ci sono nodi da entrambe le parti. Con esse:

- l'equazione dell'energia nei nodi interni diventa $a_1\theta_{i-1} + a_2\theta_i + a_1\theta_{i+1} + w_i = 0$, con $a_1 = 1/h^2$, $a_2 = -(2/h^2 + k^2)$, e alle pareti $\theta_1 = \theta_n = 0$;
- l'equazione della quantità di moto, sviluppando $(D^2 - k^2)^2 = D^4 - 2k^2D^2 + k^4$, diventa nei nodi $i = 3, \dots, n-2$

$$b_1 w_{i-2} + b_2 w_{i-1} + b_3 w_i + b_2 w_{i+1} + b_1 w_{i+2} - k^2 Ra\,\theta_i = 0,$$

  con $b_1 = 1/h^4$, $b_2 = -(4/h^4 + 2k^2/h^2)$, $b_3 = 6/h^4 + 4k^2/h^2 + k^4$;
- nei nodi 1 e n si impone $w = 0$; nei nodi 2 e $n-1$, al posto dell'equazione della quantità di moto (che richiederebbe nodi fuori dal dominio), si impone $Dw = 0$ con le formule unilatere scritte *alle pareti*.

Ordinando le incognite nel vettore $\mathbf{x} = (\theta_1, \dots, \theta_n, w_1, \dots, w_n)$ il sistema prende la forma di un **problema agli autovalori generalizzato**

$$[\mathbf{A} - Ra\,\mathbf{B}]\,\mathbf{x} = 0, \tag{8.14}$$

dove $\mathbf{B}$ contiene soltanto i termini k^2 nelle righe della quantità di moto e nelle colonne delle θ corrispondenti. Le righe delle condizioni al contorno sono state modificate sia in $\mathbf{A}$ sia in $\mathbf{B}$ (in $\mathbf{B}$ sono nulle).

Il problema agli autovalori generalizzato

La (8.14) ha soluzioni non nulle solo se $\det(\mathbf{A} - Ra\,\mathbf{B}) = 0$. Poiché $\mathbf{B}$ è singolare (ha molte righe nulle), il polinomio caratteristico ha grado minore di $2n$ e il problema ha, oltre agli autovalori finiti, autovalori "infiniti" privi di significato fisico. Le librerie numeriche (`eig(A, B)` in MATLAB, `scipy.linalg.eig(A, B)` in Python) usano l'algoritmo

QZ, che tratta direttamente la coppia $(\mathbf{A}, \mathbf{B})$ senza invertire $\mathbf{B}$ e restituisce gli autovalori infiniti come `inf`: basta scartarli e prendere il più piccolo autovalore reale positivo.

8.5 Il programma e la sua validazione

Il Programma 3.7a costruisce le matrici $\mathbf{A}$ e $\mathbf{B}$, risolve il problema generalizzato e, prima di affrontare il caso delle pareti rigide, verifica il codice sul caso delle superfici libere, per il quale la soluzione esatta (8.13) è nota. Per le superfici libere la condizione $D^2 w = 0$ si impone con un nodo fittizio, $w_0 = -w_2$ (ricavato da $w_1 = 0$ e dalla differenza centrata di $D^2 w$), e l'equazione della quantità di moto si scrive anche nei nodi 2 e $n-1$.

prog3_7a_benard.py

```
"""
prog3_7a_benard.py -- stabilita' marginale della convezione di Rayleigh-Benard
con le differenze finite: problema agli autovalori generalizzato  A x = Ra B x.

Incognite: theta_1..theta_n e w_1..w_n (ampiezze delle perturbazioni di
temperatura e di velocita' verticale sui nodi z_i = (i-1) h, h = 1/(n-1)).
Equazioni (neutralita', sigma = 0):
    (D^2 - k^2) theta + w = 0                       (energia)
    (D^2 - k^2)^2 w - k^2 Ra theta = 0              (quantita' di moto)
Pareti rigide (z = 0, 1):  theta = 0,  w = 0,  Dw = 0 (formula unilatera del
secondo ordine, che sostituisce l'equazione di w nel primo nodo interno).
Per validare il codice si risolve anche il caso con pareti libere
(theta = w = D^2 w = 0, nodo fittizio w_(-1) = -w_1), la cui soluzione esatta e'
    Ra = (pi^2 + k^2)^3 / k^2,   minimo 27 pi^4 / 4 per k = pi / sqrt(2).
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eig
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO


def matrici(n, k, pareti="rigide"):
    """Restituisce A, B (2n x 2n) con l'ordine x = (theta_1..theta_n, w_1..w_n)."""
    h = 1.0 / (n - 1)
    a1, a2 = 1 / h**2, -(2 / h**2 + k**2)
    b1 = 1 / h**4
    b2 = -(4 / h**4 + 2 * k**2 / h**2)
    b3 = 6 / h**4 + 4 * k**2 / h**2 + k**4
    A = np.zeros((2 * n, 2 * n))
    B = np.zeros((2 * n, 2 * n))
    T, W = 0, n                                     # offset dei due blocchi
    # --- equazione dell'energia (righe 0..n-1)
    A[T + 0, T + 0] = 1.0                           # theta = 0 in z = 0
    A[T + n - 1, T + n - 1] = 1.0                   # theta = 0 in z = 1
    for i in range(1, n - 1):
        A[T + i, T + i - 1] = a1
        A[T + i, T + i] = a2
        A[T + i, T + i + 1] = a1
        A[T + i, W + i] = 1.0
    # --- equazione della quantita' di moto (righe n..2n-1)
    A[W + 0, W + 0] = 1.0                           # w = 0 in z = 0
    A[W + n - 1, W + n - 1] = 1.0                   # w = 0 in z = 1
    if pareti == "rigide":                          # Dw = 0 (unilatera, O(h^2))
        A[W + 1, W + 0:W + 3] = [-3 / (2 * h), 2 / h, -1 / (2 * h)]
        A[W + n - 2, W + n - 3:W + n] = [1 / (2 * h), -2 / h, 3 / (2 * h)]
        interni = range(2, n - 2)
```

```
    else:                                          # D^2 w = 0: nodo fittizio
    w_(-1) = -w_1
        interni = range(1, n - 1)
    for i in interni:
        for d, c in zip(range(-2, 3), [b1, b2, b3, b2, b1]):
            jj = i + d
            if jj < 0:                             # fittizio sotto la parete
    inferiore
                A[W + i, W - jj] -= c
            elif jj > n - 1:                       # fittizio sopra la parete
    superiore
                A[W + i, W + 2 * (n - 1) - jj] -= c
            else:
                A[W + i, W + jj] += c
        B[W + i, T + i] = k**2
    return A, B


def Ra_minimo(n, k, pareti="rigide", vettori=False):
    A, B = matrici(n, k, pareti)
    lam, X = eig(A, B)
    ok = np.isfinite(lam) & (np.abs(lam.imag) < 1e-6 * np.abs(lam)) & (lam.real > 0)
    idx = np.nonzero(ok)[0]
    j = idx[np.argmin(lam[idx].real)]
    if vettori:
        return lam[j].real, X[:, j].real
    return lam[j].real


if __name__ == "__main__":
    print("1) Validazione con pareti libere, k = pi/sqrt(2) (esatto 27 pi^4/4 =
    %.4f):" % (27 * np.pi**4 / 4))
    kc_l = np.pi / np.sqrt(2)
    prec = None
    for n in [11, 21, 41, 81, 161]:
        r = Ra_minimo(n, kc_l, "libere")
        err = r - 27 * np.pi**4 / 4
        ordine = "" if prec is None else f"  ordine
    {np.log2(abs(prec)/abs(err)):.2f}"
        print(f"   n = {n:4d}   Ra = {r:10.4f}   errore = {err:+.3e}{ordine}")
        prec = err

    print("\n2) Pareti rigide, k = 3,117 (valore di riferimento Ra_c = 1707,76):")
    prec = None
    ris = []
    for n in [7, 13, 25, 49, 97, 151, 193]:
        r = Ra_minimo(n, 3.117)
        ris.append((n, r))
        print(f"   n = {n:4d}   Ra = {r:10.3f}")
    # estrapolazione di Richardson sugli ultimi due (ordine 2 in h)
    (n1, r1), (n2, r2) = ris[-2], ris[-1]
    h1, h2 = 1 / (n1 - 1), 1 / (n2 - 1)
    r_inf = r2 + (r2 - r1) * h2**2 / (h1**2 - h2**2)
    print(f"   Richardson (n = {n1}, {n2}): Ra = {r_inf:.2f}")

    # ------------------------------------------------------------------ figura
    A, B = matrici(7, 3.117)
    fig, ax = plt.subplots(1, 3, figsize=(7.8, 2.8))
    ax[0].spy(A, markersize=4, color=BLU)
    ax[0].set_title(r"$\mathbf{A}$ ($n=7$)")
```

```
    ax[1].spy(B, markersize=4, color=ROSSO)
    ax[1].set_title(r"$\mathbf{B}$ ($n=7$)")
    for a in ax[:2]:
        a.axhline(6.5, color=GRIGIO, lw=0.6)
        a.axvline(6.5, color=GRIGIO, lw=0.6)
        a.tick_params(labelsize=6)
    kk = np.linspace(1.0, 8.0, 60)
    ax[2].plot(kk, (np.pi**2 + kk**2) ** 3 / kk**2, color=GRIGIO, lw=2.2,
    alpha=0.6, label="esatta")
    ax[2].plot(kk[::4], [Ra_minimo(41, k, "libere") for k in kk[::4]], "o", ms=3.5,
    color=ARANCIO,
               label="diff. finite, $n=41$")
    ax[2].set_ylim(0, 6000)
    ax[2].set_xlabel("$k$")
    ax[2].set_ylabel("$Ra$")
    ax[2].set_title("pareti libere: validazione")
    ax[2].legend(fontsize=6.5)
    fig.tight_layout()
    salva(fig, "prog3_7a_benard")
```

Listato 8.1: `prog3_7a_benard.py`: problema agli autovalori di Rayleigh–Bénard con le differenze finite.

Risultati del Programma 3.7a

- *Superfici libere*, $k = \pi/\sqrt{2}$ (esatto 657,5114): $Ra = 646{,}79$; 654,81; 656,84; 657,34; 657,47 con $n = 11, 21, 41, 81, 161$. L'errore si divide per quattro a ogni raddoppio dei nodi (ordine 2,00): il codice è corretto. Anche l'intera curva neutra coincide con la (8.13) (Fig. 8.2, a destra).
- *Pareti rigide*, $k = 3{,}117$:

n	7	13	25	49	97	151	193
Ra	2372,8	1849,9	1743,5	1716,9	1710,1	1708,7	1708,4

  Con i sette nodi della formulazione più semplice l'errore è del 39%; servono circa 150 nodi per scendere sotto l'unità. L'estrapolazione di Richardson con $n = 151$ e 193 dà $Ra = 1707{,}77$, in accordo con il valore classico.

La convergenza lenta del caso rigido non dipende da un errore, ma dalla natura della formulazione: le condizioni $Dw = 0$ sono imposte con formule unilatere e la soluzione varia rapidamente vicino alle pareti. L'Esercizio 14 traccia l'intera curva neutra, trova il punto critico e le autofunzioni, confronta il risultato con una formula approssimata e lo ripete con la collocazione di Chebyshev del capitolo seguente, che con soli 17 nodi raggiunge $Ra_c = 1707{,}7618$ con tutte le cifre corrette.

Quando si mette in moto uno strato di fluido?

Dalla (8.1) la differenza di temperatura critica è $\Delta T_c = 1708\,\kappa\nu/(g\alpha H^3)$.

- *Acqua*, $H = 1\,\text{cm}$ ($\nu = 1\times10^{-6}\,\text{m}^2\,\text{s}^{-1}$, $\kappa = 1{,}43\times10^{-7}\,\text{m}^2\,\text{s}^{-1}$, $\alpha = 2{,}1\times10^{-4}\,\text{K}^{-1}$): $\Delta T_c \simeq 0{,}12\,\text{K}$. Basta un decimo di grado perché l'acqua in una vaschetta si metta in moto.
- *Aria*, $H = 1\,\text{cm}$ ($\nu = 1{,}5\times10^{-5}$, $\kappa = 2{,}1\times10^{-5}\,\text{m}^2\,\text{s}^{-1}$, $\alpha = 1/T \simeq 3{,}4\times10^{-3}\,\text{K}^{-1}$):

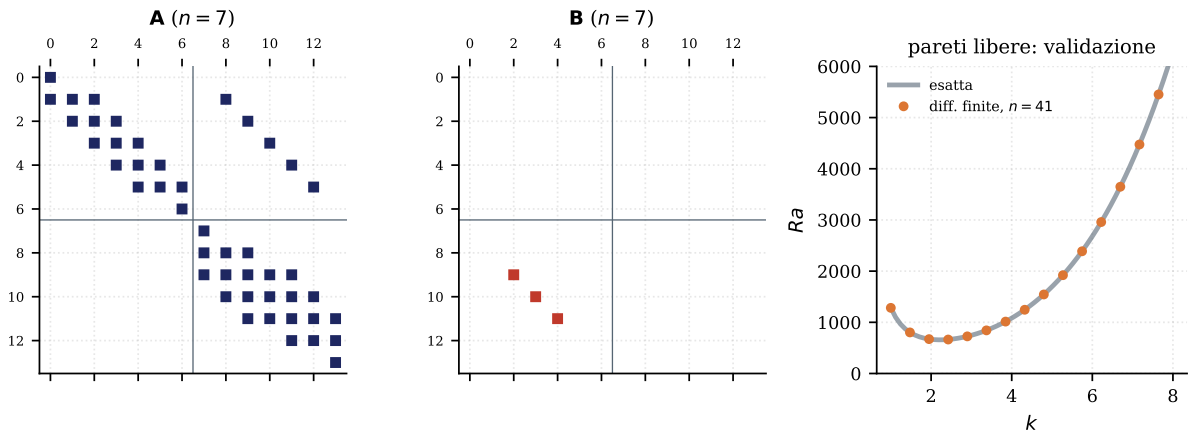


Figura 8.2: Struttura delle matrici **A** e **B** per $n = 7$ (i blocchi separano le incognite θ e w) e validazione sul caso delle superfici libere.

$\Delta T_c \simeq 16\,\mathrm{K}$; con un'intercapedine di 1,5 cm, come nei doppi vetri, la soglia scende a circa 5 K. È il motivo per cui le intercapedini troppo spesse isolano peggio: oltre la soglia la convezione si aggiunge alla conduzione.

Il numero d'onda critico $k_c = 3{,}117$ corrisponde a una lunghezza d'onda $2\pi H/k_c \simeq 2{,}0\,H$: ogni rullo, metà di una lunghezza d'onda, ha una sezione all'incirca quadrata.

8.6 Approfondimenti

8.6.1 Il principio di scambio delle stabilità: dimostrazione

Nel testo si è ammesso che σ sia reale. La dimostrazione richiede solo integrazioni per parti, e fornisce in più un risultato variazionale di grande utilità pratica.

Gli autovalori σ sono reali

Si parta dalle (8.10), con $L = D^2 - k^2$, e dalle condizioni $w' = Dw' = \theta' = 0$ alle pareti (per brevità si omettono gli apici: w, θ sono le ampiezze complesse). Si moltiplichi la prima equazione per il coniugato $\bar{w}$ e si integri su $0 \le z \le 1$. Integrando per parti, con $w = 0$ alle pareti,

$$\int_0^1 \bar{w}\, Lw \,\mathrm{d}z = -\int_0^1 \left(|Dw|^2 + k^2|w|^2\right) \mathrm{d}z = -I_1;$$

integrando per parti due volte, e usando sia $w = 0$ sia $Dw = 0$ (che annullano i termini di contorno $[\bar{w}\, D(Lw)]$ e $[D\bar{w}\, Lw]$),

$$\int_0^1 \bar{w}\, L^2 w \,\mathrm{d}z = \int_0^1 |Lw|^2 \,\mathrm{d}z = I_2.$$

Quindi la prima equazione dà $-\sigma I_1/Pr - I_2 = -k^2 Ra \int \bar{w}\theta \,\mathrm{d}z$. La seconda equazione, moltiplicata per $\bar{\theta}$ e integrata (con $\theta = 0$ alle pareti), dà $\sigma I_3 + I_4 = \int \bar{\theta} w \,\mathrm{d}z$, con $I_3 = \int |\theta|^2$ e $I_4 = \int (|D\theta|^2 + k^2|\theta|^2)$. Prendendo il coniugato di quest'ultima, $\bar{\sigma} I_3 + I_4 = \int \bar{w}\theta \,\mathrm{d}z$, e sostituendo:

$$-\frac{\sigma}{Pr} I_1 - I_2 = -k^2 Ra \left(\bar{\sigma} I_3 + I_4\right).$$

Tutte le I sono reali e positive. La parte immaginaria dell'uguaglianza è $-\sigma_I\, I_1/Pr = k^2 Ra\, \sigma_I\, I_3$, cioè $\sigma_I\,(I_1/Pr + k^2 Ra I_3) = 0$: per $Ra > 0$ la parentesi è positiva e $\sigma_I = 0$. Gli autovalori sono reali: le perturbazioni crescono o decadono in modo monotono, senza oscillare, e la soglia è $\sigma = 0$. (Per $Ra < 0$, strato riscaldato dall'alto, l'argomento non vale, ma la parte reale mostra che $\sigma < 0$: lo strato è sempre stabile.)
La parte reale, con σ reale, dà $\sigma\,(k^2 Ra I_3 - I_1/Pr) = I_2 - k^2 Ra I_4$. Alla stabilità marginale ($\sigma = 0$):

$$Ra = \frac{\int_0^1 |Lw|^2\,\mathrm{d}z}{k^2 \int_0^1 \left(|D\theta|^2 + k^2|\theta|^2\right)\mathrm{d}z}, \qquad L\theta = -w, \tag{8.15}$$

un *quoziente di Rayleigh*. Si dimostra [31] che il più piccolo autovalore è il minimo del quoziente su tutte le funzioni w ammissibili ($w = Dw = 0$ alle pareti), con θ ricavata da $L\theta = -w$: ogni funzione di prova fornisce una stima *per eccesso* di $Ra(k)$.

Il quoziente (8.15) ha anche un significato fisico: il numeratore è la dissipazione viscosa della perturbazione, il denominatore la potenza che la spinta di galleggiamento le fornisce. La formula approssimata dell'Esercizio 14 deriva proprio da un calcolo di questo tipo con una funzione di prova semplice.

8.6.2 Il Programma 3.7c: tassi di crescita, altre condizioni al contorno, stima variazionale

Il Programma 3.7c, che usa la collocazione di Chebyshev del capitolo seguente, completa l'analisi in tre direzioni: risolve il problema agli autovalori in σ (non solo marginale), ripete il calcolo del punto critico per le altre combinazioni di condizioni al contorno, e valuta il quoziente (8.15) per due funzioni di prova.

prog3_7c_benard_varianti.py

```
"""
prog3_7c_benard_varianti.py -- complementi sulla convezione di Rayleigh-Benard
con la collocazione di Chebyshev su z = (1 + y)/2.

1) Tassi di crescita: problema agli autovalori in sigma (non solo marginale)
       [sigma/Pr - L] L w = -k^2 Ra theta,   [sigma - L] theta = w,   L = D^2 - k^2,
   per verificare il principio di scambio delle stabilita' (sigma reale) e
   misurare sigma_max in funzione di Ra oltre la soglia.
2) Numero di Rayleigh critico per le tre combinazioni di condizioni al contorno:
   rigido-rigido, rigido-libero, libero-libero.
3) Stima variazionale: Ra(k) = int |L w|^2 dz / ( k^2 int (|D theta|^2 + k^2
    |theta|^2) dz ),
   con L theta = -w, per la funzione di prova w = z^2 (1 - z)^2.
"""
import numpy as np
from scipy.linalg import eig, solve
from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt
from metodi_numerici import cheb
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

N = 40
D, y = cheb(N)
z = (1 + y) / 2                 # z_0 = 1 (parete superiore), z_N = 0 (inferiore)
Dz = 2 * D
D2 = Dz @ Dz
I = np.eye(N + 1)
n1 = N + 1
```

```
28
29
30 def condizioni(A, B, sup, inf):
31     """Impone theta = 0, w = 0 e (rigido) Dw = 0 oppure (libero) D^2 w = 0 alle due
       pareti."""
32     for riga in (0, N):                                    # theta = 0
33         A[riga, :] = 0; A[riga, riga] = 1; B[riga, :] = 0
34     for riga in (0, N):                                    # w = 0
35         A[n1 + riga, :] = 0; A[n1 + riga, n1 + riga] = 1; B[n1 + riga, :] = 0
36     for riga, tipo, nodo in ((1, sup, 0), (N - 1, inf, N)):
37         A[n1 + riga, :] = 0; B[n1 + riga, :] = 0
38         A[n1 + riga, n1:] = Dz[nodo] if tipo == "rigido" else D2[nodo]
39     return A, B
40
41
42 def ra_marginale(k, sup="rigido", inf="rigido"):
43     L = D2 - k**2 * I
44     Z = np.zeros((n1, n1))
45     A = np.block([[L, I], [Z, L @ L]])
46     B = np.block([[Z, Z], [k**2 * I, Z]])
47     A, B = condizioni(A, B, sup, inf)
48     lam = eig(A, B, right=False)
49     ok = np.isfinite(lam) & (np.abs(lam.imag) < 1e-6 * np.abs(lam)) & (lam.real > 0)
50     return lam[ok].real.min()
51
52
53 def sigma_max(k, Ra, Pr=1.0, sup="rigido", inf="rigido"):
54     L = D2 - k**2 * I
55     Z = np.zeros((n1, n1))
56     A = np.block([[L, I], [-k**2 * Ra * I, L @ L]])
57     B = np.block([[I, Z], [Z, L / Pr]])
58     A, B = condizioni(A, B, sup, inf)
59     lam = eig(A, B, right=False)
60     lam = lam[np.isfinite(lam) & (np.abs(lam) < 1e6)]      # scarta gli autovalori
       spuri (infiniti)
61     # parte immaginaria massima fra i modi ben risolti (|sigma| < 1000; i modi piu'
       smorzati
62     # non sono rappresentati dalla griglia e possono risultare complessi per errore
       numerico)
63     return lam[np.argmax(lam.real)], np.max(np.abs(lam.imag[np.abs(lam) < 1e3]))
64
65
66 # ------------------------------------------------------------- 1) scambio delle
       stabilita'
67 print("Pareti rigide, k = 3,1163, Pr = 1: autovalore sigma meno stabile")
68 print("      Ra      sigma_max           max |Im sigma| (modi con |sigma| < 1000)")
69 for Ra in (1000, 1500, 1707.76, 2000, 3000, 5000):
70     s, im = sigma_max(3.1163, Ra)
71     print(f"   {Ra:8.2f}   {s.real:+9.4f} {s.imag:+.1e}i      {im:.1e}")
72 Ras = np.linspace(1500, 3000, 31)
73 sig = np.array([sigma_max(3.1163, Ra)[0].real for Ra in Ras])
74 pend = np.polyfit(Ras[:8], sig[:8], 1)
75 print(f"   vicino alla soglia: sigma ~ {pend[0]:.3e} (Ra - {-pend[1]/pend[0]:.1f});"
76       f" per Pr = 7 (acqua): sigma_max(Ra = 2000) = {sigma_max(3.1163, 2000,
       Pr=7)[0].real:.4f},"
77       f" per Pr = 0,71: {sigma_max(3.1163, 2000, Pr=0.71)[0].real:.4f}")
78
79 # ------------------------------------------------------------- 2) condizioni al
       contorno
80 print("\nNumero di Rayleigh critico per diverse condizioni al contorno (Chebyshev,
```

```
    N = 40):")
critici = {}
for sup, inf in (("rigido", "rigido"), ("rigido", "libero"), ("libero", "libero")):
    ris = minimize_scalar(lambda k: ra_marginale(k, sup, inf), bounds=(1.5, 4.5),
    method="bounded",
                          options={"xatol": 1e-5})
    critici[(sup, inf)] = (ris.x, ris.fun)
    print(f"   {sup:7s} - {inf:7s}: k_c = {ris.x:.4f}, Ra_c = {ris.fun:.3f}")
print(f"   libero-libero esatto: k_c = pi/sqrt(2) = {np.pi/np.sqrt(2):.4f}, Ra_c =
    27 pi^4/4 = {27*np.pi**4/4:.3f}")

# ------------------------------------------------------------------ 3) stima
    variazionale
def clenshaw_curtis(N):
    """Pesi di Clenshaw-Curtis sui nodi di Chebyshev-Gauss-Lobatto di [-1, 1]
    (integrazione esatta del polinomio interpolante)."""
    theta = np.pi * np.arange(N + 1) / N
    w = np.zeros(N + 1)
    ii = np.arange(1, N)
    v = np.ones(N - 1)
    if N % 2 == 0:
        w[0] = w[N] = 1.0 / (N * N - 1)
        for k in range(1, N // 2):
            v -= 2 * np.cos(2 * k * theta[ii]) / (4 * k * k - 1)
        v -= np.cos(N * theta[ii]) / (N * N - 1)
    else:
        w[0] = w[N] = 1.0 / N**2
        for k in range(1, (N - 1) // 2 + 1):
            v -= 2 * np.cos(2 * k * theta[ii]) / (4 * k * k - 1)
    w[ii] = 2 * v / N
    return w


def ra_variazionale(k, w):
    """Quoziente di Rayleigh con theta ottenuta da L theta = -w, theta = 0 alle
    pareti."""
    L = D2 - k**2 * I
    M = L.copy(); M[0, :] = 0; M[0, 0] = 1; M[N, :] = 0; M[N, N] = 1
    rhs = -w.copy(); rhs[0] = rhs[N] = 0
    th = solve(M, rhs)
    pesi = clenshaw_curtis(N) / 2                          # su [0, 1]
    num = np.sum(pesi * (L @ w)**2)
    den = k**2 * np.sum(pesi * ((Dz @ th)**2 + k**2 * th**2))
    return num / den


w_prova = z**2 * (1 - z)**2
print(f"\nVerifica dei pesi di quadratura: int_0^1 z^2 (1-z)^2 dz =
    {np.sum(clenshaw_curtis(N) / 2 * w_prova):.10f} (esatto {1/30:.10f})")
ris = minimize_scalar(lambda k: ra_variazionale(k, w_prova), bounds=(2, 4.5),
    method="bounded")
print(f"\nStima variazionale con w = z^2 (1 - z)^2: minimo Ra = {ris.fun:.2f} per k
    = {ris.x:.4f}"
      f" (esatto 1707,76 per k = 3,1163: eccesso {100*(ris.fun/1707.762-1):.2f} %)")
w2 = np.sin(np.pi * z)**2          # altra funzione di prova (ammissibile: w = Dw =
    0 alle pareti)
ris2 = minimize_scalar(lambda k: ra_variazionale(k, w2), bounds=(2, 4.5),
    method="bounded")
print(f"   con w = sin^2(pi z):          minimo Ra = {ris2.fun:.2f} per k =
    {ris2.x:.4f} (eccesso {100*(ris2.fun/1707.762-1):.2f} %)")
```

```
130
131 # ------------------------------------------------------------- figure
132 fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.9))
133 ax[0].plot(Ras, sig, color=BLU)
134 ax[0].axhline(0, color="k", lw=0.6)
135 ax[0].axvline(1707.76, color=GRIGIO, ls=":", lw=1)
136 ax[0].set_xlabel("$Ra$")
137 ax[0].set_ylabel("$\\sigma_{\\max}$  ($k=3{,}116$, $Pr=1$)")
138 ax[0].set_title("tasso di crescita oltre la soglia")
139 ks = np.linspace(1.5, 6, 46)
140 for (sup, inf), col, et in zip(critici, (BLU, ARANCIO, VERDE), ("rigido--rigido",
        "rigido--libero", "libero--libero")):
141     ax[1].plot(ks, [ra_marginale(k, sup, inf) for k in ks], color=col, label=et)
142     ax[1].plot(critici[(sup, inf)][0], critici[(sup, inf)][1], "o", color=col, ms=4)
143 ax[1].plot(ks, [ra_variazionale(k, w_prova) for k in ks], "--", color=ROSSO, lw=1,
        label="variazionale, $z^2(1-z)^2$")
144 ax[1].set_ylim(0, 6000)
145 ax[1].set_xlabel("$k$")
146 ax[1].set_ylabel("$Ra$")
147 ax[1].legend(fontsize=7)
148 ax[1].set_title("curve neutre")
149 fig.tight_layout()
150 salva(fig, "prog3_7c_benard_varianti")
```

Listato 8.2: `prog3_7c_benard_varianti.py`: autovalori σ, scambio delle stabilità, punto critico per tre tipi di pareti, stima variazionale.

Risultati del Programma 3.7c

- *Scambio delle stabilità.* Per pareti rigide, $k = 3{,}1163$ e $Pr = 1$, tutti gli autovalori ben risolti hanno parte immaginaria nulla (a meno dell'arrotondamento), e il meno stabile vale $\sigma_{\max} = -6{,}00;\ -1{,}63;\ 0{,}00;\ +2{,}15;\ +8{,}58;\ +19{,}0$ per $Ra = 1000;\ 1500;\ 1707{,}76;\ 2000;\ 3000;\ 5000$. Vicino alla soglia $\sigma \simeq 0{,}0077\,(Ra - Ra_c)$: a $Ra = 2000$ l'ampiezza delle celle raddoppia ogni 0,32 unità di tempo H^2/κ, cioè, per l'acqua con $H = 1\,\mathrm{cm}$, ogni quattro minuti. Il tasso cresce con Pr (3,10 per $Pr = 7$, 1,88 per 0,71), ma la *soglia* non dipende da Pr, come la (8.11) prevede.
- *Condizioni al contorno.*

pareti	k_c	Ra_c
rigida–rigida	3,1163	1707,762
rigida–libera	2,6823	1100,650
libera–libera	2,2214	657,511

  Il caso rigido–libero (una vaschetta d'acqua con la superficie scoperta) ha soglia quasi dimezzata rispetto a due pareti rigide; il caso libero–libero riproduce il valore esatto $27\pi^4/4$ a tutte le cifre.
- *Stima variazionale.* Con $w = z^2(1-z)^2$ il quoziente ha minimo $Ra = 1708{,}05$ per $k = 3{,}113$: appena lo 0,02% sopra il valore esatto. Con $w = \sin^2 \pi z$ si ottiene 1810 (+6%). La prima funzione di prova è così buona perché riproduce il comportamento $w \propto z^2$ vicino alle pareti, imposto da $w = Dw = 0$.

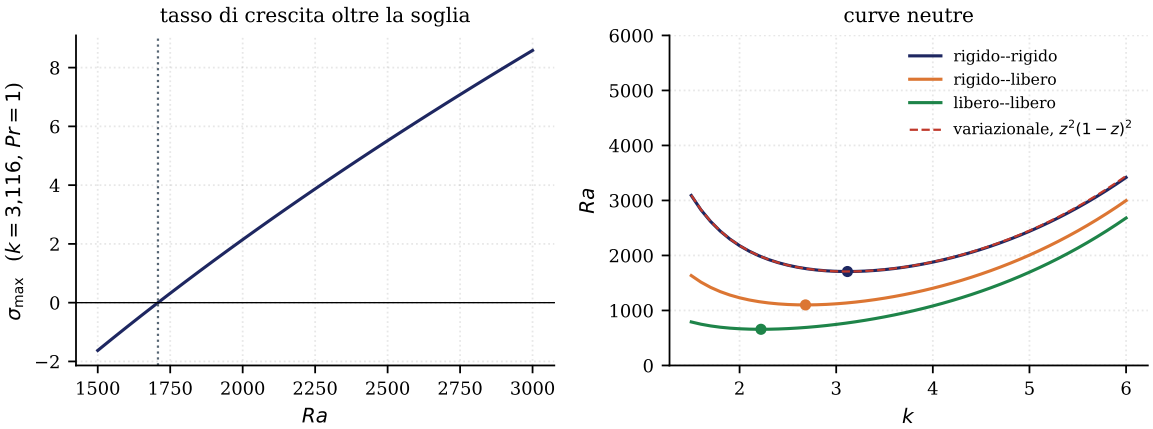


Figura 8.3: A sinistra: tasso di crescita del modo meno stabile in funzione di Ra (pareti rigide, $k = 3{,}116$, $Pr = 1$). A destra: curve neutre per i tre tipi di condizioni al contorno e stima variazionale con $w = z^2(1-z)^2$, indistinguibile dalla curva esatta.

8.6.3 Oltre la soglia

La teoria lineare dice *quando* le celle nascono e con quale lunghezza d'onda, ma non quale ampiezza raggiungono: l'ampiezza A cresce esponenzialmente finché i termini quadratici trascurati non diventano importanti. L'analisi debolmente non lineare (Landau, 1944; per questo problema Malkus e Veronis, 1958) mostra che vicino alla soglia vale un'equazione del tipo $\mathrm{d}A/\mathrm{d}t = \sigma A - \ell A^3$ con $\ell > 0$: la crescita si satura a $A^2 = \sigma/\ell \propto Ra - Ra_c$. La biforcazione è *supercritica*, cioè continua: le celle compaiono con ampiezza infinitesima e crescono con la radice quadrata dell'eccesso di Ra. Il flusso termico attraverso lo strato, misurato dal numero di Nusselt Nu (rapporto fra il flusso totale e quello conduttivo), cresce allora come $Nu - 1 \propto (Ra - Ra_c)/Ra_c$ subito oltre la soglia, e come $Ra^{1/3}$ nel regime turbolento, a Ra molto grande, in cui lo strato si riduce a due sottili strati limite termici alle pareti e un nucleo ben rimescolato. Correlazioni di questo tipo servono a stimare le perdite di calore attraverso le intercapedini: per uno strato orizzontale d'aria di 1,5 cm riscaldato dal basso con $\Delta T = 20\,\mathrm{K}$ (un lucernario a doppio vetro d'inverno) si ha $Ra \simeq 7000$ e la correlazione di Hollands dà $Nu \simeq 2$: l'intercapedine disperde il doppio di quanto farebbe per sola conduzione.

Un'ultima osservazione storica. Le celle esagonali fotografate da Bénard nel 1900 in strati sottilissimi di olio con la superficie libera non erano dovute al galleggiamento, ma alla *tensione superficiale*, che dipende dalla temperatura e trascina il fluido superficiale dalle zone calde a quelle fredde (effetto Marangoni; teoria di Pearson, 1958). Il numero di Rayleigh di quegli esperimenti era molto inferiore a 1708. Il meccanismo analizzato da Rayleigh, e in questo capitolo, è dominante negli strati spessi; nei film sottili e nelle condizioni di microgravità domina quello di Marangoni, con una soglia espressa da un numero diverso, $Ma = (\mathrm{d}\gamma/\mathrm{d}T)\,\Delta T\, H/(\mu\kappa)$, critico attorno a 80.

Capitolo 9

Stabilità idrodinamica II: metodi spettrali di Chebyshev e l'equazione di Orr–Sommerfeld

9.1 L'idea dei metodi spettrali

Le differenze finite approssimano la derivata in un nodo con i valori di pochi nodi vicini: l'interpolazione è *locale*, e l'errore decresce come una potenza fissa del passo, h^2 o h^4. I **metodi spettrali** usano invece un unico polinomio (o una serie di Fourier) che interpola i valori in *tutti* i nodi, e derivano quello. Se la soluzione è regolare, l'errore decresce più rapidamente di qualunque potenza di $1/N$, tipicamente come e^{-cN}: si parla di **accuratezza spettrale**. Con poche decine di nodi si ottengono risultati che con le differenze finite richiederebbero migliaia di nodi. Il prezzo è che le matrici diventano piene, e che il metodo funziona al meglio in geometrie semplici con soluzioni regolari: esattamente il caso dei problemi di stabilità lineare. L'uso dei metodi spettrali per la stabilità delle correnti parallele risale al lavoro di Orszag [35] sulla corrente di Poiseuille; esposizioni moderne si trovano in Trefethen [37], Moin [20] e Canuto et al. [38].

9.2 I polinomi di Chebyshev

Per funzioni non periodiche su $[-1,1]$ la base naturale è quella dei polinomi di Chebyshev,

$$T_k(y) = \cos(k\,\vartheta), \qquad y = \cos\vartheta, \tag{9.1}$$

che sono effettivamente polinomi di grado k in y, come mostra la relazione di ricorrenza ottenuta dalle formule di prostaferesi, $\cos(k+1)\vartheta + \cos(k-1)\vartheta = 2\cos\vartheta\cos k\vartheta$:

$$T_0 = 1, \quad T_1 = y, \quad T_2 = 2y^2 - 1, \quad T_3 = 4y^3 - 3y, \quad \dots, \quad T_{k+1} = 2yT_k - T_{k-1}. \tag{9.2}$$

Il polinomio T_N ha N zeri in $y_j = \cos[\pi(j-\tfrac{1}{2})/N]$ e $N+1$ estremi, di valore ± 1, nei **nodi di Chebyshev–Gauss–Lobatto**

$$y_j = \cos\frac{j\pi}{N}, \qquad j = 0, 1, \dots, N, \tag{9.3}$$

con $y_0 = 1$ e $y_N = -1$. Questi nodi non sono equispaziati: sono le proiezioni sull'asse di punti equispaziati su una semicirconferenza, e si addensano verso gli estremi, con una spaziatura

$\mathcal{O}(N^{-2})$ vicino a ± 1 contro $\mathcal{O}(N^{-1})$ al centro. Due vantaggi: l'interpolazione polinomiale su questi nodi non soffre del fenomeno di Runge (le oscillazioni che rendono inutilizzabile l'interpolazione di grado alto su nodi equispaziati), e l'addensamento cade proprio dove le correnti di parete hanno i gradienti più forti.

Una funzione si sviluppa in serie di polinomi di Chebyshev, $u(y) = \sum_{k=0}^{N} a_k T_k(y)$; per esempio $y^2 = \frac{1}{2}T_0 + \frac{1}{2}T_2$. Nei nodi (9.3), grazie all'ortogonalità discreta dei coseni, i coefficienti si calcolano dai valori nodali con

$$a_k = \frac{2}{N\bar{c}_k} \sum_{j=0}^{N} \frac{1}{\bar{c}_j} T_k(y_j)\, u_j, \qquad \bar{c}_0 = \bar{c}_N = 2, \;\; \bar{c}_j = 1 \text{ altrimenti}, \tag{9.4}$$

che è, a meno di fattori, una trasformata discreta del coseno, calcolabile con la FFT.

9.3 La matrice di differenziazione

Nel metodo di *collocazione* non si lavora con i coefficienti a_k ma direttamente con i valori nodali u_j. Il polinomio interpolante di grado N è unico; la sua derivata calcolata nei nodi è una combinazione lineare dei valori nodali, $u'(y_j) \simeq \sum_k D_{jk} u_k$. La matrice $\mathbf{D}$, di dimensione $(N+1)\times(N+1)$, è la **matrice di differenziazione di Chebyshev**.

Gli elementi della matrice

Derivando i polinomi di Lagrange associati ai nodi (9.3) si ottiene [37]

$$\begin{aligned} D_{00} &= \frac{2N^2+1}{6}, \qquad D_{NN} = -\frac{2N^2+1}{6}, \\ D_{jj} &= \frac{-y_j}{2(1-y_j^2)} \;\; (j = 1,\dots,N-1), \quad D_{jk} = \frac{\bar{c}_j}{\bar{c}_k}\frac{(-1)^{j+k}}{y_j - y_k} \;\; (j \neq k). \end{aligned} \tag{9.5}$$

Poiché la derivata di una costante è nulla, la somma degli elementi di ogni riga deve essere zero: nel calcolo conviene ricavare la diagonale come $D_{jj} = -\sum_{k\neq j} D_{jk}$, che riduce gli errori di arrotondamento. Le derivate di ordine superiore si ottengono semplicemente come potenze: $u'' \simeq \mathbf{D}^2\mathbf{u}$, $u^{\mathrm{iv}} \simeq \mathbf{D}^4\mathbf{u}$, perché la derivata di un polinomio di grado N è ancora un polinomio, interpolato esattamente.

La funzione `cheb(N)` del modulo comune (Listato A.1) costruisce $\mathbf{D}$ e i nodi in poche righe, nello stile di Trefethen. Il Programma 3.7 ne verifica due proprietà: la derivata di y^2 è esatta ($\max|\mathbf{D}\mathbf{y}^2 - 2\mathbf{y}| = 3{,}6\times10^{-15}$), e l'elemento d'angolo vale $(2N^2+1)/6$.

9.4 Un problema ai limiti con la collocazione di Chebyshev

Consideriamo il problema

$$u'' + \pi u = \pi, \qquad u(-1) = 0, \qquad u(1) = 1. \tag{9.6}$$

La soluzione generale è $u = 1 + A\cos(\sqrt{\pi}\,y) + B\sin(\sqrt{\pi}\,y)$; imponendo le condizioni agli estremi,

$$u(y) = 1 - \frac{\cos(\sqrt{\pi}\,y)}{2\cos\sqrt{\pi}} + \frac{\sin(\sqrt{\pi}\,y)}{2\sin\sqrt{\pi}}. \tag{9.7}$$

Nei nodi di collocazione l'equazione diventa il sistema lineare $(\mathbf{D}^2 + \pi\mathbf{I})\mathbf{u} = \pi\mathbf{1}$. Le righe relative ai nodi di contorno ($j = 0$ e $j = N$) non devono imporre l'equazione differenziale ma le condizioni al contorno: si sostituiscono con le righe corrispondenti della matrice identità e si pongono i termini noti uguali a 1 e a 0:

$$\begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \\ a_{1,0} & a_{1,1} & \cdots & a_{1,N-1} & a_{1,N} \\ \vdots & & \ddots & & \vdots \\ a_{N-1,0} & \cdots & & a_{N-1,N-1} & a_{N-1,N} \\ 0 & 0 & \cdots & 0 & 1 \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \\ u_N \end{bmatrix} = \begin{bmatrix} 1 \\ \pi \\ \vdots \\ \pi \\ 0 \end{bmatrix}, \qquad a_{jk} = (\mathbf{D}^2)_{jk} + \pi\delta_{jk}. \tag{9.8}$$

Si ricordi che $y_0 = +1$: la prima riga impone la condizione nell'estremo superiore.

prog3_7_chebyshev.py

```python
"""
prog3_7_chebyshev.py -- PROGRAMMA 3.7
Il metodo di collocazione di Chebyshev per un problema ai limiti:
    u'' + pi u = pi,   u(-1) = 0,   u(1) = 1,
la cui soluzione esatta e'
    u = 1 - cos(sqrt(pi) y) / (2 cos sqrt(pi)) + sin(sqrt(pi) y) / (2 sin sqrt(pi)).
Nei nodi y_j = cos(j pi/N) la derivata seconda e' D^2 u; le righe della
matrice relative ai nodi di contorno sono sostituite dalle condizioni al
contorno. Si confronta la convergenza (esponenziale) con quella delle
differenze finite centrate del secondo ordine.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import cheb
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO

s = np.sqrt(np.pi)


def esatta(y):
    return 1 - np.cos(s * y) / (2 * np.cos(s)) + np.sin(s * y) / (2 * np.sin(s))


def chebyshev(N):
    D, y = cheb(N)
    A = D @ D + np.pi * np.eye(N + 1)
    c = np.pi * np.ones(N + 1)
    A[0, :] = 0.0                    # y_0 = +1 :  u = 1
    A[0, 0] = 1.0
    c[0] = 1.0
    A[N, :] = 0.0                    # y_N = -1 :  u = 0
    A[N, N] = 1.0
    c[N] = 0.0
    return y, np.linalg.solve(A, c)


def differenze_finite(N):
    y = np.linspace(-1, 1, N + 1)
    h = y[1] - y[0]
    A = np.zeros((N + 1, N + 1))
    c = np.pi * np.ones(N + 1)
    for i in range(1, N):
        A[i, i - 1:i + 2] = [1 / h**2, -2 / h**2 + np.pi, 1 / h**2]
    A[0, 0] = A[N, N] = 1.0
    c[0], c[N] = 0.0, 1.0
    return y, np.linalg.solve(A, c)
```

```
47
48
49 # verifica della matrice di differenziazione su u = y^2 (coefficienti a0 = a2 = 1/2)
50 D, y = cheb(8)
51 print(f"Verifica: max |D y^2 - 2y| = {np.max(np.abs(D @ y**2 - 2 * y)):.1e},"
52       f"  max |D^2 y^2 - 2| = {np.max(np.abs(D @ D @ y**2 - 2)):.1e}")
53 print(f"Elementi d'angolo: D[0,0] = {D[0,0]:.4f} = (2N^2+1)/6 = {(2*64+1)/6:.4f}")
54
55 print("\n   N    errore Chebyshev    errore diff. finite")
56 Nlista = [4, 6, 8, 10, 12, 16, 20, 24, 32]
57 e_ch, e_df = [], []
58 for N in Nlista:
59     yc, uc = chebyshev(N)
60     yd, ud = differenze_finite(N)
61     e_ch.append(np.max(np.abs(uc - esatta(yc))))
62     e_df.append(np.max(np.abs(ud - esatta(yd))))
63     print(f"  {N:3d}    {e_ch[-1]:.3e}           {e_df[-1]:.3e}")
64
65 # ------------------------------------------------------------------ figura
66 fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
67 yy = np.linspace(-1, 1, 300)
68 y8, u8 = chebyshev(8)
69 ax[0].plot(yy, esatta(yy), color=GRIGIO, lw=1.8, label="esatta")
70 ax[0].plot(y8, u8, "o", color=BLU, ms=5, label="Chebyshev, 9 nodi")
71 ax[0].set_xlabel("$y$")
72 ax[0].set_ylabel("$u$")
73 ax[0].set_title(r"$u''+\pi u=\pi$")
74 ax[0].legend()
75 ax[1].semilogy(Nlista, e_ch, "o-", color=BLU, label="Chebyshev")
76 ax[1].semilogy(Nlista, e_df, "s-", color=ARANCIO, label="differenze finite")
77 ax[1].set_xlabel("$N$ (numero di intervalli)")
78 ax[1].set_ylabel("errore massimo")
79 ax[1].legend()
80 fig.tight_layout()
81 salva(fig, "prog3_7_chebyshev")
```

Listato 9.1: Programma 3.7 – il problema $u'' + \pi u = \pi$ con la collocazione di Chebyshev (`prog3_7_chebyshev.py`).

Risultati del Programma 3.7

N	errore Chebyshev	errore differenze finite
4	$8{,}277 \times 10^{-2}$	$5{,}906 \times 10^{-1}$
8	$1{,}906 \times 10^{-6}$	$1{,}686 \times 10^{-1}$
12	$3{,}203 \times 10^{-11}$	$7{,}698 \times 10^{-2}$
16	$3{,}775 \times 10^{-14}$	$4{,}372 \times 10^{-2}$
32	$2{,}189 \times 10^{-13}$	$1{,}103 \times 10^{-2}$

Con soli 9 nodi ($N = 8$) la soluzione di Chebyshev è esatta a sei cifre (Fig. 9.1); con 17 nodi raggiunge la precisione di macchina. Le differenze finite centrate, con lo stesso numero di nodi, hanno un errore del 17% e del 4% e convergono come N^{-2}. Oltre $N \simeq 20$ l'errore di Chebyshev risale leggermente: è l'arrotondamento, amplificato dal numero di condizionamento di $\mathbf{D}^2$, che cresce come N^4.

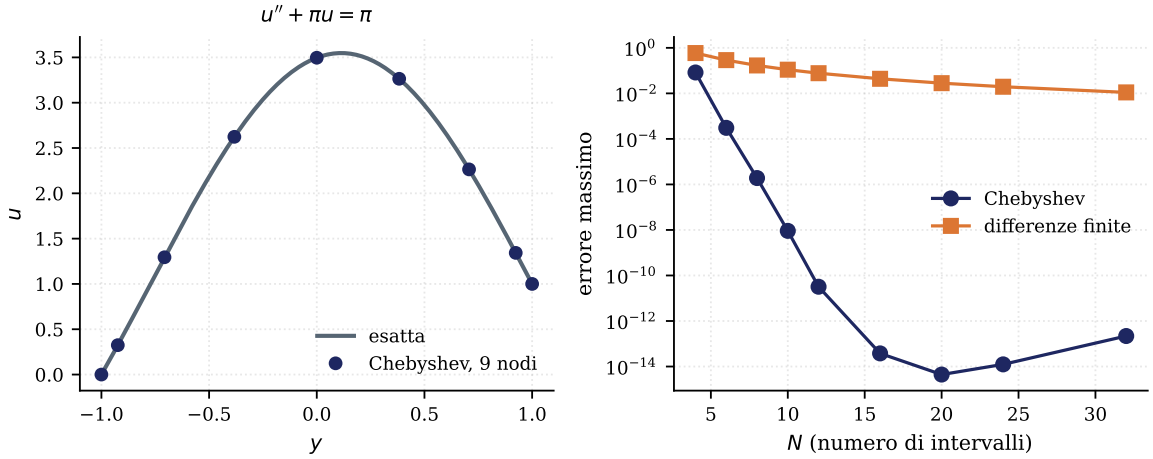


Figura 9.1: Problema $u'' + \pi u = \pi$: soluzione di Chebyshev con 9 nodi confrontata con quella esatta (a sinistra) e convergenza esponenziale della collocazione confrontata con quella algebrica delle differenze finite (a destra).

9.5 Perché i nodi di Chebyshev: il fenomeno di Runge e l'accuratezza spettrale

Le affermazioni fatte sui nodi di Chebyshev (niente fenomeno di Runge, convergenza esponenziale) meritano una giustificazione, perché su di esse si regge tutto il metodo.

L'errore di interpolazione e la scelta dei nodi

Se p_N è il polinomio di grado N che interpola u nei nodi $y_0, \dots, y_N$, per una funzione $N+1$ volte derivabile vale

$$u(y) - p_N(y) = \frac{u^{(N+1)}(\xi)}{(N+1)!} \prod_{j=0}^{N} (y - y_j), \qquad \xi \in [-1, 1].$$

Il prodotto $\omega_N(y) = \prod (y - y_j)$ dipende solo dai nodi. Con nodi equispaziati $|\omega_N|$ è piccolissimo al centro e enorme vicino agli estremi (vale circa $N!\, h^{N+1}/4$ nel primo intervallo, contro $h^{N+1}\sqrt{N}$ circa al centro): l'interpolante oscilla vicino ai bordi, e per funzioni come quella di Runge, $1/(1+25y^2)$, le cui derivate crescono come $5^N N!$, l'errore *diverge* al crescere di N. Gli zeri di T_{N+1} minimizzano $\max|\omega_N|$ su $[-1,1]$: con essi $\omega_N = 2^{-N} T_{N+1}$, che oscilla fra $\pm 2^{-N}$ uniformemente (teorema di Chebyshev sui polinomi di minima deviazione); con i nodi di Gauss–Lobatto (9.3), gli estremi di T_N, si ha $\omega_N = 2^{1-N}(y^2-1)T_N'(y)/N$, altrettanto uniforme. È la ragione dell'addensamento dei nodi verso gli estremi: dove i nodi sono fitti, ω_N resta piccolo.

Il tasso di convergenza. Per una funzione analitica il risultato è più preciso. Se u si prolunga analiticamente all'interno dell'ellisse del piano complesso con fuochi ± 1 e somma dei semiassi $\rho > 1$ (*ellisse di Bernstein*), l'errore di interpolazione nei nodi di Chebyshev decresce come ρ^{-N}. Per la funzione di Runge i poli sono in $y = \pm \mathrm{i}/5$, l'ellisse più grande che li evita ha semiasse minore $1/5$ e $\rho = 1/5 + \sqrt{1 + 1/25} = 1{,}2198$: l'errore si dimezza ogni 3,5 nodi aggiunti. Più la singolarità è lontana dall'intervallo, più rapida è la convergenza; per una funzione intera (senza singolarità al finito, come e^y o $\cos\sqrt{\pi}y$ del Programma 3.7) la convergenza è più rapida di qualunque esponenziale.

La costante di Lebesgue. L'operatore di interpolazione ha norma $\Lambda_N = \max_y \sum_j |l_j(y)|$ (l_j polinomi di Lagrange), che misura quanto gli errori sui dati vengono amplificati: $\|u - p_N\| \leq (1 + \Lambda_N)\,\|u - p_N^*\|$, con p_N^* il miglior polinomio. Per nodi equispaziati $\Lambda_N \sim 2^{N+1}/(eN \ln N)$, per quelli di Chebyshev $\Lambda_N \sim (2/\pi)\ln N$: con $N = 64$, 10^{18} contro 3,6.

prog3_7b_runge.py

```
"""
prog3_7b_runge.py -- perche' i nodi di Chebyshev: il fenomeno di Runge,
la costante di Lebesgue e la velocita' di convergenza spettrale.

1) Interpolazione polinomiale della funzione di Runge f(y) = 1/(1 + 25 y^2)
   su nodi equispaziati e su nodi di Chebyshev-Gauss-Lobatto: con i primi
   l'errore cresce con N (vicino agli estremi), con i secondi decresce come
   rho^(-N), dove rho = 1/5 + sqrt(1 + 1/25) = 1,2198 e' il parametro
   dell'ellisse di Bernstein che passa per i poli y = +-i/5.
2) Costante di Lebesgue Lambda_N (norma dell'operatore di interpolazione):
   cresce come 2^N/(e N log N) per i nodi equispaziati e come (2/pi) log N
   per quelli di Chebyshev.
3) Numero di condizionamento della matrice di differenziazione: cond(D^2) ~ N^4.
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import BarycentricInterpolator
from metodi_numerici import cheb
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

f = lambda y: 1.0 / (1 + 25 * y**2)
yy = np.linspace(-1, 1, 2001)
rho = 0.2 + np.sqrt(1 + 0.04)


def interpola(nodi, valori, y):
    return BarycentricInterpolator(nodi, valori)(y)


def lebesgue(nodi, y):
    """Lambda = max_y sum_j |l_j(y)| con i polinomi di Lagrange l_j."""
    n = len(nodi)
    somma = np.zeros_like(y)
    for j in range(n):
        e = np.zeros(n); e[j] = 1.0
        somma += np.abs(interpola(nodi, e, y))
    return somma.max()


print("Funzione di Runge 1/(1 + 25 y^2): errore massimo dell'interpolazione")
print("      N   equispaziati    Chebyshev     rho^-N      Lambda_eq
    Lambda_cheb")
Ns = [4, 8, 12, 16, 20, 24, 32, 40, 48, 64]
err_eq, err_ch, leb_eq, leb_ch = [], [], [], []
for N in Ns:
    ye = np.linspace(-1, 1, N + 1)
    _, yc = cheb(N)
    ee = np.max(np.abs(interpola(ye, f(ye), yy) - f(yy)))
    ec = np.max(np.abs(interpola(yc, f(yc), yy) - f(yy)))
    le, lc = lebesgue(ye, yy), lebesgue(yc, yy)
    err_eq.append(ee); err_ch.append(ec); leb_eq.append(le); leb_ch.append(lc)
    print(f"   {N:4d}   {ee:11.3e}   {ec:11.3e}   {rho**-N:9.2e}   {le:10.3e}
    {lc:8.3f}")
```

```
# tasso di convergenza osservato con Chebyshev
pend = -np.polyfit(Ns[2:7], np.log(err_ch[2:7]), 1)[0]
print(f"   tasso osservato (Chebyshev, 12 <= N <= 32): errore ~ exp(-{pend:.4f} N) = {np.exp(pend):.4f}^-N;"
      f" teoria: rho = {rho:.4f}")
print("\nCondizionamento della matrice di differenziazione (righe di contorno sostituite con l'identita'):")
for N in (8, 16, 32, 64):
    D, y = cheb(N)
    D2 = D @ D
    D2[0, :] = 0; D2[0, 0] = 1; D2[-1, :] = 0; D2[-1, -1] = 1
    print(f"   N = {N:3d}: cond(D^2) = {np.linalg.cond(D2):.2e}    (N^4 = {N**4:.1e})")

fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.8))
for N, col in ((8, VERDE), (16, ARANCIO)):
    ye = np.linspace(-1, 1, N + 1)
    ax[0].plot(yy, interpola(ye, f(ye), yy), color=col, label=f"equispaziati, $N={N}$")
_, yc = cheb(16)
ax[0].plot(yy, interpola(yc, f(yc), yy), "--", color=BLU, label="Chebyshev, $N=16$")
ax[0].plot(yy, f(yy), "k:", lw=1.2, label="$1/(1+25y^2)$")
ax[0].set_ylim(-0.8, 1.6)
ax[0].set_xlabel("$y$")
ax[0].legend(fontsize=6.5, loc="upper center")
ax[0].set_title("fenomeno di Runge")
ax[1].semilogy(Ns, err_eq, "s-", color=ROSSO, label="equispaziati")
ax[1].semilogy(Ns, err_ch, "o-", color=BLU, label="Chebyshev")
ax[1].semilogy(Ns, 2.0 * rho**(-np.array(Ns)), ":", color=GRIGIO, label="$\\propto\\rho^{-N}$")
ax[1].set_xlabel("$N$")
ax[1].set_ylabel("errore massimo")
ax[1].legend(fontsize=7)
ax[1].set_title("convergenza")
ax[2].semilogy(Ns, leb_eq, "s-", color=ROSSO, label="equispaziati")
ax[2].semilogy(Ns, leb_ch, "o-", color=BLU, label="Chebyshev")
ax[2].semilogy(Ns, 2 / np.pi * np.log(np.array(Ns)) + 1, ":", color=GRIGIO, label="$(2/\\pi)\\ln N+1$")
ax[2].set_xlabel("$N$")
ax[2].set_ylabel("$\\Lambda_N$")
ax[2].legend(fontsize=7)
ax[2].set_title("costante di Lebesgue")
fig.tight_layout()
salva(fig, "prog3_7b_runge")
```

Listato 9.2: `prog3_7b_runge.py`: interpolazione della funzione di Runge su nodi equispaziati e di Chebyshev, tasso di convergenza, costante di Lebesgue, condizionamento di $\mathbf{D}^2$.

Risultati del Programma 3.7b

N	errore, equispaziati	errore, Chebyshev	ρ^{-N}	Λ_N equisp.	Λ_N Cheb.
8	1,045	$2{,}047 \times 10^{-1}$	$2{,}04 \times 10^{-1}$	$1{,}095 \times 10^{1}$	2,275
16	$1{,}439 \times 10^{1}$	$3{,}671 \times 10^{-2}$	$4{,}16 \times 10^{-2}$	$9{,}345 \times 10^{2}$	2,725
32	$5{,}059 \times 10^{3}$	$1{,}618 \times 10^{-3}$	$1{,}73 \times 10^{-3}$	$2{,}431 \times 10^{7}$	3,168
64	$9{,}259 \times 10^{11}$	$2{,}865 \times 10^{-6}$	$3{,}00 \times 10^{-6}$	$7{,}105 \times 10^{18}$	3,609

Il tasso osservato fra $N = 12$ e 32 è $1{,}2171^{-N}$, contro il valore teorico $\rho = 1{,}2198$;

l'errore con i nodi di Chebyshev coincide quasi esattamente con ρ^{-N}. Il numero di condizionamento di $\mathbf{D}^2$ (con le righe di contorno sostituite) cresce come N^4: 5×10^2, 1×10^4, 2×10^5, 5×10^6 per $N=8$, 16, 32, 64. È il limite pratico della collocazione di Chebyshev con l'aritmetica a doppia precisione: con $N=64$ si perdono sei o sette delle sedici cifre, e per l'operatore del quarto ordine di Orr–Sommerfeld ($\mathbf{D}^4$, condizionamento $\sim N^8$) non conviene superare $N\simeq 100$–150.

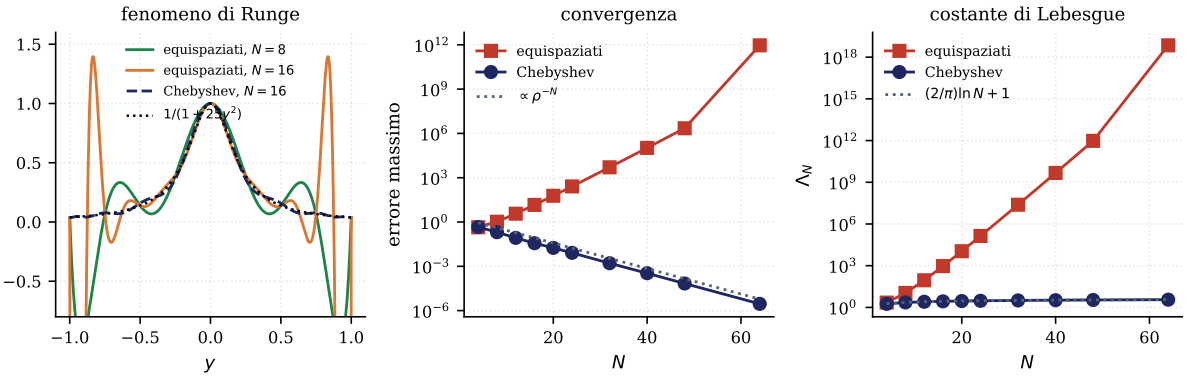


Figura 9.2: Fenomeno di Runge: interpolazione su nodi equispaziati e di Chebyshev (a sinistra); errore massimo in funzione di N con la previsione ρ^{-N} (al centro); costante di Lebesgue (a destra).

9.6 La stabilità della corrente di Poiseuille piana

Consideriamo la corrente laminare sviluppata fra due lastre parallele, prodotta da un gradiente di pressione costante (Fig. 9.3). Per uniformarsi alla notazione consueta in questo problema, l'asse verticale è ora l'asse y e le lastre sono in $y=\pm h$. Con la velocità sulla mezzeria $U_c=-(h^2/2\mu)\,\mathrm{d}p/\mathrm{d}x$ e le lunghezze riferite a h, il profilo di base è quello di Poiseuille già incontrato nel Cap. 6:

$$U(y)=1-y^2, \qquad -1\le y\le 1. \tag{9.9}$$

Segnaposto figura

file atteso: `immagini/fig_3_8_2.png`

Canale piano fra le lastre $y/h=\pm1$ con profilo di velocità parabolico $u/U_c=1-(y/h)^2$ (Fig. 3.8.2 della fonte).

Figura 9.3: La corrente di Poiseuille piana.

Si sovrappone alla corrente di base una perturbazione bidimensionale di piccola ampiezza, $u=U(y)+\hat{u}$, $v=\hat{v}$, $p=P(x)+\hat{p}$. Sostituendo nelle equazioni di Navier–Stokes, sottraendo la soluzione di base e trascurando i prodotti delle perturbazioni (con $\hat{p}$ pressione divisa per la

densità):

$$\frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y} = 0, \tag{9.10}$$

$$\frac{\partial \hat{u}}{\partial t} + U\frac{\partial \hat{u}}{\partial x} + \hat{v}\,U' + \frac{\partial \hat{p}}{\partial x} = \nu\nabla^2\hat{u}, \tag{9.11}$$

$$\frac{\partial \hat{v}}{\partial t} + U\frac{\partial \hat{v}}{\partial x} + \frac{\partial \hat{p}}{\partial y} = \nu\nabla^2\hat{v}. \tag{9.12}$$

Il termine $\hat{v}\,U'$ è quello che alimenta la perturbazione: un moto verticale trasporta fluido lento in zone veloci e viceversa, estraendo energia dal profilo di base.

L'equazione di Orr–Sommerfeld

Le equazioni sono lineari a coefficienti dipendenti solo da y: si cercano modi normali in forma di onde progressive,

$$(\hat{u},\hat{v},\hat{p}) = (\tilde{u}(y),\tilde{v}(y),\tilde{p}(y))\,e^{\mathrm{i}\alpha(x-ct)}, \tag{9.13}$$

con numero d'onda α reale e velocità d'onda $c = c_R + \mathrm{i}c_I$ complessa (problema *temporale*). Poiché $e^{-\mathrm{i}\alpha ct} = e^{-\mathrm{i}\alpha c_R t}e^{\alpha c_I t}$, c_R è la velocità di fase e αc_I il tasso di crescita: per $c_I < 0$ la perturbazione si smorza, per $c_I > 0$ cresce. Sostituendo:

$$\begin{gathered}\mathrm{i}\alpha\tilde{u} + \tilde{v}' = 0,\\ \mathrm{i}\alpha(U-c)\tilde{u} + U'\tilde{v} + \mathrm{i}\alpha\tilde{p} = \nu(\tilde{u}'' - \alpha^2\tilde{u}),\\ \mathrm{i}\alpha(U-c)\tilde{v} + \tilde{p}' = \nu(\tilde{v}'' - \alpha^2\tilde{v}).\end{gathered}$$

Dalla continuità $\tilde{u} = \mathrm{i}\tilde{v}'/\alpha$; sostituendolo nella seconda equazione si ricava $\tilde{p}$; derivando $\tilde{p}$ e sostituendolo nella terza, dopo l'adimensionalizzazione con h e U_c, si ottiene l'**equazione di Orr–Sommerfeld**

$$(U-c)\left(\tilde{v}'' - \alpha^2\tilde{v}\right) - U''\tilde{v} = -\frac{\mathrm{i}}{\alpha Re}\left(\tilde{v}^{\mathrm{iv}} - 2\alpha^2\tilde{v}'' + \alpha^4\tilde{v}\right), \qquad Re = \frac{U_c h}{\nu}, \tag{9.14}$$

con le condizioni di aderenza $\tilde{v} = \tilde{v}' = 0$ in $y = \pm 1$ (la seconda viene da $\tilde{u} = 0$). Equazione e condizioni sono omogenee: per α e Re assegnati, si hanno soluzioni non nulle solo per valori particolari di c, gli autovalori.

Il teorema di Squire (1933) assicura che, per ogni perturbazione tridimensionale instabile, ne esiste una bidimensionale instabile a un numero di Reynolds più basso: per trovare il Reynolds critico basta studiare le perturbazioni bidimensionali. Per $Re \to \infty$ la (9.14) si riduce all'equazione di Rayleigh del fluido ideale, per la quale un profilo senza punti di flesso, come quello di Poiseuille, non può essere instabile: l'instabilità che si trova è un effetto, paradossale, della viscosità.

9.6.1 Il problema agli autovalori generalizzato

Posto $\lambda = -\mathrm{i}c$ (quindi $\lambda_R = c_I$: un autovalore con parte reale positiva è un'onda crescente) e moltiplicata la (9.14) per $-\mathrm{i}$, con $U = 1 - y^2$ e $U'' = -2$, l'equazione diventa

$$\mathbf{A}\mathbf{v} = \lambda\,\mathbf{B}\mathbf{v}, \qquad \begin{aligned}\mathbf{A} &= \frac{1}{\alpha Re}\left(\mathbf{D}^4 - 2\alpha^2\mathbf{D}^2 + \alpha^4\mathbf{I}\right) - 2\mathrm{i}\,\mathbf{I} - \mathrm{i}\,\mathrm{diag}(1-y_j^2)\left(\mathbf{D}^2 - \alpha^2\mathbf{I}\right),\\ \mathbf{B} &= \mathbf{D}^2 - \alpha^2\mathbf{I},\end{aligned} \tag{9.15}$$

dove le derivate sono le matrici di Chebyshev. Le quattro condizioni al contorno si impongono sostituendo quattro righe: la prima e l'ultima di **A** con quelle dell'identità ($\tilde{v} = 0$ in $y = 1$ e in $y = -1$), la seconda e la penultima con la prima e l'ultima riga di **D** ($\tilde{v}' = 0$); le stesse righe di **B** si annullano. Il problema generalizzato si risolve con l'algoritmo QZ e gli autovalori infiniti introdotti dalle righe nulle di **B** si scartano.


prog3_8_orr_sommerfeld.py

```
"""
prog3_8_orr_sommerfeld.py -- PROGRAMMA 3.8
Stabilita' lineare della corrente di Poiseuille piana U = 1 - y^2:
equazione di Orr-Sommerfeld risolta con la collocazione di Chebyshev.

Con lam = -i c (c velocita' complessa dell'onda) l'equazione diventa il problema
agli autovalori generalizzato  A v = lam B v  con
    A = (D^4 - 2 alfa^2 D^2 + alfa^4 I)/(alfa Re) - 2 i I - i diag(1 - y^2)(D^2 -
    alfa^2 I),
    B = D^2 - alfa^2 I.
Condizioni v = v' = 0 in y = +-1: si sostituiscono la prima e l'ultima riga di A
con le righe dell'identita' e la seconda e la penultima con la prima e l'ultima
riga di D (le righe corrispondenti di B si annullano: gli autovalori
"infiniti" che ne derivano vengono scartati).
Poiche' lam = -i c = c_I - i c_R, la parte reale lam_R coincide con c_I: se e'
positiva la perturbazione cresce come exp(alfa c_I t)  (onda di
    Tollmien-Schlichting).
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eig
from metodi_numerici import cheb
from stile_grafici import salva, BLU, ROSSO, GRIGIO


def orr_sommerfeld(Re, alfa, N=100):
    D, y = cheb(N)
    D2 = D @ D
    D4 = D2 @ D2
    I = np.eye(N + 1)
    A = (D4 - 2 * alfa**2 * D2 + alfa**4 * I) / (Re * alfa) - 2j * I \
        - 1j * np.diag(1 - y**2) @ (D2 - alfa**2 * I)
    B = (D2 - alfa**2 * I).astype(complex)
    A[0, :] = 0.0
    A[0, 0] = 1.0                    # v(+1) = 0
    A[1, :] = D[0, :]                # v'(+1) = 0
    A[N, :] = 0.0
    A[N, N] = 1.0                    # v(-1) = 0
    A[N - 1, :] = D[N, :]            # v'(-1) = 0
    B[[0, 1, N - 1, N], :] = 0.0
    lam, V = eig(A, B)
    ok = np.isfinite(lam) & (np.abs(lam) < 1e3)
    return lam[ok], V[:, ok], y


if __name__ == "__main__":
    risultati = {}
    for Re in (5772.0, 7500.0):
        lam, V, y = orr_sommerfeld(Re, 1.02)
        j = np.argmax(lam.real)
        risultati[Re] = (lam, V[:, j], y)
        c = 1j * lam[j]                                  # c = i lam
        print(f"Re = {Re:6.0f}, alfa = 1,02:  (lam_R)max = {lam[j].real:+.10f}")
        print(f"     modo meno stabile: c = c_R + i c_I = {c.real:.6f}
```

```
    {c.imag:+.6f} i "
53              f"(velocita' di fase {c.real:.4f} U_c)")
54      # sensibilita' alla risoluzione
55      print("\nConvergenza in N (Re = 5772, alfa = 1,02):")
56      for N in (40, 60, 80, 100, 140):
57          lam, _, _ = orr_sommerfeld(5772.0, 1.02, N)
58          print(f"   N = {N:4d}   (lam_R)max = {lam.real.max():+.3e}   lam =
    {lam[np.argmax(lam.real)]:.8f}")
59
60      # ------------------------------------------------------------ figure
61      fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.4))
62      for a, Re in zip(ax, (5772.0, 7500.0)):
63          lam = risultati[Re][0]
64          a.plot(lam.real, lam.imag, ".", ms=4, color=BLU)
65          jm = np.argmax(lam.real)
66          a.plot(lam.real[jm], lam.imag[jm], "o", ms=7, mfc="none", color=ROSSO)
67          a.axvline(0, color=GRIGIO, lw=0.8)
68          a.set_xlim(-1, 0.2)
69          a.set_ylim(-1, 0)
70          a.set_aspect("equal")
71          a.set_xlabel(r"$\lambda_R$")
72          a.set_ylabel(r"$\lambda_I$")
73          a.set_title(f"$Re={Re:.0f}$, $\\alpha=1{{,}}02$:
    $(\\lambda_R)_{{\\max}}={lam.real.max():+.2e}$",
74                      fontsize=8)
75      fig.tight_layout()
76      salva(fig, "prog3_8_spettro")
77
78      fig, ax = plt.subplots(figsize=(4.6, 3.0))
79      lam, v, y = risultati[7500.0]
80      D, _ = cheb(len(y) - 1)
81      alfa = 1.02
82      u = 1j * (D @ v) / alfa                          # continuita': i alfa u + v' = 0
83      scala = np.max(np.abs(v))
84      c_R = (1j * lam[np.argmax(lam.real)]).real
85      y_c = np.sqrt(1 - c_R)                           # strato critico: U(y_c) = c_R
86      ax.plot(y, np.abs(v) / scala, color=BLU, label=r"$|\tilde v|$")
87      ax.plot(y, np.abs(u) / scala, "--", color=ROSSO, label=r"$|\tilde u|$")
88      for s in (-1, 1):
89          ax.axvline(s * y_c, color=GRIGIO, ls=":", lw=0.9)
90      ax.text(y_c - 0.02, 0.05, "strato\ncritico", fontsize=6.5, color=GRIGIO,
    ha="right")
91      ax.set_xlabel("$y$")
92      ax.set_ylabel("ampiezza (scala di $|\\tilde v|_{\\max}$)")
93      ax.set_title(r"onda di Tollmien-Schlichting, $Re=7500$, $\alpha=1{,}02$")
94      ax.legend(fontsize=7)
95      salva(fig, "prog3_8_autofunzione")
96      print(f"strato critico: U(y_c) = c_R per y_c = +-{y_c:.4f};  |u|max/|v|max =
    {np.abs(u).max()/scala:.2f}")
```

Listato 9.3: Programma 3.8 – spettro di Orr–Sommerfeld per la corrente di Poiseuille piana (`prog3_8_orr_sommerfeld.py`).

Risultati del Programma 3.8

Con $N = 100$ e $\alpha = 1{,}02$:

- $Re = 5772$: $(\lambda_R)_{\max} = -4{,}2 \times 10^{-7} \simeq 0$. L'onda meno smorzata è praticamente neutra, con velocità di fase $c_R = 0{,}2639\,U_c$: siamo sul punto critico.

- $Re = 7500$: $(\lambda_R)_{\max} = +0{,}002029$, con $c_R = 0{,}2521$. L'onda cresce come $e^{\alpha c_I t}$, cioè di un fattore e ogni 480 unità di tempo h/U_c: lentamente, ma esponenzialmente.
- Il risultato è già stabile con $N = 60$: $(\lambda_R)_{\max} = -4{,}22 \times 10^{-7}$ con $N = 60$, 80, 100, 140 (le variazioni nell'ultima cifra sono dovute all'arrotondamento).

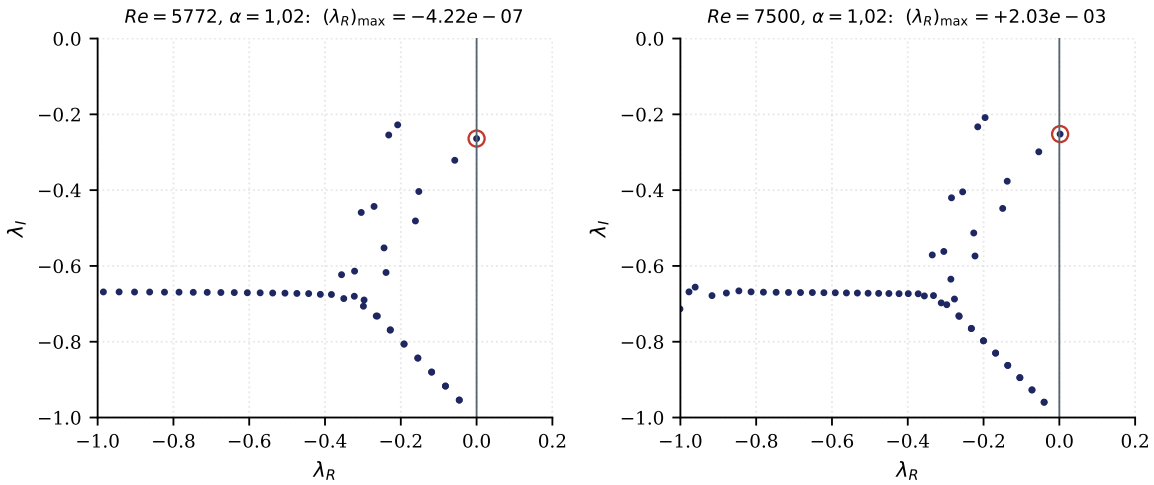


Figura 9.4: Spettro degli autovalori di Orr–Sommerfeld per $\alpha = 1{,}02$ a $Re = 5772$ e $Re = 7500$; cerchiato l'autovalore meno stabile.

Poiché $\lambda_I = -c_R$, nel piano della figura l'asse verticale misura (cambiata di segno) la velocità di fase. Lo spettro ha la caratteristica forma a "Y" coricata, con tre rami che si incontrano attorno a $c_R \simeq 2/3$: il ramo che sale verso destra (c_R piccolo) raccoglie modi concentrati vicino alle pareti, quello che scende verso destra ($c_R \to 1$) modi concentrati al centro del canale, il ramo orizzontale verso sinistra modi fortemente smorzati. Il modo instabile è in cima al ramo di parete: è l'**onda di Tollmien–Schlichting**, che viaggia a circa un quarto della velocità massima.

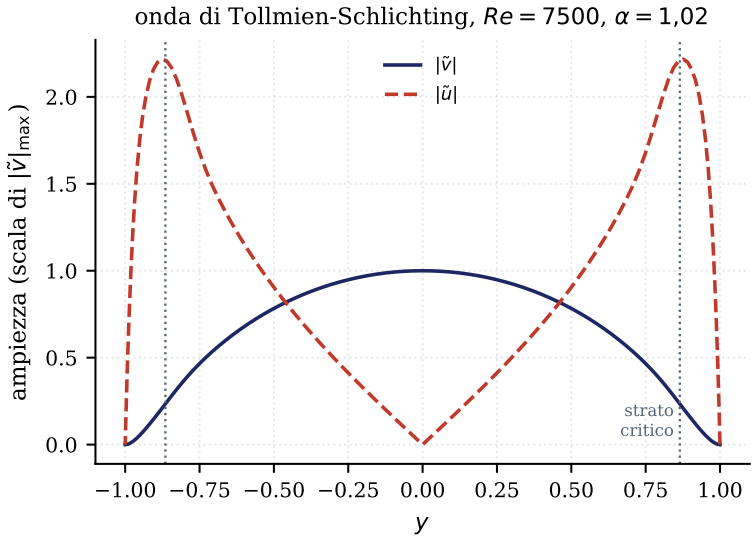


Figura 9.5: Ampiezza delle componenti della perturbazione per l'onda instabile a $Re = 7500$; le linee punteggiate indicano gli strati critici, dove $U(y_c) = c_R$.

La Fig. 9.5 mostra la struttura dell'onda: la componente $\tilde{v}$ è massima al centro, mentre

$\tilde{u} = i\tilde{v}'/\alpha$ ha due massimi vicino alle pareti, in corrispondenza degli *strati critici* $y_c = \pm 0{,}865$ dove la velocità della corrente uguaglia la velocità di fase dell'onda. È lì che la viscosità, per quanto piccola, permette lo scambio di energia fra la corrente e l'onda.

9.7 La curva di stabilità neutra

Ripetendo il calcolo su una griglia di valori di α e Re si individua la regione del piano (Re, α) in cui $c_I > 0$; il suo contorno, $c_I = 0$, è la **curva di stabilità neutra**. Il punto critico, il minimo Re della curva, si trova con più precisione risolvendo $c_{I,\max}(Re, \alpha) = 0$ per ogni α con il metodo di Brent e minimizzando poi $Re(\alpha)$.

prog3_8b_curva_neutra.py

```python
"""
prog3_8b_curva_neutra.py -- curva di stabilita' neutra della corrente di
Poiseuille piana (luogo dei punti in cui (lam_R)max = c_I = 0) e punto critico.

1) Si calcola c_I,max = max lam_R su una griglia (log Re, alfa) e si traccia la
   linea di livello zero.
2) Il punto critico (Re minimo della curva) si ottiene risolvendo, per alfa
   fissato, c_I,max(Re, alfa) = 0 con il metodo di Brent e minimizzando poi
   Re(alfa) con la ricerca della sezione aurea.
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq, minimize_scalar
from prog3_8_orr_sommerfeld import orr_sommerfeld
from stile_grafici import salva, BLU, ROSSO, GRIGIO

N = 80


def ci_max(Re, alfa):
    lam, _, _ = orr_sommerfeld(Re, alfa, N)
    return lam.real.max()


def Re_neutro(alfa):
    """Ramo inferiore della curva neutra per il numero d'onda alfa."""
    return brentq(lambda lr: ci_max(np.exp(lr), alfa), np.log(5000.0),
    np.log(12000.0), xtol=1e-7)


ris = minimize_scalar(lambda a: Re_neutro(a), bracket=(0.98, 1.02, 1.06), tol=1e-5)
alfa_c = ris.x
Re_c = np.exp(Re_neutro(alfa_c))
print(f"Punto critico: Re_c = {Re_c:.2f},  alfa_c = {alfa_c:.4f}")
lam, _, _ = orr_sommerfeld(Re_c, alfa_c, N)
c = 1j * lam[np.argmax(lam.real)]
print(f"   velocita' di fase dell'onda critica c_R = {c.real:.5f} U_c")

logRe = np.linspace(np.log10(3000), np.log10(1.0e5), 46)
alfas = np.linspace(0.3, 1.3, 41)
CI = np.zeros((len(alfas), len(logRe)))
for i, a in enumerate(alfas):
    for j, lr in enumerate(logRe):
        CI[i, j] = ci_max(10**lr, a)
print(f"c_I massimo sulla griglia: {CI.max():.4f} "
      f"(Re = {10**logRe[np.unravel_index(CI.argmax(), CI.shape)[1]]:.0f},"
      f" alfa = {alfas[np.unravel_index(CI.argmax(), CI.shape)[0]]:.2f})")
```

```
for Re in (1.0e4, 3.0e4, 1.0e5):
    j = np.argmin(np.abs(10**logRe - Re))
    inst = alfas[CI[:, j] > 0]
    if inst.size:
        print(f"   Re = {Re:8.0f}: onde instabili per {inst.min():.2f} < alfa < {inst.max():.2f}")

fig, ax = plt.subplots(figsize=(4.8, 3.4))
cs = ax.contourf(10**logRe, alfas, CI, levels=[0, 0.002, 0.004, 0.006, 0.008], cmap="Reds", alpha=0.6)
ax.contour(10**logRe, alfas, CI, levels=[0.0], colors=BLU, linewidths=1.5)
ax.plot(Re_c, alfa_c, "o", color="k", ms=4)
ax.annotate(f"({Re_c:.0f}; {alfa_c:.3f})".replace(".", ","), (Re_c, alfa_c), xytext=(7000, 1.18),
            fontsize=7, arrowprops=dict(arrowstyle="->", lw=0.6))
ax.set_xscale("log")
ax.set_xlabel("$Re=U_c h/\\nu$")
ax.set_ylabel(r"numero d'onda $\alpha$")
ax.text(3.0e4, 0.85, "instabile", fontsize=8, color=ROSSO)
ax.text(4.0e3, 0.45, "stabile", fontsize=8, color=GRIGIO)
fig.colorbar(cs, ax=ax, label="$c_I$")
salva(fig, "prog3_8b_curva_neutra")
```

Listato 9.4: `prog3_8b_curva_neutra.py`: curva di stabilità neutra della corrente di Poiseuille piana e punto critico.

Risultati del Programma 3.8b

$$Re_c = 5772{,}22, \qquad \alpha_c = 1{,}0205, \qquad c_R = 0{,}2640,$$

in accordo con il risultato classico di Orszag ($Re_c = 5772{,}22$, $\alpha_c = 1{,}02056$). Per $Re = 10^4$ sono instabili i numeri d'onda $0{,}82 < \alpha < 1{,}07$; per $Re = 10^5$ la banda si sposta verso le onde più lunghe, $0{,}50 < \alpha < 0{,}85$. La curva neutra (Fig. 9.6) ha la forma di un cuneo che si restringe al crescere di Re: a un dato numero di Reynolds solo una banda stretta di onde "risuona" con la corrente di base.

Teoria lineare ed esperimenti

In un canale piano la transizione alla turbolenza si osserva di solito già per $Re \simeq 1000$–2000, ben prima del valore critico lineare 5772; nel tubo circolare la corrente di Hagen–Poiseuille è linearmente stabile per ogni numero di Reynolds, eppure diventa turbolenta. La teoria lineare descrive bene la nascita delle onde in ambienti a bassissimo disturbo, ma la transizione "naturale" avviene spesso per vie non lineari e tridimensionali, innescate da perturbazioni di ampiezza finita. Per lo strato limite di Blasius l'analisi analoga (con il profilo di Blasius al posto di quello di Poiseuille) dà un Reynolds critico basato sullo spessore di spostamento $Re_{\delta^*} \simeq 520$, cioè $Re_x \simeq 9 \times 10^4$; le onde di Tollmien–Schlichting vi sono state osservate sperimentalmente da Schubauer e Skramstad nel 1947, e la loro amplificazione è alla base dei metodi di previsione della transizione usati nel progetto dei profili laminari.

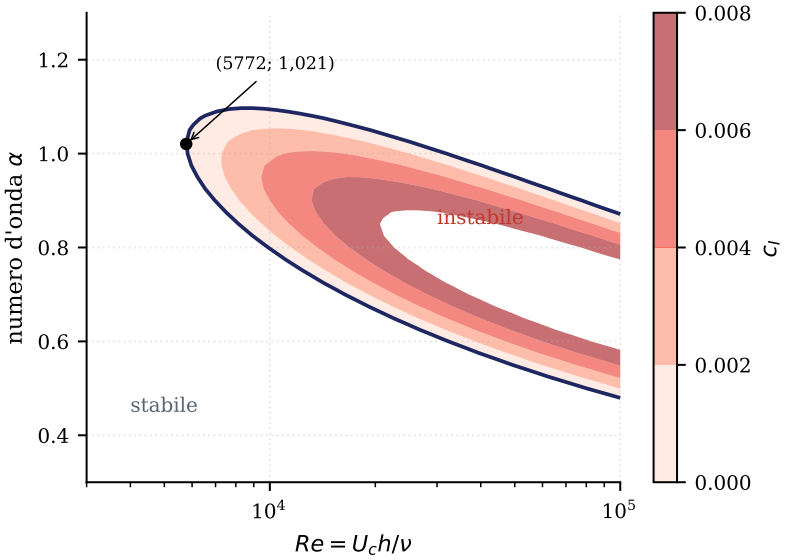


Figura 9.6: Curva di stabilità neutra ($c_I = 0$) della corrente di Poiseuille piana e tasso di crescita nella regione instabile.

9.8 Approfondimenti sulla stabilità delle correnti parallele

9.8.1 Il teorema di Squire

Nel testo si è fatto uso del teorema di Squire [32] per limitarsi alle perturbazioni piane. La dimostrazione è una trasformazione di variabili.

Le perturbazioni tridimensionali si riconducono a quelle piane

Si consideri una perturbazione tridimensionale $\propto e^{\mathrm{i}(\alpha x+\beta z-\alpha c t)}$ della corrente $U(y)$, con componenti $(\tilde{u},\tilde{v},\tilde{w},\tilde{p})$. Le equazioni linearizzate sono

$$\mathrm{i}\alpha\tilde{u}+\tilde{v}'+\mathrm{i}\beta\tilde{w}=0, \qquad \mathrm{i}\alpha(U-c)\tilde{u}+U'\tilde{v}+\mathrm{i}\alpha\tilde{p}=Re^{-1}\mathcal{L}\tilde{u},$$
$$\mathrm{i}\alpha(U-c)\tilde{v}+\tilde{p}'=Re^{-1}\mathcal{L}\tilde{v}, \qquad \mathrm{i}\alpha(U-c)\tilde{w}+\mathrm{i}\beta\tilde{p}=Re^{-1}\mathcal{L}\tilde{w},$$

con $\mathcal{L}=\mathrm{d}^2/\mathrm{d}y^2-\alpha^2-\beta^2$. Si moltiplichi la seconda per α e la quarta per β, si sommino e si divida per α; si moltiplichi la terza per $\tilde{\alpha}/\alpha$. Posto $\tilde{\alpha}=\sqrt{\alpha^2+\beta^2}$, $\tilde{\alpha}\hat{u}=\alpha\tilde{u}+\beta\tilde{w}$, $\alpha\hat{p}=\tilde{\alpha}\tilde{p}$ e $\tilde{\alpha}\,\widetilde{Re}=\alpha Re$, si ottiene

$$\mathrm{i}\tilde{\alpha}\hat{u}+\tilde{v}'=0, \qquad \mathrm{i}\tilde{\alpha}(U-c)\hat{u}+U'\tilde{v}+\mathrm{i}\tilde{\alpha}\hat{p}=\widetilde{Re}^{-1}\left(\hat{u}''-\tilde{\alpha}^2\hat{u}\right),$$

$$\mathrm{i}\tilde{\alpha}(U-c)\tilde{v}+\hat{p}'=\widetilde{Re}^{-1}\left(\tilde{v}''-\tilde{\alpha}^2\tilde{v}\right).$$

Sono esattamente le equazioni *piane* (9.10)–(9.12) per un'onda di numero d'onda $\tilde{\alpha}$ al numero di Reynolds $\widetilde{Re}=Re\,\alpha/\tilde{\alpha}<Re$, con la stessa velocità d'onda c. Quindi: se una perturbazione tridimensionale è instabile a Re, la perturbazione piana corrispondente è instabile al numero di Reynolds più basso $\widetilde{Re}$. Il minimo numero di Reynolds critico si ottiene con $\beta=0$: per determinare Re_c basta l'equazione di Orr–Sommerfeld piana.

Il teorema riguarda solo la soglia. Per $Re>Re_c$ le onde oblique possono avere tassi di crescita maggiori, e nella transizione reale sono proprio le interazioni fra onde oblique a

produrre le strutture tridimensionali (i vortici a forcina) che precedono la turbolenza.

9.8.2 Il limite non viscoso: i teoremi di Rayleigh e di Fjørtoft

Per $Re \to \infty$ l'equazione di Orr–Sommerfeld perde i termini di ordine più alto e si riduce all'**equazione di Rayleigh**

$$(U - c)\left(\tilde{v}'' - \alpha^2 \tilde{v}\right) - U''\tilde{v} = 0, \qquad \tilde{v}(\pm 1) = 0, \tag{9.16}$$

del secondo ordine, con la sola condizione di impermeabilità. Si ricava così la condizione *necessaria* di instabilità più famosa della fluidodinamica.

Teorema del punto di flesso

Si supponga $c_I \neq 0$ (modo non neutro), così che $U - c \neq 0$ ovunque. Si divida la (9.16) per $U - c$, si moltiplichi per $\bar{\tilde{v}}$ e si integri fra le pareti; integrando per parti il primo termine:

$$-\int_{-1}^{1} \left(|\tilde{v}'|^2 + \alpha^2 |\tilde{v}|^2\right) \mathrm{d}y - \int_{-1}^{1} \frac{U''}{U - c} |\tilde{v}|^2 \, \mathrm{d}y = 0.$$

Il primo integrale è reale; la parte immaginaria della relazione, con $1/(U-c) = (U - \bar{c})/|U-c|^2$, è

$$c_I \int_{-1}^{1} \frac{U'' \, |\tilde{v}|^2}{|U - c|^2} \, \mathrm{d}y = 0.$$

Se $c_I \neq 0$ l'integrale deve annullarsi, e questo è possibile solo se U'' cambia segno nell'intervallo: *un profilo senza punti di flesso non ha modi instabili non viscosi* (Rayleigh, 1880). Fjørtoft [33] rafforzò il risultato usando anche la parte reale: condizione necessaria è che $U''(U - U_s) < 0$ in qualche punto, dove U_s è la velocità nel punto di flesso, cioè che la vorticità $-U'$ abbia un *massimo* in modulo nel flesso, non un minimo.

I profili di Couette e di Poiseuille non hanno flessi: nel fluido ideale sono stabili, e l'instabilità trovata nel Programma 3.8 esiste solo grazie alla viscosità, che nello strato critico sfasa la perturbazione di velocità rispetto a quella di pressione e permette il trasferimento di energia. Ecco perché la curva neutra si richiude a grandi Re: la viscosità deve essere abbastanza piccola da non smorzare l'onda, ma non nulla. Al contrario, i profili con flesso (strati limite con gradiente avverso o con iniezione, Esercizi 2 e 3; scie; getti; strati di mescolamento) sono instabili già nel fluido ideale, con tassi di crescita molto maggiori, dell'ordine di U/δ invece che di una piccola frazione di esso: la transizione in uno strato limite che sta per distaccarsi è quasi immediata.

9.8.3 L'equazione dell'energia di Reynolds–Orr

Una perturbazione, per crescere, deve prelevare energia dalla corrente di base. Il bilancio si ottiene moltiplicando le (9.11)–(9.12) per $\hat{u}$ e $\hat{v}$, sommando e integrando su una lunghezza d'onda e sull'altezza del canale; i termini di pressione e quelli convettivi $U\partial/\partial x$ danno integrali nulli per la periodicità, e resta

$$\frac{\mathrm{d}E}{\mathrm{d}t} = -\int \hat{u}\hat{v}\, U' \, \mathrm{d}V - \frac{1}{Re}\int |\nabla \hat{\mathbf{V}}|^2 \, \mathrm{d}V, \qquad E = \int \tfrac{1}{2}\left(\hat{u}^2 + \hat{v}^2\right) \mathrm{d}V. \tag{9.17}$$

Il secondo termine è la dissipazione, sempre negativa. Il primo è la *produzione*: la perturbazione guadagna energia dove lo sforzo di Reynolds $-\hat{u}\hat{v}$ ha lo stesso segno del gradiente U', cioè

dove il moto trasversale porta fluido lento in zone veloci e viceversa, come già osservato a proposito del termine $\hat{v}U'$. Due conseguenze. La prima: una perturbazione in cui $\hat{u}$ e $\hat{v}$ sono in quadratura di fase, come nei modi non viscosi di un profilo senza flesso, ha produzione nulla e non può crescere; serve uno sfasamento, che solo la viscosità (nello strato critico) o un flesso possono fornire. La seconda, più sottile: la (9.17) è *esatta* anche per perturbazioni di ampiezza finita, perché i termini non lineari si cancellano nel bilancio dell'energia (sono di trasporto). I termini non lineari non creano energia: ridistribuiscono fra i modi quella che il meccanismo lineare produce.

9.8.4 Perché la transizione avviene prima: la non normalità

Quest'ultima osservazione sembra contraddire gli esperimenti citati nell'avviso precedente: se l'energia la produce solo il meccanismo lineare, e tutti i modi sono smorzati per $Re < 5772$, come può la corrente di Poiseuille diventare turbolenta a $Re \simeq 1000$? La risposta, compresa negli anni novanta del Novecento [37], è che gli autovalori descrivono il comportamento per $t \to \infty$, non quello transitorio. L'operatore di Orr–Sommerfeld non è *normale*: i suoi autovettori non sono ortogonali, e una combinazione di modi tutti decrescenti può crescere, anche di molto, prima di decadere. Fisicamente il meccanismo più efficace è il *lift-up*: una debole perturbazione $\hat{v}$ allungata nella direzione della corrente (vortici longitudinali) sposta fluido attraverso il gradiente U' e genera strisce di velocità $\hat{u}$ di ampiezza $\mathcal{O}(Re)$ volte maggiore, prima che la viscosità le dissipi; nel piano (Re, α) dell'analisi modale queste perturbazioni, con $\alpha = 0$, sono invisibili. L'energia può crescere transitoriamente di un fattore dell'ordine di $Re^2/1000$: per $Re = 1000$ di quasi mille volte, abbastanza perché i termini non lineari entrino in gioco e si inneschi la turbolenza. La teoria modale di questo capitolo resta esatta per quello che afferma (la crescita esponenziale asintotica), ma non esaurisce la fisica della transizione; lo studio della crescita transitoria (*crescita ottimale*) usa gli stessi strumenti numerici, matrici di Chebyshev e decomposizione ai valori singolari invece che agli autovalori.

L'Esercizio 15 estende il calcolo alla corrente di Couette–Poiseuille, in cui una delle pareti è in moto, e mostra che il moto della parete ha un forte effetto stabilizzante.

Capitolo 10

Metodi pseudo-spettrali di Fourier

10.1 Problemi periodici

Molti problemi della fluidodinamica sono periodici in una o più direzioni spaziali. Nelle simulazioni della transizione e della turbolenza in un canale si considera una sola lunghezza d'onda della perturbazione e si assume che il moto si ripeta identico a monte e a valle; spesso si assume periodica anche la direzione trasversale. Le autofunzioni di Orr–Sommerfeld del capitolo precedente possono servire come condizione iniziale di una simulazione delle equazioni di Navier–Stokes complete, che segue la crescita delle onde fino a quando le interazioni non lineari prodotte dai termini convettivi non generano la turbolenza. In queste direzioni la base naturale è la serie di Fourier, e la trasformata discreta si calcola in modo efficientissimo con la **trasformata rapida** (FFT, *fast Fourier transform*), che richiede $\mathcal{O}(N \log N)$ operazioni invece di $\mathcal{O}(N^2)$ e lavora al meglio quando il numero di nodi è una potenza di due, $N = 2^l$. Per risolvere con accuratezza minima un periodo di un'onda servono di solito almeno $N = 16$ nodi.

10.2 La trasformata discreta di Fourier

Sia $u(x)$ una funzione periodica di periodo L, nota negli N nodi equispaziati

$$x_m = m\,\Delta x, \qquad \Delta x = \frac{L}{N}, \qquad m = 0, 1, \ldots, N-1; \tag{10.1}$$

il nodo $x_N = L$ coincide, per la periodicità, con x_0 e non va contato. La funzione si rappresenta come

$$u(x_m) = \sum_{n=-N/2}^{N/2-1} \hat{u}(k_n)\, e^{\mathrm{i} k_n x_m}, \qquad k_n = \frac{2\pi n}{L}, \tag{10.2}$$

e, per l'ortogonalità discreta degli esponenziali ($\sum_{m=0}^{N-1} e^{\mathrm{i}(k_n - k_{n'})x_m} = N\delta_{nn'}$), i coefficienti si calcolano con

$$\hat{u}(k_n) = \frac{1}{N} \sum_{m=0}^{N-1} u(x_m)\, e^{-\mathrm{i} k_n x_m}. \tag{10.3}$$

Il numero d'onda più alto rappresentabile è $|k_{N/2}| = \pi/\Delta x$ (*numero d'onda di Nyquist*), corrispondente a un'onda lunga due passi di griglia: un'onda più corta, campionata sugli stessi nodi, è indistinguibile da una più lunga (*aliasing*). Per rappresentare correttamente una componente di numero d'onda k servono quindi più di due nodi per lunghezza d'onda.

Le convenzioni delle librerie

Le librerie non usano tutte la stessa normalizzazione né lo stesso ordinamento dei coefficienti. In NumPy, `np.fft.fft` calcola $\sum_m u_m e^{-\mathrm{i}k_n x_m}$ *senza* il fattore $1/N$, che è invece incluso in `np.fft.ifft`; i coefficienti sono ordinati con i numeri d'onda $n = 0, 1, \ldots, N/2-1, -N/2, \ldots, -1$, restituiti (moltiplicati per $2\pi/L$) dalla funzione `numeri_onda(N, L)` del modulo comune. Un errore di convenzione produce risultati sbagliati di un fattore N, o derivate con il segno sbagliato, senza alcun messaggio di errore.

10.3 La derivazione nello spazio dei numeri d'onda

Derivando la (10.2) termine a termine:

$$\left(\frac{\partial u}{\partial x}\right)_m = \sum_n \mathrm{i}k_n\, \hat{u}(k_n)\, e^{\mathrm{i}k_n x_m}, \qquad \left(\frac{\partial^2 u}{\partial x^2}\right)_m = \sum_n (-k_n^2)\, \hat{u}(k_n)\, e^{\mathrm{i}k_n x_m}. \tag{10.4}$$

Per derivare una funzione periodica nota nei nodi basta dunque trasformarla, moltiplicare ogni coefficiente per $\mathrm{i}k_n$ (o per $-k_n^2$) e antitrasformare: è il **metodo pseudo-spettrale** [36]. Per una funzione reale, il coefficiente del modo di Nyquist $n = -N/2$ deve essere annullato nel calcolo delle derivate di ordine dispari: quel modo, $\cos(\pi x/\Delta x)$ campionato nei nodi, vale ± 1 alternativamente e la sua derivata nei nodi è nulla, mentre la moltiplicazione per $\mathrm{i}k_{-N/2}$ produrrebbe una parte immaginaria spuria (una derivata "rumorosa"). Nella derivata seconda il fattore $-k^2$ è reale e il modo si può conservare.

Se u è regolare e periodica, i coefficienti $\hat{u}(k_n)$ decrescono più rapidamente di qualunque potenza di $1/n$ e la derivata pseudo-spettrale ha accuratezza spettrale; se u ha una derivata discontinua di ordine p, i coefficienti decrescono solo come $n^{-(p+1)}$ e la convergenza diventa algebrica; se u stessa è discontinua (o semplicemente non periodica, come una funzione che ha valori diversi agli estremi del periodo), compaiono le oscillazioni di Gibbs.

10.4 Un problema ai limiti periodico

Consideriamo il problema [20]

$$u'' - u' - 2u = 2 + 6\sin 6x - 38\cos 6x, \qquad 0 \le x \le 2\pi, \qquad u \text{ periodica}, \tag{10.5}$$

la cui soluzione esatta è $u = \cos 6x - 1$, che soddisfa anche $u(0) = u(2\pi) = 0$. (Si verifica subito: $u'' = -36\cos 6x$, $u' = -6\sin 6x$, $-2u = 2 - 2\cos 6x$, e la somma è il secondo membro.) Trasformando ambo i membri, ogni coefficiente di Fourier soddisfa un'equazione algebrica indipendente:

$$\left(-k_n^2 - \mathrm{i}k_n - 2\right)\hat{u}(k_n) = \hat{b}(k_n) \quad \Longrightarrow \quad \hat{u}(k_n) = \frac{\hat{b}(k_n)}{-k_n^2 - \mathrm{i}k_n - 2}, \tag{10.6}$$

dove $\hat{b}$ sono i coefficienti del secondo membro. Il denominatore non si annulla mai (la sua parte immaginaria è nulla solo per $k_n = 0$, dove vale -2), quindi la soluzione periodica esiste ed è unica. Le condizioni al contorno non si impongono a parte: la periodicità è incorporata nella base di Fourier. L'operatore differenziale, che nello spazio fisico accoppia i nodi vicini, nello spazio dei numeri d'onda è *diagonale*: è la proprietà che rende così efficienti i metodi di Fourier per i problemi a coefficienti costanti.

prog3_9_fourier.py

```
"""
prog3_9_fourier.py -- PROGRAMMA 3.9
Metodo di Fourier (trasformata discreta) per un problema ai limiti periodico:
    u'' - u' - 2 u = 2 + 6 sin(6x) - 38 cos(6x),   0 <= x <= 2 pi,
con soluzione esatta u = cos(6x) - 1. Nello spazio di Fourier la derivata
diventa una moltiplicazione per i k_n, quindi
    (-k_n^2 - i k_n - 2) u_hat(k_n) = b_hat(k_n)   =>   u_hat = b_hat / (-k_n^2 - i k_n - 2).
Il coefficiente del modo di Nyquist (n = -N/2) si annulla.
Si studia poi la convergenza in N per tre soluzioni di regolarita' diversa:
polinomio trigonometrico, funzione analitica periodica, funzione periodica
con derivata discontinua.
"""
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import numeri_onda
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

L = 2 * np.pi


def risolvi(b, N):
    """Risolve u'' - u' - 2u = b(x) su N nodi equispaziati di [0, 2 pi)."""
    x = L * np.arange(N) / N
    k = numeri_onda(N, L)
    bh = np.fft.fft(b(x))
    bh[N // 2] = 0.0                        # modo di Nyquist
    uh = bh / (-k**2 - 1j * k - 2.0)
    return x, np.fft.ifft(uh).real


casi = {
    "cos 6x - 1": (lambda x: np.cos(6 * x) - 1,
                   lambda x: 2 + 6 * np.sin(6 * x) - 38 * np.cos(6 * x)),
    "exp(sin x)": (lambda x: np.exp(np.sin(x)),
                   lambda x: np.exp(np.sin(x)) * (np.cos(x)**2 - np.sin(x) - np.cos(x) - 2)),
    "|sin x|^3":  (lambda x: np.abs(np.sin(x))**3,
                   None),
}
# per |sin x|^3 il secondo membro si calcola analiticamente a tratti
def b_abs3(x):
    s, c = np.sin(x), np.cos(x)
    a = np.abs(s)
    u = a**3
    du = 3 * a * s * c                      # derivata di |s|^3 = 3 |s| s c
    d2u = 3 * a * (2 * c**2 - s**2)         # derivata seconda: 6|s|c^2 - 3|s|s^2
    return d2u - du - 2 * u
casi["|sin x|^3"] = (casi["|sin x|^3"][0], b_abs3)

x64, u64 = risolvi(casi["cos 6x - 1"][1], 64)
print(f"N = 64: max |u - u_es| = {np.max(np.abs(u64 - (np.cos(6*x64) - 1))):.2e}")
Nlista = [8, 12, 14, 16, 24, 32, 48, 64, 128]
errori = {}
print("\n   N   " + "   ".join(f"{nome:>12s}" for nome in casi))
for N in Nlista:
    riga = []
    for nome, (ue, b) in casi.items():
        x, u = risolvi(b, N)
        e = np.max(np.abs(u - ue(x)))
```

```
        errori.setdefault(nome, []).append(e)
        riga.append(e)
    print(f"  {N:4d} " + "   ".join(f"{e:12.3e}" for e in riga))
e3 = errori["|sin x|^3"]
print(f"\nordine osservato per |sin x|^3 (N = 64 -> 128):
    {np.log2(e3[-2]/e3[-1]):.2f}")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
xx = np.linspace(0, L, 400)
ax[0].plot(xx, np.cos(6 * xx) - 1, color=GRIGIO, lw=1.8, label="esatta")
ax[0].plot(np.append(x64, L), np.append(u64, u64[0]), "o", ms=3, color=BLU,
    mfc="none",
           label="Fourier, $N=64$")
ax[0].set_xlabel("$x$")
ax[0].set_ylabel("$u(x)$")
ax[0].legend(fontsize=7, loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=2)
for (nome, e), col, mk in zip(errori.items(), [BLU, VERDE, ROSSO], ["o", "s", "d"]):
    ax[1].semilogy(Nlista, np.maximum(e, 1e-17), mk + "-", color=col,
    label=f"$u={nome}$".replace(" x", "\\,x"))
ax[1].set_xlabel("$N$")
ax[1].set_ylabel("errore massimo")
ax[1].legend(fontsize=7)
fig.tight_layout()
salva(fig, "prog3_9_fourier")
```

Listato 10.1: Programma 3.9 – problema periodico risolto con la trasformata di Fourier e studio della convergenza (`prog3_9_fourier.py`).

Risultati del Programma 3.9

Con $N = 64$ nodi l'errore massimo è $1{,}5 \times 10^{-14}$ (Fig. 10.1): la soluzione è esatta, perché è un polinomio trigonometrico che la base rappresenta senza errore. Il programma studia anche la convergenza al variare di N per tre soluzioni di regolarità diversa (il secondo membro è calcolato di conseguenza):

N	$u = \cos 6x - 1$	$u = e^{\sin x}$	$u = \lvert \sin x \rvert^3$
8	$4{,}400$	$6{,}211 \times 10^{-3}$	$2{,}172 \times 10^{-1}$
12	$1{,}000$	$4{,}783 \times 10^{-5}$	$7{,}644 \times 10^{-2}$
14	$2{,}465 \times 10^{-14}$	$1{,}803 \times 10^{-7}$	$6{,}437 \times 10^{-2}$
24	$1{,}688 \times 10^{-14}$	$1{,}056 \times 10^{-12}$	$2{,}205 \times 10^{-2}$
32	$1{,}651 \times 10^{-14}$	$7{,}772 \times 10^{-16}$	$1{,}271 \times 10^{-2}$
128	$6{,}106 \times 10^{-15}$	$7{,}772 \times 10^{-16}$	$8{,}306 \times 10^{-4}$

La tabella illustra le tre situazioni descritte sopra. Per $u = \cos 6x - 1$ la soluzione è esatta appena il numero d'onda 6 è rappresentato, cioè per $N > 12$; con $N = 12$ il modo $k = 6$ coincide con quello di Nyquist, viene annullato e la soluzione perde completamente la parte oscillante (errore 1). Per la funzione analitica e periodica $e^{\sin x}$ l'errore decresce esponenzialmente, raggiungendo la precisione di macchina con 32 nodi. Per $\lvert \sin x \rvert^3$, la cui derivata terza è discontinua in $x = 0$ e $x = \pi$, la convergenza è soltanto algebrica (ordine osservato 2). Il criterio pratico che si legge spesso, "servono almeno 32 nodi", non ha valore generale: il numero di nodi necessario dipende dal contenuto spettrale della soluzione.

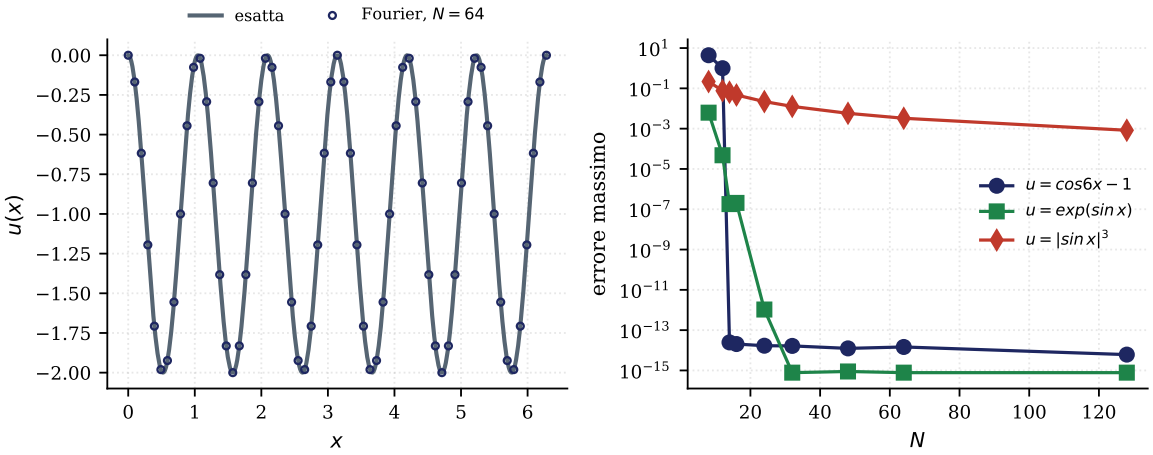


Figura 10.1: Problema periodico: soluzione con $N = 64$ (a sinistra) ed errore in funzione del numero di nodi per soluzioni di diversa regolarità (a destra).

10.5 Convezione e diffusione in un campo di moto assegnato

Un'applicazione più interessante combina le differenze finite in una direzione e il metodo di Fourier nell'altra. Si consideri l'equazione di convezione–diffusione di una temperatura $T(x, y, t)$ in un campo di velocità assegnato e "congelato" nel tempo:

$$\frac{\partial T}{\partial t} + u\frac{\partial T}{\partial x} + v\frac{\partial T}{\partial y} = \frac{1}{Pr\,Re}\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right), \qquad u = -\sin \pi x \cos 2\pi y, \quad v = \cos \pi x \cos \pi y, \tag{10.7}$$

nella striscia $0 \le x \le 1$ con $T = 0$ sulla parete fredda $x = 0$ e $T = 1$ su quella calda $x = 1$. Il campo è periodico in y con periodo $L = 2$, e sulle pareti $u = 0$: il fluido scorre lungo le pareti senza attraversarle. Il prodotto $Pe = RePr$ (numero di Péclet) misura il rapporto fra trasporto convettivo e diffusivo.

Un campo di velocità modello

Il campo assegnato non è solenoidale: $\nabla \cdot \mathbf{V} = -\pi \cos \pi x\,(\cos 2\pi y + \sin \pi y)$, con un massimo di 2π in valore assoluto. Non rappresenta quindi una corrente incomprimibile, ma è un ottimo banco di prova per il metodo numerico. Per un campo non solenoidale la forma non conservativa (10.7) e quella conservativa $\partial T/\partial t + \nabla \cdot (\mathbf{V}T) = \dots$ non sono equivalenti: qui si usa la prima.

Nel metodo misto la derivata lungo la direzione non periodica x si approssima con le differenze centrate del secondo ordine, quella lungo la direzione periodica y con il metodo pseudo-spettrale, e l'avanzamento nel tempo è quello esplicito di Eulero:

$$T^{n+1}_{i,j} = T^n_{i,j} - \Delta t\, u_{i,j}\frac{T^n_{i+1,j} - T^n_{i-1,j}}{2\Delta x} - \Delta t\, v_{i,j}\left(\frac{\partial T}{\partial y}\right)^n_{i,j} + \frac{\Delta t}{PrRe}\left[\frac{T^n_{i+1,j} - 2T^n_{i,j} + T^n_{i-1,j}}{\Delta x^2} + \left(\frac{\partial^2 T}{\partial y^2}\right)^n_{i,j}\right], \tag{10.8}$$

dove le derivate in y si calcolano per ogni x_i con la (10.4). Poiché l'integrazione è esplicita, il passo temporale deve rispettare i vincoli di stabilità diffusivo e convettivo. Il primo, ricavato con l'analisi di von Neumann come nel Cap. 5 ma con il contributo della direzione y, è

$$\Delta t \le \frac{2\,PrRe}{4/\Delta x^2 + k^2_{\max}}, \qquad k_{\max} = \frac{\pi N_y}{L}, \tag{10.9}$$

ed è tanto più severo quanto più piccolo è $PrRe$, cioè quanto più intensa è la diffusione.

Dati del Programma 3.10

$Re = 1$; $Pr = 0{,}71$ (aria) e $Pr = 25$; 41 nodi in x ($\Delta x = 0{,}025$), 32 nodi in y su un periodo; $\Delta t = 10^{-4}$ per $Pr = 0{,}71$ e 10^{-3} per $Pr = 25$ (limiti diffusivi $1{,}6 \times 10^{-4}$ e $5{,}6 \times 10^{-3}$); condizione iniziale $T = x$ (profilo conduttivo); integrazione fino a $\max|\partial T/\partial t| < 10^{-4}$. La soluzione stazionaria è verificata risolvendo direttamente il sistema lineare stazionario con le stesse discretizzazioni (matrice di differenziazione di Fourier in y).

prog3_10_convezione_diffusione.py

```
"""
prog3_10_convezione_diffusione.py -- PROGRAMMA 3.10
Convezione-diffusione bidimensionale di una temperatura in un campo di
velocita' assegnato, periodico in y (periodo 2):
    T_t + u T_x + v T_y = (T_xx + T_yy) / (Pr Re),
    u = -sin(pi x) cos(2 pi y),   v = cos(pi x) cos(pi y),
    T = 0 per x = 0 (parete fredda),  T = 1 per x = 1 (parete calda).
Metodo misto: differenze centrate in x (direzione non periodica), derivate in y
con la trasformata di Fourier (direzione periodica), Eulero esplicito nel tempo.
Si integra fino allo stato stazionario per Pr = 0,71 (aria) e Pr = 25 e si
confronta il risultato con la soluzione stazionaria calcolata direttamente
(sistema lineare con la matrice di differenziazione di Fourier).
"""
import time
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import numeri_onda
from stile_grafici import salva, BLU, ROSSO

RE = 1.0
NX, NY = 41, 32                         # nodi in x (pareti incluse) e in y (un
    periodo)
LY = 2.0
x = np.linspace(0.0, 1.0, NX)
y = LY * np.arange(NY) / NY
dx = x[1] - x[0]
X, Y = np.meshgrid(x, y, indexing="ij")
U = -np.sin(np.pi * X) * np.cos(2 * np.pi * Y)
V = np.cos(np.pi * X) * np.cos(np.pi * Y)
k = numeri_onda(NY, LY)
k1 = k.copy()
k1[NY // 2] = 0.0                       # modo di Nyquist nullo per la derivata prima


def dy(T):
    return np.fft.ifft(1j * k1 * np.fft.fft(T, axis=1), axis=1).real


def dyy(T):
    return np.fft.ifft(-k**2 * np.fft.fft(T, axis=1), axis=1).real


def evolvi(Pr, dt, toll=1e-4, nmax=400000):
    """Eulero esplicito fino a max|dT/dt| < toll. Restituisce T, passi, tempo."""
    a = 1.0 / (Pr * RE)
    T = np.tile(x[:, None], (1, NY))    # condizione iniziale: profilo conduttivo
    for n in range(1, nmax + 1):
        Tx = np.zeros_like(T)
        Txx = np.zeros_like(T)
```

```
        Tx[1:-1] = (T[2:] - T[:-2]) / (2 * dx)
        Txx[1:-1] = (T[2:] - 2 * T[1:-1] + T[:-2]) / dx**2
        dTdt = -U * Tx - V * dy(T) + a * (Txx + dyy(T))
        dTdt[0] = dTdt[-1] = 0.0       # pareti a temperatura imposta
        T = T + dt * dTdt
        if np.max(np.abs(dTdt)) < toll:
            return T, n, n * dt
    raise RuntimeError("stato stazionario non raggiunto")


def stazionario(Pr):
    """Soluzione diretta di u T_x + v T_y = a lap T con le stesse discretizzazioni."""
    a = 1.0 / (Pr * RE)
    # matrice di differenziazione di Fourier (prima e seconda) applicata a vettori in y
    I = np.eye(NY)
    Dy = np.fft.ifft(1j * k1[:, None] * np.fft.fft(I, axis=0), axis=0).real
    Dyy = np.fft.ifft(-(k**2)[:, None] * np.fft.fft(I, axis=0), axis=0).real
    n = (NX - 2) * NY
    A = np.zeros((n, n))
    b = np.zeros(n)
    idx = lambda i, jj: (i - 1) * NY + jj
    for i in range(1, NX - 1):
        for jj in range(NY):
            r = idx(i, jj)
            # u T_x - a T_xx
            coef = {i - 1: -U[i, jj] / (2 * dx) - a / dx**2,
                    i: 2 * a / dx**2,
                    i + 1: U[i, jj] / (2 * dx) - a / dx**2}
            for ii, c in coef.items():
                if ii == 0:
                    b[r] -= c * 0.0
                elif ii == NX - 1:
                    b[r] -= c * 1.0
                else:
                    A[r, idx(ii, jj)] += c
            # v T_y - a T_yy
            A[r, idx(i, 0):idx(i, 0) + NY] += V[i, jj] * Dy[jj] - a * Dyy[jj]
    Tin = np.linalg.solve(A, b)
    T = np.zeros((NX, NY))
    T[-1] = 1.0
    T[1:-1] = Tin.reshape(NX - 2, NY)
    return T


if __name__ == "__main__":
    risultati = {}
    for Pr, dt in ((0.71, 1.0e-4), (25.0, 1.0e-3)):
        a = 1 / (Pr * RE)
        lim_diff = 2.0 / (a * (4 / dx**2 + np.max(k**2)))
        t0 = time.perf_counter()
        T, passi, tfin = evolvi(Pr, dt)
        durata = time.perf_counter() - t0
        Ts = stazionario(Pr)
        risultati[Pr] = T
        print(f"Pr = {Pr:5.2f}: dt = {dt:.0e} (limite diffusivo ~ {lim_diff:.1e}), {passi} passi,"
              f" t = {tfin:.2f}, {durata:.1f} s")
        print(f"           scarto dalla soluzione stazionaria diretta:
```

```
    {np.max(np.abs(T - Ts)):.1e};"
106             f"  T(x = 0,5) fra {T[NX//2].min():.3f} e {T[NX//2].max():.3f}")
107     print(f"\nNumero di Peclet Pe = Re Pr: {RE*0.71:.2f} e {RE*25:.0f}")
108     div = -np.pi * np.cos(np.pi * X) * np.cos(2 * np.pi * Y) - np.pi * np.cos(np.pi
    * X) * np.sin(np.pi * Y)
109     print(f"Divergenza del campo assegnato: max |div V| = {np.max(np.abs(div)):.2f}
    (il campo non e' solenoidale)")
110
111     # ------------------------------------------------------------------ figure
112     xe = np.concatenate([y, [LY]])
113     fig, ax = plt.subplots(1, 3, figsize=(7.8, 3.6))
114     s = 2
115     ax[0].quiver(X[::s, ::2], Y[::s, ::2], U[::s, ::2], V[::s, ::2], color=BLU,
    scale=25, width=0.006)
116     ax[0].set_title("campo di velocita'")
117     for a_, Pr in zip(ax[1:], (0.71, 25.0)):
118         T = risultati[Pr]
119         Te = np.concatenate([T, T[:, :1]], axis=1)
120         cs = a_.contour(x, xe, Te.T, levels=np.linspace(0.05, 0.95, 10),
    cmap="coolwarm", linewidths=1.0)
121         a_.clabel(cs, fmt=lambda v: f"{v:.2f}".replace(".", ","), fontsize=5.5)
122         a_.set_title(f"isoterme, $Pr={Pr:g}$".replace(".", "{,}"))
123     for a_ in ax:
124         a_.set_xlim(0, 1)
125         a_.set_ylim(0, 2)
126         a_.set_aspect("equal")
127         a_.set_xlabel("$x$")
128     ax[0].set_ylabel("$y$")
129     fig.tight_layout()
130     salva(fig, "prog3_10_convezione_diffusione")
```

Listato 10.2: Programma 3.10 – convezione–diffusione bidimensionale con differenze finite in x e metodo pseudo-spettrale in y (`prog3_10_convezione_diffusione.py`).

Risultati del Programma 3.10

- $Pr = 0{,}71$: stato stazionario dopo 1476 passi ($t = 0{,}15$); scarto dalla soluzione stazionaria diretta $4{,}7 \times 10^{-6}$. Le isoterme sono quasi rette verticali: a $x = 0{,}5$ la temperatura varia soltanto fra 0,486 e 0,514. Con $Pe = 0{,}71$ la conduzione domina e il campo di moto la perturba appena.
- $Pr = 25$: stato stazionario dopo 11 316 passi ($t = 11{,}3$); scarto dalla soluzione diretta $1{,}6 \times 10^{-4}$, dell'ordine della tolleranza di arresto. Le isoterme sono fortemente deformate: a $x = 0{,}5$ la temperatura varia fra 0,074 e 0,607. Dove $u > 0$ il fluido freddo è trascinato verso la parete calda, dove $u < 0$ il fluido caldo è portato verso quella fredda.

L'interpretazione corretta del confronto è questa: aumentando il numero di Prandtl, a Re fissato, *diminuisce* il coefficiente di diffusione $1/(PrRe)$. La diffusione non è più in grado di uniformare le temperature trasversalmente e il campo di temperatura viene "scolpito" dal moto: il calore della parete calda penetra più in profondità verso la parete fredda lungo le correnti dirette verso di essa, e il fluido freddo si spinge verso la parete calda lungo le altre. Per la stessa ragione il transitorio dura molto di più ($t = 11$ contro 0,15): il tempo di diffusione attraverso la striscia è proporzionale a $PrRe$.

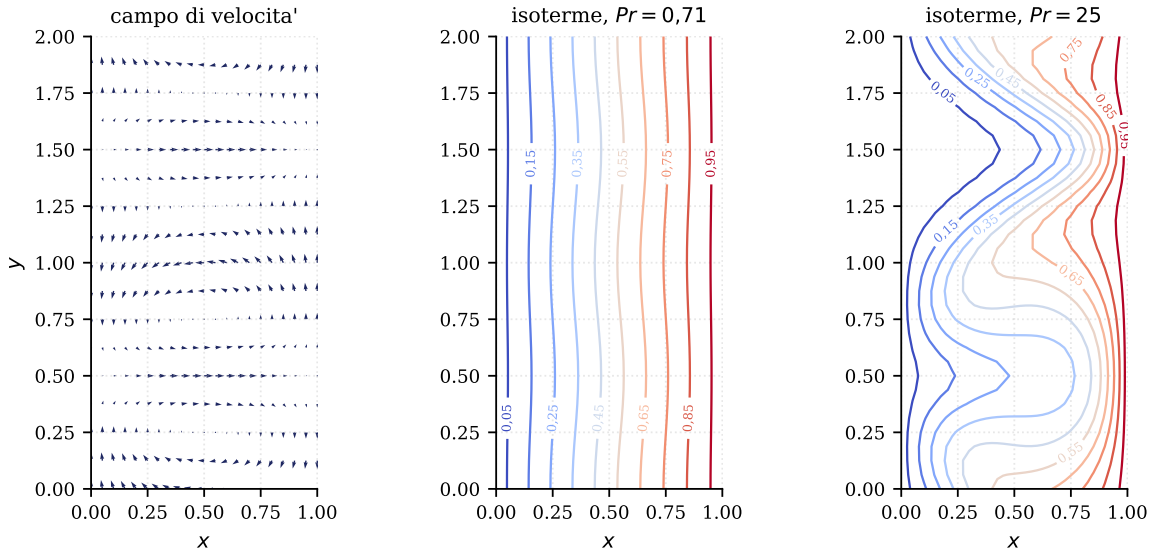


Figura 10.2: Campo di velocità assegnato (a sinistra) e isoterme stazionarie per $Pr = 0{,}71$ (al centro) e $Pr = 25$ (a destra); parete fredda a $x = 0$, calda a $x = 1$.

Il passo temporale e la scelta dello schema

Con $Pr = 0{,}71$ il passo è limitato dalla diffusione a $1{,}6 \times 10^{-4}$, e servono quasi 1500 passi per un transitorio che dura 0,15 unità di tempo. Il rimedio è quello del Cap. 6: trattare in modo implicito i termini diffusivi, lasciando espliciti quelli convettivi (schemi *IMEX*, implicito–esplicito). Nella direzione periodica la trasformata di Fourier rende il problema implicito particolarmente semplice: per ogni numero d'onda k si ottiene un sistema tridiagonale indipendente in x. L'Esercizio 16 realizza questo schema e mostra che il passo temporale può crescere di oltre mille volte.

10.6 Approfondimenti

10.6.1 Perché i coefficienti di Fourier decrescono così in fretta

L'affermazione che la regolarità di u governa il decadimento dei coefficienti si dimostra con una sola integrazione per parti, ripetuta.

Regolarità e decadimento dei coefficienti

Per u periodica di periodo 2π, con derivate continue fino all'ordine p, il coefficiente $\hat{u}_n = \frac{1}{2\pi}\int_0^{2\pi} u\, e^{-inx}\mathrm{d}x$ si integra per parti: il termine di contorno $[u\, e^{-inx}]_0^{2\pi}$ è nullo per la periodicità, e resta $\hat{u}_n = \frac{1}{in}\cdot\frac{1}{2\pi}\int u' e^{-inx}\mathrm{d}x$. Ripetendo p volte,

$$|\hat{u}_n| = \frac{1}{|n|^p}\left|\frac{1}{2\pi}\int_0^{2\pi} u^{(p)} e^{-\mathrm{i}nx}\mathrm{d}x\right| \le \frac{\max|u^{(p)}|}{|n|^p}.$$

Se u ha derivate continue di ogni ordine, $|\hat{u}_n|$ decresce più in fretta di qualunque potenza di $1/n$. Se u è *analitica* in una striscia $|\mathrm{Im}\, x| < a$ del piano complesso, spostando il cammino di integrazione su $\mathrm{Im}\, x = \pm a$ si ottiene $|\hat{u}_n| \le C\, e^{-a|n|}$: decadimento esponenziale, con esponente dato dalla distanza della singolarità più vicina dall'asse reale. È l'analogo

dell'ellisse di Bernstein del § 9.5. Per $e^{\sin x}$, intera, il decadimento è più che esponenziale (i coefficienti sono funzioni di Bessel modificate, $|\hat{u}_n| = I_n(1) \sim (1/2)^n/n!$); per $|\sin x|^3$, la cui derivata terza è discontinua, l'integrazione per parti si ferma a $p = 3$ e la derivata quarta è una somma di delta: $|\hat{u}_n| \sim n^{-4}$, e l'errore della derivata seconda (moltiplicata per n^2) decresce come n^{-2}, come osservato nel Programma 3.9.

10.6.2 L'aliasing e la regola dei 3/2

Nei problemi non lineari si calcolano prodotti, e il prodotto di due polinomi trigonometrici ha grado maggiore dei fattori. Qui nasce l'errore specifico dei metodi pseudo-spettrali.

L'errore di aliasing nel prodotto

Siano u e v rappresentate sui modi $|n| \le N/2$. Il prodotto $w = uv$ ha coefficienti esatti $\hat{w}_n = \sum_{m+l=n} \hat{u}_m \hat{v}_l$ per $|n| \le N$. Se però w viene calcolato nello spazio fisico, nei nodi $x_j = 2\pi j/N$, e poi trasformato, la trasformata discreta non distingue il modo n dal modo $n \pm N$ (nei nodi, $e^{\mathrm{i}(n\pm N)x_j} = e^{\mathrm{i}nx_j}$): i coefficienti calcolati sono

$$\tilde{w}_n = \hat{w}_n + \hat{w}_{n+N} + \hat{w}_{n-N},$$

cioè i modi oltre quello di Nyquist si "ripiegano" sui modi bassi. Per la convezione $u\,\partial u/\partial x$ l'errore non è solo di accuratezza: lo schema aliasato non conserva l'energia e può diventare instabile, come si vedrà.
La regola dei 3/2. Si riempiano di zeri i coefficienti di u e v fino a M modi, si calcoli il prodotto nei nodi della griglia più fine di M punti e si troncino i coefficienti risultanti a $|n| \le N/2$. I termini di aliasing sono ora $\hat{w}_{n\pm M}$, e $\hat{w}_k$ è nullo per $|k| > N$: perché nessun termine spurio ricada nei modi conservati occorre $|n \pm M| > N$ per ogni $|n| \le N/2$, cioè $M - N/2 > N$, $M > 3N/2$. Con $M = 3N/2$ il prodotto troncato è *esatto* (è la proiezione di Galerkin del prodotto): il costo è una trasformata su una griglia una volta e mezzo più fine, molto meno del calcolo diretto della convoluzione, che costerebbe $\mathcal{O}(N^2)$.

Il Programma 3.9b applica il metodo all'equazione di Burgers periodica, $u_t + uu_x = \nu u_{xx}$ con $u(x,0) = \sin x$, già studiata nel modulo precedente con le differenze finite. Il termine diffusivo, diagonale nello spazio dei numeri d'onda, è trattato esattamente con il *fattore integrante* $e^{-\nu k^2 t}$, che elimina il vincolo diffusivo sul passo; il termine convettivo è integrato con Runge–Kutta del quarto ordine. La soluzione esatta si ottiene dalla trasformazione di Cole–Hopf, $u = -2\nu\,\varphi_x/\varphi$, con φ soluzione dell'equazione del calore, calcolata per quadratura.

```
"""
prog3_9b_burgers.py -- equazione di Burgers con il metodo pseudo-spettrale di
Fourier: aliasing, regola dei 3/2 e trasformata rapida.

    u_t + u u_x = nu u_xx,   0 <= x < 2 pi periodico,   u(x, 0) = sin x.

Nello spazio dei numeri d'onda:  d u^_k/dt = -(u u_x)^_k - nu k^2 u^_k.
Il termine non lineare si calcola nello spazio fisico (prodotto u * u_x) e si
ritrasforma: il prodotto di due polinomi trigonometrici di grado N/2 ha grado N,
e i modi oltre quello di Nyquist si "ripiegano" (aliasing) sui modi bassi. Con
la regola dei 3/2 (Orszag) il prodotto si calcola su una griglia di M = 3N/2
nodi, riempiendo di zeri i coefficienti aggiunti, e si troncano poi i modi
oltre N/2: il risultato e' esente da aliasing.
Avanzamento nel tempo: Runge-Kutta del quarto ordine con fattore integrante
```

prog3_9b_burgers.py

```
exp(-nu k^2 t) per il termine diffusivo (nessun vincolo diffusivo sul passo).
Soluzione esatta: trasformazione di Cole-Hopf, u = -2 nu phi_x/phi, con phi
soluzione dell'equazione del calore, calcolata per quadratura.
"""
import numpy as np
import time
import matplotlib.pyplot as plt
from scipy.integrate import quad
from metodi_numerici import numeri_onda
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

NU = 0.05


def esatta(x, t, nu=NU):
    """Cole-Hopf: phi(x, t) = int phi0(y) G(x - y, t) dy, phi0 = exp(-(1 - cos
    y)/(2 nu))
    (dato iniziale u0 = sin x), G nucleo del calore periodico."""
    def phi_e_phix(xx):
        def nucleo(y):                      # somma delle immagini periodiche del
    nucleo gaussiano
            s = sum(np.exp(-(xx - y - 2 * np.pi * m)**2 / (4 * nu * t)) for m in
    range(-3, 4))
            return s
        def dnucleo(y):
            return sum(-(xx - y - 2 * np.pi * m) / (2 * nu * t) * np.exp(-(xx - y -
    2 * np.pi * m)**2 / (4 * nu * t))
                       for m in range(-3, 4))
        f0 = lambda y: np.exp(-(1 - np.cos(y)) / (2 * nu))
        p = quad(lambda y: f0(y) * nucleo(y), 0, 2 * np.pi, limit=200)[0]
        px = quad(lambda y: f0(y) * dnucleo(y), 0, 2 * np.pi, limit=200)[0]
        return p, px
    u = np.empty_like(x)
    for i, xx in enumerate(x):
        p, px = phi_e_phix(xx)
        u[i] = -2 * nu * px / p
    return u


def burgers(N, t_fin, dt, dealias, nu=NU):
    x = 2 * np.pi * np.arange(N) / N
    k = numeri_onda(N, 2 * np.pi)
    uh = np.fft.fft(np.sin(x))
    M = 3 * N // 2
    kM = numeri_onda(M, 2 * np.pi)

    def nonlin(uh):
        """Trasformata di -(u u_x), con o senza de-aliasing (regola dei 3/2)."""
        if not dealias:
            u = np.fft.ifft(uh).real
            ux = np.fft.ifft(1j * k * uh).real
            return -np.fft.fft(u * ux)
        uhM = np.zeros(M, dtype=complex)                  # riempimento con zeri: N
    -> M modi
        uhM[:N // 2] = uh[:N // 2]
        uhM[M - N // 2:] = uh[N // 2:]
        uM = np.fft.ifft(uhM).real * M / N
        uxM = np.fft.ifft(1j * kM * uhM).real * M / N
        wM = np.fft.fft(uM * uxM) * N / M
        w = np.zeros(N, dtype=complex)                    # troncamento: M -> N modi
```

```
        w[:N // 2] = wM[:N // 2]
        w[N // 2:] = wM[M - N // 2:]
        return -w

    E = np.exp(-nu * k**2 * dt / 2)                        # fattore integrante su mezzo passo
    passi = int(round(t_fin / dt))
    with np.errstate(over="ignore", invalid="ignore"):    # un calcolo instabile produce inf/nan
        for _ in range(passi):
            a = dt * nonlin(uh)
            b = dt * nonlin(E * (uh + a / 2))
            c = dt * nonlin(E * uh + b / 2)
            d = dt * nonlin(E * E * uh + E * c)
            uh = E * E * uh + (E * E * a + 2 * E * (b + c) + d) / 6
            if not np.all(np.isfinite(uh)):
                return x, None
    return x, np.fft.ifft(uh).real


# ------------------------------------------------------------- convergenza (nu = 0,05)
t_fin = 1.5
xr = np.linspace(0, 2 * np.pi, 65)[:-1]
u_rif = esatta(xr, t_fin)
print(f"Burgers, nu = {NU}, t = {t_fin}: pendenza massima della soluzione esatta "
      f"{np.max(np.abs(np.gradient(u_rif, xr))):.2f} (iniziale 1)")
print("     N     dt      errore senza de-aliasing   errore con regola 3/2")
Ns = (16, 32, 64, 128, 256)
ea, ed = [], []
for N in Ns:
    dt = 0.5 / N
    x, ua = burgers(N, t_fin, dt, dealias=False)
    x, ud = burgers(N, t_fin, dt, dealias=True)
    ue = esatta(x[::max(N // 64, 1)], t_fin)
    ea.append(np.max(np.abs(ua[::max(N // 64, 1)] - ue)))
    ed.append(np.max(np.abs(ud[::max(N // 64, 1)] - ue)))
    print(f"   {N:4d}   {dt:.4f}      {ea[-1]:11.3e}                {ed[-1]:11.3e}")
# spettro a t = 1,5
N = 128
x, u = burgers(N, t_fin, 0.5 / N, dealias=True)
uh = np.fft.fft(u) / N
print(f"   coefficienti |u^_k| a t = {t_fin} (N = 128): k = 1: {abs(uh[1]):.3e}, 8: {abs(uh[8]):.3e},"
      f" 16: {abs(uh[16]):.3e}, 32: {abs(uh[32]):.3e}, 48: {abs(uh[48]):.3e}, 63: {abs(uh[63]):.3e}")

# ------------------------------------------------------------- aliasing a viscosita' piccola
print("\nViscosita' piccola (nu = 0,001), t = 2 (l'urto si forma per t = 1):")
print("     N     senza de-aliasing              regola dei 3/2")
for N in (64, 128, 256, 512):
    riga = f"   {N:4d}"
    for da in (False, True):
        x, u = burgers(N, 2.0, 0.25 / N, da, nu=0.001)
        riga += "     instabile                " if u is None else f"     max|u| = {np.max(np.abs(u)):.3f}, E = {np.sum(u**2)/N:.4f}"
    print(riga)
print("   (E = media di u^2; vale 0,5 a t = 0 e, per nu -> 0 con l'urto, decresce lentamente)")
```

```
122
123 # ------------------------------------------------------------- trasformata
    rapida "a mano"
124 def fft_ricorsiva(f):
125     """FFT radix-2 (Cooley-Tukey): T(N) = 2 T(N/2) + O(N)  =>  O(N log N)."""
126     N = len(f)
127     if N == 1:
128         return f.astype(complex)
129     pari, dispari = fft_ricorsiva(f[0::2]), fft_ricorsiva(f[1::2])
130     w = np.exp(-2j * np.pi * np.arange(N // 2) / N)
131     return np.concatenate([pari + w * dispari, pari - w * dispari])
132
133
134 def dft_diretta(f):
135     N = len(f)
136     n = np.arange(N)
137     return np.exp(-2j * np.pi * np.outer(n, n) / N) @ f
138
139
140 print("\nTrasformata rapida ricorsiva contro somma diretta:")
141 for N in (64, 256, 1024, 4096):
142     f = np.random.default_rng(0).standard_normal(N)
143     t0 = time.perf_counter(); F1 = dft_diretta(f); t1 = time.perf_counter() - t0
144     t0 = time.perf_counter(); F2 = fft_ricorsiva(f); t2 = time.perf_counter() - t0
145     t0 = time.perf_counter(); F3 = np.fft.fft(f); t3 = time.perf_counter() - t0
146     print(f"   N = {N:5d}: scarto max {np.max(np.abs(F2 - F3)):.1e}; tempi: diretta
    {t1*1e3:7.2f} ms,"
147           f" ricorsiva {t2*1e3:6.2f} ms, numpy {t3*1e6:6.1f} us;  N^2/(N log2 N) =
    {N/np.log2(N):.0f}")
148
149 # ------------------------------------------------------------------ figure
150 fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.8))
151 xe = np.linspace(0, 2 * np.pi, 201)
152 for t, col in zip((0.0, 0.5, 1.0, 1.5, 3.0), (GRIGIO, VERDE, ARANCIO, ROSSO, BLU)):
153     ax[0].plot(xe, esatta(xe, t) if t > 0 else np.sin(xe), color=col,
    label=f"$t={t:g}$".replace(".", "{,}"))
154 x, u = burgers(32, 1.5, 0.5 / 32, dealias=True)
155 ax[0].plot(x, u, "o", color=ROSSO, ms=2.5, mfc="none")
156 ax[0].set_xlabel("$x$")
157 ax[0].set_ylabel("$u$")
158 ax[0].legend(fontsize=6.5)
159 ax[0].set_title("soluzione esatta e $N=32$ (cerchi)")
160 ax[1].semilogy(Ns, np.maximum(ea, 1e-16), "s--", color=ROSSO, label="senza
    de-aliasing")
161 ax[1].semilogy(Ns, np.maximum(ed, 1e-16), "o-", color=BLU, label="regola dei 3/2")
162 ax[1].set_xlabel("$N$")
163 ax[1].set_ylabel("errore massimo a $t=1{,}5$")
164 ax[1].legend(fontsize=7)
165 ax[1].set_title("convergenza")
166 x2, u2 = burgers(256, 2.0, 0.25 / 256, True, nu=0.001)
167 x1, u1 = burgers(64, 2.0, 0.25 / 64, False, nu=0.001)
168 ax[2].plot(x2, u2, color=BLU, lw=1.0, label="$N=256$, 3/2")
169 ax[2].plot(x1, u1, "--", color=ROSSO, lw=1.0, label="$N=64$, con aliasing")
170 ax[2].set_xlabel("$x$")
171 ax[2].set_ylabel("$u$")
172 ax[2].legend(fontsize=7)
173 ax[2].set_title("$\\nu=0{,}001$, $t=2$")
174 fig.tight_layout()
175 salva(fig, "prog3_9b_burgers")
```

Listato 10.3: `prog3_9b_burgers.py`: equazione di Burgers con il metodo pseudo-spettrale, regola dei 3/2, confronto con la soluzione di Cole–Hopf e trasformata rapida ricorsiva.

Risultati del Programma 3.9b

- $\nu = 0{,}05$, $t = 1{,}5$ (il fronte ha pendenza massima 6,5, contro 1 iniziale):

N	Δt	errore, con aliasing	errore, regola dei 3/2
16	0,0312	$9{,}348 \times 10^{-2}$	$1{,}253 \times 10^{-1}$
32	0,0156	$2{,}280 \times 10^{-2}$	$3{,}637 \times 10^{-2}$
64	0,0078	$6{,}707 \times 10^{-4}$	$2{,}056 \times 10^{-3}$
128	0,0039	$1{,}052 \times 10^{-6}$	$6{,}420 \times 10^{-6}$
256	0,0020	$9{,}431 \times 10^{-10}$	$9{,}938 \times 10^{-10}$

  La convergenza è spettrale: da $N = 64$ a 256 l'errore scende di sei ordini di grandezza, e con $N = 256$ si raggiunge la precisione del passo temporale. Lo spettro a $t = 1{,}5$ decade esponenzialmente ($|\hat{u}_k| = 0{,}37$; $2{,}6 \times 10^{-2}$; $5{,}8 \times 10^{-3}$; $3{,}3 \times 10^{-4}$; $1{,}4 \times 10^{-6}$ per $k = 1$; 8; 16; 32; 63): il fronte viscoso ha spessore $\sim \nu$, e per risolverlo servono $k \sim 1/\nu$ modi. A questa viscosità l'aliasing è innocuo (l'errore è perfino un po' minore, per compensazione casuale di errori).

- $\nu = 0{,}001$, $t = 2$ (urto quasi discontinuo, formato per $t = 1$): con l'aliasing il calcolo con $N = 128$ produce valori di $|u|$ fino a 11 e quelli con $N = 256$ e 512 divergono; con la regola dei 3/2 tutti restano limitati (max $|u|$ fra 1,13 e 1,17, energia $\sum u^2/N = 0{,}33$), con le oscillazioni di Gibbs attorno all'urto non risolto (Fig. 10.3, a destra). È la prova sperimentale di quanto affermato sopra: l'aliasing del termine convettivo può distruggere la stabilità, e il de-aliasing la ripristina perché restituisce al termine non lineare la sua proprietà di conservare l'energia.

- *Trasformata rapida.* Una FFT ricorsiva di dieci righe (Cooley–Tukey, radice 2) coincide con quella di NumPy entro 10^{-13}; per $N = 4096$ impiega 57 ms contro 875 ms della somma diretta $\mathcal{O}(N^2)$, e la libreria, scritta in C, 0,14 ms.

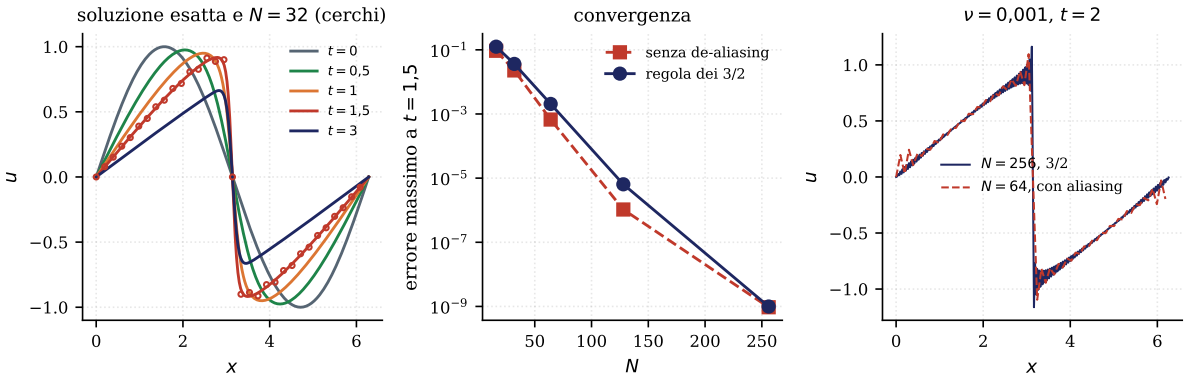


Figura 10.3: Equazione di Burgers pseudo-spettrale: soluzione esatta a vari istanti e soluzione con $N = 32$ (a sinistra); convergenza spettrale con e senza de-aliasing per $\nu = 0{,}05$ (al centro); soluzioni a viscosità piccola, con la regola dei 3/2 e con aliasing (a destra).

10.6.3 Perché la trasformata rapida costa $N \log N$

La somma (10.3) richiede N moltiplicazioni complesse per ciascuno degli N coefficienti: N^2 operazioni. L'osservazione di Cooley e Tukey [26], già nota a Gauss, è che per N pari la somma si spezza nei termini di indice pari e dispari:

$$\hat{u}_n = \sum_{m=0}^{N/2-1} u_{2m}\, e^{-2\pi \mathrm{i} n m/(N/2)} + e^{-2\pi \mathrm{i} n/N} \sum_{m=0}^{N/2-1} u_{2m+1}\, e^{-2\pi \mathrm{i} n m/(N/2)} = E_n + w^n O_n,$$

dove E e O sono trasformate di lunghezza $N/2$ e $w = e^{-2\pi \mathrm{i}/N}$. Poiché $E_{n+N/2} = E_n$ e $w^{n+N/2} = -w^n$, con due trasformate di lunghezza $N/2$ e $N/2$ moltiplicazioni si ottengono tutti gli N coefficienti: $T(N) = 2T(N/2) + \mathcal{O}(N)$, da cui, per $N = 2^l$, $T(N) = \mathcal{O}(N \log_2 N)$. Il guadagno rispetto alla somma diretta è $N/\log_2 N$: 340 volte per $N = 4096$, 5×10^4 per $N = 10^6$. È questa riduzione che ha reso possibili le simulazioni spettrali della turbolenza, nelle quali a ogni passo temporale si eseguono decine di trasformate tridimensionali su griglie di miliardi di punti.

Appendice A

Simboli, listati di supporto e istruzioni

A.1 Simboli principali

A.2 Listati di supporto


metodi_numerici.py

```
"""
metodi_numerici.py -- strumenti comuni della monografia 3 (flussi viscosi).

Contiene:
  * trid(C)              eliminazione di Gauss per sistemi tridiagonali (forma a 4 colonne)
  * thomas(a, b, c, d)   algoritmo di Thomas in memoria compatta
  * rk4_passo(f, x, y, h) un passo di Runge-Kutta del quarto ordine per y' = f(x, y)
  * rk4_integra(...)     integrazione a passo costante con memorizzazione della soluzione
  * liebmann(F, Q, h, errmax)  Gauss-Seidel per lap(F) = Q su griglia quadrata
  * sor_poisson(...)     sovrarilassamento successivo per lap(F) = Q
  * omega_ottimo(m, n)   fattore di sovrarilassamento ottimo per un rettangolo
  * cheb(N)              matrice di differenziazione di Chebyshev e nodi di Gauss-Lobatto
  * numeri_onda(N, L)    numeri d'onda della trasformata discreta di Fourier
"""
import numpy as np


# ---------------------------------------------------------------------------
# 1. Sistemi tridiagonali
# ---------------------------------------------------------------------------
def trid(C):
    """Risolve il sistema tridiagonale scritto riga per riga come
         C[i,0]*f[i-1] + C[i,1]*f[i] + C[i,2]*f[i+1] = C[i,3],  i = 0..n-1
    (C[0,0] e C[n-1,2] devono essere gia' stati azzerati dalle condizioni al
    contorno). Eliminazione in avanti per combinazione di righe, poi
    sostituzione all'indietro. Restituisce il vettore f (C non viene alterata).
    """
    C = np.array(C, dtype=float)          # copia di lavoro
    n = C.shape[0]
    for i in range(1, n - 1):             # eliminazione di C[i,0], i = 1..n-2
        c2 = C[i, 1] * C[i - 1, 1] - C[i, 0] * C[i - 1, 2]
        c3 = C[i, 2] * C[i - 1, 1]
        c4 = C[i, 3] * C[i - 1, 1] - C[i, 0] * C[i - 1, 3]
        C[i, 1], C[i, 2], C[i, 3] = c2, c3, c4
    f = np.zeros(n)
```

```
    # ultime due equazioni risolte insieme
    f[n - 1] = ((C[n - 1, 3] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 3]) /
                (C[n - 1, 1] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 2]))
    for j in range(n - 2, -1, -1):        # sostituzione all'indietro
        f[j] = (C[j, 3] - C[j, 2] * f[j + 1]) / C[j, 1]
    return f


def thomas(a, b, c, d):
    """Algoritmo di Thomas per  a[i] x[i-1] + b[i] x[i] + c[i] x[i+1] = d[i].
    a[0] e c[-1] non vengono usati. Funziona anche con coefficienti complessi."""
    n = len(d)
    tipo = np.result_type(a, b, c, d, float)
    cp = np.zeros(n, dtype=tipo)
    dp = np.zeros(n, dtype=tipo)
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]
    for i in range(1, n):                 # eliminazione in avanti
        den = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / den if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / den
    x = np.zeros(n, dtype=tipo)
    x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):        # sostituzione all'indietro
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


# ---------------------------------------------------------------------------
# 2. Runge-Kutta del quarto ordine per sistemi  y' = f(x, y)
# ---------------------------------------------------------------------------
def rk4_passo(f, x, y, h):
    """Un passo RK4: restituisce y(x + h)."""
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0


def rk4_integra(f, y0, x_fin, h, x0=0.0):
    """Integra y' = f(x, y) da x0 a x_fin con passo h costante.
    Restituisce i vettori x (n+1) e la matrice Y (n+1, dim)."""
    n = int(round((x_fin - x0) / h))
    x = x0 + h * np.arange(n + 1)
    Y = np.zeros((n + 1, len(y0)))
    Y[0] = y0
    for i in range(n):
        Y[i + 1] = rk4_passo(f, x[i], Y[i], h)
    return x, Y


# ---------------------------------------------------------------------------
# 3. Equazione di Poisson  F_xx + F_yy = Q  su griglia di passo h.
#    F[i, j] = valore nel nodo (x_i, y_j); i valori sul contorno restano fissi.
# ---------------------------------------------------------------------------
def liebmann(F, Q, h, errmax, azzera=True):
    """Metodo di Liebmann (Gauss-Seidel) sui nodi interni di un rettangolo;
    si ferma quando la somma dei |variazioni| di un'iterazione e' <= errmax.
    Se azzera=True i nodi interni partono da zero (come la subroutine LIEBMN).
```

```
    Restituisce (F, numero di iterazioni)."""
    M, N = F.shape
    if azzera:
        F[1:-1, 1:-1] = 0.0
    it = 0
    while True:
        it += 1
        err = 0.0
        for i in range(1, M - 1):
            for j in range(1, N - 1):
                nuovo = 0.25 * (F[i - 1, j] + F[i + 1, j] + F[i, j - 1] +
                                F[i, j + 1] - h * h * Q[i, j])
                err += abs(nuovo - F[i, j])
                F[i, j] = nuovo
        if err <= errmax:
            return F, it


def sor_poisson(F, Q, h, omega, errmax, nodi, prima_del_nodo=None, it_max=100000):
    """Sovrarilassamento successivo sui nodi 'nodi' (lista di coppie (i, j),
    visitati nell'ordine dato). prima_del_nodo(F, i, j), se fornita, viene
    chiamata prima di aggiornare ogni nodo (serve ad aggiornare i nodi fittizi
    delle condizioni di derivata). Restituisce (F, iterazioni)."""
    for it in range(1, it_max + 1):
        err = 0.0
        for i, j in nodi:
            if prima_del_nodo is not None:
                prima_del_nodo(F, i, j)
            gs = 0.25 * (F[i - 1, j] + F[i + 1, j] + F[i, j - 1] + F[i, j + 1] - h
    * h * Q[i, j])
            nuovo = (1.0 - omega) * F[i, j] + omega * gs
            err += abs(nuovo - F[i, j])
            F[i, j] = nuovo
        if err <= errmax:
            return F, it
    raise RuntimeError("SOR: convergenza non raggiunta")


def omega_ottimo(m, n):
    """Fattore di sovrarilassamento ottimo per un rettangolo con m x n linee di
    griglia (come nei programmi originali):
    omega = (8 - 4 sqrt(4 - a^2)) / a^2,  a = cos(pi/m) + cos(pi/n)."""
    a = np.cos(np.pi / m) + np.cos(np.pi / n)
    return (8.0 - 4.0 * np.sqrt(4.0 - a * a)) / (a * a)


# ---------------------------------------------------------------------------
# 4. Metodi spettrali
# ---------------------------------------------------------------------------
def cheb(N):
    """Nodi di Chebyshev-Gauss-Lobatto y_j = cos(j pi / N), j = 0..N (da +1 a -1)
    e matrice di differenziazione D (N+1 x N+1): (D u)_j approssima u'(y_j).
    Elementi fuori diagonale: D_jk = c_j (-1)^(j+k) / (c_k (y_j - y_k)), c_0 = c_N
    = 2;
    la diagonale si ottiene imponendo che la derivata di una costante sia nulla
    (somma per righe nulla), che e' anche la scelta numericamente piu' stabile."""
    if N == 0:
        return np.zeros((1, 1)), np.array([1.0])
    j = np.arange(N + 1)
    y = np.cos(np.pi * j / N)
```

```
    c = np.ones(N + 1)
    c[0] = c[-1] = 2.0
    c = c * (-1.0) ** j
    Y = np.tile(y, (N + 1, 1)).T
    dY = Y - Y.T
    D = np.outer(c, 1.0 / c) / (dY + np.eye(N + 1))
    D = D - np.diag(D.sum(axis=1))
    return D, y


def numeri_onda(N, L):
    """Numeri d'onda k_n = 2 pi n / L nell'ordine usato da numpy.fft
    (n = 0, 1, ..., N/2-1, -N/2, ..., -1)."""
    return 2.0 * np.pi * np.fft.fftfreq(N, d=L / N)
```

Listato A.1: `metodi_numerici.py`: sistemi tridiagonali, Runge–Kutta, metodi iterativi per l'equazione di Poisson, matrice di Chebyshev, numeri d'onda di Fourier.

stile_grafici.py

```
"""
stile_grafici.py -- impostazioni grafiche comuni (palette "della casa").
Importato da tutti gli script della monografia.
"""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Palette coerente con raucci-aero.sty
BLU      = "#1E2761"
ARANCIO  = "#DC7633"
ROSSO    = "#C0392B"
VERDE    = "#1E8449"
GIALLO   = "#B7950B"
VIOLA    = "#6C3483"
GRIGIO   = "#566573"
PALETTE  = [BLU, ARANCIO, VERDE, VIOLA, ROSSO, GIALLO, GRIGIO]

plt.rcParams.update({
    "font.family": "serif",
    "font.size": 9,
    "axes.labelsize": 9,
    "axes.titlesize": 9,
    "legend.fontsize": 8,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "axes.prop_cycle": matplotlib.cycler(color=PALETTE),
    "axes.grid": True,
    "grid.alpha": 0.3,
    "grid.linestyle": ":",
    "lines.linewidth": 1.4,
    "figure.dpi": 120,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.02,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "legend.frameon": False,
})

# Le figure vengono salvate nella cartella "figure" accanto a quella degli script
# (creata se manca): cosi' ogni programma funziona anche se scaricato da solo.
FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "figure")
```

```
44
45
46 def salva(fig, nome):
47     """Salva la figura in PDF nella cartella delle figure del progetto LaTeX."""
48     os.makedirs(FIG_DIR, exist_ok=True)
49     from matplotlib.ticker import NullFormatter
50     for ax in fig.get_axes():              # niente etichette secondarie sugli assi
       logaritmici
51         if ax.get_xscale() == "log":
52             ax.xaxis.set_minor_formatter(NullFormatter())
53         if ax.get_yscale() == "log":
54             ax.yaxis.set_minor_formatter(NullFormatter())
55     fig.savefig(os.path.join(FIG_DIR, f"{nome}.pdf"))
56     plt.close(fig)
57     print(f"  -> figura salvata: {nome}.pdf")
```

Listato A.2: `stile_grafici.py`: impostazioni comuni di Matplotlib.

A.3 Come riprodurre i risultati

Istruzioni

1. Requisiti: Python $\geq$ 3.9 con `numpy`, `scipy` e `matplotlib` (`pip install numpy scipy matplotlib`); una distribuzione TEX con `pdflatex` e i pacchetti `tcolorbox`, `pgfplots`, `siunitx`, `listings`, `qrcode`, `marginnote`, `babel-italian` (tutti inclusi in TEX Live / MacTeX completo).
2. Dalla cartella `codice/`: `bash esegui_tutti.sh` esegue tutti i programmi e gli esercizi (circa cinque minuti, di cui un minuto per la curva neutra del Programma 3.8b e uno per l'Esercizio 15) e rigenera le figure in `figure/`. Ogni programma si può anche eseguire da solo con `python3 nome.py`; per gli esercizi, `python3 esercizi_svolti.py 5 12` esegue solo il quinto e il dodicesimo.
3. Dalla cartella principale: `pdflatex main.tex` due volte (o `latexmk -pdf main.tex`).

A.4 Codici in rete e QR code

I programmi sono pubblicati all'indirizzo

`https://braucci.github.io/CFD/monografia3/codici/`

La pagina elenca tutti i file con le rispettive dipendenze, li mostra con l'evidenziazione della sintassi e permette di scaricarli singolarmente o in un unico archivio. Il QR code nel margine accanto a ogni listato codifica l'indirizzo del file corrispondente e, nel PDF, è anche un collegamento cliccabile.

Per l'autore: configurazione e pubblicazione

1. In `main.tex` la macro `\nomemonografia` (qui `monografia3`) determina tutti gli indirizzi: `\urlcodici` vale `https://braucci.github.io/CFD/<nome>/codici/` e

`\urlfile{file.py}` aggiunge il nome del file.

2. Un listato completo si inserisce con `\listatopy[opzioni]{file.py}` (legge `codice/file.py` e mette il QR nel margine); prima di un estratto in `lstlisting` si scrive `\qrlistato{file.py}`.
3. `python3 pubblica_github.py` crea `github/monografia3/` con il PDF, la cartella `codici/`, la pagina `index.html` e l'archivio zip: va copiata così com'è nella cartella `CFD/` del repository, accanto a `monografia1/` e `monografia2/`.

A.5 Come sostituire le figure

Le figure del documento sono di due tipi, gestite da due macro definite in `main.tex`.

`\figuraesterna[larghezza]{nome}{testo}` per le figure schematiche: geometrie, griglie e molecole di calcolo. Se nella cartella `immagini/` esiste il file `nome.png` (oppure `.pdf` o `.jpg`) viene incluso con la larghezza indicata; altrimenti compare un *segnaposto* grigio con il nome del file atteso e una breve descrizione. Basta copiare in `immagini/` le immagini con i nomi della Tab. A.2 e ricompilare.

`\figuragenerata[larghezza]{nome}` grafici prodotti dagli script Python, che il documento legge dalla cartella `figure/`. Anche questi si possono sostituire: se esiste `immagini/nome.png`, ha la precedenza sul PDF generato.

Tabella A.1: Principali simboli della monografia e nomi corrispondenti nei programmi.

Simbolo	Programma	Significato
Grandezze generali (Cap. 1)		
μ, ν	`mu`, `NU`	viscosità dinamica e cinematica
Re, *Pr*, *Ra*	`RE`, `PR`, `Ra`	numeri di Reynolds, Prandtl, Rayleigh
ψ, ζ	`psi`, `zeta`	funzione di corrente e vorticità, $\nabla^2\psi = -\zeta$
Φ	–	funzione di dissipazione
P	`P`	parametro del gradiente di pressione nella corrente di Couette–Poiseuille
Strato limite (Cap. 2, 3)		
η	`eta`	variabile di similitudine $y\sqrt{U/(\nu x)}$
f, $p = f'$, $q = f''$	`f`, `p`, `q`	funzione di Blasius e sue derivate
$q_0 = f''(0)$	`q0`	sforzo adimensionale alla parete (0,332057)
$\eta_{\max}$, ε	`ETAMAX`, `EPSLON`	estremo del dominio e tolleranza della condizione all'infinito
δ, δ^*, θ, H	–	spessore convenzionale, di spostamento, di quantità di moto; fattore di forma
C_f, C_D	–	coefficiente d'attrito locale e di resistenza della lamina
$\theta(\eta)$	`theta`	temperatura adimensionale $(T - T_1)/(U^2/2c_p)$
r	–	fattore di recupero $\theta(0)$
Nu_x	–	numero di Nusselt locale
$C_{i1}, \ldots, C_{i4}$	`C[i, 0:4]`	coefficienti del sistema tridiagonale
Condotti ed equazioni paraboliche (Cap. 4–6)		
U	`U`	velocità assiale adimensionale nel canale
ω	`omega`	fattore di sovrarilassamento
Q	`portata`	portata volumetrica adimensionale
$R = \nu\tau/h^2$	`R`	parametro di stabilità degli schemi parabolici
τ, h	`TAU`, `H`	passo temporale e spaziale
G	–	fattore di amplificazione di von Neumann
W	`W`	differenza fra il profilo di regime e quello attuale
ϑ	`theta`	peso dello schema ϑ (1: implicito; 1/2: Crank–Nicolson)
Stokes, stabilità, metodi spettrali (Cap. 7–10)		
ERPSI, ERZETA	`erpsi`, `erzeta`	somme delle variazioni di ψ e ζ in un'iterazione
k	`k`	numero d'onda orizzontale (Bénard) o di Fourier
θ', w'	`th`, `w`	ampiezze delle perturbazioni di temperatura e velocità verticale
$\mathbf{A}$, $\mathbf{B}$	`A`, `B`	matrici del problema agli autovalori generalizzato
$\mathbf{D}$, y_j	`D`, `y`	matrice di differenziazione e nodi di Chebyshev
α, $c = c_R + \mathrm{i}c_I$	`alfa`, `c`	numero d'onda e velocità complessa dell'onda (Orr–Sommerfeld)
$\lambda = -\mathrm{i}c$	`lam`	autovalore di Orr–Sommerfeld, $\lambda_R = c_I$
k_n, $\hat{u}$	`k`, `uh`	numeri d'onda e coefficienti di Fourier

Tabella A.2: Figure con segnaposto e nome del file atteso in `immagini/`.

Figura	File	Contenuto
2.1	`fig_3_2_1.png`	strato limite su una lamina piana
2.2	`fig_3_2_2.png`	metodo del dimezzamento per il problema di Blasius
B.2	`fig_3_2_4.png`	lamina con iniezione di fluido (Esercizio 2)
B.6	`fig_3_3_1.png`	fluido attorno a un cilindro rotante (Esercizio 5)
4.1	`fig_3_4_1.png`	canale inclinato a pelo libero
4.2	`fig_3_4_2.png`	aree di competenza dei nodi per la portata
B.19	`fig_3_4_4.png`	sezioni di tubi diverse (Esercizio 17)
5.1	`fig_3_5_1.png`	griglia del metodo esplicito e dominio di dipendenza
6.2	`fig_3_6_2.png`	molecola di calcolo di Crank–Nicolson
7.1	`fig_3_7_1.png`	cavità mossa da una lastra e nodi della vorticità di parete
8.1	`fig_3_8_1.png`	schema del problema di Bénard
9.3	`fig_3_8_2.png`	corrente di Poiseuille piana

Appendice B

Esercizi svolti

Gli esercizi completano i capitoli: alcuni migliorano un algoritmo del testo, altri estendono i programmi a situazioni nuove, due sono veri e propri progetti. Ogni esercizio è svolto con lo stesso schema dei moduli precedenti: *testo*, *impostazione* (che cosa ci si aspetta e perché, prima di calcolare), *svolgimento* passo per passo con il codice essenziale, *risultato* e *commento*. Il codice completo è nel file `esercizi_svolti.py` (Listato B.1), in cui ogni esercizio è la funzione `es_NN()`; i numeri riportati sono quelli stampati dal programma.

B.1 Esercizio 1 – Il problema di Blasius senza iterazioni

Testo

Il Programma 3.1 impiega 31 iterazioni, quasi tutte spese a cercare per tentativi il valore di $f''(0)$. Si introducano la trasformazione lineare $\eta = kz$, con k costante da determinare, e la funzione $g(z) = k\,f(\eta)$. Mostrare che il problema di Blasius si trasforma in $g''' + \frac{1}{2} g g'' = 0$, $g(0) = g'(0) = 0$, $g'(\infty) = k^2$; che si può scegliere $g''(0) = 1$; e che allora $f''(0) = k^{-3}$. Scrivere un programma che ricavi $f''(0)$ con una sola integrazione.

Impostazione. L'equazione di Blasius e le sue due condizioni nell'origine non contengono alcuna scala: se f è una soluzione, lo è anche una sua versione opportunamente "stirata". La condizione all'infinito fissa la scala, ma si può imporla *dopo* l'integrazione, scegliendo lo stiramento giusto. È il metodo di Töpfer (1912), un primo esempio dell'uso dei gruppi di trasformazioni nelle equazioni differenziali.

Svolgimento.

1. Da $f(\eta) = g(z)/k$ con $z = \eta/k$ si ha $\mathrm{d}/\mathrm{d}\eta = k^{-1}\mathrm{d}/\mathrm{d}z$, quindi

$$\frac{\mathrm{d}^n f}{\mathrm{d}\eta^n} = \frac{1}{k^{n+1}} \frac{\mathrm{d}^n g}{\mathrm{d}z^n}, \qquad n \geq 1.$$

   Sostituendo: $f''' + \frac{1}{2} f f'' = k^{-4}\left(g''' + \frac{1}{2} g g''\right) = 0$. Le condizioni $f(0) = f'(0) = 0$ diventano $g(0) = g'(0) = 0$, e $f'(\infty) = 1$ diventa $g'(\infty) = k^2$.

2. Poiché k è arbitrario e compare solo nella condizione all'infinito, il valore di $g''(0)$ non è vincolato: si sceglie $g''(0) = 1$.

3. Si integra g da $z = 0$ con $g = g' = 0$, $g'' = 1$ fino a un z abbastanza grande; l'ultimo valore di g' fornisce $k = \sqrt{g'(\infty)}$ e, dalla relazione fra le derivate, $f''(0) = g''(0)/k^3 = k^{-3}$.

Risultato

Con un integratore adattivo di ordine 8:

$$g'(\infty) = 2{,}0854091764, \qquad k = 1{,}44409459, \qquad f''(0) = k^{-3} = 0{,}3320573362,$$

identico, in tutte le cifre, al valore di riferimento del Programma 3.1b. Con $z_{\max} = 6$ (cioè $\eta_{\max} = kz_{\max} = 8{,}7$) si ottiene già 0,3320575. Il profilo $f'(\eta) = g'(z)/k^2$ coincide con quello ottenuto integrando la (2.7) con $f''(0) = k^{-3}$ entro 7×10^{-8} (Fig. B.1).

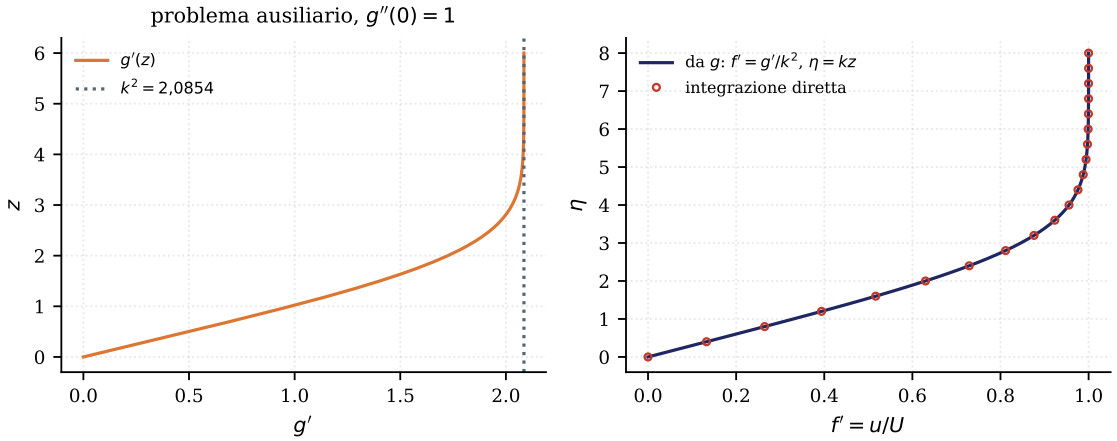


Figura B.1: Esercizio 1: soluzione del problema ausiliario con $g''(0) = 1$ (a sinistra) e profilo di Blasius ricostruito per riscalamento (a destra).

Commento. Il metodo funziona perché il problema è invariante per il cambio di scala. Non funziona più per la lamina con iniezione dell'Esercizio 2, dove la condizione $f(0) = -2C \neq 0$ diventa $g(0) = -2Ck$: la costante incognita compare anche nell'origine e $g''(0)$ non si può più scegliere liberamente. In quei casi resta il tiro, possibilmente con il metodo di Newton.

B.2 Esercizio 2 – Lamina con iniezione e aspirazione

Testo

Attraverso una lamina porosa si inietta ($C > 0$) o si aspira ($C < 0$) fluido con velocità normale $v_0 = C\sqrt{\nu U/x}$ (Fig. B.2). Mostrare che il moto resta autosimile, con la sola condizione alla parete modificata in $f(0) = -2C$. Scrivere un programma con il metodo del dimezzamento, modificato in modo da funzionare qualunque sia il verso di variazione di $p_{\max}$ con q_0, ed eseguirlo per C da $-0{,}6$ a $0{,}6$ a passi di $0{,}1$. Stimare lo spessore dello strato limite, tracciare δ e $f''(0)$ in funzione di C, dare l'interpretazione fisica di $f''(0) = 0$, spiegare le difficoltà che nascono per $C > 0{,}6$ e mostrare che per $C > 0$ il profilo di velocità ha un punto di flesso.

Segnaposto figura

file atteso: `immagini/fig_3_2_4.png`

Lamina piana porosa con iniezione di fluido di velocità $v_0(x)$ e profilo di velocità con punto di flesso (Fig. 3.2.4 della fonte).

Figura B.2: Lamina con iniezione di fluido.

Impostazione. L'aspirazione rimuove il fluido rallentato vicino alla parete: ci si aspetta uno strato più sottile e uno sforzo maggiore. L'iniezione aggiunge fluido fermo: lo strato si ispessisce, lo sforzo diminuisce e, per un'iniezione abbastanza forte, lo strato viene "soffiato via" dalla parete. La similitudine è possibile solo se v_0 decresce come $x^{-1/2}$, come la velocità normale dello strato di Blasius.

Svolgimento.

1. Dalla (2.8), alla parete ($\eta = 0$, $f' = 0$) $v = -\frac{1}{2}\sqrt{\nu U/x}\, f(0)$. Uguagliando a $v_0 = C\sqrt{\nu U/x}$, x si semplifica e resta $f(0) = -2C$: equazione e condizione all'infinito sono quelle di Blasius, la soluzione dipende ancora dalla sola η. Con una velocità di iniezione uniforme, invece, x resterebbe nella condizione e il moto non sarebbe autosimile.
2. Nel dimezzamento del Programma 3.1 il verso di Δq_0 si inverte quando $p_{\max}$ "scavalca" 1, ipotizzando che $p_{\max}$ cresca con q_0. Per un criterio indipendente da questa ipotesi si confrontano l'errore attuale e quello dell'iterazione precedente: si inverte e si dimezza il passo se cambiano segno (si è scavalcata la radice) oppure se l'errore è aumentato in modulo (si sta andando nella direzione sbagliata).

```
if err_prec is not None and (err * err_prec < 0 or abs(err_prec) - abs(err) <
    0):
    dq0 = -dq0 / 2.0
err_prec = err
```

esercizi_svolti.py

3. Il risultato si verifica con il metodo di Newton su $\eta_{\max} = 14$; lo spessore η_{99} è la quota dove $f' = 0{,}99$.
4. *Punto di flesso.* Dalla (2.7), $f''' = -\frac{1}{2} f f''$, e $f'' > 0$ nello strato: la curvatura del profilo $u(y)$, proporzionale a f''', si annulla dove $f = 0$. Per $C \leq 0$ si ha $f \geq 0$ ovunque e non ci sono flessi; per $C > 0$ f parte da $-2C < 0$ e cresce, quindi si annulla in un punto interno, dove il profilo ha un flesso.

Risultato

C	$f''(0)$ dimezz.	iter.	$f''(0)$ Newton	η_{99}	η del flesso
$-0{,}6$	0,814 427	33	0,814 427	3,404	–
$-0{,}5$	0,728 867	30	0,728 867	3,593	–
$-0{,}4$	0,644 969	32	0,644 969	3,801	–
$-0{,}3$	0,562 994	31	0,562 995	4,031	–
$-0{,}2$	0,483 253	35	0,483 253	4,288	–
$-0{,}1$	0,406 120	43	0,406 120	4,578	–
0,0	0,332 057	34	0,332 057	4,910	–
0,1	0,261 637	41	0,261 637	5,297	1,22
0,2	0,195 588	39	0,195 588	5,763	1,92
0,3	0,134 871	34	0,134 871	6,347	2,64
0,4	0,080 824	48	0,080 822	7,140	3,52
0,5	0,035 570	42	0,035 520	8,400	4,84
0,6	0,006 587	41	0,003 367	12,036	8,52

Proseguendo per continuazione in C con $\eta_{\max} = 40$: $f''(0) = 1{,}0 \times 10^{-3}$ per $C = 0{,}612$, $1{,}3 \times 10^{-4}$ per $C = 0{,}618$; per estrapolazione $f''(0) = 0$ per $C = 0{,}619$.

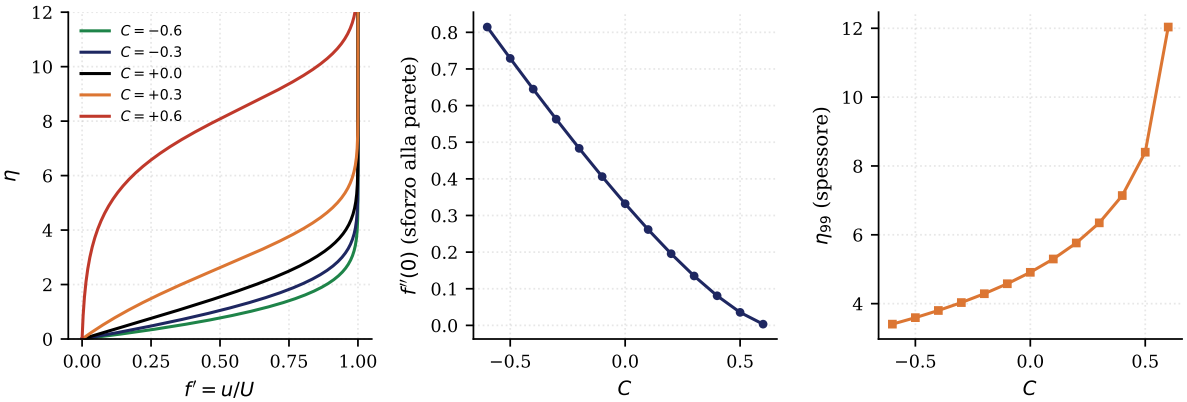


Figura B.3: Esercizio 2: profili di velocità per diverse intensità di aspirazione e iniezione (a sinistra), sforzo alla parete (al centro) e spessore dello strato limite (a destra) in funzione di C.

Commento. L'aspirazione con $C = -0{,}6$ riduce lo spessore del 31% e aumenta lo sforzo di un fattore 2,45: il profilo diventa più "pieno" e più stabile, ed è il principio del controllo laminare per aspirazione sperimentato sulle ali di alcuni velivoli. L'iniezione fa l'opposto: lo sforzo cala rapidamente e si annulla per $C \simeq 0{,}619$. Lo sforzo nullo alla parete significa che lo strato limite è stato staccato dalla parete (*blowing-off*): un cuscinetto di fluido iniettato separa la corrente dalla lamina. È il principio del raffreddamento per traspirazione degli ugelli e degli scudi termici. Per $C > 0{,}6$ il calcolo diventa difficile per tre ragioni: lo strato si ispessisce tanto che $\eta_{\max} = 10$ non basta più (a $C = 0{,}6$ il dimezzamento con $\eta_{\max} = 10$ dà 0,0066, il doppio del valore corretto); $f''(0)$ è piccolissimo e il residuo all'infinito diventa estremamente sensibile; oltre $C = 0{,}619$ non esistono più soluzioni attaccate. Il punto di flesso, infine, è un indizio di instabilità: secondo il criterio di Rayleigh un profilo con flesso è instabile già in fluido ideale, e con l'iniezione ci si aspetta una transizione precoce alla turbolenza.

B.3 Esercizio 3 – Strati limite con gradiente di pressione: Falkner–Skan

Testo

Se la corrente esterna lungo la lamina è $U_e = U_0 x^m$, il gradiente di pressione vale $\partial p/\partial x = -m\rho U_0^2 x^{2m-1}$: favorevole per $m > 0$, avverso per $m < 0$. Aggiungendo il termine di pressione all'equazione dello strato limite e usando le trasformazioni

$$\eta = y\sqrt{\frac{m+1}{2}\frac{U_0 x^{m-1}}{\nu}}, \qquad \psi = \sqrt{\frac{2\nu U_0 x^{m+1}}{m+1}}\, f(\eta),$$

mostrare che si ottiene l'equazione di Falkner–Skan

$$f''' + f f'' + \beta\left(1 - f'^2\right) = 0, \qquad \beta = \frac{2m}{m+1},$$

con le condizioni di Blasius. Risolverla per $m = 0$; 0,5; 1; $-0{,}05$; $-0{,}09$, e verificare che $f''(0)$ è quasi nullo per $m = -0{,}0904$.

Impostazione. Per $m > 0$ la $U_e = U_0 x^m$ è la velocità sulla parete di un cuneo di angolo $\pi\beta$ investito simmetricamente da una corrente uniforme; $m = 1$ ($\beta = 1$, angolo di 180°) è il ristagno su una parete piana normale alla corrente (corrente di Hiemenz). Un gradiente favorevole accelera il fluido vicino alla parete, assottiglia lo strato e aumenta lo sforzo; uno avverso lo rallenta fino al distacco, dove lo sforzo si annulla.

Svolgimento.

1. La sostituzione è analoga a quella del Cap. 2: con $u = U_e f'(\eta)$ i termini d'inerzia producono $-f f''$ e $\beta f'^2$, il termine di pressione $U_e\,\mathrm{d}U_e/\mathrm{d}x = m U_0^2 x^{2m-1}$ produce β, quello viscoso f'''; tutte le potenze di x si semplificano e resta l'equazione indicata.
2. Per $m = 0$ l'equazione non è identica a quella di Blasius, perché le variabili sono scalate diversamente: $\eta_{FS} = \eta_B/\sqrt{2}$ e $f_{FS} = f_B/\sqrt{2}$, da cui $f''_{FS}(0) = \sqrt{2}\, f''_B(0) = 0{,}46960$. È un controllo del programma.
3. La soluzione si trova con il metodo di Newton e le equazioni variazionali (ora $Q' = -(Fq + fQ) + 2\beta pP$), partendo per ogni m dalla soluzione del caso precedente (*continuazione*).
4. Il punto di distacco si trova imponendo $f''(0) = 0$ e cercando, con il metodo di Brent, il valore di β per cui $f'(\eta_{\max}) = 1$.

Risultato

m	β	$f''(0)$	geometria
1,00	1,0000	1,232 59	ristagno piano (180°)
0,50	0,6667	1,038 90	cuneo di 120°
0,00	0,0000	0,469 60	lamina piana
$-0{,}05$	$-0{,}1053$	0,309 75	gradiente avverso
$-0{,}09$	$-0{,}1978$	0,027 98	vicino al distacco

Distacco: $\beta_s = -0{,}198838$, cioè $m_s = \beta_s/(2-\beta_s) = -0{,}090429$, indipendente da $\eta_{\max}$ fra 10 e 14.

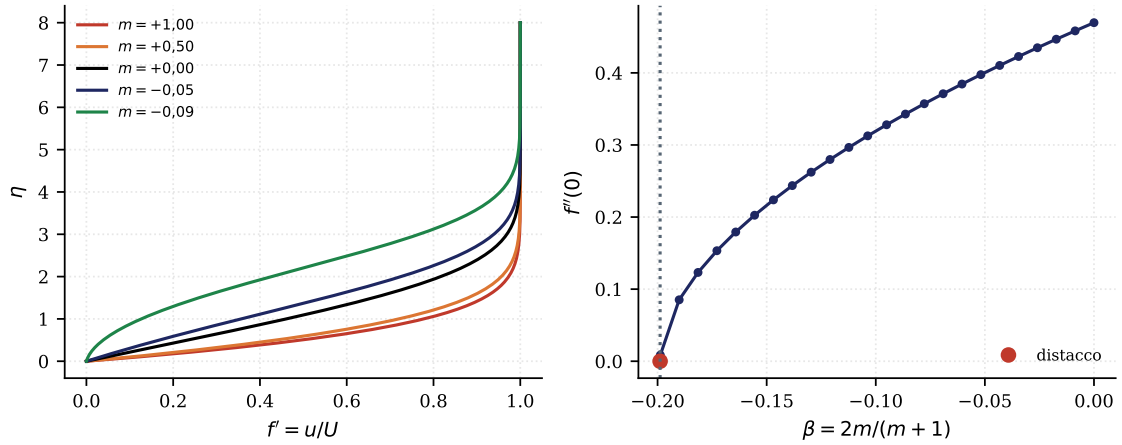


Figura B.4: Esercizio 3: profili di Falkner–Skan (a sinistra) e sforzo alla parete in funzione di β fino al distacco (a destra).

Commento. Il gradiente avverso è severissimo: basta una corrente esterna che decresce come $x^{-0{,}09}$ (una decelerazione del 6% quando x raddoppia) per portare lo strato limite laminare al distacco. Dall'equazione alla parete, $f'''(0) = -\beta$: con $\beta < 0$ la curvatura del profilo vicino alla parete è positiva e, poiché più in alto è negativa, esiste un flesso, come nell'Esercizio 2. Per $\beta < 0$ esiste anche un secondo ramo di soluzioni con corrente inversa vicino alla parete; per restare sul ramo fisico conviene procedere per continuazione da $\beta = 0$, come si è fatto qui. L'analogia con la corrente di Couette–Poiseuille del Cap. 1 ($P < -1$) è evidente.

B.4 Esercizio 4 – L'entalpia totale nello strato limite termico

Testo

L'entalpia totale per unità di massa è $H = c_p T + \frac{1}{2}u^2$ (il contributo di v è trascurabile). Mostrare che nello strato limite della lamina adiabatica

$$\frac{H - H_1}{U^2/2} = \theta + f'^2 - 1,$$

tracciarla in funzione di η per acqua, aria e mercurio e interpretare il risultato.

Impostazione. Fuori dallo strato limite $H = H_1 = c_p T_1 + \frac{1}{2}U^2$. Dalla (3.4), $c_p T = c_p T_1 + \frac{1}{2}U^2\theta$, e $u = Uf'$: la formula segue subito. Ci si aspetta che la parete, adiabatica e ferma, non possa né fornire né sottrarre energia al fluido: l'entalpia totale può solo essere *ridistribuita*.

Svolgimento.

1. Si risolve l'equazione dell'energia con lo schema del Programma 3.2, ma con $\eta_{\max} = 12$ per acqua e aria e 60 per il mercurio (il dominio deve contenere lo strato termico, § 3.6), e passo 0,02.

2. *Bilancio integrale.* Riscritta per H, l'equazione dello strato limite termico ha la forma $\rho(uH_x + vH_y) = \partial_y \left[(k/c_p)H_y + \mu(1 - Pr^{-1})uu_y\right]$. Integrandola attraverso lo strato, il flusso a secondo membro si annulla alla parete ($u = 0$ e $T_y = 0$) e all'infinito; quindi $\int_0^\infty \rho u(H - H_1)\,\mathrm{d}y$ non dipende da x, ed è nullo perché lo è al bordo d'attacco. In forma di similitudine:
$$\int_0^\infty f' \,\left(\theta + f'^2 - 1\right)\mathrm{d}\eta = 0.$$

3. Per $Pr = 1$ il termine $\mu(1 - Pr^{-1})uu_y$ scompare e H soddisfa un'equazione omogenea con condizioni omogenee: $H = H_1$ in tutto lo strato (relazione di Crocco–Busemann).

Risultato

fluido	alla parete	minimo	massimo	$\int f'(H - H_1)\mathrm{d}\eta$
acqua	1,4862	$-0{,}250$ ($\eta = 2{,}66$)	$+1{,}486$ (parete)	$-4{,}8 \times 10^{-5}$
aria	$-0{,}1559$	$-0{,}156$ (parete)	$+0{,}036$ ($\eta = 3{,}78$)	$-1{,}1 \times 10^{-5}$
mercurio	$-0{,}7987$	$-0{,}799$ (parete)	$+0{,}105$ ($\eta = 5{,}28$)	$-8{,}8 \times 10^{-7}$
$Pr = 1$	0,0000	$\lvert H - H_1\rvert < 5 \times 10^{-5}$ ovunque		$-1{,}4 \times 10^{-5}$

Gli integrali sono nulli entro l'errore di discretizzazione.

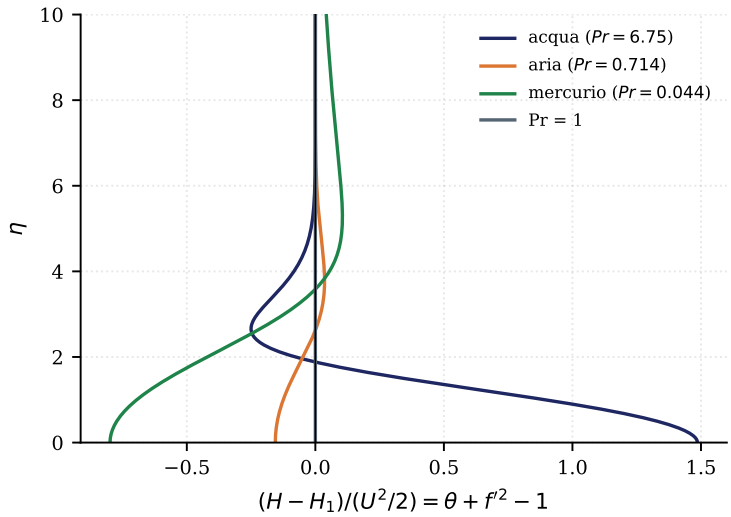


Figura B.5: Esercizio 4: differenza di entalpia totale adimensionale nello strato limite della lamina adiabatica.

Commento. Per $Pr < 1$ (aria, mercurio) vicino alla parete c'è un difetto di energia, compensato più in alto da un eccesso: la conduzione, più efficace della diffusione di quantità di moto, porta via dalla parete il calore prodotto per attrito e lo deposita negli strati esterni. Per $Pr > 1$ (acqua) accade il contrario: l'energia si accumula vicino alla parete, dove il calore prodotto non riesce ad allontanarsi, ed è sottratta agli strati esterni. In tutti i casi il bilancio integrale è nullo: la viscosità e la conduzione non creano energia, la spostano.

B.5 Esercizio 5 – Il fluido attorno a un cilindro rotante

Testo

Uno strato di fluido circonda un cilindro indefinito di raggio r_b; forze gravitazionali radiali lo mantengono in equilibrio con una superficie libera di raggio r_a (Fig. B.6). Il cilindro ruota con velocità periferica v_b. A regime la componente azimutale dell'equazione di Navier–Stokes si riduce, con $R = r/r_b$ e $V = v_\theta/v_b$, a

$$V'' + \frac{V'}{R} - \frac{V}{R^2} = 0, \qquad V(1) = 1, \qquad \frac{\mathrm{d}V}{\mathrm{d}R} = 0 \text{ in } R = R_a = \frac{r_a}{r_b},$$

dove la seconda condizione dovrebbe esprimere l'assenza di sforzo sulla superficie libera. Dividere lo strato in 50 intervalli e calcolare V per $R_a = 2$ con TRID, confrontando con la soluzione esatta $V = \frac{R_a}{1+R_a^2}\left(\frac{R_a}{R} + \frac{R}{R_a}\right)$.

Segnaposto figura

file atteso:
`immagini/fig_3_3_1.png`

Cilindro di raggio r_b che ruota con velocità periferica v_b, circondato da uno strato di fluido con superficie libera di raggio r_a; velocità azimutale v_θ (Fig. 3.3.1 della fonte).

Figura B.6: Fluido attorno a un cilindro rotante.

Impostazione. È un problema ai limiti lineare, con una condizione di Dirichlet sul cilindro e una di Neumann sulla superficie libera: la tecnica del nodo fittizio del Cap. 3, questa volta all'estremo esterno. Prima di calcolare conviene però chiedersi se la condizione $\mathrm{d}V/\mathrm{d}R = 0$ esprima davvero l'assenza di sforzo.

Svolgimento.

1. Con la forma generale (3.6), $A = 1/R$, $B = -1/R^2$, $D = 0$; i nodi sono $R_0 = 1, \dots, R_n = R_a$ e il nodo fittizio è R_{n+1}. La condizione $V'(R_a) = 0$ scritta con la differenza centrata dà $V_{n+1} = V_{n-1}$, che si incorpora con $C_{n1} \leftarrow C_{n1} + C_{n3}$, $C_{n3} \leftarrow 0$.
2. *Lo sforzo in coordinate polari.* In un moto puramente azimutale lo sforzo tangenziale non è $\mu\,\mathrm{d}v_\theta/\mathrm{d}r$, ma
$$\tau_{r\theta} = \mu\, r\,\frac{\mathrm{d}}{\mathrm{d}r}\left(\frac{v_\theta}{r}\right) = \mu\left(\frac{\mathrm{d}v_\theta}{\mathrm{d}r} - \frac{v_\theta}{r}\right):$$
in una rotazione rigida, $v_\theta = \Omega r$, le particelle non si deformano e lo sforzo deve essere nullo, mentre $\mathrm{d}v_\theta/\mathrm{d}r = \Omega \neq 0$. La condizione corretta di superficie libera è quindi $\mathrm{d}(V/R)/\mathrm{d}R = 0$, cioè $V' = V/R$, che nel nodo fittizio diventa $V_{n+1} = V_{n-1} + 2hV_n/R_a$.

```python
if condizione == "dV/dR":                          # V_(n+1) = V_(n-1)
    C[n - 1, 0] += C[n - 1, 2]
else:                                              # V_(n+1) = V_(n-1) + 2h V_n / Ra
    C[n - 1, 0] += C[n - 1, 2]
    C[n - 1, 1] += C[n - 1, 2] * 2 * h / Ra
C[n - 1, 2] = 0.0
```

esercizi_svolti.py

3. *Argomento di equilibrio.* A regime il momento torcente trasmesso attraverso un cilindro di raggio r, $2\pi r^2 \tau_{r\theta}$ per unità di lunghezza, deve essere lo stesso per ogni r (altrimenti il fluido compreso fra due raggi accelererebbe). Sulla superficie libera è nullo, quindi è nullo ovunque: $\tau_{r\theta} = 0$, v_θ / r costante, rotazione rigida.

Risultato

- Con $\mathrm{d}V/\mathrm{d}R = 0$ l'errore percentuale rispetto alla soluzione esatta indicata è al massimo 0,0052%, sulla superficie libera ($V = 0{,}80004$ contro 0,80000): la tecnica del nodo fittizio funziona perfettamente.
- Con questa soluzione, però, lo sforzo sulla superficie libera vale $\tau_{r\theta} = -0{,}402\,\mu v_b / r_b$: non è nullo.
- Con la condizione corretta di sforzo nullo il programma restituisce $V = R$ con uno scarto di 10^{-13}: lo strato ruota rigidamente con il cilindro (Fig. B.7).

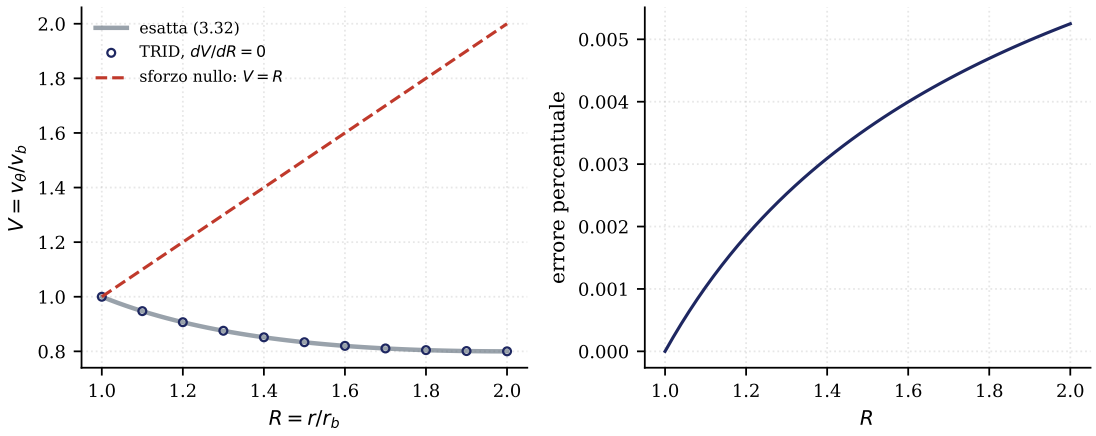


Figura B.7: Esercizio 5: velocità azimutale con le due condizioni sulla superficie libera (a sinistra) ed errore percentuale della soluzione con $\mathrm{d}V/\mathrm{d}R = 0$ (a destra).

Commento. L'esercizio insegna due cose. Sul piano numerico, il nodo fittizio impone qualunque condizione sulla derivata con accuratezza del secondo ordine. Sul piano fisico, e più importante: una condizione al contorno va ricavata dalla fisica, non dalla forma dell'equazione. Con $\mathrm{d}V/\mathrm{d}R = 0$ si conclude che lo strato non ruoterebbe mai come un corpo rigido; con la corretta condizione di sforzo nullo si conclude l'opposto: in assenza di altri momenti, il fluido finisce per ruotare insieme al cilindro. Per la stessa ragione un'atmosfera senza riscaldamento differenziale e senza altre cause motrici ruoterebbe rigidamente con il pianeta: i venti sono prodotti da quelle cause, non dalla viscosità.

B.6 Esercizio 6 – Canale con parete superiore mobile

Testo

Un canale di sezione quadrata $L \times L$ ha la parete superiore mobile con velocità u_0 parallela all'asse. Calcolare la distribuzione di velocità e la portata della corrente stazionaria. Le fessure fra la parete mobile e quelle laterali sono più piccole del passo h: sulla riga superiore la velocità vale u_0 in tutti i nodi tranne gli spigoli, dove è nulla.

Impostazione. Senza gradiente di pressione né gravità la (4.1) diventa l'equazione di Laplace per $U = u/u_0$, con $U = 1$ sulla parete superiore e $U = 0$ sulle altre. Prima di calcolare: esiste un argomento che dà la portata senza risolvere nulla?

Svolgimento.

1. *Simmetria.* Si considerino i quattro problemi in cui si muove, a turno, una sola delle quattro pareti. Per la simmetria del quadrato hanno tutti la stessa portata Q. Sommandoli (l'equazione è lineare) si ottiene il problema con $U = 1$ su tutto il contorno, la cui soluzione è $U \equiv 1$ e la cui portata è 1. Quindi $4Q = 1$: $Q = u_0 L^2/4$, e per lo stesso motivo la velocità al centro è esattamente $u_0/4$.
2. *Soluzione in serie.* Sviluppando $U = 1$ sulla parete superiore in serie di seni, $U = \sum_{n \text{ dispari}} \frac{4}{n\pi} \sin(n\pi z) \frac{\sinh(n\pi y)}{\sinh n\pi}$ e $Q = \sum_{n \text{ dispari}} \frac{8}{(n\pi)^3} \tanh \frac{n\pi}{2}$, che vale proprio 0,25.
3. La soluzione discreta si calcola con un metodo diretto per matrici sparse e la portata con la regola dei trapezi del Cap. 4.

Risultato

h	$Q/(u_0 L^2)$	errore	U al centro
1/20	0,249 375	$-6{,}25 \times 10^{-4}$	0,250 00
1/40	0,249 844	$-1{,}56 \times 10^{-4}$	0,250 00
1/80	0,249 961	$-3{,}91 \times 10^{-5}$	0,250 00
1/160	0,249 990	$-9{,}77 \times 10^{-6}$	0,250 00

L'errore vale esattamente $-h^2/4$, e la velocità al centro è 0,25 su ogni griglia.

Commento. Anche i risultati discreti si spiegano con la simmetria. La formula a cinque punti è invariante per rotazione di 90°, quindi l'argomento del punto 1 vale nodo per nodo: la somma dei quattro problemi discreti ha $U = 1$ in tutti i nodi di contorno tranne i quattro spigoli, che non entrano in nessuna equazione interna; la soluzione interna è $U \equiv 1$ e la velocità al centro è esattamente 1/4. Con la regola dei trapezi la portata della somma è 1 meno il contributo mancante dei quattro spigoli, $4 \cdot (h^2/4) \cdot 1 = h^2$; dividendo per quattro, $Q_h = \frac{1}{4} - \frac{h^2}{4}$, esattamente come nella tabella. Quando un errore numerico ha una forma così semplice, conviene sempre cercarne la ragione: qui è tutto dovuto alla convenzione sugli spigoli.

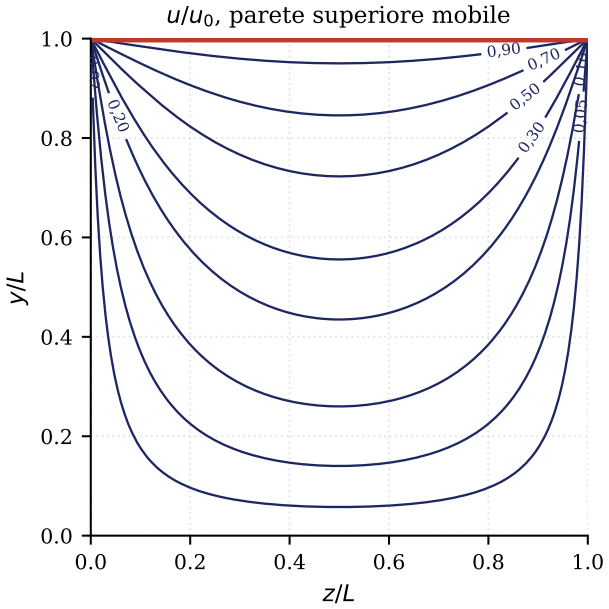


Figura B.8: Esercizio 6: isotache u/u_0 nel canale con la parete superiore mobile (in rosso).

B.7 Esercizio 7 – Lastra mobile di larghezza parziale

Testo

Si sostituisca la parete mobile dell'Esercizio 6 con una lastra di larghezza $L/2$, lasciando libera la parte scoperta della superficie del fluido. Studiare la variazione della portata con la posizione orizzontale della lastra, mantenendo la stessa velocità u_0.

Impostazione. Sulla riga superiore le condizioni sono ora miste: $U = 1$ sotto la lastra, $\partial U/\partial y = 0$ sulla superficie libera (nodo fittizio). Ci si aspetta che la lastra trascini più fluido quando è lontana dalle pareti laterali, che frenano il moto.

Svolgimento.

1. Si indica con s l'ascissa del bordo sinistro della lastra, $0 \le s \le 1/2$; per simmetria basta $s \le 1/4$, ma il calcolo su tutto l'intervallo verifica la simmetria.
2. Si assembla il sistema sparso: le incognite comprendono i nodi della riga superiore che appartengono alla superficie libera, nei quali l'equazione a cinque punti usa come vicino fittizio il nodo sottostante.

Risultato

Con $h = 1/80$: $Q/(u_0 L^2) = 0{,}17373$ con la lastra contro una parete ($s = 0$) e 0,20114 con la lastra centrata ($s = 1/4$), che è il massimo; la curva $Q(s)$ è simmetrica (Fig. B.9). La lastra intera dell'Esercizio 6 dava 0,25.

Commento. Una lastra larga la metà, ben centrata, produce l'80% della portata della lastra intera: la superficie libera non frena il fluido, mentre la parte della lastra intera vicina agli

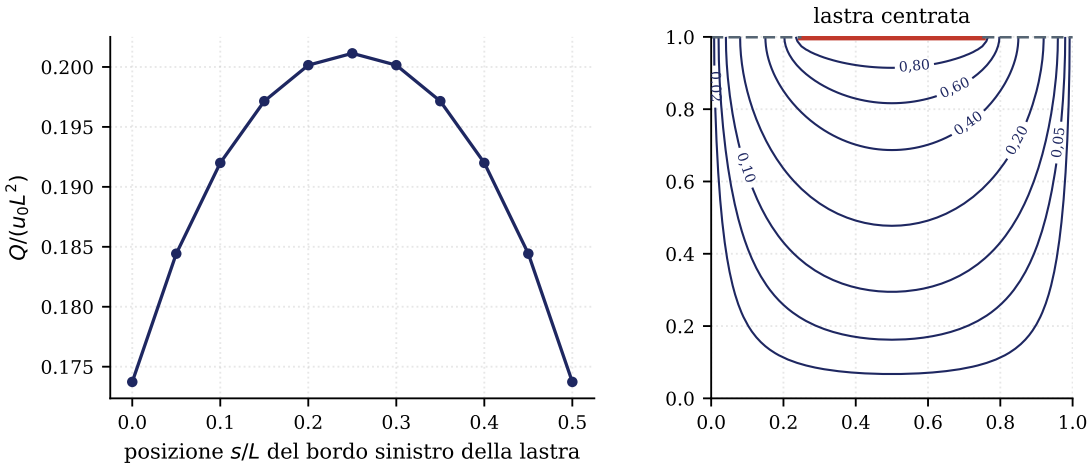


Figura B.9: Esercizio 7: portata in funzione della posizione della lastra (a sinistra) e isotache con la lastra centrata (a destra).

spigoli lavora contro l'attrito delle pareti laterali. Con la lastra contro una parete se ne perde il 14%. È lo stesso effetto frenante delle pareti laterali visto nel canale a pelo libero.

B.8 Esercizio 8 – Lo schema esplicito oltre il limite di stabilità

Testo

Eseguire il Programma 3.4 con un valore di R maggiore di 0,5 e osservare la crescita della soluzione fino a valori irrealistici, a conferma del criterio di stabilità (5.6).

Impostazione. La componente più pericolosa è l'onda più corta rappresentabile. Con 20 intervalli e pareti con valori assegnati, la soluzione si scompone nei modi $\sin(n\pi y)$, $n = 1, \ldots, 19$; il modo $n = 19$ ha $kh = 19\pi/20$ e il fattore di amplificazione (5.5) più grande in modulo, $|1 - 4R\sin^2(19\pi/40)|$. Il dato iniziale discontinuo contiene tutti i modi, quindi anche quello.

Svolgimento. Si ripete il calcolo per $R = 0{,}5$, 0,55 e 0,6 per 200 passi, registrando $\max|u|$.

Risultato

R	50 passi	100 passi	200 passi	$\lvert G\rvert$ del modo $n = 19$
0,50	1,000	1,000	1,000	0,9877
0,55	$2{,}392 \times 10^{1}$	$1{,}055 \times 10^{5}$	$2{,}788 \times 10^{12}$	1,1865
0,60	$5{,}667 \times 10^{4}$	$5{,}652 \times 10^{11}$	$7{,}927 \times 10^{25}$	1,3852

Il rapporto di crescita misurato fra il passo 50 e il passo 100 è 1,183 per passo con $R = 0{,}55$ e 1,380 con $R = 0{,}6$, in accordo con il fattore teorico del modo più corto.

Commento. Con $R = 0{,}55$, appena il 10% oltre il limite, dopo 200 passi (poco più di tre giorni di tempo simulato, con $\tau = 1375\,\mathrm{s}$) la velocità calcolata vale 10^{12} m/s, contro $1\,\mathrm{m\,s^{-1}}$

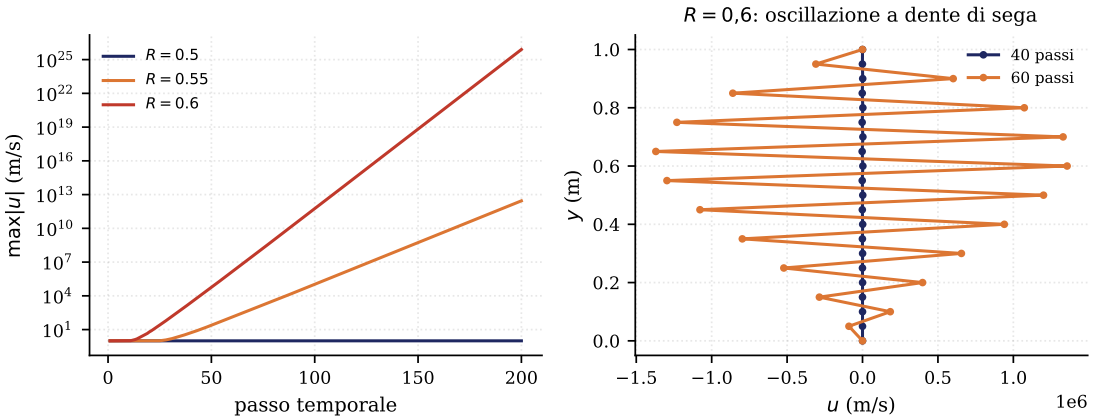


Figura B.10: Esercizio 8: crescita di $\max|u|$ (a sinistra) e oscillazione a dente di sega della soluzione instabile (a destra).

della lastra. L'instabilità si riconosce dalla forma a dente di sega: i valori nei nodi adiacenti si alternano in segno, la firma dell'onda di lunghezza $2h$. Con $R = 0{,}5$ la soluzione resta limitata, anche se il modo più corto si smorza appena ($|G| = 0{,}988$): lo schema è stabile, ma le irregolarità del dato iniziale sopravvivono a lungo.

B.9 Esercizio 9 – La messa in rotazione del fluido attorno a un cilindro

Testo

Trovare la distribuzione di velocità a tempi crescenti nel fluido, inizialmente in quiete, attorno al cilindro con superficie libera dell'Esercizio 5, dopo che al cilindro è stata impressa bruscamente una rotazione con velocità periferica v_b. Per tempi molto grandi la soluzione deve tendere a quella di regime.

Impostazione. L'equazione è la componente azimutale di Navier–Stokes non stazionaria, $\partial V/\partial T = V_{RR} + V_R/R - V/R^2$, con $T = \nu t/r_b^2$: un'equazione parabolica in coordinate cilindriche. Il regime è quello dell'Esercizio 5, e dipende dalla condizione sulla superficie libera: si calcolano entrambi i casi.

Svolgimento.

1. Schema esplicito con differenze centrate: $V_i^{j+1} = V_i^j + \tau\left[\frac{V_{i+1}-2V_i+V_{i-1}}{h^2} + \frac{V_{i+1}-V_{i-1}}{2hR_i} - \frac{V_i}{R_i^2}\right]^j$, con $h = 1/50$ e $\tau = 0{,}25\,h^2$. Il termine $-V/R^2$ è dissipativo e non peggiora la stabilità; quello in V_R/R è piccolo rispetto alla derivata seconda quando $h \ll R$.
2. Condizioni: $V = 1$ sul cilindro per $T > 0$; sulla superficie libera il nodo fittizio dell'Esercizio 5, nella versione $\mathrm{d}V/\mathrm{d}R = 0$ oppure in quella di sforzo nullo.

Risultato

- Con $\mathrm{d}V/\mathrm{d}R = 0$ lo scarto dal regime (Esercizio 5) decade come $e^{-2{,}24\,T}$ e scende sotto l'1% per $T > 2{,}1$.
- Con la condizione di sforzo nullo il fluido tende alla rotazione rigida $V = R$, più

lentamente: lo scarto decade come $e^{-0{,}97\,T}$ e scende sotto l'1% per $T > 5{,}5$.

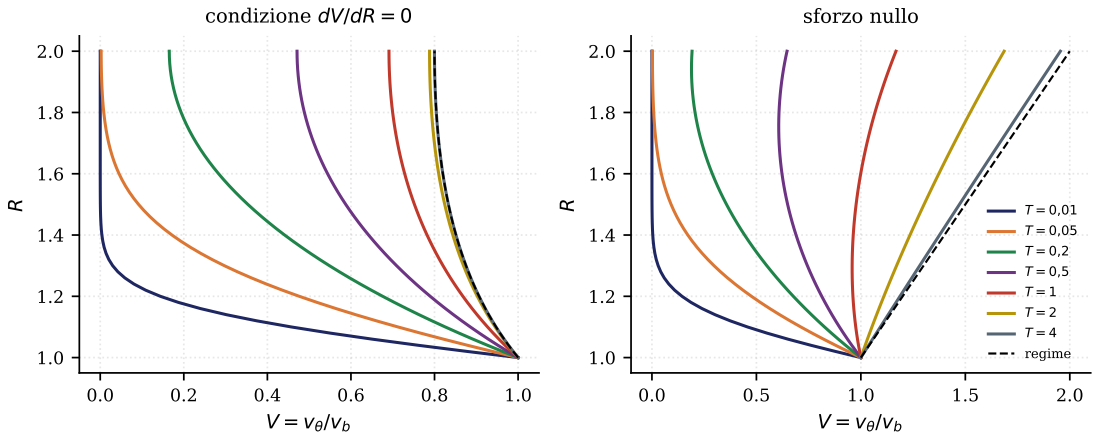


Figura B.11: Esercizio 9: messa in rotazione del fluido per $T = 0{,}01;\ 0{,}05;\ 0{,}2;\ 0{,}5;\ 1;\ 2;\ 4$, con le due condizioni sulla superficie libera.

Commento. Il regime con sforzo nullo si raggiunge più lentamente perché tutto il fluido, fino alla superficie libera, deve essere portato alla velocità Ωr, che cresce verso l'esterno. In tempo dimensionale, per acqua con $r_b = 5\,\text{cm}$, l'unità r_b^2/ν vale 2500 s: per la sola diffusione viscosa servirebbero quasi quattro ore. La messa in rotazione del caffè in una tazza è molto più rapida perché avviene per un meccanismo diverso, la circolazione secondaria prodotta dallo strato limite sul fondo (strato di Ekman), che il modello monodimensionale non contiene.

B.10 Esercizio 10 – La lastra oscillante

Testo

Trovare la distribuzione di velocità nel canale del Programma 3.4 quando la lastra superiore è sostituita da una lastra che oscilla con velocità $u_0 \sin \omega t$, con $u_0 = 1\,\text{m}\,\text{s}^{-1}$ e $\omega = 1 \times 10^{-3}\,\text{s}^{-1}$.

Impostazione. È il secondo problema di Stokes. Una lastra oscillante in un fluido viscoso genera un'onda di velocità che si propaga nel fluido smorzandosi in uno strato di spessore $\delta_s = \sqrt{2\nu/\omega}$. Con i dati, $\delta_s = 4{,}47\,\text{cm}$: la griglia del Programma 3.4, con $h = 5\,\text{cm}$, avrebbe *un solo* nodo nello strato, e il passo temporale di 625 s darebbe appena dieci passi per periodo ($2\pi/\omega = 6283\,\text{s}$). Bisogna raffinare.

Svolgimento.

1. *Soluzione periodica esatta.* Cercando $u = \text{Im}\,[F(y)e^{\text{i}\omega t}]$ si ha $\text{i}\omega F = \nu F''$, quindi $F = A \sinh \kappa y$ con $\kappa = (1+\text{i})/\delta_s$; imponendo $F(d) = u_0$:

$$u = u_0\,\text{Im}\left[e^{\text{i}\omega t}\frac{\sinh \kappa y}{\sinh \kappa d}\right] \;\simeq\; u_0\, e^{-(d-y)/\delta_s} \sin\left(\omega t - \frac{d-y}{\delta_s}\right) \qquad (d \gg \delta_s)\,:$$

un'onda smorzata che penetra nel fluido con un ritardo di fase crescente.

2. Si usa lo schema esplicito con $R = 1/4$ e tre griglie: $h = 5$, 1 e 0,5 cm ($\tau = 625$, 25 e 6,25 s), partendo dalla quiete, e si confronta con la soluzione periodica dopo 10, 20 e 40 periodi.

Risultato

h	passi per periodo	10 periodi	20 periodi	40 periodi
5 cm	10	$2{,}69 \times 10^{-2}$	–	–
1 cm	251	$3{,}84 \times 10^{-3}$	$1{,}83 \times 10^{-3}$	$1{,}01 \times 10^{-3}$
0,5 cm	1005	$3{,}85 \times 10^{-3}$	$1{,}83 \times 10^{-3}$	$5{,}26 \times 10^{-4}$

(errore massimo in $\mathrm{m\,s^{-1}}$ rispetto alla soluzione periodica).

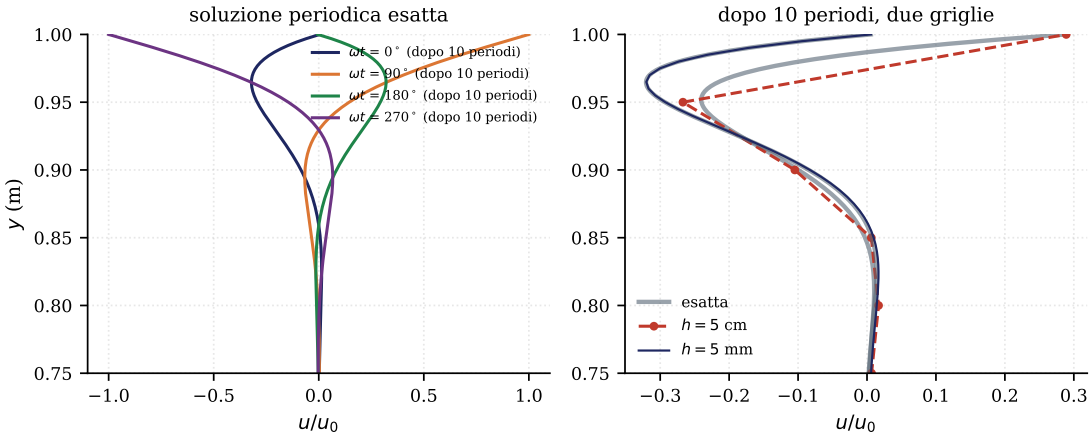


Figura B.12: Esercizio 10: soluzione periodica esatta in quattro fasi (a sinistra) e confronto fra le due griglie dopo dieci periodi (a destra).

Commento. La griglia grossolana sbaglia del 3% dell'ampiezza e non descrive affatto la forma dello strato oscillante. Sulle griglie fini l'errore dopo 10 periodi è lo stesso ($3{,}8 \times 10^{-3}$) e non dipende dalla griglia: non è un errore di discretizzazione, ma il residuo del *transitorio di avviamento*. Partendo dalla quiete, la lastra trasferisce al fluido nel primo semiperiodo una quantità di moto non compensata, che diffonde lentamente verso l'interno del canale (con la scala di tempo d^2/ν, circa 160 periodi) e decade solo gradualmente: dopo 40 periodi lo scarto è sceso a 5×10^{-4}. Prima di confrontare una simulazione con una soluzione periodica bisogna essere sicuri di aver raggiunto il regime.

B.11 Esercizio 11 – Lo schema di DuFort–Frankel

Testo

Usando la differenza centrata anche nel tempo e sostituendo nella (5.4) il valore $u_{i,j}$ del termine centrale con la media $(u_{i,j-1} + u_{i,j+1})/2$ si ottiene lo schema a tre livelli

$$(1 + 2R)\, u_{i,j+1} = 2R\,(u_{i-1,j} + u_{i+1,j}) + (1 - 2R)\, u_{i,j-1}.$$

Dimostrare che è incondizionatamente stabile.

Impostazione. Lo schema è esplicito, perché $u_{i,j+1}$ si ricava senza risolvere sistemi, e tuttavia, come si dimostrerà, stabile per ogni R: sembra il meglio dei due mondi. C'è un prezzo nascosto, che si cerca nella consistenza.

Svolgimento.

1. *Stabilità.* Con $u_{i,j} = G^j e^{\mathrm{i}kih}$ e $\vartheta = kh$: $(1+2R)G^2 - 4R\cos\vartheta\, G - (1-2R) = 0$, da cui

$$G = \frac{2R\cos\vartheta \pm \sqrt{1 - 4R^2\sin^2\vartheta}}{1+2R}.$$

   Se il radicando è non negativo le radici sono reali e $|G| \le (2R|\cos\vartheta| + 1)/(1+2R) \le 1$. Se è negativo le radici sono complesse coniugate e $|G|^2$ è il loro prodotto, $(2R-1)/(2R+1) < 1$. In ogni caso $|G| \le 1$: lo schema è incondizionatamente stabile. Il programma lo verifica numericamente: $\max|G| = 1{,}000000$ (raggiunto per $\vartheta = 0$) su $0{,}01 \le R \le 1000$.

2. *Consistenza.* Sviluppando con Taylor, $u_{i,j+1} + u_{i,j-1} = 2u + \tau^2 u_{tt} + \ldots$; sostituendo, lo schema approssima

$$u_t = \nu u_{yy} - \nu\left(\frac{\tau}{h}\right)^2 u_{tt} + \mathcal{O}\left(\tau^2, h^2\right).$$

   Il termine in $(\tau/h)^2$ tende a zero solo se $\tau/h \to 0$. Se si raffina tenendo fisso $\tau/h = c$, lo schema converge alla soluzione di un'altra equazione, $u_t + \nu c^2 u_{tt} = \nu u_{yy}$, iperbolica (equazione del telegrafo).

3. *Verifica.* Con $u(y,0) = \sin\pi y$ (soluzione esatta $e^{-\pi^2 t}\sin\pi y$, primo passo dalla soluzione esatta) si raffina la griglia in due modi: con $\tau = h^2/4$ e con $\tau = h/10$.

Risultato

h	$\tau = h^2/4$	$\tau = h/10$
$1/10$	$5{,}870 \times 10^{-4}$	$2{,}537 \times 10^{-2}$
$1/20$	$1{,}424 \times 10^{-4}$	$2{,}764 \times 10^{-2}$
$1/40$	$3{,}533 \times 10^{-5}$	$2{,}830 \times 10^{-2}$
$1/80$	$8{,}815 \times 10^{-6}$	$2{,}850 \times 10^{-2}$
$1/160$	$2{,}203 \times 10^{-6}$	$2{,}858 \times 10^{-2}$

(errore massimo a $t = 0{,}2$). Con $\tau = h/10$ il modo $\sin\pi y$ dell'equazione modificata decade come $e^{-11{,}10\,t}$ invece di $e^{-9{,}87\,t}$, e l'errore previsto per $h \to 0$ è $2{,}86 \times 10^{-2}$: è proprio il valore a cui tende la colonna di destra.

Commento. Il teorema di equivalenza di Lax afferma che, per un problema lineare ben posto, uno schema converge se e solo se è stabile e consistente [21]. DuFort–Frankel è stabile per ogni R, ma consistente solo se $\tau/h \to 0$: con passi temporali "grandi" produce una soluzione stabile, liscia e sbagliata. È un'insidia tanto più pericolosa quanto più il risultato sembra plausibile.

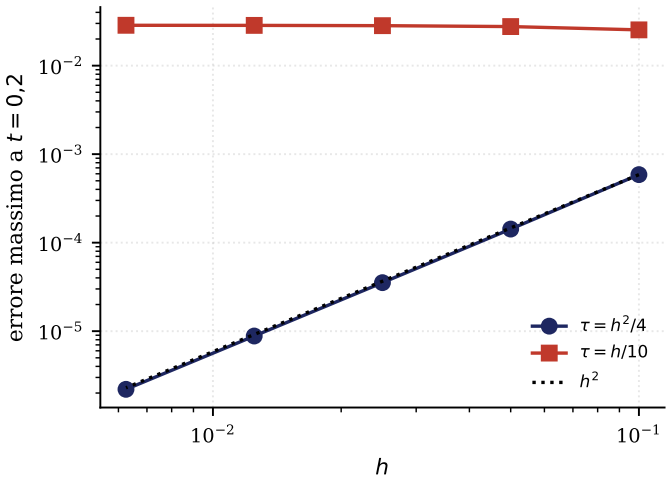


Figura B.13: Esercizio 11: errore dello schema di DuFort–Frankel con due leggi di raffinamento.

B.12 Esercizio 12 – L'avviamento della corrente in un tubo

Testo

Per la corrente non stazionaria in un tubo indefinito di sezione circolare l'equazione del moto è $\partial u/\partial t = -\rho^{-1}\partial p/\partial x + \nu(\partial^2 u/\partial r^2 + r^{-1}\partial u/\partial r)$. Ricavare uno schema implicito analogo alla (6.2), studiarne la stabilità e usarlo per trovare la corrente prodotta dall'applicazione brusca di un gradiente di pressione costante in un tubo di raggio A, confrontando con la soluzione analitica in serie di Fourier–Bessel.

Impostazione. Con $R = r/A$, $T = \nu t/A^2$, $U = u/[-(A^2/4\mu)\,\mathrm{d}p/\mathrm{d}x]$ il regime è $U = 1 - R^2$ e la differenza $W = (1 - R^2) - U$ soddisfa $W_T = W_{RR} + W_R/R$, con $W(1, T) = 0$, W regolare sull'asse e $W(R, 0) = 1 - R^2$: lo stesso schema del Programma 3.5, in coordinate cilindriche. Il tubo ha la parete tutto intorno: ci si aspetta un transitorio più rapido di quello del canale piano.

Svolgimento.

1. Nei nodi interni $R_i = ih$, con le differenze centrate:
$$-r\left(1 - \frac{1}{2i}\right) W_{i-1}^n + (1 + 2r)\, W_i^n - r\left(1 + \frac{1}{2i}\right) W_{i+1}^n = W_i^{n-1}, \qquad r = \frac{\tau}{h^2}.$$
2. *Asse.* Per $R \to 0$, $W_R/R \to W_{RR}$ (regola di de l'Hôpital, con $W_R(0) = 0$), quindi l'equazione diventa $W_T = 2W_{RR}$; con il nodo fittizio simmetrico $W_{-1} = W_1$: $(1 + 4r)W_0^n - 4rW_1^n = W_0^{n-1}$.
3. *Stabilità.* I coefficienti variano con i e l'analisi di von Neumann non si applica direttamente; si usa il principio del massimo. La matrice ha diagonale positiva $1 + 2r$ (o $1 + 4r$) e fuori diagonale elementi non positivi la cui somma in modulo vale $2r$ (o $4r$). Nel nodo i dove $|W^n|$ è massimo, M_n: $(1 + 2r)M_n \le 2rM_n + |W_i^{n-1}|$, cioè $M_n \le M_{n-1}$. La soluzione non può crescere, qualunque sia τ: lo schema è incondizionatamente stabile.
4. *Soluzione esatta.* Separando le variabili, $W = \sum_n A_n J_0(\lambda_n R) e^{-\lambda_n^2 T}$, con λ_n zeri di J_0 (2,405; 5,520; …) e, per l'ortogonalità delle funzioni di Bessel, $A_n = 8/[\lambda_n^3 J_1(\lambda_n)]$ [5].

Risultato

Con 40 intervalli: velocità sull'asse a $T = 1$ pari a 0,99598 con $\tau = 0{,}01$ e 0,99647 con $\tau = 0{,}002$, contro il valore esatto 0,99659; l'errore massimo scende da $6{,}0 \times 10^{-4}$ a $1{,}2 \times 10^{-4}$, proporzionalmente a τ (primo ordine). L'asse raggiunge il 90% della velocità di regime per $T = 0{,}417$ e il 99% per $T = 0{,}814$; nel canale piano servono $T = 0{,}947$ e 1,881.

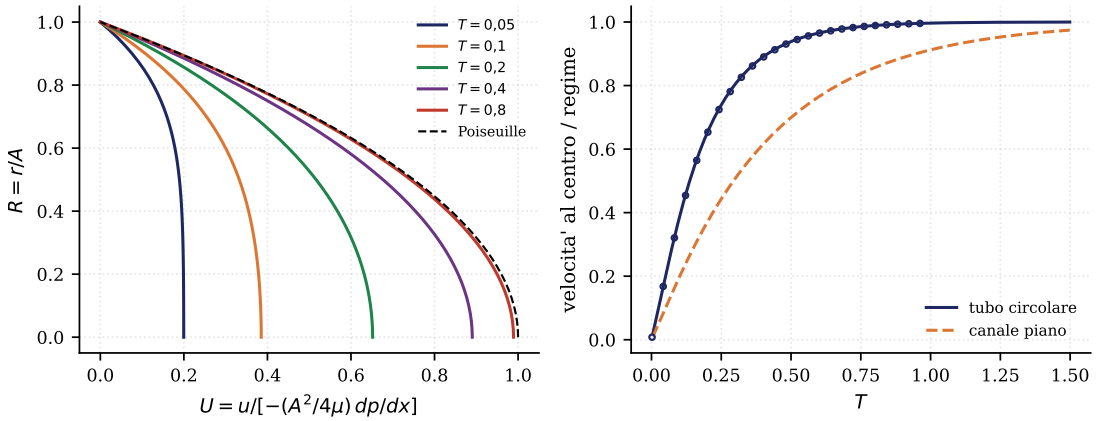


Figura B.14: Esercizio 12: profili di velocità nel tubo durante l'avviamento (a sinistra) e velocità sull'asse confrontata con quella del canale piano (a destra; i cerchi sono la soluzione numerica).

Commento. Il transitorio del tubo è governato dal primo autovalore, $\lambda_1^2 = 5{,}783$, più del doppio di quello del canale, $(\pi/2)^2 = 2{,}467$: con la stessa distanza fra asse e parete, il tubo ha "più parete" per unità di volume e la vorticità vi diffonde da tutte le direzioni. Per acqua in un tubo di raggio 1 cm, $A^2/\nu = 100$ s e la corrente si stabilisce al 99% in circa 80 secondi.

B.13 Esercizio 13 – Cavità con superficie libera e fondo mobile

Testo

Risolvere il flusso di Stokes nella cavità quadrata della Fig. 7.1 con la superficie superiore CD libera e il fondo AB in moto con velocità unitaria verso x positivo.

Impostazione. Cambiano le condizioni di parete sulla vorticità. Sul fondo mobile vale una formula analoga alla (7.8); sulla superficie libera piana $\psi = 0$ (impermeabilità) e lo sforzo tangenziale $\mu\,\partial u/\partial y$ è nullo; poiché anche $\partial v/\partial x = 0$ lungo la superficie, la vorticità $\zeta = \partial v/\partial x - \partial u/\partial y$ è nulla: la superficie libera impone direttamente $\zeta = 0$.

Svolgimento.

1. *Fondo mobile.* Ripetendo la derivazione della (7.8) con gli sviluppi verso l'alto, $\psi_{i,1+n} = \psi + nh\psi_y + \tfrac{1}{2}n^2h^2\psi_{yy}$, cambia solo il segno del termine di velocità:

$$\zeta_{i,1} = \frac{1}{h^2}\left(-\psi_{i-1,2} + \frac{8}{3}\psi_{i,2} - \psi_{i+1,2} - \frac{2}{3}\psi_{i,3}\right) + \frac{2}{3h}\left(\frac{\partial\psi}{\partial y}\right)_{i,1}, \qquad \left(\frac{\partial\psi}{\partial y}\right)_{i,1} = 1.$$

Il segno è quello atteso: sopra il fondo che si muove verso destra $\partial u/\partial y < 0$, quindi $\zeta > 0$.

2. *Superficie libera.* $\zeta = 0$ e $\psi = 0$ sulla riga superiore.
3. Il sistema lineare accoppiato si risolve con il metodo diretto del Programma 3.6b, su griglie 11×11, 41×41 e 161×161.
4. *Un'osservazione di simmetria.* Prolungando la soluzione oltre la superficie libera con $\psi(x, 2-y) = -\psi(x,y)$, le condizioni $\psi = 0$ e $\psi_{yy} = 0$ sulla superficie sono rispettate automaticamente: la cavità con superficie libera è la metà inferiore di una cavità 1×2 con il fondo e il coperchio in moto *nello stesso verso*.

Risultato

- $\psi_{\max} = +0{,}0955$ sulla griglia 11×11, $+0{,}1008$ sulla 41×41, $+0{,}1010$ sulla 161×161, con il centro del vortice sulla mezzeria a $y = 0{,}237$.
- Sulla superficie libera, al centro, la velocità vale $-0{,}080$: il fluido in superficie si muove in verso opposto al fondo.
- Per confronto, la cavità con il coperchio mobile della stessa griglia ha $\psi_{\min} = -0{,}1001$, con il centro a 0,235 dalla parete mobile.

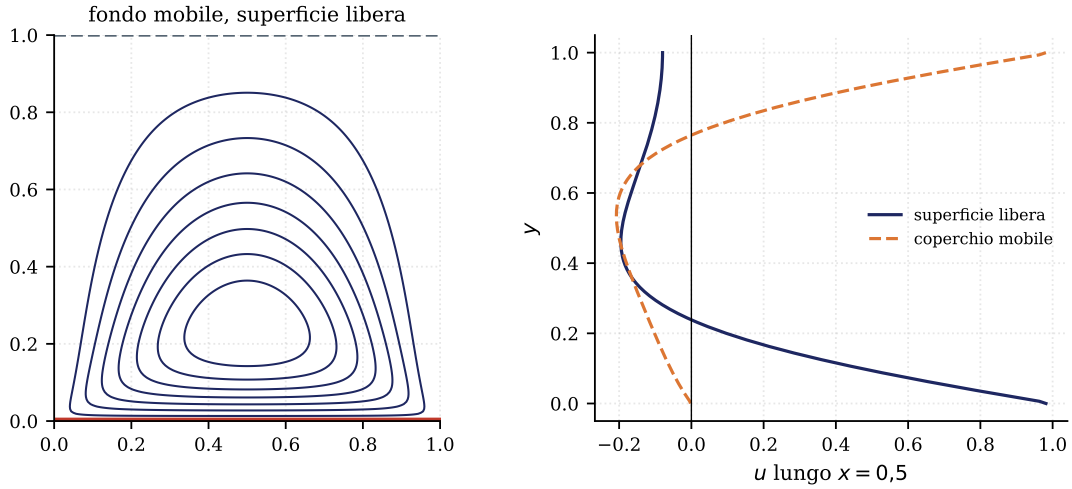


Figura B.15: Esercizio 13: linee di corrente nella cavità con fondo mobile (in rosso) e superficie libera (tratteggiata), e profilo di velocità lungo la mezzeria confrontato con quello della cavità con coperchio mobile.

Commento. Il vortice resta vicino alla parete che lo alimenta, a una distanza praticamente uguale nei due casi (0,237 e 0,235): il tipo di condizione sul lato opposto (aderenza o sforzo nullo) influenza poco la regione vicina alla parete mobile, e cambia invece la corrente di ritorno, che sulla superficie libera può scorrere senza attrito. È una situazione analoga a quella di un fiume o di un canale mossi dal vento in superficie, rovesciata.

B.14 Esercizio 14 – La curva di stabilità neutra di Bénard

Testo

Calcolare la curva di stabilità neutra della convezione di Bénard fra pareti rigide con la formulazione del Cap. 8, per $0{,}5 \le k \le 10$ a passi di 0,1; per ogni k si sceglie il minimo autovalore. (a) Tracciare la curva; (b) trovare il numero di Rayleigh critico; (c) tracciare le autofunzioni corrispondenti; (d) confrontare con la formula approssimata

$$Ra = \frac{(\pi^2 + k^2)^3}{\left[1 - \dfrac{\pi^2}{(\pi^2 + k^2)^2 \kappa}\right] k^2}, \qquad \kappa = \frac{1}{8k}\tanh\frac{k}{2} + \frac{1}{16}\operatorname{sech}^2\frac{k}{2}.$$

Impostazione. Dal Cap. 8 si sa che con le differenze finite servono circa 150 nodi per un risultato accurato vicino al minimo; la curva intera si può tracciare con meno nodi e poi raffinare il minimo. La formula approssimata deriva da un metodo variazionale con una funzione di prova semplice e, come tutte le stime variazionali, dovrebbe sovrastimare l'autovalore.

Svolgimento.

1. Curva con $n = 40$ nodi; minimo raffinato con $n = 150$ con la ricerca della sezione aurea in k.
2. Per avere una curva di riferimento "esatta" si risolve lo stesso problema con la collocazione di Chebyshev del Cap. 9, sulla variabile $z = (1 + y)/2$ (quindi $\mathrm{d}/\mathrm{d}z = 2\,\mathrm{d}/\mathrm{d}y$), imponendo $\theta' = w' = Dw' = 0$ alle pareti con la sostituzione di sei righe.
3. Le autofunzioni sono le componenti θ' e w' dell'autovettore del valore critico, normalizzate al loro massimo.

Risultato

- Differenze finite: con $n = 40$ il minimo della curva è a $k = 3{,}1$, $Ra = 1721{,}6$; con $n = 150$, $k_c = 3{,}1163$, $Ra_c = 1708{,}74$.
- Chebyshev a $k = 3{,}1163$: $Ra = 1707{,}7566$ con $N = 10$, $1707{,}7618$ con $N = 16$, e lo stesso valore per $N = 20, 30, 40$. Il punto critico è

$$Ra_c = 1707{,}762, \qquad k_c = 3{,}1163.$$

- La formula approssimata ha il minimo 1715,1 per $k = 3{,}114$ e si discosta dalla curva esatta al massimo dello 0,52% per $1 < k < 8$, sempre per eccesso.

Commento. Il confronto fra i due metodi è eloquente: la collocazione di Chebyshev con 17 nodi dà sette cifre esatte, le differenze finite con 150 nodi ne danno tre. Le autofunzioni mostrano la struttura delle celle: la velocità verticale è massima a metà altezza e si annulla con tangente orizzontale alle pareti rigide; la perturbazione di temperatura ha forma simile ed è positiva dove il fluido sale. La formula approssimata, costruita a mano con una sola funzione di prova, sbaglia di mezzo punto percentuale: un ottimo esempio di quanto valga un buon metodo variazionale.

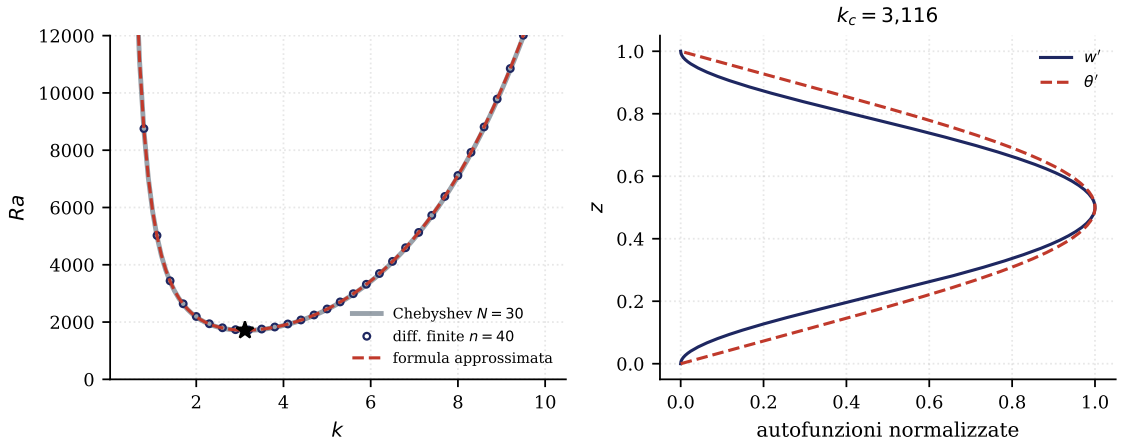


Figura B.16: Esercizio 14: curva di stabilità neutra di Bénard fra pareti rigide (a sinistra; la stella è il punto critico) e autofunzioni critiche (a destra).

B.15 Esercizio 15 – La stabilità della corrente di Couette–Poiseuille

Testo

Modificare il Programma 3.8 per ottenere la curva di stabilità neutra della corrente di Couette–Poiseuille, prodotta da un gradiente di pressione in un canale la cui parete superiore si muove con velocità costante. Il profilo, riferito alla velocità della parete, è

$$U(y) = \tfrac{1}{2}(1+y) + \tfrac{B}{2}(1-y^2), \qquad -1 \le y \le 1.$$

Ottenere le curve neutre per $B = 1, 3, 7, 25, 100$. Per B grande la curva tende a quella della corrente di Poiseuille? Tracciare il Reynolds critico in funzione di B e dire se il moto della parete rende la corrente più o meno stabile. Per $B = 0$ esistono modi instabili?

Impostazione. Nell'equazione di Orr–Sommerfeld cambiano soltanto U e $U'' = -B$: nella matrice $\mathbf{A}$ della (9.15) il termine $-2\mathrm{i}\mathbf{I}$ diventa $-\mathrm{i}B\,\mathbf{I}$, e il profilo non è più simmetrico. Una questione di scala: con la velocità della parete come riferimento, al crescere di B la parte di Poiseuille cresce senza limite, e il Reynolds critico basato sulla parete tende a zero. Per confrontare i casi conviene usare la velocità massima della parte di Poiseuille, $U_P = B/2$, e il Reynolds $Re_P = Re\,B/2$: il profilo diventa $\tilde{U} = \frac{r}{2}(1+y) + (1-y^2)$, con $r = 2/B$ rapporto fra la velocità della parete e U_P. Per $B \to \infty$ ($r \to 0$) si ritrova la corrente di Poiseuille.

Svolgimento.

1. Per ogni α si cerca su una griglia di Re_P il primo valore instabile e si raffina con il metodo di Brent (ramo inferiore della curva neutra); poi si minimizza $Re_P(\alpha)$.
2. Per $B = 0$ (Couette pura, $U = \frac{1}{2}(1+y)$) si cerca il massimo di λ_R su una griglia di Re e α.
3. Per $B = 3$ e $B = 1$ si estende la ricerca fino a $Re_P = 10^6$, con più punti di collocazione ad alto Reynolds.

Risultato

B	$r = 2/B$	$Re_{P,c}$	α_c	Re_c (parete)
∞	0	5772	1,021	–
100	0,020	6025	1,008	120,5
25	0,080	10 389	0,859	831,1
7	0,286	16 480	0,580	4708,6
3	0,667	nessun modo instabile fino a $Re_P = 10^6$		
1	2,000	$(\lambda_R)_{\max} = -0{,}0032$ fino a $Re_P = 10^6$		

Per $B = 0$ il massimo di λ_R su $10^3 \le Re \le 10^5$ e $0{,}3 \le \alpha \le 1{,}5$ vale $-0{,}013$: nessun modo instabile.

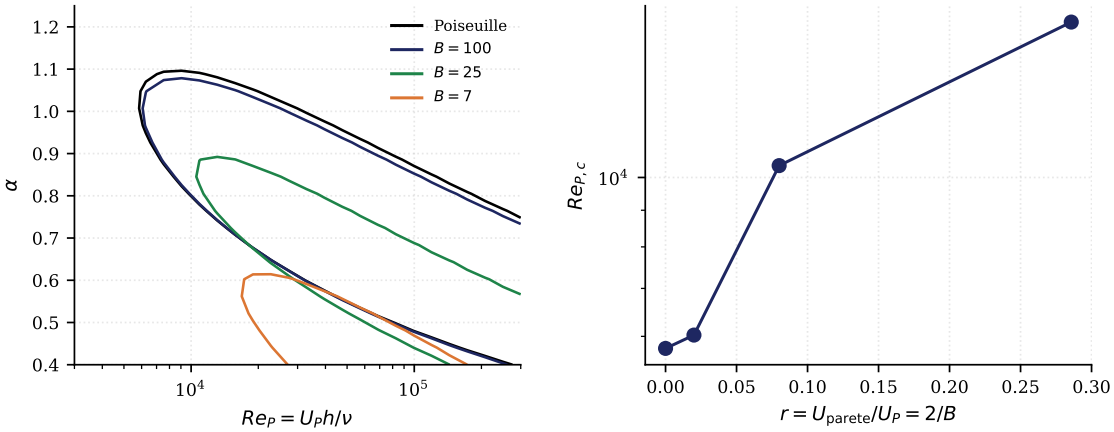


Figura B.17: Esercizio 15: curve neutre della corrente di Couette–Poiseuille nel piano (Re_P, α) (a sinistra) e Reynolds critico in funzione del rapporto r fra la velocità della parete e quella di Poiseuille (a destra).

Commento. Per $B = 100$ la curva neutra è quasi sovrapposta a quella di Poiseuille ($Re_{P,c}$ maggiore del 4%): la risposta alla prima domanda è sì, purché si usi la scala di velocità appropriata. Il moto della parete è fortemente stabilizzante: basta una velocità della parete pari al 29% di quella di Poiseuille ($B = 7$) per quasi triplicare il Reynolds critico e restringere la regione instabile a onde più lunghe; per $r \simeq 0{,}7$ ($B = 3$) l'instabilità non compare più nel campo esplorato. La corrente di Couette pura, infine, è linearmente stabile per ogni numero di Reynolds (è un risultato classico della teoria della stabilità), eppure negli esperimenti e nelle simulazioni dirette diventa turbolenta già per numeri di Reynolds di qualche centinaio: un altro esempio di transizione che la teoria lineare non può prevedere.

B.16 Esercizio 16 – Convezione esplicita e diffusione implicita

Testo

Risolvere l'equazione (10.7) trattando in modo esplicito i termini convettivi e in modo implicito quelli diffusivi, con differenze centrate del secondo ordine in x e trasformata di Fourier nella direzione periodica y. Si può usare TRID per i sistemi tridiagonali che risultano dalle differenze finite in x.

Impostazione. Il vincolo diffusivo dello schema esplicito, che con la griglia del Programma 3.10 vale $1{,}6 \times 10^{-4}$ per $Pr = 0{,}71$ e $5{,}6 \times 10^{-3}$ per $Pr = 25$, scompare; resta quello dovuto alla convezione esplicita. Con la convezione centrata ed esplicita, senza diffusione, lo schema di Eulero sarebbe instabile per ogni passo; la diffusione implicita smorza le onde e si può stimare, con l'analisi di von Neumann per le onde lunghe, un limite del tipo $\Delta t \lesssim 2a/|\mathbf{V}|^2_{\max}$, con $a = 1/(PrRe)$ e $|\mathbf{V}|^2_{\max} = 2$.

Svolgimento.

1. Nello spazio di Fourier in y lo schema è, per ogni numero d'onda k,

$$\left[\frac{1}{\Delta t} - a\left(\delta_{xx} - k^2\right)\right]\hat{T}^{n+1} = \frac{\hat{T}^n}{\Delta t} + \widehat{N}(T^n), \qquad N = -u\,T_x - v\,T_y,$$

   dove δ_{xx} è la differenza centrata seconda in x e $\widehat{N}$ è la trasformata dei termini convettivi, calcolati nello spazio fisico. Per ogni k si risolve un sistema tridiagonale complesso in x; le condizioni $T = 0$ e $T = 1$ alle pareti, trasformate, valgono 0 per tutti i modi tranne il modo $k = 0$ sulla parete calda.

2. I sistemi dei diversi modi hanno la stessa struttura e differiscono solo per la diagonale: l'algoritmo di Thomas si esegue su tutti contemporaneamente, operando su matrici invece che su vettori.

```
bb = 1 + 2 * r + a * dt * k**2              # diagonale (dipende dal modo)
d = rhs[1:-1].copy()
d[-1] += r * Th_bordo
cp[0] = -r / bb
dp[0] = d[0] / bb
for i in range(1, n_int):
    den = bb + r * cp[i - 1]
    cp[i] = -r / den
    dp[i] = (d[i] + r * dp[i - 1]) / den
```

esercizi_svolti.py

Risultato

Pr	Δt	passi	t finale	scarto dallo stazionario
0,71	0,001	153	0,15	$4{,}6 \times 10^{-6}$
0,71	0,05	7	0,35	$2{,}7 \times 10^{-6}$
0,71	2,0	3	6,00	$1{,}7 \times 10^{-6}$
25	0,01	1135	11,35	$1{,}6 \times 10^{-4}$
25	0,1	116	11,60	$1{,}6 \times 10^{-4}$
25	0,3	118	35,40	$2{,}7 \times 10^{-5}$
25	1,0		instabile dopo 58 passi	

La stima $\Delta t < 2a/|\mathbf{V}|^2_{\max}$ vale 1,4 per $Pr = 0{,}71$ e 0,04 per $Pr = 25$.

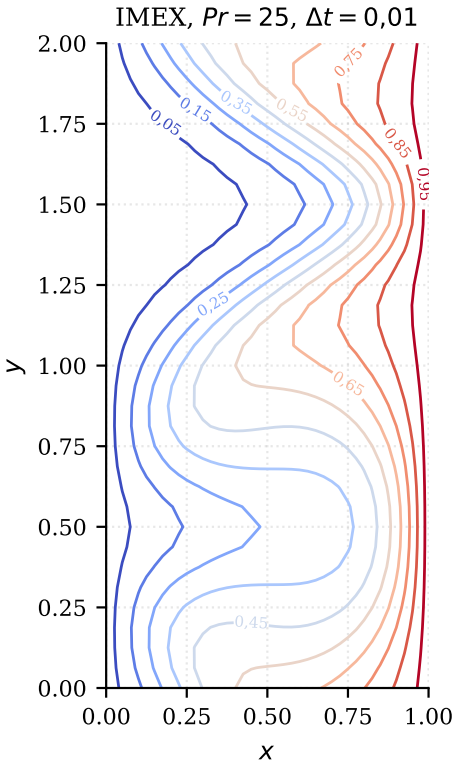


Figura B.18: Esercizio 16: isoterme stazionarie per $Pr = 25$ calcolate con lo schema misto.

Commento. Il guadagno è enorme. Per $Pr = 0{,}71$ lo schema esplicito del Programma 3.10 impiegava 1476 passi con $\Delta t = 10^{-4}$; lo schema misto arriva allo stato stazionario in 7 passi con $\Delta t = 0{,}05$, e resta stabile perfino con $\Delta t = 2$. Per $Pr = 25$ si passa da 11 316 passi a 116. La stima di stabilità per le onde lunghe si rivela prudente: per $Pr = 25$ lo schema è stabile ancora con $\Delta t = 0{,}3$ (anche se la convergenza allo stazionario diventa oscillante e più lenta) e diventa instabile fra 0,3 e 1. La struttura del metodo, implicito nella direzione non periodica e diagonale nei numeri d'onda della direzione periodica, è quella dei codici di simulazione diretta della turbolenza nei canali.

B.17 Esercizio 17 – Progetto: condotti di varie sezioni

Testo

In un tubo lungo è stata applicata da molto tempo una differenza di pressione costante. Calcolare la distribuzione di velocità e la portata di un fluido incomprimibile nelle sezioni della Fig. B.19:

1. un tubo circolare di raggio R, calcolando solo il primo quadrante (con derivate normali nulle sui due lati rettilinei) e trattando il contorno curvo con la tecnica dei bracci disuguali; confrontare con la soluzione di Poiseuille $Q = -\pi R^4/(8\mu)\,\mathrm{d}p/\mathrm{d}x$;
2. un tubo triangolare le cui due pareti inclinate formano angoli di 45° con la terza;
3. un tubo rettangolare $3a \times 2a$ che contiene un tubo interno quadrato di lato a.

Segnaposto figura

file atteso: `immagini/fig_3_4_4.png`

(a) Tubo circolare di raggio R con il primo quadrante tratteggiato e condizioni $\partial u/\partial z = 0$, $\partial u/\partial y = 0$ sui lati; (b) tubo triangolare con base L e pareti inclinate a 45°; (c) tubo rettangolare $3a \times 2a$ con tubo interno quadrato di lato a, distante $a/2$ dalla parete destra e $a/4$ dal fondo (Fig. 3.4.4 della fonte).

Figura B.19: Sezioni di tubi diverse.

Impostazione. In tutti i casi si risolve $\nabla^2 U = -1$ con $U = u/[-(\mathrm{d}p/\mathrm{d}x)\ell^2/\mu]$, dove ℓ è una lunghezza di riferimento (R per il tubo circolare, metà della base per il triangolo, a per il rettangolo), con $U = 0$ sulle pareti. Per confrontare sezioni diverse si usa il prodotto $f \cdot Re = 2D_h^2 A/Q$ (in grandezze adimensionali), con $D_h = 4A/P$ diametro idraulico, A area e P perimetro bagnato: vale 64 per il cerchio e 56,9 per il quadrato.

Svolgimento.

1. *Tubo circolare.* Griglia quadrata sul quarto di cerchio. Nei nodi accanto alla circonferenza, dove il braccio verso il contorno è una frazione $a < 1$ del passo, la derivata seconda si scrive con la formula a bracci disuguali del modulo precedente (di Shortley–Weller):

$$\frac{\partial^2 U}{\partial x^2} \simeq \frac{2}{h^2}\left[\frac{U_E}{a_E(a_E + a_W)} + \frac{U_W}{a_W(a_E + a_W)} - \frac{U_P}{a_E a_W}\right],$$

   con $U = 0$ nel punto di contorno. Sugli assi la simmetria si impone con nodi fittizi speculari. La portata si calcola integrando in coordinate polari il campo interpolato.

2. *Triangolo.* Con la base di lunghezza 2 sul lato superiore e il vertice in basso, le pareti inclinate $y = |x - 1|$ passano per i nodi di una griglia quadrata: la formula a cinque punti ordinaria basta, e la regola dei trapezi dà la portata.

3. *Rettangolo con tubo interno.* Dominio a connessione multipla: i nodi del tubo interno e del suo contorno sono esclusi ($U = 0$). Con $h = a/8$ o più piccolo tutti i lati passano per i nodi.
4. In tutti i casi si usano più griglie e l'estrapolazione di Richardson.

Risultato

- *Tubo circolare*: $U(0) = 0{,}250000$ (esatto) su tutte le griglie; $Q = 0{,}391282$, $0{,}392363$, $0{,}392618$, $0{,}392678$ con $n = 10$, 20, 40, 80 intervalli sul raggio, contro il valore esatto $\pi/8 = 0{,}392699$ (errore del secondo ordine, 2×10^{-5} con $n = 80$).
- *Triangolo*: $Q = 0{,}025959$; $0{,}026057$; $0{,}026081$; $0{,}026088$ con $n = 20$, 40, 80, 160; per estrapolazione $Q = 0{,}026090$; $U_{\max} = 0{,}05905$. Con $A = 1$, $P = 2 + 2\sqrt{2}$, $D_h = 0{,}8284$: $f \cdot Re = 52{,}61$.
- *Rettangolo con tubo interno*: $Q = 0{,}39476$ contro $1{,}17456$ del rettangolo senza tubo interno: il tubo interno, che occupa il 16,7% della sezione, riduce la portata del 66%.

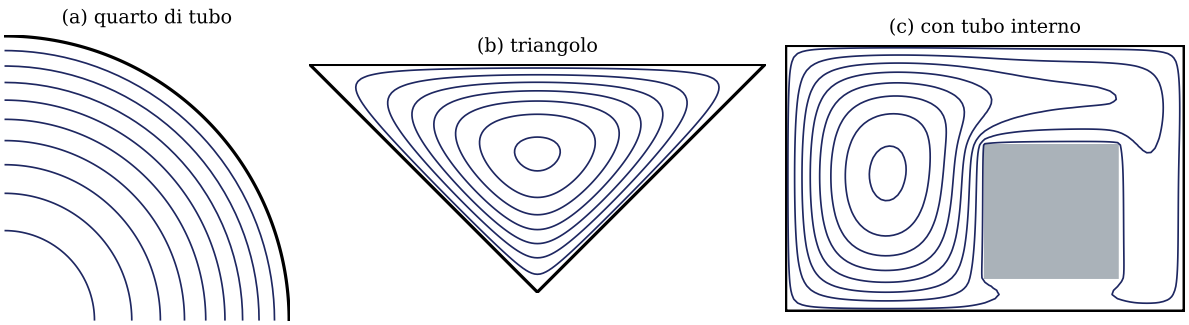


Figura B.20: Esercizio 17: isotache nel quarto di tubo circolare, nel tubo triangolare e nel tubo rettangolare con tubo interno.

Commento. Nel tubo circolare i valori nodali sono *esatti* su qualunque griglia: la soluzione $U = (1 - r^2)/4$ è un polinomio di secondo grado, e sia la formula a cinque punti sia quella a bracci disuguali derivano esattamente i polinomi di secondo grado. L'unico errore è quello della quadratura vicino al contorno curvo. La sezione triangolare ha $f \cdot Re = 52{,}6$, meno del cerchio (64) e del quadrato (56,9): negli angoli acuti il fluido è quasi fermo, e una parte del perimetro "lavora" poco. Il tubo interno, infine, mostra quanto conti la posizione degli ostacoli: occupa la regione dove la velocità sarebbe stata più alta e aggiunge perimetro bagnato, e la portata scende a un terzo. Gli stessi calcoli, con l'analogia di Prandtl richiamata nel Cap. 4, danno la rigidezza torsionale di travi prismatiche con queste sezioni.

B.18 Esercizio 18 – Progetto: la corrente pulsante in un tubo

Testo

Trovare la corrente in un tubo di raggio A prodotta da un gradiente di pressione alternato $\partial p/\partial x = -K\cos\omega t$. Con $T = \omega t$, $R = r/A$, $U = u/(KA^2/4\mu)$ l'equazione diventa

$$\Omega\frac{\partial U}{\partial T} = 4\cos T + \frac{\partial^2 U}{\partial R^2} + \frac{1}{R}\frac{\partial U}{\partial R}, \qquad \Omega = \frac{\omega\rho A^2}{\mu}.$$

Con lo schema dell'Esercizio 12 trovare $U(R,T)$ per $\Omega = 0{,}1$, 7 e 35 e confrontare con la soluzione analitica.

Impostazione. Il parametro Ω confronta il periodo dell'oscillazione con il tempo di diffusione viscosa sul raggio, A^2/ν; la sua radice quadrata, $\alpha_W = A\sqrt{\omega/\nu}$, è il *numero di Womersley*. Per $\Omega \ll 1$ la viscosità ha tutto il tempo di adattare il profilo: in ogni istante si avrà il profilo di Poiseuille corrispondente al gradiente istantaneo. Per $\Omega \gg 1$ domina l'inerzia: il nucleo della corrente oscillerà come un pistone, in ritardo di fase sul gradiente, e la viscosità agirà solo in uno strato sottile vicino alla parete. È il problema del flusso sanguigno pulsante nelle arterie, studiato da Womersley nel 1955 [16].

Svolgimento.

1. *Soluzione periodica esatta.* Con $U = \mathrm{Re}[F(R)e^{\mathrm{i}T}]$: $\mathrm{i}\Omega F = 4 + F'' + F'/R$. Una soluzione particolare è $F = 4/(\mathrm{i}\Omega)$; l'omogenea, regolare sull'asse, è $J_0(\varkappa R)$ con $\varkappa^2 = -\mathrm{i}\Omega$, cioè $\varkappa = \mathrm{i}^{3/2}\sqrt{\Omega}$. Imponendo $F(1) = 0$:
$$U(R,T) = \mathrm{Re}\left\{\frac{4}{\mathrm{i}\Omega}\left[1 - \frac{J_0(\mathrm{i}^{3/2}\sqrt{\Omega}\,R)}{J_0(\mathrm{i}^{3/2}\sqrt{\Omega})}\right]e^{\mathrm{i}T}\right\}.$$
2. *Schema numerico.* Crank–Nicolson in coordinate cilindriche, con il trattamento dell'asse dell'Esercizio 12 e il termine forzante valutato a metà passo; 60 intervalli sul raggio, 400 passi per periodo. Si parte dalla quiete e si integra per 12 periodi ($\Omega = 0{,}1$ e 7) o 25 periodi ($\Omega = 35$), finché il transitorio non si è estinto.
3. Dalla soluzione esatta si calcolano l'ampiezza della portata, riferita al valore quasi stazionario, e il suo ritardo di fase rispetto al gradiente di pressione.

Risultato

Ω	α_W	errore numerico	portata / quasi stazionaria	ritardo (gradi)
0,1	0,32	$3{,}1\times10^{-5}$	1,00	1
7	2,65	$6{,}9\times10^{-5}$	0,64	48
35	5,92	$2{,}7\times10^{-5}$	0,18	75

Commento. Per $\Omega = 0{,}1$ i profili sono parabole che crescono e decrescono in fase con il gradiente: la corrente è quasi stazionaria. Per $\Omega = 35$ il nucleo si muove quasi in blocco, con un ritardo che tende a 90° (l'accelerazione, non la velocità, è in fase con la forza) e un'ampiezza

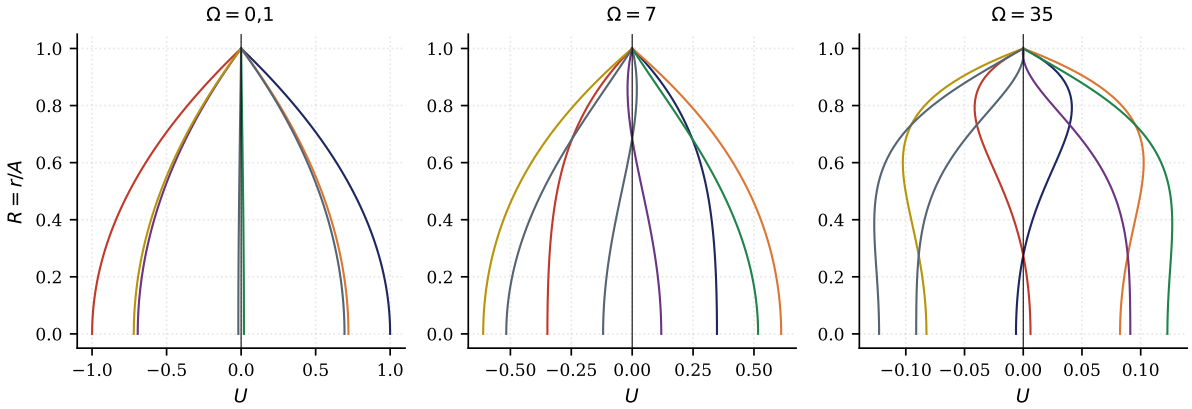


Figura B.21: Esercizio 18: profili di velocità in otto fasi del periodo per $\Omega = 0{,}1$, 7 e 35 (soluzione di Womersley).

ridotta a meno di un quinto; i massimi di velocità si spostano vicino alla parete (effetto anulare di Richardson), dove la vorticità resta confinata in uno strato di Stokes di spessore relativo $\sqrt{2/\Omega}$. Nell'aorta umana α_W vale 15–20, nelle arterie femorali circa 4, nei capillari molto meno di 1. Il modello di arteria del modulo precedente usava una legge d'attrito quasi stazionaria, cioè di Poiseuille: questo esercizio mostra che è giustificata solo nei vasi piccoli, mentre nei grandi vasi lo sforzo alla parete è sfasato e diverso da quello di Poiseuille.

Listato completo degli esercizi

esercizi_svolti.py

```
"""
esercizi_svolti.py -- codice degli esercizi svolti della monografia 3 (flussi viscosi).

Ogni esercizio e' una funzione es_NN(); senza argomenti si eseguono tutti,
altrimenti solo quelli indicati:   python3 esercizi_svolti.py 3 7
Le figure vengono salvate come es_NN.pdf nella cartella figure/.
"""
import sys
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.integrate import solve_ivp, simpson
from scipy.linalg import eig
from scipy.optimize import brentq, minimize_scalar
from scipy.special import jv, jn_zeros, erfc
from metodi_numerici import trid, thomas, rk4_integra, cheb, numeri_onda
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, VIOLA, GIALLO, GRIGIO, PALETTE

Q0_BLASIUS = 0.332057336215


def blasius(eta, z):
    f, p, q = z
    return np.array([p, q, -0.5 * f * q])


def soluzione_blasius(etamax, h=0.01):
    """f, f', f'' su [0, etamax] (oltre eta = 10 si prolunga con f = eta - 1,7208)."""
    e10, Z = rk4_integra(blasius, np.array([0.0, 0.0, Q0_BLASIUS]), min(10.0, etamax), h)
    if etamax <= 10.0:
        return e10, Z[:, 0], Z[:, 1], Z[:, 2]
    eta = np.arange(0.0, etamax + h / 2, h)
    f = np.where(eta <= 10, np.interp(eta, e10, Z[:, 0]), Z[-1, 0] + (eta - 10.0))
    p = np.where(eta <= 10, np.interp(eta, e10, Z[:, 1]), 1.0)
    q = np.where(eta <= 10, np.interp(eta, e10, Z[:, 2]), 0.0)
    return eta, f, p, q


# ============================================================================
def es_01():
    """Trasformazione di Topfer: f''(0) con una sola integrazione."""
    print("\n=== Esercizio 1: f''(0) di Blasius senza iterazioni")
    def g_eq(z, y):
        g, gp, gpp = y
        return [gp, gpp, -0.5 * g * gpp]
    for zmax in (6.0, 8.0, 10.0, 15.0):
        s = solve_ivp(g_eq, (0, zmax), [0, 0, 1.0], rtol=1e-12, atol=1e-13, method="DOP853")
        k = np.sqrt(s.y[1, -1])
        print(f"   z_max = {zmax:5.1f}:  g'(z_max) = {s.y[1,-1]:.10f}  k = {k:.8f}  "
              f"f''(0) = k^-3 = {k**-3:.10f}")
    # profili: f(eta) = g(z)/k con eta = k z, confronto con l'integrazione diretta
```

```
    s = solve_ivp(g_eq, (0, 10.0), [0, 0, 1.0], rtol=1e-12, atol=1e-13, dense_output=True)
    k = np.sqrt(s.y[1, -1])
    eta = np.linspace(0, 8, 81)
    fp_topfer = s.sol(eta / k)[1] / k**2          # f' = g'/k^2
    _, Z = rk4_integra(blasius, np.array([0, 0, k**-3]), 8.0, 0.1)
    print(f"   scarto massimo fra f' di Topfer e quella integrata con f''(0) = k^-3: "
          f"{np.max(np.abs(fp_topfer - Z[:, 1])):.1e}")
    zz = np.linspace(0, 6, 200)
    fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.9))
    ax[0].plot(s.sol(zz)[1], zz, color=ARANCIO, label="$g'(z)$")
    ax[0].axvline(k**2, color=GRIGIO, ls=":", label=f"$k^2={k**2:.4f}$".replace(".", "{,}"))
    ax[0].set_xlabel("$g'$")
    ax[0].set_ylabel("$z$")
    ax[0].legend(fontsize=7)
    ax[0].set_title("problema ausiliario, $g''(0)=1$")
    ax[1].plot(fp_topfer, eta, color=BLU, label="da $g$: $f'=g'/k^2$, $\\eta=kz$")
    ax[1].plot(Z[::4, 1], eta[::4], "o", ms=3, mfc="none", color=ROSSO, label="integrazione diretta")
    ax[1].set_xlabel("$f'=u/U$")
    ax[1].set_ylabel("$\\eta$")
    ax[1].legend(fontsize=7)
    fig.tight_layout()
    salva(fig, "es_01")


# ============================================================================
def _tiro_dimezzamento(fw, etamax=10.0, h=0.05, q0=1.0, dq0=0.2, eps=1e-6, itmax=400):
    """Dimezzamento dell'intervallo con il criterio generale: si inverte e si
    dimezza il passo quando error*error' < 0 oppure |error'| - |error| < 0."""
    err_prec = None
    for it in range(1, itmax + 1):
        q0 += dq0
        _, Z = rk4_integra(blasius, np.array([fw, 0.0, q0]), etamax, h)
        err = 1.0 - Z[-1, 1]
        if not np.isfinite(err):
            err = -1e3
        if 0.0 <= err <= eps:
            return q0, it
        if err_prec is not None and (err * err_prec < 0 or abs(err_prec) - abs(err) < 0):
            dq0 = -dq0 / 2.0
        err_prec = err
    raise RuntimeError("dimezzamento non convergente")


def _tiro_newton(fw, q0, etamax, h=0.02, tol=1e-12):
    def F(eta, z):
        f, p, q, Fs, P, Q = z
        return np.array([p, q, -0.5 * f * q, P, Q, -0.5 * (Fs * q + f * Q)])
    for _ in range(60):
        eta, Z = rk4_integra(F, np.array([fw, 0, q0, 0, 0, 1.0]), etamax, h)
        r = Z[-1, 1] - 1.0
        if abs(r) < tol:
            return q0, eta, Z
        q0 = q0 - r / Z[-1, 4]
    raise RuntimeError("Newton non convergente")
```

```
107
108
109 def es_02():
110     """Iniezione e aspirazione: f(0) = -2C."""
111     print("\n=== Esercizio 2: lamina piana con iniezione (C > 0) o aspirazione (C <
    0)")
112     print("     C     f''(0) dimezz.  iter   f''(0) Newton   delta_99   eta_flesso")
113     Cs = np.round(np.arange(-0.6, 0.61, 0.1), 2)
114     q0s, deltas, profili = [], [], {}
115     q_guess = 0.33
116     for C in Cs:
117         q_dim, it = _tiro_dimezzamento(-2 * C)
118         q_n, eta, Z = _tiro_newton(-2 * C, q_dim, etamax=14.0)
119         p, q = Z[:, 1], Z[:, 2]
120         d99 = np.interp(0.99, p[: np.argmax(p > 0.995)], eta[: np.argmax(p >
    0.995)])
121         # punto di flesso del profilo: f''' = -(1/2) f f'' = 0  <=>  f = 0 (con f''
    > 0)
122         f = Z[:, 0]
123         fl = eta[np.argmax(f >= 0)] if C > 0 else np.nan
124         q0s.append(q_n)
125         deltas.append(d99)
126         profili[C] = (eta, p)
127         print(f"   {C:+.1f}   {q_dim:11.6f}   {it:4d}    {q_n:11.6f}    {d99:7.3f}
      "
128               + (f"{fl:6.3f}" if C > 0 else "   -"))
129     # limite di "soffiamento" (blowing-off): f''(0) -> 0
130     print("   continuazione in C con eta_max = 40 (Newton):")
131     q = q0s[-1]
132     ultimi = []
133     for C in np.arange(0.60, 0.6301, 0.002):
134         try:
135             q, _, _ = _tiro_newton(-2 * C, q, etamax=40.0, h=0.02)
136         except RuntimeError:
137             break
138         ultimi.append((C, q))
139         if q < 2e-4:
140             break
141     for C, q in ultimi[-4:]:
142         print(f"      C = {C:.3f}   f''(0) = {q:.6f}")
143     (C1, q1), (C2, q2) = ultimi[-2], ultimi[-1]
144     print(f"   estrapolazione lineare: f''(0) = 0 per C = {C2 - q2 * (C2 - C1) /
    (q2 - q1):.4f}")
145     fig, ax = plt.subplots(1, 3, figsize=(7.8, 2.8))
146     for C, col in zip([-0.6, -0.3, 0.0, 0.3, 0.6], [VERDE, BLU, "k", ARANCIO,
    ROSSO]):
147         eta, p = profili[C]
148         ax[0].plot(p, eta, color=col, label=f"$C={C:+.1f}$")
149     ax[0].set_ylim(0, 12)
150     ax[0].set_xlabel("$f'=u/U$")
151     ax[0].set_ylabel("$\\eta$")
152     ax[0].legend(fontsize=6.5)
153     ax[1].plot(Cs, q0s, "o-", color=BLU, ms=3)
154     ax[1].set_xlabel("$C$")
155     ax[1].set_ylabel("$f''(0)$ (sforzo alla parete)")
156     ax[2].plot(Cs, deltas, "s-", color=ARANCIO, ms=3)
157     ax[2].set_xlabel("$C$")
158     ax[2].set_ylabel("$\\eta_{99}$ (spessore)")
159     fig.tight_layout()
160     salva(fig, "es_02")
```

```
161
162
163 # ============================================================================
164 def _falkner_skan(beta, q0, etamax=10.0):
165     """f''' + f f'' + beta (1 - f'^2) = 0; Newton sul valore iniziale f''(0)."""
166     def F(eta, z):
167         f, p, q, Fs, P, Q = z
168         return [p, q, -f * q - beta * (1 - p * p), P, Q, -(Fs * q + f * Q) + 2 *
    beta * p * P]
169     for _ in range(50):
170         s = solve_ivp(F, (0, etamax), [0, 0, q0, 0, 0, 1.0], rtol=1e-11,
    atol=1e-12, method="DOP853")
171         r = s.y[1, -1] - 1.0
172         if abs(r) < 1e-10:
173             return q0
174         q0 -= r / s.y[4, -1]
175     raise RuntimeError(f"Falkner-Skan: Newton non converge per beta = {beta}")
176
177
178 def es_03():
179     """Equazione di Falkner-Skan."""
180     print("\n=== Esercizio 3: soluzioni di Falkner-Skan, U = U0 x^m, beta =
    2m/(m+1)")
181     print("      m        beta      f''(0)    angolo del cuneo")
182     ms = [1.0, 0.5, 0.0, -0.05, -0.09]
183     q = 1.2
184     ris = {}
185     for m in ms:
186         beta = 2 * m / (m + 1)
187         q = _falkner_skan(beta, q, etamax=10.0 if m >= 0 else 14.0)
188         ris[m] = q
189         cuneo = f"{beta*180:7.1f} gradi" if m >= 0 else "  (gradiente avverso)"
190         print(f"   {m:+6.3f}   {beta:+7.4f}   {q:9.5f}    {cuneo}")
191     # distacco: si impone f''(0) = 0 e si cerca beta tale che f'(eta_max) = 1
192     def residuo(beta, etamax=12.0):
193         s = solve_ivp(lambda e, z: [z[1], z[2], -z[0] * z[2] - beta * (1 -
    z[1]**2)],
194                       (0, etamax), [0, 0, 0.0], rtol=1e-12, atol=1e-13,
    method="DOP853")
195         return s.y[1, -1] - 1.0
196     for emax in (10.0, 12.0, 14.0):
197         beta_s = brentq(lambda b: residuo(b, emax), -0.21, -0.18, xtol=1e-12)
198         print(f"   distacco (f''(0) = 0), eta_max = {emax:4.1f}: beta_s =
    {beta_s:.6f},"
199               f"  m_s = beta/(2 - beta) = {beta_s/(2 - beta_s):.6f}")
200     betas = np.linspace(beta_s + 1e-4, 0.0, 24)
201     qs = []
202     q = 0.05
203     for b in betas:
204         q = _falkner_skan(b, max(q, 0.02), etamax=14.0)
205         qs.append(q)
206     qs = np.array(qs)
207     fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
208     eta = np.linspace(0, 8, 300)
209     for m, col in zip(ms, [ROSSO, ARANCIO, "k", BLU, VERDE]):
210         beta = 2 * m / (m + 1)
211         s = solve_ivp(lambda e, z: [z[1], z[2], -z[0] * z[2] - beta * (1 -
    z[1]**2)],
212                       (0, 8), [0, 0, ris[m]], rtol=1e-10, atol=1e-12,
    dense_output=True)
```

```
        ax[0].plot(s.sol(eta)[1], eta, color=col,
    label=f"$m={m:+.2f}$".replace(".", "{,}"))
    ax[0].set_xlabel("$f'=u/U$")
    ax[0].set_ylabel("$\\eta$")
    ax[0].legend(fontsize=6.5)
    ax[1].plot(betas, qs, "o-", ms=3, color=BLU)
    ax[1].plot([beta_s], [0.0], "o", color=ROSSO, label="distacco")
    ax[1].axvline(beta_s, color=GRIGIO, ls=":")
    ax[1].set_xlabel("$\\beta=2m/(m+1)$")
    ax[1].set_ylabel("$f''(0)$")
    ax[1].legend(fontsize=7)
    fig.tight_layout()
    salva(fig, "es_03")


# ============================================================================
def _theta_adiabatica(Pr, etamax, n):
    eta, f, p, q = soluzione_blasius(etamax, h=etamax / n)
    h = eta[1]
    C = np.zeros((n, 4))
    C[:, 0] = 1 - h / 4 * Pr * f[:n]
    C[:, 1] = -2.0
    C[:, 2] = 1 + h / 4 * Pr * f[:n]
    C[:, 3] = -2 * h * h * Pr * q[:n] ** 2
    C[0, 2] += C[0, 0]
    C[0, 0] = 0.0
    C[n - 1, 2] = 0.0
    th = np.zeros(n + 1)
    th[:n] = trid(C)
    return eta, p, th


def es_04():
    """Distribuzione dell'entalpia totale nello strato limite."""
    print("\n=== Esercizio 4: entalpia totale (H - H1)/(U^2/2) = theta + f'^2 - 1")
    fig, ax = plt.subplots(figsize=(4.8, 3.2))
    for (nome, Pr), col in zip([("acqua", 6.75), ("aria", 0.714), ("mercurio",
    0.044), ("Pr = 1", 1.0)],
                               [BLU, ARANCIO, VERDE, GRIGIO]):
        etamax = 12.0 if Pr > 0.5 else 60.0
        eta, p, th = _theta_adiabatica(Pr, etamax, int(etamax / 0.02))
        dH = th + p**2 - 1
        bilancio = simpson(p * dH, x=eta)
        i_min, i_max = np.argmin(dH), np.argmax(dH)
        print(f"   {nome:9s}: parete {dH[0]:+.4f}; min {dH[i_min]:+.4f} (eta =
    {eta[i_min]:.2f}),"
              f" max {dH[i_max]:+.4f} (eta = {eta[i_max]:.2f});  int f'(H-H1) deta
    = {bilancio:+.1e}")
        ax.plot(dH, eta, color=col, label=f"{nome}" + (f" ($Pr={Pr}$)" if "Pr" not
    in nome else ""))
    ax.axvline(0, color="k", lw=0.6)
    ax.set_ylim(0, 10)
    ax.set_xlabel("$(H-H_1)/(U^2/2)=\\theta+f'^2-1$")
    ax.set_ylabel("$\\eta$")
    ax.legend(fontsize=7)
    salva(fig, "es_04")


# ============================================================================
def _cilindro_rotante(Ra, n, condizione="dV/dR"):
```

```
    """V'' + V'/R - V/R^2 = 0 su [1, Ra], V(1) = 1; in Ra:
       'dV/dR'  : dV/dR = 0;      'sforzo' : d(V/R)/dR = 0, cioe' dV/dR = V/R."""
    R = np.linspace(1.0, Ra, n + 1)
    h = R[1] - R[0]
    # incognite V_1..V_n (V_0 = 1 noto); nodo fittizio V_(n+1)
    C = np.zeros((n, 4))
    Ri = R[1:]
    C[:, 0] = 1 / h**2 - 1 / (2 * h * Ri)
    C[:, 1] = -2 / h**2 - 1 / Ri**2
    C[:, 2] = 1 / h**2 + 1 / (2 * h * Ri)
    C[0, 3] = -C[0, 0] * 1.0
    C[0, 0] = 0.0
    if condizione == "dV/dR":                          # V_(n+1) = V_(n-1)
        C[n - 1, 0] += C[n - 1, 2]
    else:                                              # V_(n+1) = V_(n-1) + 2h V_n /
    Ra
        C[n - 1, 0] += C[n - 1, 2]
        C[n - 1, 1] += C[n - 1, 2] * 2 * h / Ra
    C[n - 1, 2] = 0.0
    V = np.concatenate([[1.0], trid(C)])
    return R, V


def es_05():
    """Fluido attorno a un cilindro rotante, con superficie libera."""
    print("\n=== Esercizio 5: cilindro rotante con strato di fluido a superficie
    libera (Ra = 2)")
    Ra, n = 2.0, 50
    R, V = _cilindro_rotante(Ra, n)
    Ves = Ra / (1 + Ra**2) * (Ra / R + R / Ra)
    err = 100 * (V - Ves) / Ves
    print(f"   condizione dV/dR = 0: errore percentuale massimo
    {np.max(np.abs(err)):.4f} %"
          f" (in R = {R[np.argmax(np.abs(err))]:.2f}); V(Ra) = {V[-1]:.5f} (esatto
    {Ves[-1]:.5f})")
    for i in range(0, n + 1, 10):
        print(f"      R = {R[i]:.2f}   V = {V[i]:.6f}   V_es = {Ves[i]:.6f}
    errore = {err[i]:+.4f} %")
    R2, V2 = _cilindro_rotante(Ra, n, "sforzo")
    print(f"   condizione di sforzo nullo d(V/R)/dR = 0: max |V - R| =
    {np.max(np.abs(V2 - R2)):.1e}"
          " (rotazione rigida)")
    tau_a = (np.gradient(V, R) - V / R)[-1]
    print(f"   con dV/dR = 0 lo sforzo sulla superficie libera NON e' nullo:
    tau/(mu v_b/r_b) = {tau_a:+.4f}")
    fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.9))
    ax[0].plot(R, Ves, color=GRIGIO, lw=2, alpha=0.6, label="esatta (3.32)")
    ax[0].plot(R[::5], V[::5], "o", ms=3.5, color=BLU, mfc="none", label="TRID,
    $dV/dR=0$")
    ax[0].plot(R2, V2, "--", color=ROSSO, label="sforzo nullo: $V=R$")
    ax[0].set_xlabel("$R=r/r_b$")
    ax[0].set_ylabel("$V=v_\\theta/v_b$")
    ax[0].legend(fontsize=7)
    ax[1].plot(R, err, color=BLU)
    ax[1].set_xlabel("$R$")
    ax[1].set_ylabel("errore percentuale")
    fig.tight_layout()
    salva(fig, "es_05")


```

```
# ============================================================================
def _laplace_canale(n, dirichlet_sup, neumann_sup):
    """lap U = 0 nel quadrato [0,1]^2 (nodi i: z, j: y), U = 0 su fondo e pareti;
    sulla riga superiore j = n: dirichlet_sup[i] (valore) dove neumann_sup[i] e'
    False,
    altrimenti dU/dy = 0 (nodo fittizio). Metodo diretto per matrici sparse."""
    nz = n - 1
    incognite = {}
    for j in range(1, n + 1):
        for i in range(1, n):
            if j < n or neumann_sup[i]:
                incognite[(i, j)] = len(incognite)
    A = sp.lil_matrix((len(incognite), len(incognite)))
    b = np.zeros(len(incognite))
    def val(i, j):
        if j == n and not neumann_sup[i]:
            return dirichlet_sup[i]
        return 0.0
    for (i, j), r in incognite.items():
        A[r, r] = -4.0
        vicini = [(i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)]
        if j == n:                                      # riga superiore libera:
    fittizio = (i, n-1)
            vicini[3] = (i, j - 1)
        for (a, c) in vicini:
            if (a, c) in incognite:
                A[r, incognite[(a, c)]] += 1.0
            elif 0 < a < n and 0 < c <= n:
                b[r] -= val(a, c)
    u = spla.spsolve(A.tocsr(), b)
    U = np.zeros((n + 1, n + 1))
    for i in range(1, n):
        U[i, n] = val(i, n)
    for (i, j), r in incognite.items():
        U[i, j] = u[r]
    return U


def _portata(U, h):
    w = np.ones_like(U)
    w[0, :] *= 0.5
    w[-1, :] *= 0.5
    w[:, 0] *= 0.5
    w[:, -1] *= 0.5
    return np.sum(w * U) * h * h


def es_06():
    """Canale quadrato con parete superiore mobile."""
    print("\n=== Esercizio 6: canale quadrato con parete superiore mobile
    (velocita' u0)")
    def serie_Q(nmax=4001):
        Q = 0.0
        for k in range(1, nmax, 2):
            Q += 8 / (k * np.pi) ** 3 * np.tanh(k * np.pi / 2)    # (cosh x -
    1)/sinh x = tanh(x/2)
        return Q
    print(f"   portata esatta (serie): Q/(u0 L^2) = {serie_Q():.6f}  (argomento di
    simmetria: 1/4)")
    for n in (20, 40, 80, 160):
```

```
        sup = np.ones(n + 1)
        sup[0] = sup[-1] = 0.0                    # spigoli a velocita' nulla
        U = _laplace_canale(n, sup, np.zeros(n + 1, dtype=bool))
        Q = _portata(U, 1.0 / n)
        print(f"   h = 1/{n:<4d}  Q = {Q:.6f}   errore = {Q - 0.25:+.2e};  U al centro = {U[n//2, n//2]:.5f}")
    zz = np.linspace(0, 1, n + 1)
    fig, ax = plt.subplots(figsize=(3.8, 3.5))
    cs = ax.contour(zz, zz, U.T, levels=[0.02, 0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9], colors=BLU,
                    linewidths=0.9)
    ax.clabel(cs, fmt=lambda v: f"{v:.2f}".replace(".", ","), fontsize=6)
    ax.plot([0, 1], [1, 1], color=ROSSO, lw=3)
    ax.set_aspect("equal")
    ax.set_xlabel("$z/L$")
    ax.set_ylabel("$y/L$")
    ax.set_title("$u/u_0$, parete superiore mobile")
    salva(fig, "es_06")


# ============================================================================
def es_07():
    """Lastra mobile larga L/2 su un canale a pelo libero."""
    print("\n=== Esercizio 7: lastra mobile di larghezza L/2, resto della superficie libero")
    n = 80
    h = 1.0 / n
    pos = np.linspace(0.0, 0.5, 11)              # ascissa del bordo sinistro della lastra
    Qs = []
    for s in pos:
        z = np.linspace(0, 1, n + 1)
        sotto = (z >= s - 1e-12) & (z <= s + 0.5 + 1e-12)
        sup = np.where(sotto, 1.0, 0.0)
        neu = ~sotto
        neu[0] = neu[-1] = False
        sup[0] = sup[-1] = 0.0
        U = _laplace_canale(n, sup, neu)
        Qs.append(_portata(U, h))
        if s in (0.0, 0.25):
            print(f"   lastra fra z = {s:.2f} e {s+0.5:.2f}: Q/(u0 L^2) = {Qs[-1]:.5f}")
    Qs = np.array(Qs)
    print(f"   massimo per lastra centrata (s = {pos[np.argmax(Qs)]:.2f}): Q = {Qs.max():.5f};"
          f" minimo con la lastra contro una parete: Q = {Qs.min():.5f}")
    # confronto: lastra larga quanto il canale (Esercizio 6) -> 0,25
    s = 0.25
    z = np.linspace(0, 1, n + 1)
    sotto = (z >= s - 1e-12) & (z <= s + 0.5 + 1e-12)
    sup = np.where(sotto, 1.0, 0.0)
    neu = ~sotto
    neu[0] = neu[-1] = False
    U = _laplace_canale(n, sup, neu)
    fig, ax = plt.subplots(1, 2, figsize=(7.0, 3.0))
    ax[0].plot(pos, Qs, "o-", ms=3.5, color=BLU)
    ax[0].set_xlabel("posizione $s/L$ del bordo sinistro della lastra")
    ax[0].set_ylabel("$Q/(u_0L^2)$")
    cs = ax[1].contour(z, z, U.T, levels=[0.02, 0.05, 0.1, 0.2, 0.4, 0.6, 0.8], colors=BLU,
```

```
                          linewidths=0.9)
    ax[1].clabel(cs, fmt=lambda v: f"{v:.2f}".replace(".", ","), fontsize=6)
    ax[1].plot([0.25, 0.75], [1, 1], color=ROSSO, lw=3)
    ax[1].plot([0, 0.25], [1, 1], "--", color=GRIGIO)
    ax[1].plot([0.75, 1], [1, 1], "--", color=GRIGIO)
    ax[1].set_aspect("equal")
    ax[1].set_title("lastra centrata")
    fig.tight_layout()
    salva(fig, "es_07")


# ============================================================================
def es_08():
    """Instabilita' dello schema esplicito per R > 1/2."""
    print("\n=== Esercizio 8: schema esplicito con R > 1/2")
    M = 21
    fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.9))
    for R, col in ((0.5, BLU), (0.55, ARANCIO), (0.6, ROSSO)):
        u = np.zeros(M)
        u[-1] = 1.0
        storia = []
        for j in range(1, 201):
            u[1:-1] = u[1:-1] + R * (u[:-2] - 2 * u[1:-1] + u[2:])
            storia.append(np.max(np.abs(u)))
        print(f"   R = {R:.2f}: max|u| dopo 50, 100, 200 passi = {storia[49]:.3e},
    {storia[99]:.3e},"
              f" {storia[199]:.3e};  |G|max = |1 - 4R sin^2(pi/2 * 19/20)| ="
              f" {abs(1 - 4*R*np.sin(np.pi*19/40)**2):.4f}")
        ax[0].semilogy(np.arange(1, 201), storia, color=col, label=f"$R={R}$")
        if R == 0.6:
            for j in (40, 60):
                u2 = np.zeros(M)
                u2[-1] = 1.0
                for _ in range(j):
                    u2[1:-1] = u2[1:-1] + R * (u2[:-2] - 2 * u2[1:-1] + u2[2:])
                ax[1].plot(u2, np.linspace(0, 1, M), "o-", ms=2.5, label=f"{j}
    passi")
    ax[0].set_xlabel("passo temporale")
    ax[0].set_ylabel("$\\max|u|$ (m/s)")
    ax[0].legend(fontsize=7)
    ax[1].set_xlabel("$u$ (m/s)")
    ax[1].set_ylabel("$y$ (m)")
    ax[1].set_title("$R=0{,}6$: oscillazione a dente di sega")
    ax[1].legend(fontsize=7)
    fig.tight_layout()
    salva(fig, "es_08")



# ============================================================================
def _spin_up(Ra, n, T_fin, condizione, istanti, Rd=0.25):
    """Schema esplicito per V_T = V_RR + V_R/R - V/R^2 su [1, Ra]."""
    R = np.linspace(1.0, Ra, n + 1)
    h = R[1] - R[0]
    tau = Rd * h * h
    V = np.zeros(n + 1)
    V[0] = 1.0                                      # il cilindro parte bruscamente
    out = {}
    t = 0.0
    k = 0
```

```
    istanti = sorted(istanti)
    while t < T_fin - 1e-12:
        Vg = np.empty(n + 2)                          # con il nodo fittizio V_(n+1)
        Vg[:n + 1] = V
        if condizione == "dV/dR":
            Vg[n + 1] = V[n - 1]
        else:                                         # dV/dR = V/R in R = Ra
            Vg[n + 1] = V[n - 1] + 2 * h * V[n] / Ra
        Ri = R[1:]
        lap = (Vg[2:] - 2 * Vg[1:-1] + Vg[:-2]) / h**2 + (Vg[2:] - Vg[:-2]) / (2 *
    h * Ri) - Vg[1:-1] / Ri**2
        V[1:] = V[1:] + tau * lap
        t += tau
        while k < len(istanti) and t >= istanti[k] - 1e-12:
            out[istanti[k]] = V.copy()
            k += 1
    return R, out


def es_09():
    """Messa in rotazione del fluido attorno al cilindro (spin-up)."""
    print("\n=== Esercizio 9: avviamento della rotazione (Ra = 2, T = nu t /
    r_b^2)")
    istanti = [0.01, 0.05, 0.2, 0.5, 1.0, 2.0, 4.0]
    fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
    for a_, cond, titolo in zip(ax, ("dV/dR", "sforzo"), ("condizione $dV/dR=0$",
    "sforzo nullo")):
        R, out = _spin_up(2.0, 50, 4.0, cond, istanti)
        fin = (2.0 / 5.0 * (2.0 / R + R / 2.0)) if cond == "dV/dR" else R
        for T, col in zip(istanti, PALETTE):
            a_.plot(out[T], R, color=col, label=f"$T={T:g}$".replace(".", "{,}"))
        a_.plot(fin, R, "k--", lw=1.0, label="regime")
        a_.set_xlabel("$V=v_\\theta/v_b$")
        a_.set_ylabel("$R$")
        a_.set_title(titolo)
        scarti = [np.max(np.abs(out[T] - fin)) for T in istanti]
        print(f"   {cond:6s}: scarto dal regime " + ", ".join(f"T={T:g}: {s:.2e}"
    for T, s in zip(istanti, scarti)))
    ax[1].legend(fontsize=6, loc="lower right")
    fig.tight_layout()
    salva(fig, "es_09")
    # costante di tempo: primo autovalore del problema omogeneo (decadimento
    esponenziale)
    for cond in ("dV/dR", "sforzo"):
        R, out = _spin_up(2.0, 50, 4.0, cond, [2.0, 3.0])
        fin = (2.0 / 5.0 * (2.0 / R + R / 2.0)) if cond == "dV/dR" else R
        e2, e3 = np.max(np.abs(out[2.0] - fin)), np.max(np.abs(out[3.0] - fin))
        lam = np.log(e2 / e3)
        print(f"   {cond:6s}: decadimento exp(-lam T) con lam = {lam:.3f};"
              f"  scarto < 1% per T > {np.log(e3/0.01)/lam + 3.0:.2f}")


# ============================================================================
def es_10():
    """Lastra superiore oscillante: secondo problema di Stokes in un canale."""
    print("\n=== Esercizio 10: lastra superiore con velocita' u0 sin(omega t)")
    nu, d, u0, om = 1.0e-6, 1.0, 1.0, 1.0e-3
    delta = np.sqrt(2 * nu / om)
    periodo = 2 * np.pi / om
```

```
    print(f"   spessore dello strato di Stokes sqrt(2 nu/omega) = {delta*100:.2f} cm; periodo = {periodo:.0f} s")
    kap = (1 + 1j) / delta
    def esatta(y, t):                                   # soluzione periodica di regime
        return u0 * np.imag(np.exp(1j * om * t) * np.sinh(kap * y) / np.sinh(kap * d))
    risultati = {}
    for h in (0.05, 0.01, 0.005):
        M = int(round(d / h)) + 1
        y = np.linspace(0, d, M)
        R = 0.25
        tau = R * h * h / nu
        u = np.zeros(M)
        t = 0.0
        errori = []
        for per in (10, 20, 40):
            while t < per * periodo - 1e-9:
                t += tau
                u[1:-1] = u[1:-1] + R * (u[:-2] - 2 * u[1:-1] + u[2:])
                u[-1] = u0 * np.sin(om * t)
            errori.append(np.max(np.abs(u - esatta(y, t))))
            if per == 10:
                risultati[h] = (y, u.copy(), t)
            if h == 0.05:
                break
        print(f"   h = {h:5.3f} m  tau = {tau:7.2f} s ({periodo/tau:6.1f} passi per periodo):"
              f"  max|u - u_es| dopo 10, 20, 40 periodi = " + ", ".join(f"{e:.2e}" for e in errori))
    fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
    y, u, t = risultati[0.005]
    yy = np.linspace(0.75, 1.0, 400)
    for k, col in enumerate(PALETTE[:4]):
        tt = t + k * periodo / 4
        ax[0].plot(esatta(yy, tt), yy, color=col, label=f"$\\omega t$ = {k*90}$^\\circ$ (dopo 10 periodi)")
    ax[0].set_ylim(0.75, 1.0)
    ax[0].set_xlabel("$u/u_0$")
    ax[0].set_ylabel("$y$ (m)")
    ax[0].legend(fontsize=6)
    ax[0].set_title("soluzione periodica esatta")
    yc, uc, tc = risultati[0.05]
    ax[1].plot(esatta(yy, tc), yy, color=GRIGIO, lw=2, alpha=0.6, label="esatta")
    ax[1].plot(uc, yc, "o--", ms=3, color=ROSSO, label="$h=5$ cm")
    yf, uf, tf = risultati[0.005]
    ax[1].plot(esatta(yy, tf), yy, color=GRIGIO, lw=2, alpha=0.6)
    ax[1].plot(uf, yf, "-", color=BLU, lw=1.0, label="$h=5$ mm")
    ax[1].set_ylim(0.75, 1.0)
    ax[1].set_xlabel("$u/u_0$")
    ax[1].legend(fontsize=7)
    ax[1].set_title("dopo 10 periodi, due griglie")
    fig.tight_layout()
    salva(fig, "es_10")


# ============================================================================
def es_11():
    """Schema di DuFort-Frankel: stabilita' e consistenza."""
    print("\n=== Esercizio 11: DuFort-Frankel su u_t = u_yy, u(y,0) = sin(pi y)")
    # fattore di amplificazione: radici di (1+2R) G^2 - 4R cos(theta) G - (1-2R) = 0
```

```
    Gmax = 0.0
    for R in np.logspace(-2, 3, 60):
        for th in np.linspace(0, np.pi, 91):
            r = np.roots([1 + 2 * R, -4 * R * np.cos(th), -(1 - 2 * R)])
            Gmax = max(Gmax, np.max(np.abs(r)))
    print(f"   max |G| su 0,01 <= R <= 1000 e 0 <= kh <= pi: {Gmax:.6f}")
    def df(h, tau, T):
        n = int(round(1 / h))
        y = np.linspace(0, 1, n + 1)
        R = tau / h**2
        u_old = np.sin(np.pi * y)
        u = np.exp(-np.pi**2 * tau) * np.sin(np.pi * y)  # primo passo dalla soluzione esatta
        passi = int(round(T / tau))
        for _ in range(passi - 1):
            u_new = np.zeros_like(u)
            u_new[1:-1] = (2 * R * (u[:-2] + u[2:]) + (1 - 2 * R) * u_old[1:-1]) / (1 + 2 * R)
            u_old, u = u, u_new
        tt = passi * tau
        return np.max(np.abs(u - np.exp(-np.pi**2 * tt) * np.sin(np.pi * y)))
    print("        h      tau = h^2/4 (R fisso)     tau = h/10 (tau/h fisso)")
    passi_h = [1 / 10, 1 / 20, 1 / 40, 1 / 80, 1 / 160]
    e1, e2 = [], []
    for h in passi_h:
        e1.append(df(h, h * h / 4, 0.2))
        e2.append(df(h, h / 10, 0.2))
        print(f"    1/{int(round(1/h)):<4d}       {e1[-1]:.3e}       {e2[-1]:.3e}")
    # equazione modificata: u_t + (tau/h)^2 u_tt = u_yy  ->  decadimento del modo sin(pi y)
    c = 0.1
    s = (-1 + np.sqrt(1 - 4 * c**2 * np.pi**2)) / (2 * c**2)
    print(f"   con tau/h = {c}: il metodo converge a u_t + {c**2:.2f} u_tt = u_yy, il cui modo decade come exp({s:.4f} t)"
          f" invece di exp({-np.pi**2:.4f} t)")
    s2 = (-1 - np.sqrt(1 - 4 * c**2 * np.pi**2)) / (2 * c**2)
    A = (-np.pi**2 - s2) / (s - s2)                  # da u(0) = 1 e u'(0) = -pi^2
    print(f"   secondo modo (spurio) exp({s2:.1f} t); ampiezza del modo lento A = {A:.4f}")
    print(f"   errore previsto a t = 0,2: {abs(A*np.exp(s*0.2) - np.exp(-np.pi**2*0.2)):.3e}")
    fig, ax = plt.subplots(figsize=(4.2, 3.0))
    hh = np.array(passi_h)
    ax.loglog(hh, e1, "o-", color=BLU, label="$\\tau=h^2/4$")
    ax.loglog(hh, e2, "s-", color=ROSSO, label="$\\tau=h/10$")
    ax.loglog(hh, e1[0] * (hh / hh[0]) ** 2, ":", color="k", label="$h^2$")
    ax.set_xlabel("$h$")
    ax.set_ylabel("errore massimo a $t=0{,}2$")
    ax.legend(fontsize=7)
    salva(fig, "es_11")


# ============================================================================
def es_12():
    """Avviamento della corrente in un tubo circolare (metodo implicito)."""
    print("\n=== Esercizio 12: tubo circolare, gradiente di pressione applicato bruscamente")
    N = 40
    h = 1.0 / N
```

```
    Rn = np.linspace(0, 1, N + 1)
    lam = jn_zeros(0, 200)
    def W_es(R, T):
        return sum(8 * jv(0, l * R) / (l**3 * jv(1, l)) * np.exp(-l**2 * T) for l
    in lam)
    for tau in (0.01, 0.002):
        r = tau / h**2
        # nodi 0..N-1 incogniti (W_N = 0); in R = 0: W_T = 4 (W_1 - W_0)/h^2
        a = np.zeros(N); b = np.zeros(N); c = np.zeros(N)
        b[0] = 1 + 4 * r
        c[0] = -4 * r
        for i in range(1, N):
            a[i] = -r * (1 - 1 / (2 * i))
            b[i] = 1 + 2 * r
            c[i] = -r * (1 + 1 / (2 * i))
        W = 1 - Rn**2
        T = 0.0
        storia = []
        while T < 1.0 - 1e-12:
            W[:N] = thomas(a, b, c, W[:N])
            W[N] = 0.0
            T += tau
            storia.append((T, 1 - W[0]))
        err = np.max(np.abs(W - W_es(Rn, T)))
        print(f"   tau = {tau}: U al centro per T = 1: {1 - W[0]:.5f} (esatta {1 -
    W_es(np.array([0.0]), 1.0)[0]:.5f});"
              f" errore massimo {err:.1e}")
    Ts = np.linspace(0.005, 3.0, 1200)
    Uc = np.array([1 - W_es(np.array([0.0]), T)[0] for T in Ts])
    # canale piano (Programma 3.5) per confronto
    def Wc(T):
        return sum(4 * (-1) ** (m + 1) / ((2*m-1)*np.pi/2) ** 3 *
    np.exp(-((2*m-1)*np.pi/2) ** 2 * T)
                   for m in range(1, 200))
    Ucan = np.array([1 - Wc(T) for T in Ts])
    for f in (0.9, 0.99):
        print(f"   U_centro = {f}: tubo T = {Ts[np.argmax(Uc >= f)]:.3f};  canale
    piano T = {Ts[np.argmax(Ucan >= f)]:.3f}")
    print(f"   primo autovalore: tubo lam_1^2 = {lam[0]**2:.3f}, canale (pi/2)^2 =
    {(np.pi/2)**2:.3f}")
    fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.9))
    for T, col in zip([0.05, 0.1, 0.2, 0.4, 0.8], PALETTE):
        ax[0].plot(1 - Rn**2 - W_es(Rn, T), Rn, color=col,
    label=f"$T={T:g}$".replace(".", "{,}"))
    ax[0].plot(1 - Rn**2, Rn, "k--", lw=1, label="Poiseuille")
    ax[0].set_xlabel("$U=u/[-(A^2/4\\mu)\\,dp/dx]$")
    ax[0].set_ylabel("$R=r/A$")
    ax[0].legend(fontsize=6.5)
    sel = Ts <= 1.5
    ax[1].plot(Ts[sel], Uc[sel], color=BLU, label="tubo circolare")
    ax[1].plot(Ts[sel], Ucan[sel], "--", color=ARANCIO, label="canale piano")
    ax[1].plot([s[0] for s in storia][::20], [s[1] for s in storia][::20], "o",
    ms=2.5, color=BLU, mfc="none")
    ax[1].set_xlabel("$T$")
    ax[1].set_ylabel("velocita' al centro / regime")
    ax[1].legend(fontsize=7)
    fig.tight_layout()
    salva(fig, "es_12")
```

```
# ============================================================================
def _cavita_generale(n, fondo_u=0.0, coperchio_u=1.0, superficie_libera=False):
    """Cavita' di Stokes con il solutore diretto: fondo e coperchio con velocita'
    tangenziale assegnata, oppure coperchio sostituito da una superficie libera
    (psi = 0, zeta = 0)."""
    h = 1.0 / n
    ip = -np.ones((n + 1, n + 1), dtype=int)
    iz = -np.ones((n + 1, n + 1), dtype=int)
    k = 0
    for i in range(1, n):
        for j in range(1, n):
            ip[i, j] = k; k += 1
    for i in range(n + 1):
        for j in range(n + 1):
            if not ((i in (0, n)) and (j in (0, n))):
                iz[i, j] = k; k += 1
    rr, cc, vv = [], [], []
    b = np.zeros(k)
    r = 0
    def add(a_, c_, v_):
        rr.append(a_); cc.append(c_); vv.append(v_)
    for i in range(1, n):
        for j in range(1, n):
            add(r, ip[i, j], -4 / h**2)
            for (a_, c_) in ((i-1, j), (i+1, j), (i, j-1), (i, j+1)):
                if ip[a_, c_] >= 0:
                    add(r, ip[a_, c_], 1 / h**2)
            add(r, iz[i, j], 1.0); r += 1
            add(r, iz[i, j], -4.0)
            for (a_, c_) in ((i-1, j), (i+1, j), (i, j-1), (i, j+1)):
                add(r, iz[a_, c_], 1.0)
            r += 1
    pareti = []
    for t_ in range(1, n):
        pareti += [((0, t_), (1, 0), 0.0), ((n, t_), (-1, 0), 0.0), ((t_, 0), (0,
    1), fondo_u)]
        pareti.append(((t_, n), (0, -1), coperchio_u))
    for (i, j), (ni, nj), uw in pareti:
        add(r, iz[i, j], 1.0)
        if superficie_libera and j == n:
            r += 1                                      # zeta = 0 sulla superficie
    libera
            continue
        ti, tj = nj, ni
        i1, j1, i2, j2 = i + ni, j + nj, i + 2 * ni, j + 2 * nj
        for (a_, c_, w) in ((i1 - ti, j1 - tj, -1.0), (i1, j1, 8 / 3), (i1 + ti, j1
    + tj, -1.0), (i2, j2, -2 / 3)):
            if ip[a_, c_] >= 0:
                add(r, ip[a_, c_], -w / h**2)
        # termine di velocita': -2/(3h) dpsi/dy sul coperchio (normale -y), +2/(3h)
    dpsi/dy sul fondo
        b[r] = (-2 / (3 * h) * uw) if nj == -1 else (2 / (3 * h) * uw if nj == 1
    else 0.0)
        r += 1
    A = sp.csr_matrix((vv, (rr, cc)), shape=(k, k))
    x = spla.spsolve(A, b)
    psi = np.zeros((n + 1, n + 1)); zeta = np.zeros((n + 1, n + 1))
    m = ip >= 0; psi[m] = x[ip[m]]
    m = iz >= 0; zeta[m] = x[iz[m]]
    return psi, zeta
```

```
755
756
757 def es_13():
758     """Cavita' con superficie libera superiore e fondo mobile."""
759     print("\n=== Esercizio 13: cavita' di Stokes con fondo mobile (u = 1) e
    superficie libera")
760     for n in (10, 40, 160):
761         psi, zeta = _cavita_generale(n, fondo_u=1.0, coperchio_u=0.0,
    superficie_libera=True)
762         i, j = np.unravel_index(np.argmax(np.abs(psi)), psi.shape)
763         u_sup = (psi[n // 2, n] - psi[n // 2, n - 1]) / (1.0 / n)   # stima di u in
    superficie (O(h))
764         print(f"   griglia {n+1}x{n+1}: psi estremo = {psi[i, j]:+.5f} in x =
    {i/n:.3f}, y = {j/n:.3f}")
765     psiL, _ = _cavita_generale(160, fondo_u=0.0, coperchio_u=1.0)
766     print(f"   per confronto, cavita' con coperchio mobile: psi_min =
    {psiL.min():+.5f}")
767     n = 160
768     x = np.linspace(0, 1, n + 1)
769     # velocita' orizzontale lungo la verticale di mezzeria
770     u_mid = np.gradient(psi[n // 2, :], x)
771     print(f"   velocita' sulla superficie libera (x = 0,5): u = {u_mid[-1]:+.4f};"
772           f" inversione del moto a y = {x[np.argmax(np.diff(np.sign(u_mid)) !=
    0)]:.3f}")
773     fig, ax = plt.subplots(1, 2, figsize=(7.0, 3.2))
774     lv = np.linspace(psi.max(), 0, 9)[:-1][::-1] if psi.max() > 0 else
    np.linspace(psi.min(), 0, 9)[:-1]
775     ax[0].contour(x, x, psi.T, levels=np.sort(lv), colors=BLU, linewidths=0.9,
    linestyles="solid")
776     ax[0].plot([0, 1], [0, 0], color=ROSSO, lw=3)
777     ax[0].plot([0, 1], [1, 1], "--", color=GRIGIO)
778     ax[0].set_aspect("equal")
779     ax[0].set_title("fondo mobile, superficie libera")
780     ax[1].plot(u_mid, x, color=BLU, label="superficie libera")
781     ax[1].plot(np.gradient(psiL[n // 2, :], x), x, "--", color=ARANCIO,
    label="coperchio mobile")
782     ax[1].axvline(0, color="k", lw=0.6)
783     ax[1].set_xlabel("$u$ lungo $x=0{,}5$")
784     ax[1].set_ylabel("$y$")
785     ax[1].legend(fontsize=7)
786     fig.tight_layout()
787     salva(fig, "es_13")
788
789
790 # ============================================================================
791 def _benard_cheb(N, k):
792     """Benard con pareti rigide, collocazione di Chebyshev su z = (1 + y)/2."""
793     D, y = cheb(N)
794     Dz = 2 * D
795     D2 = Dz @ Dz
796     I = np.eye(N + 1)
797     L = D2 - k**2 * I
798     Z = np.zeros((N + 1, N + 1))
799     A = np.block([[L, I], [Z, L @ L]])
800     B = np.block([[Z, Z], [k**2 * I, Z]])
801     n1 = N + 1
802     for rr, riga in ((0, None), (N, None)):          # theta = 0
803         A[rr, :] = 0; A[rr, rr] = 1; B[rr, :] = 0
804     A[n1 + 0, :] = 0; A[n1 + 0, n1 + 0] = 1; B[n1 + 0, :] = 0          # w(1) = 0
805     A[n1 + N, :] = 0; A[n1 + N, n1 + N] = 1; B[n1 + N, :] = 0          # w(0) = 0
```

```
    A[n1 + 1, :] = 0; A[n1 + 1, n1:] = Dz[0]; B[n1 + 1, :] = 0          # w'(1) = 0
    A[n1 + N - 1, :] = 0; A[n1 + N - 1, n1:] = Dz[N]; B[n1 + N - 1, :] = 0  # w'(0) = 0
    lam = eig(A, B, right=False)
    ok = np.isfinite(lam) & (np.abs(lam.imag) < 1e-6 * np.abs(lam)) & (lam.real > 0)
    return lam[ok].real.min()


def es_14():
    """Curva di stabilita' neutra di Rayleigh-Benard (pareti rigide)."""
    print("\n=== Esercizio 14: curva neutra di Benard, pareti rigide")
    from prog3_7a_benard import Ra_minimo
    ks = np.round(np.arange(0.5, 10.01, 0.1), 2)
    Ra40 = np.array([Ra_minimo(40, k) for k in ks])
    i = np.argmin(Ra40)
    print(f"   n = 40: minimo della curva per k = {ks[i]:.1f}, Ra = {Ra40[i]:.1f}")
    ris = minimize_scalar(lambda k: Ra_minimo(150, k), bracket=(2.9, 3.1, 3.3), tol=1e-6)
    print(f"   n = 150, minimo raffinato: k_c = {ris.x:.4f}, Ra_c = {ris.fun:.2f}")
    for N in (10, 16, 20, 30, 40):
        print(f"   Chebyshev N = {N:2d}: Ra(k = 3,1163) = {_benard_cheb(N, 3.1163):.4f}")
    rc = minimize_scalar(lambda k: _benard_cheb(30, k), bracket=(3.0, 3.1, 3.2), tol=1e-8)
    print(f"   Chebyshev N = 30: k_c = {rc.x:.4f}, Ra_c = {rc.fun:.4f}")
    def jeffreys(k):
        kap = np.tanh(k / 2) / (8 * k) + (1 / np.cosh(k / 2)) ** 2 / 16
        return (np.pi**2 + k**2) ** 3 / ((1 - np.pi**2 / ((np.pi**2 + k**2) ** 2 * kap)) * k**2)
    kj = minimize_scalar(jeffreys, bracket=(2.5, 3.1, 3.8))
    print(f"   formula approssimata: minimo Ra = {kj.fun:.1f} per k = {kj.x:.3f}")
    Ra_exact = np.array([_benard_cheb(30, k) for k in ks])
    print(f"   scarto massimo fra la formula approssimata e la curva 'esatta' (Chebyshev) per 1 < k < 8: "
          f"{np.max(np.abs(jeffreys(ks[5:76]) / Ra_exact[5:76] - 1))*100:.2f} %")
    Ra_c, vec = Ra_minimo(150, ris.x, vettori=True)
    n = 150
    th, w = vec[:n], vec[n:]
    z = np.linspace(0, 1, n)
    fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
    ax[0].plot(ks, Ra_exact, color=GRIGIO, lw=2.2, alpha=0.6, label="Chebyshev $N=30$")
    ax[0].plot(ks[::3], Ra40[::3], "o", ms=3, color=BLU, mfc="none", label="diff. finite $n=40$")
    ax[0].plot(ks, jeffreys(ks), "--", color=ROSSO, label="formula approssimata")
    ax[0].plot(ris.x, ris.fun, "k*", ms=8)
    ax[0].set_ylim(0, 12000)
    ax[0].set_xlabel("$k$")
    ax[0].set_ylabel("$Ra$")
    ax[0].legend(fontsize=6.5)
    ax[1].plot(w / np.max(np.abs(w)), z, color=BLU, label="$w'$")
    ax[1].plot(th / np.max(np.abs(th)), z, "--", color=ROSSO, label="$\\theta'$")
    ax[1].set_xlabel("autofunzioni normalizzate")
    ax[1].set_ylabel("$z$")
    ax[1].legend(fontsize=7)
    ax[1].set_title(f"$k_c={ris.x:.3f}$".replace(".", "{,}"))
    fig.tight_layout()
    salva(fig, "es_14")


```

```
# ============================================================================
def _os_cp(ReP, alfa, r, N=64):
    """Orr-Sommerfeld per la corrente di Couette-Poiseuille scritta nella scala
    della velocita' massima della parte di Poiseuille:
        U~ = (r/2)(1 + y) + (1 - y^2),   r = U_parete / U_P = 2/B,   U~'' = -2,
    con Re_P = U_P h / nu = Re B / 2. Restituisce max lam_R (= max c_I)."""
    D, y = cheb(N)
    D2 = D @ D
    D4 = D2 @ D2
    I = np.eye(N + 1)
    U = 0.5 * r * (1 + y) + (1 - y**2)
    A = (D4 - 2 * alfa**2 * D2 + alfa**4 * I) / (ReP * alfa) - 2j * I \
        - 1j * np.diag(U) @ (D2 - alfa**2 * I)
    Bm = (D2 - alfa**2 * I).astype(complex)
    A[0, :] = 0; A[0, 0] = 1; A[1, :] = D[0]
    A[N, :] = 0; A[N, N] = 1; A[N - 1, :] = D[N]
    Bm[[0, 1, N - 1, N], :] = 0
    lam = eig(A, Bm, right=False)
    lam = lam[np.isfinite(lam) & (np.abs(lam) < 1e3)]
    return lam.real.max()


def es_15():
    """Curve neutre della corrente di Couette-Poiseuille."""
    print("\n=== Esercizio 15: corrente di Couette-Poiseuille U = (1+y)/2 +
    (B/2)(1-y^2)")
    # B = 0: Couette pura (nella scala della parete: U = (1+y)/2, U'' = 0)
    def os_couette(Re, alfa, N=80):
        D, y = cheb(N); D2 = D @ D; D4 = D2 @ D2; I = np.eye(N + 1)
        A = (D4 - 2 * alfa**2 * D2 + alfa**4 * I) / (Re * alfa) - 1j * np.diag(0.5
    * (1 + y)) @ (D2 - alfa**2 * I)
        Bm = (D2 - alfa**2 * I).astype(complex)
        A[0, :] = 0; A[0, 0] = 1; A[1, :] = D[0]; A[N, :] = 0; A[N, N] = 1; A[N -
    1, :] = D[N]
        Bm[[0, 1, N - 1, N], :] = 0
        lam = eig(A, Bm, right=False)
        return lam[np.isfinite(lam) & (np.abs(lam) < 1e3)].real.max()
    mx = max(os_couette(Re, a) for Re in (1e3, 1e4, 1e5) for a in (0.3, 0.6, 1.0,
    1.5))
    print(f"   B = 0 (Couette pura): max lam_R per Re = 1e3..1e5, alfa = 0,3..1,5:
    {mx:+.4f} -> stabile")
    risultati = {0.0: (5772.22, 1.0206)}
    griglia_Re = np.logspace(np.log10(2e3), np.log10(1e6), 30)
    def Re_n(a, r):
        """Ramo inferiore della curva neutra: primo Re_P instabile sulla griglia,
    poi Brent."""
        prec = None
        for Re in griglia_Re:
            N = 90 if Re < 2e5 else 150
            g = _os_cp(Re, a, r, N)
            if g > 0:
                if prec is None:
                    return Re
                return np.exp(brentq(lambda lr: _os_cp(np.exp(lr), a, r, N),
    np.log(prec), np.log(Re),
                                     xtol=1e-6))
            prec = Re
        return np.inf
    for B in (100.0, 25.0, 7.0, 3.0):
        r = 2.0 / B
```

```
911         al = np.arange(0.3, 1.21, 0.05)
912         vals = np.array([Re_n(a, r) for a in al])
913         if np.all(~np.isfinite(vals)):
914             print(f"   B = {B:5.1f} (r = {r:.3f}): nessun modo instabile per Re_P
    <= 1e6")
915             continue
916         j = int(np.argmin(vals))
917         ris = minimize_scalar(lambda a: Re_n(a, r), bounds=(al[max(j - 1, 0)],
    al[min(j + 1, len(al) - 1)]),
918                               method="bounded", options={"xatol": 1e-4})
919         risultati[r] = (ris.fun, ris.x)
920         print(f"   B = {B:5.1f} (r = U_par/U_P = {r:.3f}): Re_P,c = {ris.fun:9.1f},
    alfa_c = {ris.x:.3f};"
921               f"  nella scala della parete Re_c = 2 Re_P,c / B =
    {2*ris.fun/B:8.1f}")
922     r = 2.0
923     mxB = max(_os_cp(Re, a, r, 100 if Re < 2e5 else 160)
924               for Re in (1e4, 3e4, 1e5, 3e5, 1e6) for a in np.linspace(0.1, 1.2,
    12))
925     print(f"   B =   1.0 (r = 2,000): max lam_R per Re_P = 1e4..1e6, alfa =
    0,1..1,2: {mxB:+.5f}")
926     # curve neutre nel piano (Re_P, alfa)
927     fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
928     lRP = np.linspace(np.log10(3000), np.log10(3.0e5), 26)
929     al = np.linspace(0.4, 1.25, 22)
930     for r, col, et in ((0.0, "k", "Poiseuille"), (0.02, BLU, "$B=100$"), (0.08,
    VERDE, "$B=25$"),
931                        (2 / 7, ARANCIO, "$B=7$")):
932         G = np.array([[_os_cp(10**l, a, r, 64) for l in lRP] for a in al])
933         ax[0].contour(10**lRP, al, G, levels=[0.0], colors=[col], linewidths=1.2)
934         ax[0].plot([], [], color=col, label=et)
935     ax[0].set_xscale("log")
936     ax[0].set_xlabel("$Re_P=U_Ph/\\nu$")
937     ax[0].set_ylabel("$\\alpha$")
938     ax[0].legend(fontsize=7)
939     rr = sorted(risultati)
940     ax[1].plot(rr, [risultati[x][0] for x in rr], "o-", color=BLU)
941     ax[1].set_xlabel("$r=U_{\\mathrm{parete}}/U_P=2/B$")
942     ax[1].set_ylabel("$Re_{P,c}$")
943     ax[1].set_yscale("log")
944     fig.tight_layout()
945     salva(fig, "es_15")
946
947
948 # ============================================================================
949 def es_16():
950     """Convezione-diffusione con schema misto esplicito-implicito."""
951     print("\n=== Esercizio 16: convezione esplicita, diffusione implicita (modo per
    modo in y)")
952     import prog3_10_convezione_diffusione as cd
953     x, NX, NY, dx, k, U, V = cd.x, cd.NX, cd.NY, cd.dx, cd.k, cd.U, cd.V
954     import time
955     def imex(Pr, dt, toll=1e-4, nmax=200000):
956         a = 1.0 / (Pr * cd.RE)
957         T = np.tile(x[:, None], (1, NY))
958         Th_bordo = np.zeros(NY, dtype=complex)
959         Th_bordo[0] = NY                                  # trasformata di T = 1 su x
    = 1
960         n_int = NX - 2
961         for n in range(1, nmax + 1):
```

```
            Tx = np.zeros_like(T)
            Tx[1:-1] = (T[2:] - T[:-2]) / (2 * dx)
            conv = -U * Tx - V * cd.dy(T)
            rhs = np.fft.fft(T + dt * conv, axis=1)      # (T^n + dt N^n) nello spazio di Fourier
            Tn = np.empty_like(rhs)
            Tn[0] = 0.0
            Tn[-1] = Th_bordo
            r = a * dt / dx**2
            # algoritmo di Thomas eseguito contemporaneamente su tutti i modi m
            bb = 1 + 2 * r + a * dt * k**2               # diagonale (dipende dal modo)
            d = rhs[1:-1].copy()
            d[-1] += r * Th_bordo
            cp = np.zeros((n_int, NY), dtype=complex)
            dp = np.zeros((n_int, NY), dtype=complex)
            cp[0] = -r / bb
            dp[0] = d[0] / bb
            for i in range(1, n_int):
                den = bb + r * cp[i - 1]
                cp[i] = -r / den
                dp[i] = (d[i] + r * dp[i - 1]) / den
            sol = np.zeros((n_int, NY), dtype=complex)
            sol[-1] = dp[-1]
            for i in range(n_int - 2, -1, -1):
                sol[i] = dp[i] - cp[i] * sol[i + 1]
            Tn[1:-1] = sol
            Tnew = np.fft.ifft(Tn, axis=1).real
            var = np.max(np.abs(Tnew - T)) / dt
            T = Tnew
            if not np.isfinite(var) or var > 1e6:
                return None, n, n * dt
            if var < toll:
                return T, n, n * dt
        return T, nmax, nmax * dt
    for Pr, dts in ((0.71, [1e-3, 1e-2, 0.05, 0.5, 2.0]), (25.0, [1e-2, 0.1, 0.3, 1.0])):
        Ts = cd.stazionario(Pr)
        a = 1 / (Pr * cd.RE)
        print(f"   Pr = {Pr}: limite di stabilita' stimato dt < 2a/|V|max^2 = {2*a/2.0:.3f}")
        for dt in dts:
            t0 = time.perf_counter()
            T, n, tf = imex(Pr, dt)
            durata = time.perf_counter() - t0
            if T is None:
                print(f"      dt = {dt:6.3f}: INSTABILE dopo {n} passi")
            else:
                print(f"      dt = {dt:6.3f}: {n:6d} passi, t = {tf:6.2f}, {durata:5.2f} s,"
                      f" scarto dallo stazionario {np.max(np.abs(T - Ts)):.1e}")
    T, _, _ = imex(25.0, 0.01)
    xe = np.concatenate([cd.y, [cd.LY]])
    fig, ax = plt.subplots(figsize=(3.0, 4.2))
    Te = np.concatenate([T, T[:, :1]], axis=1)
    cs = ax.contour(x, xe, Te.T, levels=np.linspace(0.05, 0.95, 10), cmap="coolwarm", linewidths=1.0)
    ax.clabel(cs, fmt=lambda v: f"{v:.2f}".replace(".", ","), fontsize=5.5)
    ax.set_aspect("equal")
    ax.set_xlabel("$x$")
```

```
    ax.set_ylabel("$y$")
    ax.set_title("IMEX, $Pr=25$, $\\Delta t=0{,}01$")
    salva(fig, "es_16")


# ============================================================================
def _poisson_maschera(dentro, h, bracci=None, simmetria=()):
    """Risolve lap U = -1 sui nodi 'dentro' (matrice booleana), U = 0 altrove.
    bracci[(i,j)] = (aE, aW, aN, aS) frazioni di h per i nodi irregolari
    (formula di Shortley-Weller). simmetria: 'x' e/o 'y' -> dU/dn = 0 sulle
    linee i = 0 / j = 0 (nodi fittizi speculari)."""
    idx = -np.ones(dentro.shape, dtype=int)
    idx[dentro] = np.arange(dentro.sum())
    A = sp.lil_matrix((dentro.sum(), dentro.sum()))
    b = -np.ones(dentro.sum())
    for (i, j) in zip(*np.nonzero(dentro)):
        r = idx[i, j]
        aE, aW, aN, aS = bracci.get((i, j), (1, 1, 1, 1)) if bracci else (1, 1, 1, 1)
        cx = 2 / (h * h * (aE + aW)); cy = 2 / (h * h * (aN + aS))
        A[r, r] = -cx * (1 / aE + 1 / aW) * (aE + aW) / 2 * 0 - 2 / (h * h * aE * aW) - 2 / (h * h * aN * aS)
        for (a_, c_, coef) in ((i + 1, j, cx / aE), (i - 1, j, cx / aW), (i, j + 1, cy / aN), (i, j - 1, cy / aS)):
            if a_ < 0 and "x" in simmetria:
                a_ = -a_
            if c_ < 0 and "y" in simmetria:
                c_ = -c_
            if 0 <= a_ < dentro.shape[0] and 0 <= c_ < dentro.shape[1] and dentro[a_, c_] and \
                    not ((a_, c_) != (i + 1, j) and coef == cx / aE and aE < 1):
                A[r, idx[a_, c_]] += coef
    u = spla.spsolve(A.tocsr(), b)
    Uf = np.zeros(dentro.shape)
    Uf[dentro] = u
    return Uf


def _cerchio_quarto(n):
    """Quarto di cerchio di raggio 1 con simmetria sugli assi (Shortley-Weller)."""
    h = 1.0 / n
    xs = np.arange(n + 1) * h
    X, Y = np.meshgrid(xs, xs, indexing="ij")
    dentro = X**2 + Y**2 < 1 - 1e-12
    bracci = {}
    for (i, j) in zip(*np.nonzero(dentro)):
        aE = aN = 1.0
        if not (i + 1 <= n and dentro[i + 1, j]):
            aE = (np.sqrt(1 - Y[i, j]**2) - X[i, j]) / h
        if not (j + 1 <= n and dentro[i, j + 1]):
            aN = (np.sqrt(1 - X[i, j]**2) - Y[i, j]) / h
        if aE < 1 or aN < 1:
            bracci[(i, j)] = (aE, 1.0, aN, 1.0)
    # assemblaggio diretto (piu' leggibile della routine generale)
    idx = -np.ones(dentro.shape, dtype=int)
    idx[dentro] = np.arange(dentro.sum())
    A = sp.lil_matrix((dentro.sum(), dentro.sum()))
    b = -np.ones(dentro.sum())
    for (i, j) in zip(*np.nonzero(dentro)):
        r = idx[i, j]
```

```
        aE, aW, aN, aS = bracci.get((i, j), (1.0, 1.0, 1.0, 1.0))
        A[r, r] = -2 / (h * h * aE * aW) - 2 / (h * h * aN * aS)
        vicini = (((i + 1, j), 2 / (h * h * aE * (aE + aW)), aE),
                  ((i - 1, j), 2 / (h * h * aW * (aE + aW)), aW),
                  ((i, j + 1), 2 / (h * h * aN * (aN + aS)), aN),
                  ((i, j - 1), 2 / (h * h * aS * (aN + aS)), aS))
        for (a_, c_), coef, braccio in vicini:
            if braccio < 1.0:
                continue                                # il vicino e' il contorno: U = 0
            a_, c_ = abs(a_), abs(c_)                   # simmetria: U(-x) = U(x), U(-y) = U(y)
            A[r, idx[a_, c_]] += coef
    u = spla.spsolve(A.tocsr(), b)
    U = np.zeros(dentro.shape)
    U[dentro] = u
    # portata del quarto: integrazione in coordinate polari del campo interpolato
    from scipy.interpolate import RegularGridInterpolator
    f = RegularGridInterpolator((xs, xs), U)
    rr = np.linspace(0, 1, 401)
    tt = np.linspace(0, np.pi / 2, 201)
    Rg, Tg = np.meshgrid(rr, tt, indexing="ij")
    vals = f(np.stack([np.clip(Rg * np.cos(Tg), 0, 1), np.clip(Rg * np.sin(Tg), 0, 1)], axis=-1))
    Q = simpson(simpson(vals * Rg, x=tt, axis=1), x=rr)
    return xs, U, Q


def _dominio_maschera(dentro, h):
    idx = -np.ones(dentro.shape, dtype=int)
    idx[dentro] = np.arange(dentro.sum())
    A = sp.lil_matrix((dentro.sum(), dentro.sum()))
    for (i, j) in zip(*np.nonzero(dentro)):
        r = idx[i, j]
        A[r, r] = -4 / h**2
        for (a_, c_) in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
            if dentro[a_, c_]:
                A[r, idx[a_, c_]] = 1 / h**2
    u = spla.spsolve(A.tocsr(), -np.ones(dentro.sum()))
    U = np.zeros(dentro.shape)
    U[dentro] = u
    return U


def es_17():
    """Progetto: condotti di sezione circolare, triangolare, rettangolare con tubo interno."""
    print("\n=== Esercizio 17: correnti di Poiseuille in condotti di varie sezioni")
    print("   (a) tubo circolare, quarto di sezione con simmetria; U = u/[-(dp/dx) R^2/mu]")
    for n in (10, 20, 40, 80):
        xs, Uc, Q = _cerchio_quarto(n)
        print(f"      n = {n:3d}: U(0) = {Uc[0,0]:.6f} (esatto 0,25), Q_tubo = 4 Q_quarto = {4*Q:.6f}"
              f" (esatto pi/8 = {np.pi/8:.6f})")
    print("   (b) triangolo rettangolo isoscele (ipotenusa L = 2 in alto, vertice in basso)")
    ris_t = []
    for n in (20, 40, 80, 160):
        h = 1.0 / n
```

```
        xs = np.arange(2 * n + 1) * h
        ys = np.arange(n + 1) * h
        X, Y = np.meshgrid(xs, ys, indexing="ij")
        dentro = (Y > np.abs(X - 1) + 1e-12) & (Y < 1 - 1e-12)
        U = _dominio_maschera(dentro, h)
        Q = np.sum(U) * h * h                          # U = 0 sul contorno: regola dei trapezi
        ris_t.append(Q)
        print(f"      n = {n:3d}: Q = {Q:.6f},  U_max = {U.max():.6f}")
    Qt = ris_t[-1] + (ris_t[-1] - ris_t[-2]) / 3
    A_t, P_t = 1.0, 2 + 2 * np.sqrt(2)
    Dh = 4 * A_t / P_t
    print(f"      Richardson: Q = {Qt:.6f};  D_h = {Dh:.4f};  f Re = 2 D_h^2 A / Q = {2*Dh**2*A_t/Qt:.2f}")
    print("   (c) rettangolo 3a x 2a con tubo interno a x a (a = 1)")
    ris_r = []
    for n in (8, 16, 32, 64):
        h = 1.0 / n
        xs = np.arange(3 * n + 1) * h
        ys = np.arange(2 * n + 1) * h
        X, Y = np.meshgrid(xs, ys, indexing="ij")
        interno = (X >= 1.5 - 1e-12) & (X <= 2.5 + 1e-12) & (Y >= 0.25 - 1e-12) & (Y <= 1.25 + 1e-12)
        dentro = (X > 1e-12) & (X < 3 - 1e-12) & (Y > 1e-12) & (Y < 2 - 1e-12) & ~interno
        U = _dominio_maschera(dentro, h)
        pieno = (X > 1e-12) & (X < 3 - 1e-12) & (Y > 1e-12) & (Y < 2 - 1e-12)
        U0 = _dominio_maschera(pieno, h)
        ris_r.append((np.sum(U) * h * h, np.sum(U0) * h * h))
        print(f"      n = {n:3d}: Q = {ris_r[-1][0]:.6f}  (senza tubo interno {ris_r[-1][1]:.6f})")
    Qr = ris_r[-1][0] + (ris_r[-1][0] - ris_r[-2][0]) / 3
    Q0 = ris_r[-1][1] + (ris_r[-1][1] - ris_r[-2][1]) / 3
    print(f"      Richardson: Q = {Qr:.5f}, senza tubo {Q0:.5f}: il tubo interno riduce la portata del"
          f" {100*(1-Qr/Q0):.1f} % occupando il {100/6:.1f} % della sezione")
    xs_c, Uc, _ = _cerchio_quarto(80)
    fig, ax = plt.subplots(1, 3, figsize=(8.0, 2.9), gridspec_kw={"width_ratios": [1, 1.6, 1.4]})
    ax[0].contour(xs_c, xs_c, Uc.T, levels=np.linspace(0.025, 0.225, 9), colors=BLU, linewidths=0.8)
    tt = np.linspace(0, np.pi / 2, 100)
    ax[0].plot(np.cos(tt), np.sin(tt), "k", lw=1.5)
    ax[0].set_aspect("equal")
    ax[0].set_title("(a) quarto di tubo")
    n = 80; h = 1 / n
    xs = np.arange(2 * n + 1) * h; ys = np.arange(n + 1) * h
    X, Y = np.meshgrid(xs, ys, indexing="ij")
    Ut = _dominio_maschera((Y > np.abs(X - 1) + 1e-12) & (Y < 1 - 1e-12), h)
    ax[1].contour(xs, ys, Ut.T, levels=np.linspace(0.005, Ut.max() * 0.97, 7), colors=BLU, linewidths=0.8)
    ax[1].plot([0, 1, 2, 0], [1, 0, 1, 1], "k", lw=1.5)
    ax[1].set_aspect("equal")
    ax[1].set_title("(b) triangolo")
    n = 32; h = 1 / n
    xs = np.arange(3 * n + 1) * h; ys = np.arange(2 * n + 1) * h
    X, Y = np.meshgrid(xs, ys, indexing="ij")
    interno = (X >= 1.5 - 1e-12) & (X <= 2.5 + 1e-12) & (Y >= 0.25 - 1e-12) & (Y <= 1.25 + 1e-12)
```

```
    Ur = _dominio_maschera((X > 1e-12) & (X < 3 - 1e-12) & (Y > 1e-12) & (Y < 2 - 1e-12) & ~interno, h)
    ax[2].contour(xs, ys, Ur.T, levels=np.linspace(0.01, Ur.max() * 0.97, 8), colors=BLU, linewidths=0.8)
    ax[2].plot([0, 3, 3, 0, 0], [0, 0, 2, 2, 0], "k", lw=1.5)
    ax[2].fill([1.5, 2.5, 2.5, 1.5], [0.25, 0.25, 1.25, 1.25], color=GRIGIO, alpha=0.5)
    ax[2].set_aspect("equal")
    ax[2].set_title("(c) con tubo interno")
    for a_ in ax:
        a_.set_xticks([]); a_.set_yticks([])
        a_.spines[:].set_visible(False)
    fig.tight_layout()
    salva(fig, "es_17")


# ============================================================================
def es_18():
    """Progetto: corrente pulsante in un tubo (soluzione di Womersley)."""
    print("\n=== Esercizio 18: gradiente di pressione alternato, Omega U_T = 4 cos T + U_RR + U_R/R")
    N = 60
    h = 1.0 / N
    Rn = np.linspace(0, 1, N + 1)
    def esatta(R, T, Om):
        k = np.exp(3j * np.pi / 4) * np.sqrt(Om)      # i^(3/2) sqrt(Omega)
        F = 4 / (1j * Om) * (1 - jv(0, k * R) / jv(0, k))
        return np.real(F * np.exp(1j * T))
    fig, ax = plt.subplots(1, 3, figsize=(8.0, 2.9))
    for a_, Om in zip(ax, (0.1, 7.0, 35.0)):
        dT = 2 * np.pi / 400
        r = dT / (Om * h * h)
        # Crank-Nicolson: (I - r/2 L) U^(n+1) = (I + r/2 L) U^n + (dT/Om) 4 cos(T_(n+1/2))
        a = np.zeros(N); b = np.zeros(N); c = np.zeros(N)
        b[0] = 1 + 2 * r; c[0] = -2 * r                  # in R = 0: L U = 4 (U_1 - U_0)/h^2 (x h^2)
        for i in range(1, N):
            a[i] = -r / 2 * (1 - 1 / (2 * i)); b[i] = 1 + r; c[i] = -r / 2 * (1 + 1 / (2 * i))
        U = np.zeros(N + 1)
        T = 0.0
        periodi = 12 if Om < 10 else 25
        for nstep in range(400 * periodi):
            LU = np.zeros(N)
            LU[0] = 4 * (U[1] - U[0])
            i = np.arange(1, N)
            LU[1:] = (U[2:] - 2 * U[1:-1] + U[:-2]) + (U[2:] - U[:-2]) / (2 * i)
            rhs = U[:N] + r / 2 * LU + dT / Om * 4 * np.cos(T + dT / 2)
            U[:N] = thomas(a, b, c, rhs)
            U[N] = 0.0
            T += dT
        err = np.max(np.abs(U - esatta(Rn, T, Om)))
        # ampiezza e sfasamento della portata rispetto al gradiente (cos T)
        fasi = np.linspace(0, 2 * np.pi, 361)
        Qs = np.array([2 * simpson(esatta(Rn, T0, Om) * Rn, x=Rn) for T0 in fasi])
        sfas = np.degrees(fasi[np.argmax(Qs)])
        print(f"   Omega = {Om:5.1f} (alfa_W = {np.sqrt(Om):.2f}): errore CN dopo {periodi} periodi = {err:.1e};"
```

```
                f" portata max {Qs.max():.4f} (quasi stazionaria: {2*0.25:.4f}),
    ritardo di fase {sfas:.1f} gradi")
        for k, col in zip(range(8), PALETTE + [GRIGIO]):
            T0 = k * np.pi / 4
            a_.plot(esatta(Rn, T0, Om), Rn, color=col, lw=1.0)
        a_.axvline(0, color="k", lw=0.5)
        a_.set_title(f"$\\Omega={Om:g}$".replace(".", "{,}"))
        a_.set_xlabel("$U$")
    ax[0].set_ylabel("$R=r/A$")
    fig.tight_layout()
    salva(fig, "es_18")


if __name__ == "__main__":
    scelti = [int(a) for a in sys.argv[1:]] or range(1, 19)
    for k in scelti:
        nome = f"es_{k:02d}"
        if nome in globals():
            globals()[nome]()
```

Listato B.1: `esercizi_svolti.py`: codice di tutti gli esercizi svolti.

Bibliografia

Testi di riferimento del modulo

[1] S. Biringen, C.-Y. Chow, *An Introduction to Computational Fluid Mechanics by Example*, Wiley, Hoboken, 2011. Capitolo 3: *Viscous Fluid Flows*, pp. 145–214. Programmi MATLAB su `www.wiley.com/go/biringen`.

[2] C.-Y. Chow, *An Introduction to Computational Fluid Mechanics*, Wiley, New York, 1979 (edizione originale, con programmi FORTRAN).

Meccanica dei fluidi viscosi e strato limite

[3] H. Schlichting, K. Gersten, *Boundary-Layer Theory*, 9ª ed., Springer, Berlin, 2017. Cap. 5–7: soluzioni esatte e soluzioni di similitudine dello strato limite (Blasius, Falkner–Skan, aspirazione e iniezione); Cap. 9: strato limite termico; Cap. 15: stabilità e onde di Tollmien–Schlichting.

[4] F. M. White, *Viscous Fluid Flow*, 3ª ed., McGraw-Hill, New York, 2006. Cap. 2: equazioni di Navier–Stokes e funzione di dissipazione; Cap. 3: soluzioni esatte, condotti di varie sezioni, correnti non stazionarie; Cap. 4: strato limite laminare e fattore di recupero; Cap. 5: stabilità.

[5] G. K. Batchelor, *An Introduction to Fluid Dynamics*, Cambridge University Press, 1967. Cap. 3: equazioni del moto dei fluidi viscosi; Cap. 4: soluzioni esatte, correnti di Couette e di Poiseuille, avviamento delle correnti, problemi di Rayleigh e di Stokes, moto lento di Stokes.

[6] P. K. Kundu, I. M. Cohen, D. R. Dowling, *Fluid Mechanics*, 6ª ed., Academic Press, Waltham, 2016. Cap. 8–10: correnti viscose laminari, strato limite, instabilità.

[7] H. Blasius, *Grenzschichten in Flüssigkeiten mit kleiner Reibung*, Zeitschrift für Mathematik und Physik **56** (1908) 1–37.

[8] L. Howarth, *On the solution of the laminar boundary layer equations*, Proceedings of the Royal Society of London A **164** (1938) 547–579.

[9] V. M. Falkner, S. W. Skan, *Some approximate solutions of the boundary layer equations*, Philosophical Magazine **12** (1931) 865–896.

[10] D. R. Hartree, *On an equation occurring in Falkner and Skan's approximate treatment of the equations of the boundary layer*, Proceedings of the Cambridge Philosophical Society **33** (1937) 223–239.

[11] E. Pohlhausen, *Der Wärmeaustausch zwischen festen Körpern und Flüssigkeiten mit kleiner Reibung und kleiner Wärmeleitung*, Zeitschrift für Angewandte Mathematik und Mechanik **1** (1921) 115–121.

[12] R. K. Shah, A. L. London, *Laminar Flow Forced Convection in Ducts*, Academic Press, New York, 1978 (raccolta di soluzioni per condotti di sezione rettangolare, triangolare, anulare e di forma qualsiasi).

[13] H. K. Moffatt, *Viscous and resistive eddies near a sharp corner*, Journal of Fluid Mechanics **18** (1964) 1–18.

[14] K. Pohlhausen, *Zur näherungsweisen Integration der Differentialgleichung der laminaren Grenzschicht*, Zeitschrift für Angewandte Mathematik und Mechanik **1** (1921) 252–268.

[15] C. W. Oseen, *Über die Stokes'sche Formel und über eine verwandte Aufgabe in der Hydrodynamik*, Arkiv för Matematik, Astronomi och Fysik **6** (1910) n. 29.

[16] J. R. Womersley, *Method for the calculation of velocity, rate of flow and viscous drag in arteries when the pressure gradient is known*, Journal of Physiology **127** (1955) 553–563.

Metodi numerici

[17] J. C. Tannehill, D. A. Anderson, R. H. Pletcher, *Computational Fluid Mechanics and Heat Transfer*, 2ª ed., Taylor & Francis, Washington, 1997. Cap. 3: analisi di stabilità di von Neumann, equazione modificata; Cap. 4: equazione del calore, schemi esplicito, implicito, di Crank–Nicolson e di DuFort–Frankel; Cap. 7: equazioni dello strato limite.

[18] J. H. Ferziger, M. Perić, *Computational Methods for Fluid Dynamics*, 3ª ed., Springer, Berlin, 2002. Cap. 3: differenze finite; Cap. 5: metodi iterativi; Cap. 6: problemi non stazionari e schemi a più livelli; Cap. 7: formulazione in funzione di corrente e vorticità.

[19] S. C. Chapra, R. P. Canale, *Numerical Methods for Engineers*, 7ª ed., McGraw-Hill, New York, 2015. Cap. 25 e 27: metodi di Runge–Kutta e metodo del tiro; Cap. 30: equazioni paraboliche.

[20] P. Moin, *Fundamentals of Engineering Numerical Analysis*, 2ª ed., Cambridge University Press, 2010. Cap. 4–5: equazioni differenziali ordinarie e paraboliche; Cap. 6: metodi spettrali di Fourier e di Chebyshev, schemi misti espliciti–impliciti.

[21] R. J. LeVeque, *Finite Volume Methods for Hyperbolic Problems*, Cambridge University Press, 2002. Cap. 8: convergenza, consistenza e stabilità; teorema di equivalenza di Lax.

[22] J. Crank, P. Nicolson, *A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type*, Proceedings of the Cambridge Philosophical Society **43** (1947) 50–67.

[23] E. C. DuFort, S. P. Frankel, *Stability conditions in the numerical treatment of parabolic differential equations*, Mathematical Tables and Other Aids to Computation **7** (1953) 135–152.

[24] D. W. Peaceman, H. H. Rachford, *The numerical solution of parabolic and elliptic differential equations*, Journal of the Society for Industrial and Applied Mathematics **3** (1955) 28–41.

[25] R. Rannacher, *Finite element solution of diffusion problems with irregular data*, Numerische Mathematik **43** (1984) 309–327.

[26] J. W. Cooley, J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Mathematics of Computation **19** (1965) 297–301.

[27] A. Thom, *The flow past circular cylinders at low speeds*, Proceedings of the Royal Society of London A **141** (1933) 651–669.

Stabilità idrodinamica e metodi spettrali

[28] P. G. Drazin, W. H. Reid, *Hydrodynamic Stability*, 2ª ed., Cambridge University Press, 2004. Cap. 2: convezione di Bénard; Cap. 4: stabilità delle correnti parallele, equazione di Orr–Sommerfeld, criterio di Rayleigh.

[29] S. Chandrasekhar, *Hydrodynamic and Hydromagnetic Stability*, Clarendon Press, Oxford, 1961 (ristampa Dover, New York, 1981). Cap. II: il problema di Bénard con pareti rigide e libere.

[30] Lord Rayleigh, *On convection currents in a horizontal layer of fluid, when the higher temperature is on the under side*, Philosophical Magazine **32** (1916) 529–546.

[31] A. Pellew, R. V. Southwell, *On maintained convective motion in a fluid heated from below*, Proceedings of the Royal Society of London A **176** (1940) 312–343.

[32] H. B. Squire, *On the stability for three-dimensional disturbances of viscous fluid flow between parallel walls*, Proceedings of the Royal Society of London A **142** (1933) 621–628.

[33] R. Fjørtoft, *Application of integral theorems in deriving criteria of stability for laminar flows and for the baroclinic circular vortex*, Geofysiske Publikasjoner **17** (1950) 1–52.

[34] G. B. Schubauer, H. K. Skramstad, *Laminar boundary-layer oscillations and transition on a flat plate*, NACA Report 909, 1948.

[35] S. A. Orszag, *Accurate solution of the Orr–Sommerfeld stability equation*, Journal of Fluid Mechanics **50** (1971) 689–703.

[36] S. A. Orszag, *Comparison of pseudospectral and spectral approximation*, Studies in Applied Mathematics **51** (1972) 253–259.

[37] L. N. Trefethen, *Spectral Methods in MATLAB*, SIAM, Philadelphia, 2000. Cap. 1–3: differenziazione di Fourier, aliasing; Cap. 5: fenomeno di Runge, nodi di Chebyshev, ellisse di Bernstein; Cap. 6–7: matrice di differenziazione di Chebyshev, problemi ai limiti; Cap. 9 e 14: problemi agli autovalori e di quarto ordine. Si veda anche L. N. Trefethen, A. E. Trefethen, S. C. Reddy, T. A. Driscoll, *Hydrodynamic stability without eigenvalues*, Science **261** (1993) 578–584, per la crescita transitoria.

[38] C. Canuto, M. Y. Hussaini, A. Quarteroni, T. A. Zang, *Spectral Methods: Fundamentals in Single Domains*, Springer, Berlin, 2006. Cap. 2: approssimazioni di Fourier e di Chebyshev; Cap. 4: metodi di collocazione.